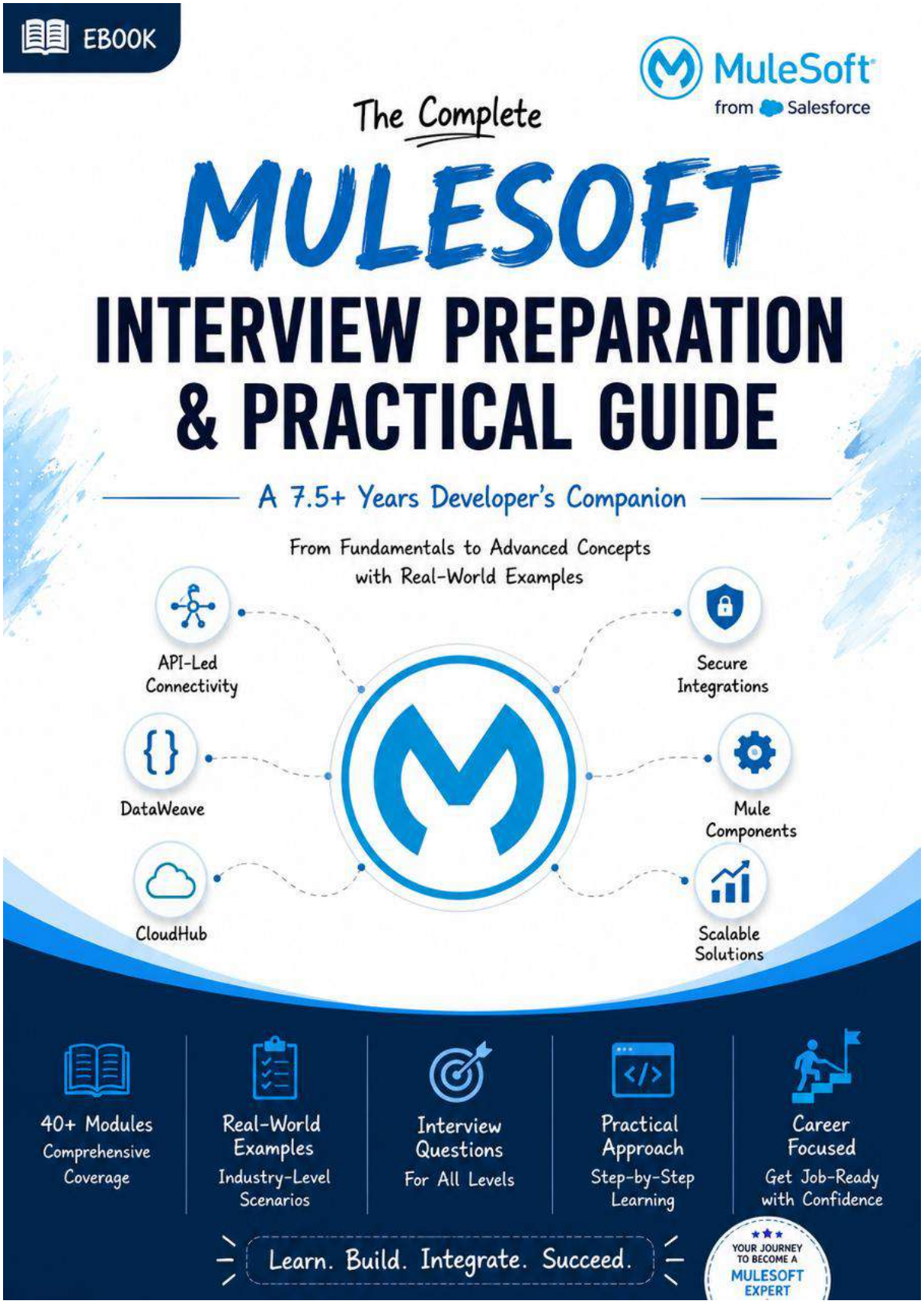

EBOOK
MuleSoft
from Salesforce
The Complete
MULESOFT
INTERVIEW PREPARATION
& PRACTICAL GUIDE
A 7.5+ Years Developer's Companion
From Fundamentals to Advanced Concepts
with Real-World Examples
API-Led Connectivity
Secure Integrations
DataWeave
Mule Components
CloudHub
Scalable Solutions
40+ Modules
Comprehensive Coverage
Real-World Examples
Industry-Level Scenarios
Interview Questions
For All Levels
Practical Approach
Step-by-Step Learning
Career Focused
Get Job-Ready with Confidence
Learn. Build. Integrate. Succeed.
YOUR JOURNEY TO BECOME A MULESOFT EXPERT

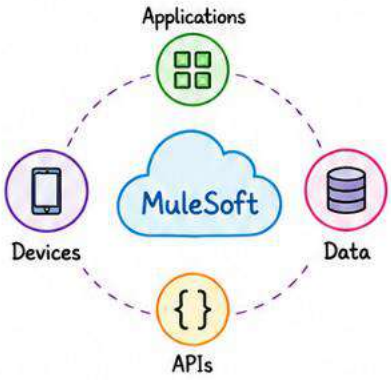
1.1 WHAT IS MULESOFT?

MuleSoft is a leading integration and automation platform that helps connect applications, data, and devices across cloud and on-premises environments.

MuleSoft from salesforce

KEY POINTS

- MuleSoft is an iPaaS (Integration Platform as a Service).
- It enables organizations to build, manage and compose APIs and integrations.
- It provides a unified platform – Anypoint Platform to design, build, deploy and manage integrations.
- It supports both cloud and on-premises deployments.
- It helps achieve real-time integration and process automation.



WHY MULESOFT?

CONNECT ANYTHING
Connect applications, data, devices and APIs with ease.

BUILD FASTER
Reusable components and accelerators speed up development.

SCALE ANYWHERE
Deploy anywhere – CloudHub, on-premises or edge.

SECURE BY DESIGN
Built-in security features to protect your integrations.

REDUCE COST
Lower operational costs and improve business agility.

KEY FEATURES

- ☑ API-led connectivity
- ☑ Rich set of connectors
- ☑ Visual development (Anypoint Studio)
- ☑ Data transformation (DataWeave)
- ☑ Management & monitoring
- ☑ Security & governance
- ☑ Scalability & high availability
- ☑ Reusable assets (Exchange)

MULESOFT ECOSYSTEM – ANYPOINT PLATFORM

Design Center
Design and mock APIs visually

Anypoint Studio
Build integrations and APIs

Runtime Manager
Deploy and manage applications

Access Manager
Manage users and permissions

API Manager
Manage, secure and analyze APIs

Exchange
Reuse connectors, templates and APIs

REAL-WORLD USE CASES

Banking
Connect core banking systems with digital channels.

Retail & eCommerce
Integrate inventory, orders, payments and shipping systems.

Healthcare
Connect patient data, devices and healthcare applications.

Travel & Hospitality
Integrate booking, payments, notifications and loyalty systems.

Manufacturing
Connect machines, sensors and enterprise applications.

MuleSoft helps organizations to CONNECT, AUTOMATE and TRANSFORM their business with APIs and integrations.

1.2 API-LED CONNECTIVITY

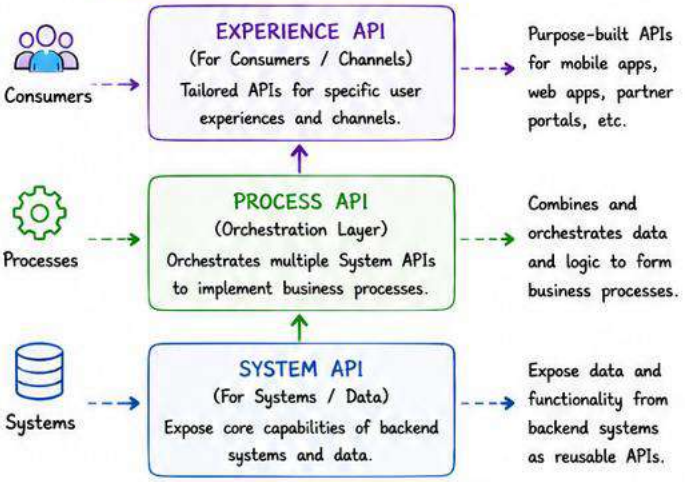
MuleSoft from salesforce

API-Led Connectivity is an architectural approach that uses **APIs** as the **building blocks** to connect applications, data, and services in a **reusable, scalable** and **secure** way.

WHAT IS IT?

In API-Led Connectivity, we expose and consume capabilities as **APIs** in a layered manner. Each API layer has a **specific purpose** and can be **reused** by other applications.

THREE-LAYER ARCHITECTURE



WHY API-LED CONNECTIVITY?

- Promotes **reusability** of APIs
- Improves **scalability** and **agility**
- Encourages **consistency** and **standardization**
- Enhances **maintainability**
- Speeds up development and **time-to-market**
- Supports **secure** and **governed** integrations

KEY PRINCIPLES

- Everything is an **API**
- Design for **reuse**
- Use **loose coupling**
- Think at **scale**
- **Security** and **governance** by design

BENEFITS

REUSABILITY
APIs are reusable across multiple applications.

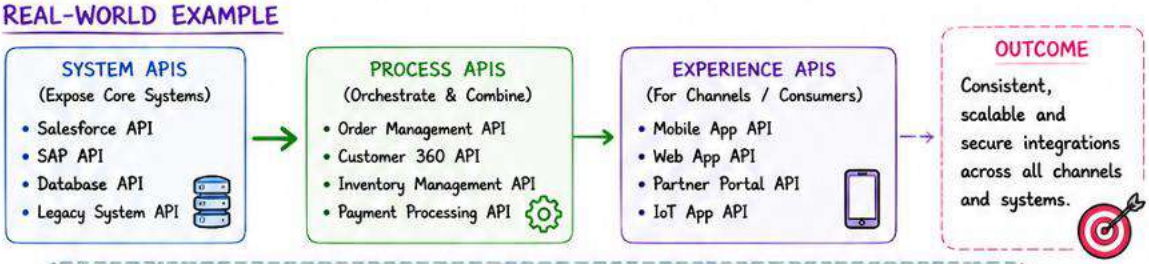
SCALABILITY
Easily scale by reusing existing APIs.

FASTER DELIVERY
Build new applications faster using existing building blocks.

SECURITY
Centralized security and consistent policies.

MAINTAINABILITY
Changes in one layer have minimal impact on others.

REAL-WORLD EXAMPLE



API-Led Connectivity helps organizations build a **flexible** and **future-ready** integration architecture that grows with business needs.

1.3 THREE-LAYER ARCHITECTURE

MuleSoft from salesforce

The Three-Layer Architecture is a core principle of API-Led Connectivity. It organizes APIs into three logical layers, each with a specific role and purpose.

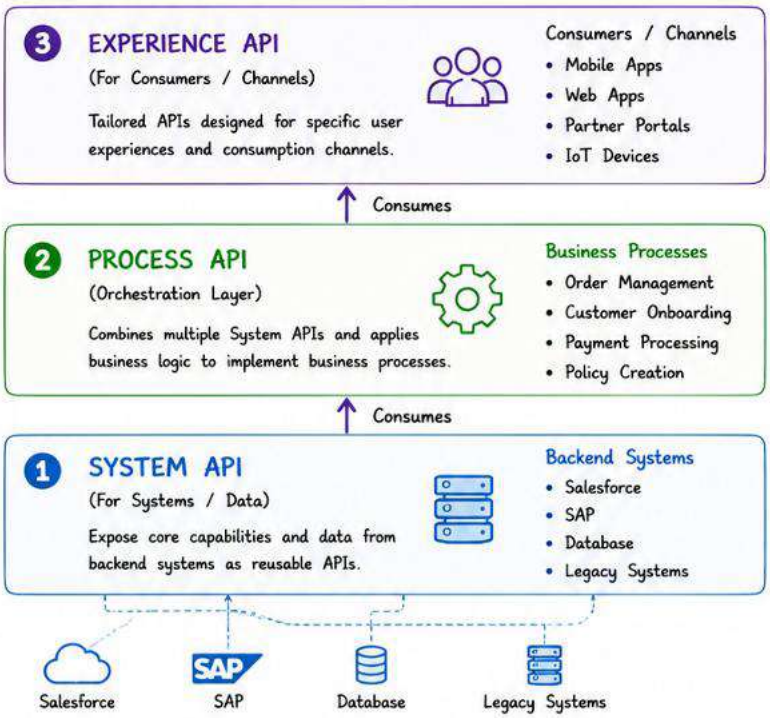
OVERVIEW

This architecture promotes separation of concerns, reusability, scalability and maintainability of APIs and integrations.

LAYERS AT A GLANCE

1. **System API**
   Expose data and capabilities from backend systems.
2. **Process API**
   Orchestrate and combine System APIs to implement business processes.
3. **Experience API**
   Expose Process APIs to consumers through different channels.

ARCHITECTURE DIAGRAM



LAYER DETAILS

1 SYSTEM API (For Systems / Data)

- Expose data and functions from backend systems.
- Technology-focused.
- Reusable across multiple processes.
- Examples: GET /customers, GET /products

2 PROCESS API (Orchestration Layer)

- Orchestrate and combine multiple System APIs.
- Implement business logic and rules.
- Reusable across different experience layers.
- Examples: /processOrder, /createCustomer

3 EXPERIENCE API (For Consumers / Channels)

- Designed for specific consumer needs.
- Optimized for user experience.
- May aggregate multiple Process APIs.
- Examples: /mobile/orders, /partner/quotes

KEY BENEFITS

REUSABILITY	APIs in lower layers are reused across multiple processes and channels.
SCALABILITY	Changes in one layer have minimal impact on other layers.
SEPARATION OF CONCERNS	Each layer has a specific responsibility, making it easier to design and maintain.
FLEXIBILITY	New channels or systems can be added without affecting existing layers.
GOVERNANCE & SECURITY	Centralized policies, security and governance can be applied.

BEST PRACTICES

- Keep APIs in their appropriate layer.
- Avoid exposing System APIs directly to consumers.
- Design APIs to be reusable and loosely coupled.
- Use consistent naming and versioning strategies.
- Apply security and governance at all layers.

The Three-Layer Architecture is the foundation of API-Led Connectivity. It helps build reusable, scalable, secure and future-ready integrations.

1.4 MULE APPLICATION LIFECYCLE

MuleSoft from salesforce

The Mule Application Lifecycle represents the end-to-end journey of a Mule application from initial design to ongoing maintenance in production.

LIFECYCLE OVERVIEW

1 DESIGN
Understand requirements and design APIs and integrations using API-led connectivity approach.

2 DEVELOPMENT
Build the Mule application using Anypoint Studio, implement business logic, and configure connectors.

3 TESTING
Test the application using MUnit, functional testing, and integration testing to ensure quality.

4 PACKAGING
Package the application as a deployable artifact (JAR) with all dependencies and resources.

5 DEPLOYMENT
Deploy the packaged application to the target environment (CloudHub, On-Prem, RTF, etc.).

6 RUNNING
Application is executed by Mule Runtime. It starts processing events and messages as per configuration.

MONITORING & MAINTENANCE
Monitor performance, logs, and alerts. Apply updates, fix issues and improve continuously.

DETAILS OF EACH STAGE

1 DESIGN
- Analyze business requirements.
- Identify systems, data, and processes.
- Design APIs using RAML in Design Center.
- Define flows, integrations, and error scenarios.

2 DEVELOPMENT
- Build flows and sub-flows in Anypoint Studio.
- Implement business logic and transformations (DataWeave).
- Configure connectors and global elements.
- Follow best practices and coding standards.

3 TESTING
- Write unit tests using MUnit.
- Perform functional testing of flows and APIs.
- Run integration/system testing.
- Ensure code coverage and fix defects.

4 PACKAGING
- Build the project using Maven.
- Package as a deployable JAR/WAR file.
- Ensure all dependencies and configurations are included.
- Validate the packaged artifact.

5 DEPLOYMENT
- Deploy to target environment (CloudHub / On-Prem / RTF).
- Configure environment properties and secure properties.
- Use Runtime Manager or CI/CD pipeline for deployment.
- Verify connectivity and configurations.

6 RUNNING
- Mule Runtime starts the application.
- Listens on configured listeners (HTTP, MQ, etc.).
- Processes incoming events/messages.
- Handles transactions, errors and security.

7 MONITORING & MAINTENANCE
- Monitor logs, metrics and performance in Anypoint Monitoring.
- Set alerts and notifications.
- Analyze and troubleshoot issues.
- Apply patches, updates and enhancements.
- Continuously improve and optimize.

KEY POINTS

- The lifecycle is iterative, not always linear.
- Feedback from monitoring helps improve design and development.
- Automation (CI/CD) helps in faster and reliable delivery.
- Following best practices at each stage ensures high-quality and scalable integrations.

TOOLS & COMPONENTS USED

Design	→ Anypoint Design Center
Development	→ Anypoint Studio
Testing	→ MUnit, Postman, DataWeave
Packaging	→ Maven
Deployment	→ Runtime Manager, CloudHub, On-Prem, RTF, Kubernetes
Monitoring	→ Anypoint Monitoring, Runtime Manager

BENEFITS OF FOLLOWING LIFECYCLE

- ☑ Ensures a structured and standard approach.
- ☑ Improves quality, reliability and performance.
- ☑ Supports collaboration across teams.
- ☑ Enables faster time-to-market.
- ☑ Simplifies maintenance and future enhancements.

Following the Mule Application Lifecycle ensures that applications are well-designed, thoroughly tested, securely deployed and continuously improved for business success.

1.5 MULE EVENT

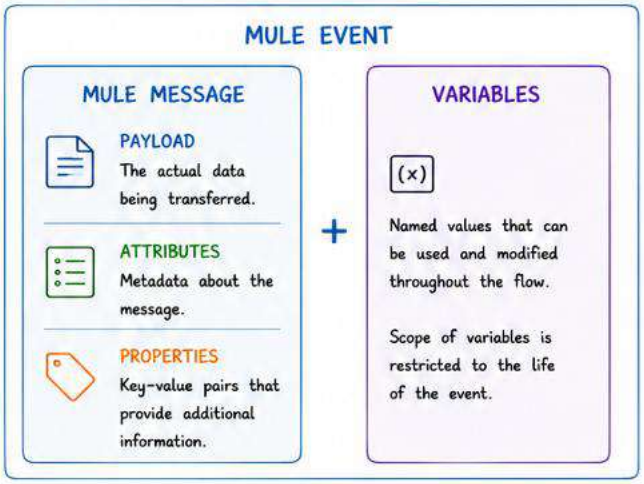
MuleSoft from salesforce

A Mule Event is the fundamental unit of processing in Mule runtime. It represents a message that flows through the application from the source to the destination.

WHAT IS A MULE EVENT?

- It is an object that carries data through the flow.
- It consists of a Mule Message and associated variables.
- It is created by a source and processed by processors in the flow.
- It is passed from one processor to another until it reaches the final destination.

COMPONENTS OF MULE EVENT



EVENT LIFECYCLE

Event Created
A source creates a Mule Event.

Event Processed
The event is passed through various processors.

Event Transformed
Payload, attributes, properties or variables may be modified.

Event Routed
The event is routed to the next processor or flow.

Event Completed
The event reaches the final destination and processing completes.

MULE EVENT STRUCTURE

KEY POINTS

- Mule Event flows through the application and holds all the data.
- It is immutable in structure but content can be modified.
- Each processor receives an event, operates on it and passes it on.
- Variables live only for the lifetime of the event.
- Understanding Mule Event is key to understanding Mule applications.

EXAMPLE

Consider an HTTP Listener receives a request.

```
Mule Event
{
  "message": {
    "payload": { "id": 101, "name": "John" },
    "attributes": {
      "httpStatus": 200,
      "method": "GET",
      "headers": { "content-type": "application/json" }
    },
    "properties": {
      "source": "HTTP Listener",
      "correlationId": "12345"
    }
  },
  "variables": {
    "userId": 101,
    "isPremium": true
  }
}
```

IMPORTANT

Everything in Mule is event-driven. Sources create events, processors transform events, and sinks consume events.

1.6 MULE MESSAGE

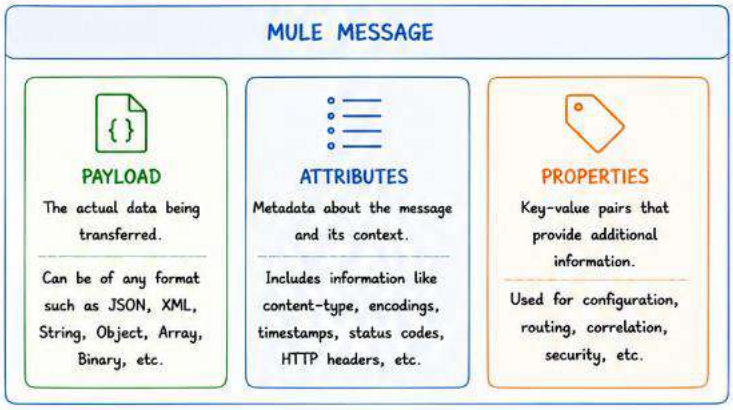
MuleSoft from salesforce

A Mule Message is the core part of a Mule Event. It carries the data that flows through the application along with its metadata.

WHAT IS A MULE MESSAGE?

- It is an immutable data container that holds the data and metadata in a Mule Event.
- It travels through the flow and may be modified by processors.
- It consists of Payload, Attributes and Properties.

STRUCTURE OF MULE MESSAGE



COMPONENTS IN DETAIL

1. PAYLOAD

- Contains the business data.
- This is what we actually want to transfer or process.
- It is the primary data of the message.
- Example: JSON, XML, CSV, Text, Binary, Object.

2. ATTRIBUTES

- Contains metadata about the message.
- Helps in routing, transformation and processing.
- Common attributes: content-type, content-encoding, httpStatus, headers, mediaType, characterEncoding.
- Automatically set by connectors or processors.

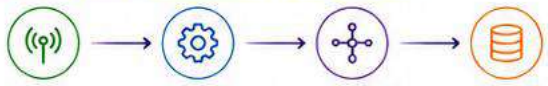
3. PROPERTIES

- User-defined key-value pairs.
- Used to store additional information for processing.
- Not transmitted as part of the payload.
- Example: correlationId, userId, config values.

EXAMPLE OF MULE MESSAGE

```
{
  "payload": {                              // Business Data
    "id": 101,
    "name": "John",
    "amount": 2500
  },
  "attributes": {
    "content-type": "application/json",     // Media type
    "content-encoding": "UTF-8",            // Encoding
    "httpStatus": 200,                      // HTTP Status
    "headers": {                            // HTTP Headers
      "host": "api.example.com",
      "accept": "application/json"
    }
  },
  "properties": {
    "correlationId": "12345",
    "sourceSystem": "CRM",
    "retryCount": 0                         // Custom property
  }
}
```

MESSAGE FLOW IN A MULE APPLICATION



COMMONLY USED ATTRIBUTES

ATTRIBUTE	DESCRIPTION
content-type	MIME type of the payload
content-encoding	Character encoding (e.g., UTF-8)
httpStatus	HTTP status code
headers	Map of HTTP headers
mediaType	Media type of the payload
characterEncoding	Character encoding of the payload

KEY POINTS

- Mule Message is part of Mule Event.
- It contains Payload, Attributes and Properties.
- It is immutable, but a new message is created when modified.
- Understanding Mule Message is key to effective data flow and integration.

BEST PRACTICES

- Use attributes for standard metadata.
- Use properties for custom information needed across the flow.
- Avoid storing large data in properties.
- Maintain message immutability and follow transformation best practices.

A Mule Message carries the right data, at the right time, in the right format to the right destination.

1.7 MULE RUNTIME

MuleSoft from salesforce

Mule Runtime is the powerful engine that executes Mule applications. It provides the environment, services, and infrastructure required to run, manage and monitor applications.

WHAT IS MULE RUNTIME?

- Mule Runtime is the core engine of the Mule platform.
- It executes Mule applications and manages events as they flow through the application.
- It provides infrastructure services such as threading, connectivity, transaction management, security, logging and more.
- Mule applications are deployed on Mule Runtime.

KEY RESPONSIBILITIES

EXECUTION
Executes flows, sub-flows and processors.

EVENT MANAGEMENT
Creates, manages and routes events through the flow.

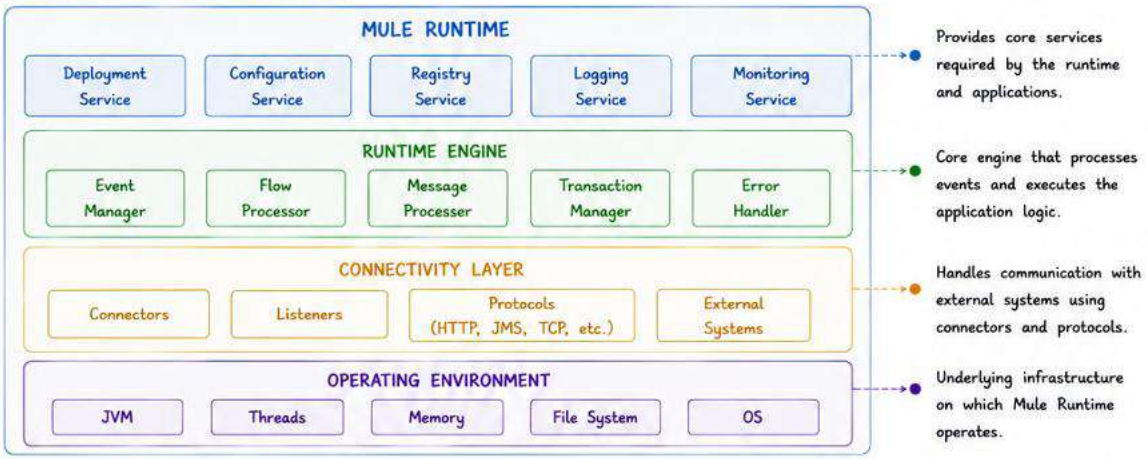
CONNECTIVITY
Manages connectors, listeners and external connections.

SECURITY
Provides authentication, authorization and encryption support.

MONITORING & LOGGING
Collects metrics, logs and enables monitoring.

RESOURCE MANAGEMENT
Manages threads, memory, transactions and other resources.

MULE RUNTIME ARCHITECTURE



HOW IT WORKS (EVENT FLOW)

1. **EVENT RECEIVED**
   A source (e.g., HTTP Listener) receives a request and creates a Mule Event.
2. **EVENT PROCESSING**
   Runtime Engine processes the event through flows and processors.
3. **ROUTING & TRANSFORMATION**
   Event is routed, transformed or enriched as per the flow logic.
4. **EXTERNAL INTERACTION**
   Connectors are used to interact with external systems if required.
5. **RESPONSE SENT**
   The response is returned to the source, completing the flow.

RUNTIME SERVICES (IMPORTANT)

SERVICE	DESCRIPTION
Deployment Service	Deploys and undeploys applications.
Configuration Service	Loads and provides configuration to applications.
Registry Service	Manages connectors and shared resources.
Logging Service	Provides logging framework for applications.
Monitoring Service	Exposes metrics and health of applications.
Object Store Service	Stores and manages persistent data if required.
Scheduler Service	Handles scheduled tasks in applications.

KEY FEATURES

- Lightweight and high performance.
- Scalable and supports clustering.
- Built-in security and message management.
- Extensible with connectors and modules.
- Cloud-ready and on-premise capable.

BENEFITS

- Faster development and deployment.
- High reliability and availability.
- Better resource utilization.
- Centralized management and monitoring.
- Supports enterprise integration needs.

RUNTIME MODES

- **Standalone** : Runs Mule application as a standalone process.
- **Clustered** : Multiple Mule runtimes run together for high availability and scaling.
- **Managed Cloud** : Runtime is managed by Anypoint Platform (CloudHub).

Mule Runtime is the heart of MuleSoft, providing a robust, secure and scalable execution environment for building and running integration solutions.

1.8 MULE DOMAINS

MuleSoft from salesforce

A Mule Domain is a logical container that provides a dedicated runtime environment for deploying and running Mule applications. It isolates applications, configurations, resources and policies from other domains.

WHAT IS A MULE DOMAIN?

- A Mule Domain is a runtime environment that hosts Mule applications.
- It provides isolation, resource management, and configuration for the applications deployed in it.
- Each domain has its own configuration, policies, security settings, logging, threading and resources.
- Multiple domains can coexist in the same Mule Runtime instance.

WHY USE MULE DOMAINS?

ISOLATION
Ensure applications are isolated from each other for stability and security.

CONFIGURATION
Each domain has its own configuration, policies and properties.

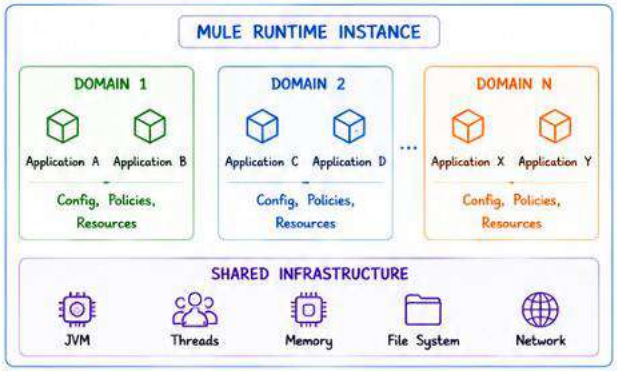
RESOURCE MANAGEMENT
Allocate and manage resources like threads, memory and connections per domain.

SECURITY
Apply domain-level security, authentication and access control.

MONITORING
Monitor and track metrics, logs and performance per domain.

SCALABILITY
Run multiple domains in a single runtime for better utilization and scaling.

MULE DOMAIN ARCHITECTURE



KEY CHARACTERISTICS

FEATURE	DESCRIPTION
Logical Container	A domain is a logical container within Mule Runtime.
Isolation	Applications in one domain are isolated from other domains.
Independent Config	Each domain has its own configuration, properties and policies.
Dedicated Resources	Resources (threads, memory, connection pools, etc.) are managed per domain.
Multiple Applications	Multiple Mule applications can be deployed in the same domain.
Lifecycle Management	Domains can be started, stopped, deployed, undeployed independently.
Monitoring & Logging	Metrics and logs are captured per domain for better visibility.

TYPES OF MULE DOMAINS

DEFAULT DOMAIN
- Created by default in every Mule Runtime.
- Used when no specific domain is configured.
- Usually named "default".

CUSTOM DOMAINS
- Created manually as per application needs.
- Can be many in one runtime instance.
- Have custom configurations and policies.

HOW DOMAINS WORK

1. CREATE DOMAIN – Create a domain with desired configurations, resources and policies.
2. DEPLOY APPLICATION – Deploy Mule applications to the target domain.
3. START DOMAIN – Mule Runtime starts the domain and initializes resources.
4. PROCESS EVENTS – Applications in the domain process events and interact with external systems.
5. STOP/UNDEPLOY DOMAIN – Stop or undeploy the domain as needed.

EXAMPLE SCENARIO

MULE RUNTIME INSTANCE

Domain: default
- Application: Order API
- Application: Product API

Domain: finance
- Application: Invoice API
- Application: Payment API

Domain: hr
- Application: Employee API
- Application: Leave API

In this setup:
- Order and Product APIs run in the default domain.
- Invoice and Payment APIs run in the finance domain.
- Employee and Leave APIs run in the hr domain.

All domains share the same Mule Runtime instance but remain isolated.

BEST PRACTICES

- Create separate domains for different applications or teams.
- Use meaningful domain names based on business or environment.
- Allocate resources (threads, memory, connection pools) appropriately per domain.
- Externalize configurations and use domain-level properties.
- Apply security policies at the domain level.
- Monitor domains separately for better visibility and troubleshooting.
- Regularly review and optimize domain configurations and resources.

Mule Domains provide isolation, control and flexibility to run multiple applications securely and efficiently within the same Mule Runtime instance.

1.9 MULE PROJECT STRUCTURE

MuleSoft from salesforce

A Mule project is an Anypoint Studio project that contains all the resources, configuration files and assets required to build, run and deploy a Mule application.

TYPICAL MULE PROJECT STRUCTURE

- my-mule-project
  - src — Holds all the source code and Mule components.
    - main — Contains the main application code.
      - mule — Mule flows, sub-flows, configurations.
      - java — Custom Java classes.
      - resources — Property files, schemas, static resources.
    - test — Contains test flows and test resources.
  - target — Build output, compiled classes and packaged application.
  - pom.xml — Maven Project Object Model file.
  - mule-artifact.json — Mule application descriptor.
  - README.md — Project information and documentation.
  - .gitignore — Git ignore configuration.

KEY DIRECTORIES AND FILES

ITEM	DESCRIPTION
src/main/mule	• Contains Mule flows, sub-flows and configuration files. • Includes global elements and API implementations.
src/main/java	• Contains custom Java classes, utilities and modules. • Used when custom logic beyond DataWeave is required.
src/main/resources	• Stores property files, schemas, JSON/XML samples, certificates, static content, etc. • Used by the application at runtime.
src/test	• Contains test flows, mock data and test resources. • Used for automated testing with MUnit.
target	• Output directory generated by Maven build. • Contains compiled classes and packaged application (JAR/WAR).
pom.xml	• Defines project dependencies, plugins and build configurations. • Standard Maven configuration file.
mule-artifact.json	• Describes the Mule application. • Contains the app name, version, min Mule version, secure properties, etc.
README.md	• Provides project overview, setup and usage instructions.
.gitignore	• Specifies files and folders to be ignored by Git.

IMPORTANT FILES EXPLAINED

pom.xml	• Manages dependencies (Mule libraries, connectors, etc.). • Defines build lifecycle and plugins. • Supports profiles for different environments.
mule-artifact.json	• Unique descriptor for every Mule application. • Contains app name, version and secure properties. • Used by Mule Runtime for deployment.
application.properties / application.yaml	• Application configuration properties. • Supports externalized configuration.
flows	• XML files defining Mule flows and sub-flows. • Core of the application logic.

HOW IT WORKS

1. **DEVELOP** — Developer builds flows, configurations and custom logic in the project structure.
2. **BUILD** — Maven builds the project and resolves dependencies.
3. **PACKAGE** — Mule application is packaged as a JAR/WAR in the target folder.
4. **DEPLOY** — Packaged application is deployed to Mule Runtime.
5. **RUN & MONITOR** — Application runs, processes events and can be monitored via Anypoint Platform.

BEST PRACTICES

- Follow a consistent folder structure across projects.
- Keep flows modular; use sub-flows and global elements.
- Externalize configurations using property files.
- Store sensitive information in secure properties.
- Write tests in src/test using MUnit.
- Keep dependencies updated and avoid unused libraries.
- Maintain clean and meaningful project documentation.
- Use version control (Git) to manage your projects.

A well-structured Mule project ensures code organization, maintainability, reusability and smooth development to deployment.

1.10 MULE-ARTIFACT.JSON

MuleSoft from salesforce

mule-artifact.json is a descriptor file that contains metadata about a Mule application. It is used by Anypoint Platform and Mule Runtime to identify, configure and run the application.

WHAT IS MULE-ARTIFACT.JSON?

- It is a JSON descriptor file.
- It contains information about the application such as name, version, min Mule version, secure properties, required plugins, etc.
- It is mandatory in every Mule application.
- It is located in the root directory of the Mule project.
- Mule Runtime reads this file to deploy and run the application.

PURPOSE

- Identifies the application to Mule Runtime and Anypoint Platform.
- Holds configuration about the application and its dependencies.
- Provides information about secure properties and their encryption.
- Helps in packaging, deploying and managing the application.

TYPICAL mule-artifact.json STRUCTURE

```
{
  "name": "my-mule-app",                    // Application name
  "minMuleVersion": "4.4.0",                // Minimum Mule version
  "secureProperties": [
    {
      "key": "db.password",
      "value": "![123456]"
    }
  ],                                        // Encrypted secure properties
  "requiredPlugins": [
    {
      "groupId": "org.mule.connectors",
      "artifactId": "mule-http-connector",
      "version": "1.8.1"
    }
  ],                                        // Required plugins
  "muleVersion": "4.4.0",                   // Target Mule version
  "classLoaderModelLoaderDescriptor": {
    "id": "mule",                           // Classloader model
    "version": "1.3.0"
  }
}
```

FIELD DESCRIPTION

FIELD	DESCRIPTION
name	The name of the Mule application.
minMuleVersion	The minimum Mule Runtime version required to run this application.
secureProperties	List of secure properties with their encrypted values.
requiredPlugins	List of plugins (connectors, modules) required by the application.
muleVersion	The target Mule version for which the application is built.
classLoaderModelLoaderDescriptor	Defines the classloader model used by the application.

KEY POINTS

- Always present in the root directory.
- Automatically generated when you create a new Mule project.
- Updated when you add plugins or change secure properties.
- Do not modify it manually unless required.
- Used during build, deploy and runtime.

mule-artifact.json

EXAMPLE (EXPLAINED)

- name : my-mule-app → The application name.
- minMuleVersion : 4.4.0 → Requires at least Mule 4.4.0.
- secureProperties : Contains encrypted sensitive data (e.g., database passwords, API keys).
- requiredPlugins : Lists connectors used in the application (e.g., HTTP connector).
- muleVersion : The Mule version used to build the app.
- classLoaderModelLoaderDescriptor : Defines how classes and dependencies are loaded at runtime.

LOCATION IN PROJECT

```
my-mule-app/
  src/
    mule-artifact.json  ← Located in root
    pom.xml
    README.md
```

LIFECYCLE USAGE

DEVELOP — Developer builds the application and adds flows, configurations and resources.

BUILD — Maven reads mule-artifact.json to resolve plugins and dependencies.

DEPLOY — Application is packaged (JAR/WAR) and deployed to Mule Runtime / CloudHub.

RUN — Mule Runtime reads this file at startup to run the application correctly.

BEST PRACTICES

- Keep secure data only in secureProperties.
- Let Anypoint Studio manage this file automatically.
- Do not store plain text passwords or secrets.
- Keep plugins list minimal and up to date.
- Commit this file to version control (Git).

NOTE

mule-artifact.json is small but powerful. It tells the platform everything it needs to know about your Mule application.

mule-artifact.json is the identity card of your Mule application. It ensures your app is built, deployed and run in the right way.

2.1 WHAT IS ANYPOINT PLATFORM?

MuleSoft from salesforce

Anypoint Platform is MuleSoft's integration platform that helps organizations connect applications, data, and devices securely. It provides a complete set of tools to design, build, test, deploy, manage and monitor APIs and integrations.

OVERVIEW

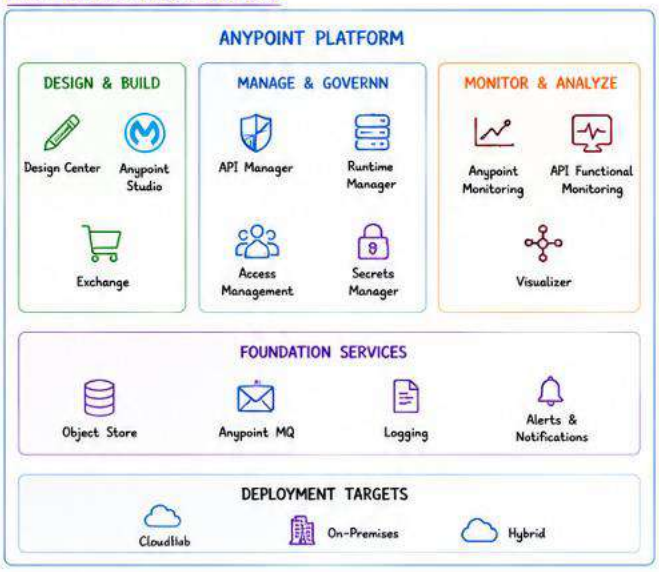
Anypoint Platform is a unified platform for API-led connectivity. It enables you to build reusable APIs and integrations and manage them across their entire lifecycle.

- It is a cloud-based platform (with on-premises runtime option).
- It provides a single interface to all tools and capabilities.
- It supports the entire API lifecycle.
- It promotes reuse, standardization, security and governance.
- It is built on the principles of API-led connectivity.

KEY COMPONENTS

COMPONENT	PURPOSE
Design Center	Design and document APIs using RAML.
Anypoint Studio	Build, test and debug Mule applications.
Exchange	Discover and share reusable assets.
API Manager	Secure, manage and analyze APIs.
Runtime Manager	Deploy, manage and monitor applications.
Access Management	Manage users, roles and permissions.
Anypoint Monitoring	Monitor performance and health.
Visualizer	Visualize API dependencies and flows.
Secrets Manager	Securely store and manage sensitive data.
Object Store	Store and retrieve data persistently.
Anypoint MQ	Reliable messaging for applications.

HIGH-LEVEL ARCHITECTURE



BENEFITS

USE CASES

HOW IT WORKS

DESIGN — Design APIs in Design Center using RAML.

BUILD & TEST — Build and test applications in Anypoint Studio.

DEPLOY — Deploy to Runtime Manager (CloudHub or On-Premises).

MANAGE & SECURE — Manage via API Manager and Access Management. Apply policies and control access.

MONITOR — Monitor performance, health and usage in Anypoint Monitoring.

ANYPOINT PLATFORM – AT A GLANCE

- A unified platform for API-led connectivity.
- Covers the entire API lifecycle – Design, Build, Deploy, Manage, Monitor.
- Provides powerful tools and services to accelerate integration.
- Enables secure, scalable and reliable connectivity.
- Integrates with any system, any data, at any time.

Anypoint Platform is the foundation for building, managing and scaling API-led connectivity in the enterprise.

2.2 DESIGN CENTER

MuleSoft from salesforce

Anypoint Design Center is a cloud-based API design tool that lets you design, document and collaborate on APIs using RAML. It enables you to create high-quality APIs quickly with built-in validation, mocking and rich documentation.

WHAT IS DESIGN CENTER?

- A web-based API design platform on Anypoint Platform.
- Used to design and document APIs using RAML API specification.
- Provides tools for validation, mocking, documentation and collaboration.
- Eliminates guesswork for developers by providing a contract-first approach.
- APIs designed here can be exported, published to Exchange or used in implementations.

KEY CAPABILITIES

API DESIGN
Design APIs using RAML with intelligent autocmeplete and syntax assistance.

VALIDATION
Real-time validation ensures RAML compliance and best practices.

MOCKING
Generate mocks from RAML and simulate API responses.

DOCUMENTATION
Auto-generate interactive API documentation (API Console).

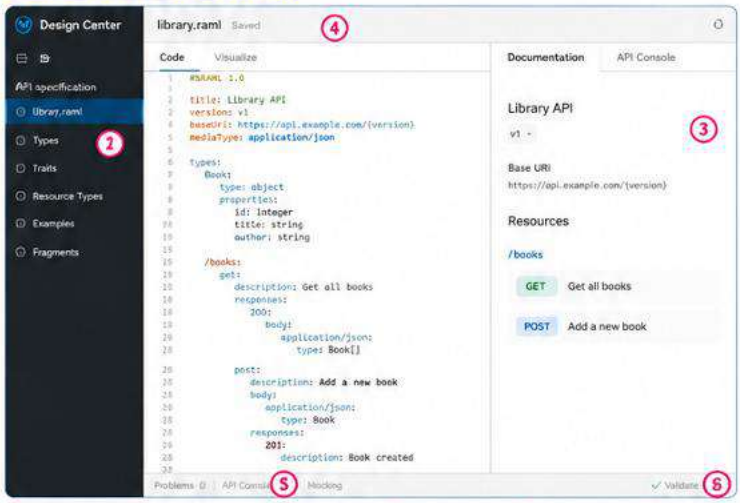
COLLABORATION
Collaborate in real time with team members.

VERSION CONTROL
Track changes and manage versions of API specifications.

IMPORT / EXPORT
Import / export RAML files and sync with other tools.

PUBLISH
Publish APIs or assets to Anypoint Exchange for reuse.

DESIGN CENTER WORKSPACE



① Navigator — Organize and navigate RAML assets.
② RAML Editor — Write and edit RAML with smart assistance.
③ Documentation — Auto-generated documentation.
④ View Switch — Switch between Code and Visualize views.
⑤ Tools — Access problems, mock and API console.
⑥ Validation — Validate RAML syntax and best practices.

SUPPORTED RAML VERSIONS

- RAML 1.0
- RAML 0.8 (Legacy)

DESIGN CENTER OUTPUTS

RAML File
API specification in RAML format.

API Console
Interactive documentation for testing.

Mock Service
Mock endpoints for API simulation.

Exchange Asset
Publish RAML to Exchange for reuse.

BENEFITS

- Speeds up API design and documentation.
- Ensures consistency and standardization.
- Enables early collaboration and feedback.
- Reduces development time with mocking.
- Promotes reusability through Exchange.
- Improves API quality with validation.

DESIGN CENTER WORKFLOW

CREATE — Create a new API specification or import existing RAML file.

DESIGN — Define resources, methods, types, traits, and examples.

VALIDATE — Validate RAML syntax and best practices in real time.

MOCK — Generate and test mock responses.

DOCUMENT — Auto-generate documentation and API console.

PUBLISH — Publish to Exchange or export for implementation.

BEST PRACTICES

- Design APIs using the contract-first approach.
- Follow RAML best practices and naming conventions.
- Use data types, traits and resource types for reusability.
- Validate frequently to maintain high-quality specifications.
- Collaborate and review with stakeholders early.
- Publish reusable assets to Exchange.

USE CASES

- Designing REST and SOAP APIs.
- Creating reusable API specifications for teams.
- Mocking APIs for parallel development.
- Generating documentation for consumers.
- Publishing assets for organization-wide reuse.

Design Center helps you design better APIs, faster and together!
Contract First. Design Smart. Deliver Quality APIs.

2.3 ANYPOINT STUDIO

MuleSoft from salesforce

Anypoint Studio is the official IDE for building Mule applications and APIs. It provides tools to design, develop, test, debug and deploy integrations on Anypoint Platform.

WHAT IS ANYPOINT STUDIO?

- Anypoint Studio is an Eclipse-based integrated development environment (IDE).
- Used to develop Mule applications, APIs and integrations.
- Provides visual flow designer and XML editor.
- Supports debugging, testing and deployment.
- Integrates seamlessly with Anypoint Platform.

KEY CAPABILITIES

VISUAL FLOW DESIGNER
Drag and drop components to build integrations visually.

XML EDITOR
View and edit the Mule configuration in XML format.

DEBUGGING
Debug flows with breakpoints, step over, step into and inspect variables.

RUN & TEST
Run applications locally, test flows and validate behavior.

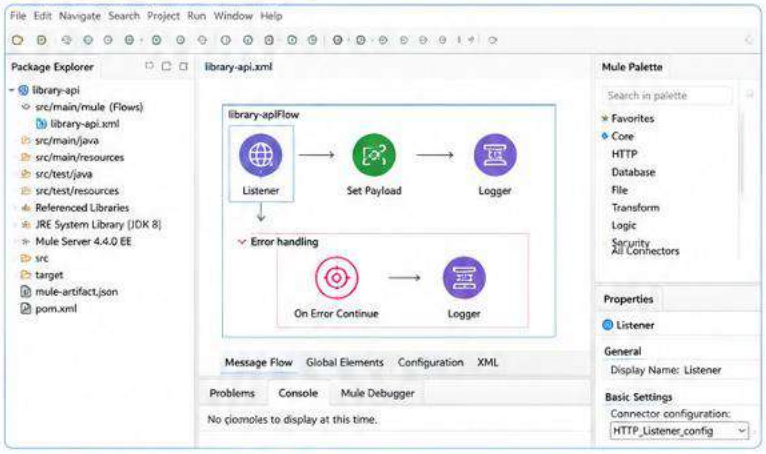
BUILD & PACKAGE
Build and package Mule applications as JAR/WAR files.

CONNECTORS
Access and configure Anypoint connectors easily.

GLOBAL ELEMENTS
Create and reuse global elements like Config, Error Handler, etc.

DEPLOYMENT
Deploy apps to Mule Runtime, CloudHub or any runtime environment.

STUDIO PERSPECTIVE OVERVIEW



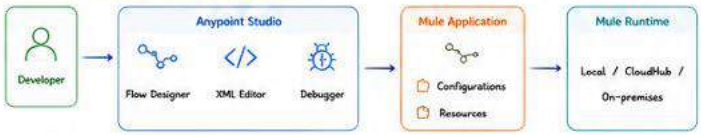
KEY FEATURES

- Graphical Flow Designer
- Rich set of connectors and modules
- XML configuration for advanced use
- Code auto-completion and validation
- Error highlighting and quick fixes
- Built-in debugger and breakpoints
- MUnit testing framework support
- Easy dependency management
- Integration with Git and CI/CD tools

SUPPORTS

- Mule 4.x
- Java (8 and above)
- MUnit for unit testing
- DataWeave 2.0
- CloudHub & On-premises deployment

ANYPOINT STUDIO ARCHITECTURE



SYSTEM REQUIREMENTS

Component	Minimum Requirement
OS	Windows 10/11, macOS 10.14+, Linux (64-bit)
RAM	8 GB (16 GB recommended)
Disk Space	5 GB (10 GB recommended)
JDK	Java 8 or 11 (64-bit)
Anypoint Studio	Latest version

STUDIO WORKFLOW

Create Project
Create a new Mule project or import existing project.

Design & Develop
Build flows using visual designer or XML editor.

Test & Debug
Run and debug the application locally using Mule Runtime.

Build
Build and package the application (JAR/WAR).

Deploy
Deploy to CloudHub or on-premises runtime.

Monitor
Monitor logs, metrics and performance on Anypoint Platform.

BEST PRACTICES

- Use global elements for reusability.
- Externalize configurations using properties.
- Follow consistent naming conventions.
- Write unit tests using MUnit.
- Use error handling to manage exceptions.
- Keep flows small and focused.
- Validate and test before deployment.
- Use connectors instead of custom code.
- Version control your projects (Git).

USE CASES

- Build REST and SOAP APIs
- System integration and data transformation
- Event-driven and real-time integrations
- File, DB and legacy system integration
- Enterprise application connectivity

Anypoint Studio provides everything a developer needs to build, test, debug and deploy powerful integrations on Anypoint Platform.

2.4 EXCHANGE

MuleSoft from salesforce

Anypoint Exchange is the central marketplace for APIs, connectors, templates and other assets on the Anypoint Platform. It helps you discover, share and reuse assets across teams and organizations to accelerate integration development.

WHAT IS EXCHANGE?

- Exchange is a centralized repository for APIs and integration assets.
- It promotes reusability and collaboration across teams and organizations.
- You can publish your assets or reuse assets built by MuleSoft or other community members.
- Assets can be public or private (organization scoped).
- It is accessible from Anypoint Platform and Anypoint Studio.

ASSETS AVAILABLE IN EXCHANGE

APIs
Reusable APIs published by MuleSoft or organizations.

CONNECTORS
Pre-built connectors for various SaaS and on-premise systems.

TEMPLATES
Application templates to quickly start common integration use cases.

POLICIES
Reusable API Manager policies to secure and manage APIs.

EXAMPLES
Sample flows and configurations for reference.

FRAGMENTS
Reusable XML or DataWeave snippets to speed up development.

KEY BENEFITS

- Accelerates development with reusable assets.
- Ensures consistency and standardization.
- Encourages collaboration and knowledge sharing.
- Reduces duplicate efforts and rework.
- Easy discovery of MuleSoft certified connectors and best practices.
- Improves governance and security with organization scoped assets.

EXCHANGE ACCESS

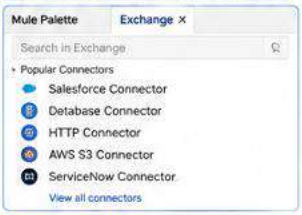
1. Log in to Anypoint Platform
2. Go to Exchange
3. Search / Browse Assets
4. View Details & Download
5. Use in Studio or Publish New

SEARCH & FILTER

PUBLISH ASSET TO EXCHANGE

1. Build your asset (API / Connector / Template etc.)
2. Add asset details, description, tags and metadata
3. Choose visibility (Public / Organization / Private)
4. Publish to Exchange
5. Other users can search and use your asset

EXCHANGE IN ANYPOINT STUDIO



You can search and add connectors, examples and templates directly from Anypoint Studio using the Exchange view.

ASSET VISIBILITY OPTIONS

VISIBILITY	DESCRIPTION
Public	Visible to everyone in the Anypoint Exchange.
Organization	Visible only to users within your organization.
Private	Visible only to specific users or groups.

BEST PRACTICES

- Use meaningful names and descriptions.
- Add relevant tags and attributes for easy discovery.
- Maintain versioning for your assets.
- Validate assets before publishing.
- Follow security best practices.
- Reuse official MuleSoft assets whenever possible.

USE CASES

- Reuse a connector to integrate with a system quickly.
- Use a template to bootstrap a common integration pattern.
- Apply a policy from Exchange to secure an API.
- Share custom APIs and accelerators with your organization.

Exchange empowers developers to discover, share and reuse high-quality assets to build integrations faster and smarter.

2.5 API MANAGER

MuleSoft from salesforce

API Manager is a powerful gateway and policy management layer on Anypoint Platform. It allows organizations to secure, manage, control and analyze API traffic and enforce policies at runtime.

WHAT IS API MANAGER?

- A runtime service that sits in front of your APIs as a proxy.
- Enforces policies to secure and manage API access and traffic.
- Provides traffic management, access control, throttling, threat protection and analytics.
- Part of Anypoint Platform, deployed on CloudHub or on-premises.
- Implements API-led connectivity principles (Autonomy, Loose Coupling, Reusability, Discoverability, Composability).

KEY CAPABILITIES

SECURITY
Protect APIs with OAuth, SAML, API Key, IP Filtering and more.

ACCESS CONTROL
Control who can access APIs using Client ID, Scopes and Roles.

TRAFFIC MANAGEMENT
Throttle traffic, set quotas and manage SLA tiers.

POLICY ENFORCEMENT
Apply and enforce policies on requests and responses.

THREAT PROTECTION
Detect and block threats like SQL injection, cross-site scripting.

ANALYTICS
Monitor API usage, performance and business metrics.

VERSION MANAGEMENT
Manage API versions and lifecycle effectively.

DEVELOPER EXPERIENCE
Provide self-service portal for developers and consumers.

HOW API MANAGER WORKS

All requests go through API Manager where policies are applied before reaching the Mule application or backend.

COMMON POLICIES

- Authentication – Validate API callers.
- Authorization – Check scopes and roles.
- Rate Limiting – Limit request rate.
- Quota – Limit number of requests over a period.
- IP Filtering – Allow or block IPs.
- Threat Protection – Protect from common attacks.
- Message Validation – Validate request/response.
- Transform – Modify headers, body or status.
- CORS – Enable cross-origin access.

API MANAGER ARCHITECTURE

SLA TIERS

TIER	DESCRIPTION	EXAMPLE USE CASE
Gold	Highest level. More limits, priority support.	Premium customers, critical APIs
Silver	Medium level. Standard limits and support.	Regular customers, business APIs
Bronze	Basic level. Lower limits, best-effort support.	Internal users, low priority APIs

DEPLOYMENT OPTIONS

CloudHub
Fully managed, elastic, high availability.

On-Premises
Deploy API Manager on your own infrastructure.

API MANAGER COMPONENTS

Runtime
Handles API traffic and enforces policies.

Control Plane
Manages configuration, policies and analytics.

Analytics
Provides metrics, reports and insights.

Portal
Developer portal for self-service.

BENEFITS

- Centralized API management and security.
- Fine-grained access control and authentication.
- Better performance and traffic management.
- Threat protection against common attacks.
- Detailed analytics for better decision making.
- Enforces API governance and compliance.
- Improves developer experience with portals.

USE CASES

- Expose APIs to partners and customers securely.
- Enforce usage limits and manage SLA.
- Control access to sensitive data and services.
- Monitor API usage and performance.
- Protect APIs from malicious attacks.

API Manager acts as the front door for your APIs – securing, managing and monitoring every API call with policies and analytics.

2.6 RUNTIME MANAGER

MuleSoft from salesforce

Runtime Manager is the operations and deployment solution of MuleSoft. It allows you to deploy, manage, monitor and scale Mule applications in the cloud (CloudHub) or on-premises environments.

WHAT IS RUNTIME MANAGER?

- A centralized application management and monitoring solution.
- Deploy and manage Mule applications across multiple environments.
- Provides operational insights, alerts and logs.
- Supports CloudHub and on-premises deployments.

KEY CAPABILITIES

DEPLOYMENT
Deploy applications to CloudHub or on-premises with ease.

APPLICATION MANAGEMENT
Start, stop, restart applications and manage versions.

MONITORING
Real-time monitoring, metrics, logs and alerts.

ALERTING
Create custom alerts and get notified on critical events.

ENVIRONMENT MANAGEMENT
Manage servers, worker nodes and environments.

ROLE-BASED ACCESS
Secure access with roles and permissions.

LOG MANAGEMENT
Search, view and download logs for troubleshooting.

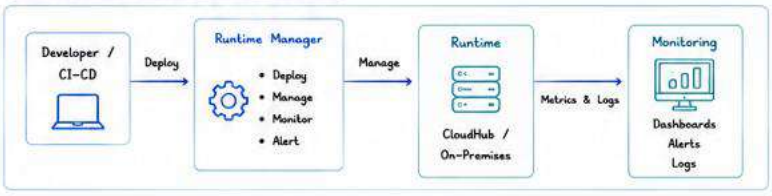
AUTOMATION
Automate deployments using APIs and CI/CD tools.

SUPPORTED DEPLOYMENTS

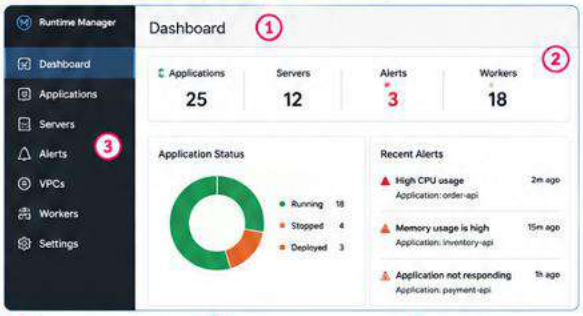
CloudHub
Fully managed, elastic runtime on MuleSoft Cloud infrastructure.

On-Premises
Deploy and manage applications on your own infrastructure.

RUNTIME MANAGER ARCHITECTURE



RUNTIME MANAGER CONSOLE OVERVIEW



1. Dashboard — Overview of applications, servers, alerts and workers.
2. Summary — Key metrics and health at a glance.
3. Navigation — Access to all Runtime Manager features.

COMMON OPERATIONS

OPERATION	DESCRIPTION
Deploy Application	Deploy new or updated applications.
Start / Stop / Restart	Control application lifecycle.
Update Application	Update to a new version.
View Logs	Search and download application logs.
View Metrics	Monitor performance and health metrics.
Configure Alerts	Create alerts based on rules and thresholds.
Manage Workers	Scale and manage worker nodes.
Manage Servers	Add, edit or delete on-premises servers.

ENVIRONMENTS

Use environments to promote applications through different stages of the SDLC with controlled access.

MONITORING & ALERTING

Monitoring
- Real-time dashboards
- Application metrics
- Custom dashboards
- Health status
- Log search

Alerting
- Create alert policies
- Set thresholds
- Email / Webhook notifications
- Acknowledgement and escalation

BENEFITS

- Centralized management of all Mule applications.
- Improved operational visibility and faster issue resolution.
- Automated deployments and environment management.
- Scalable and highly available infrastructure.
- Better performance monitoring and proactive alerting.
- Supports DevOps and CI/CD practices.

BEST PRACTICES

- Use environments to promote applications.
- Configure alerts for critical metrics.
- Regularly review logs and metrics.
- Use application properties for configuration.
- Automate deployments using APIs / CLI.
- Follow naming conventions for applications and environments.

USE CASES

- Deploy APIs to CloudHub for high availability.
- Manage on-prem applications in data centers.
- Monitor API performance and SLA.
- Troubleshoot issues using logs and metrics.
- Automate deployments in CI/CD pipelines.

Runtime Manager helps you deploy, manage and monitor your Mule applications with efficiency, visibility and control.

2.7 ACCESS MANAGEMENT

MuleSoft from salesforce

Access Management in Anypoint Platform is used to manage users, roles, permissions and control access to Anypoint Platform resources. It provides secure access for individuals and teams based on their responsibilities.

WHAT IS ACCESS MANAGEMENT?

- Access Management is a security service that controls who can access Anypoint Platform and what they can do.
- It helps organizations manage users, roles, permissions, and resources in a centralized way.
- It ensures the right access to the right people for the right resources.
- It is a foundation for governance, security and compliance.

KEY CAPABILITIES

USER MANAGEMENT
Create and manage users in the organization.

ROLE MANAGEMENT
Create roles and define permissions.

PERMISSION MANAGEMENT
Control access to resources based on roles.

BUSINESS GROUPS
Segment the organization into isolated groups.

TEAMS
Organize users and resources within business groups.

POLICY ENFORCEMENT
Enforce permissions across Anypoint Platform services.

ACCESS MANAGEMENT HIERARCHY

Organization → **Business Groups** → **Teams** → **Users**

- **Organization**: Top level container. One organization per tenant.
- **Business Groups**: Isolate resources and users. (e.g., IT, Finance, HR, Projects)
- **Teams**: Group of users within a business group. (e.g., Developers Team)
- **Users**: Individual users who access the platform.

Resources (APIs, applications, environments, etc.) are owned by Business Groups.

ACCESS MANAGEMENT COMPONENTS

COMPONENT	DESCRIPTION
Users	Individuals who can log in to Anypoint Platform.
Roles	Collection of permissions. Assigned to users.
Permissions	Specific actions allowed on resources.
Business Groups	Logical container to organize resources and users.
Teams	Sub-groups within business groups.
Client Applications	Applications registered for API access (connected apps).

COMMON ROLES (EXAMPLES)

ROLE	DESCRIPTION
Organization Administrator	Full access to the organization. Manages users, roles, business groups, etc.
Business Group Administrator	Full access within a business group.
API Developer	Can design, build and manage APIs.
API Consumer	Can view and consume APIs.
Viewer	Read-only access to resources.
Custom Role	Role with specific permissions.

HOW ACCESS IS GRANTED

User → **Role** → **Permissions** → **Resources**

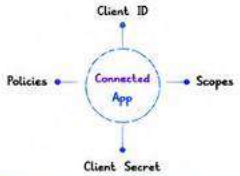
- **User**: A user is created in Access Management.
- **Role**: User is assigned one or more roles.
- **Permissions**: Roles contain permissions to perform specific actions.
- **Resources**: Permissions are applied to platform resources.

Example:
A user assigned the "API Developer" role can create, edit and deploy APIs in the business group.

CONNECTED APPS (OAUTH CLIENTS)

Access Management is also used to create Connected Apps (OAuth Clients) that allow applications to access APIs securely.

- Register a Client ID
- Define scopes
- Generate Client Secret
- Use for OAuth authentication



BEST PRACTICES

- Follow least privilege principle.
- Create custom roles instead of using out-of-the-box roles.
- Regularly review users, roles and permissions.
- Use Business Groups to isolate teams and environments.
- Remove inactive users and unnecessary access.
- Use Connected Apps with appropriate scopes.
- Enable MFA for users.

ACCESS MANAGEMENT BENEFITS

Enhanced Security
Ensure only authorized users access the right resources.

Better Governance
Centralized control over users, roles and permissions.

Compliance
Helps meet regulatory and security requirements.

Operational Efficiency
Simplifies user and access management across the platform.

Team Isolation
Business Groups keep resources and teams separated.

Scalability
Easily manage access as your organization grows.

Access Management ensures the right users have the right access to the right resources at the right time.

2.8 ANYPOINT MONITORING

MuleSoft from salesforce

Anypoint Monitoring provides real-time visibility into the health, performance and availability of your APIs and applications. It helps you monitor, analyze and troubleshoot issues quickly using dashboards, metrics, alerts and logs.

WHAT IS ANYPOINT MONITORING?

- A unified monitoring service for APIs and Mule applications.
- Provides real-time and historical insights on performance, availability and usage.
- Helps in proactive issue detection and faster troubleshooting.
- Part of Anypoint Platform and works with Runtime Manager, API Manager and other services.

KEY CAPABILITIES

REAL-TIME VISIBILITY
Monitor your applications and APIs in real time with live metrics and dashboards.

PERFORMANCE MONITORING
Track throughput, response time, latency, errors and resource usage.

ALERTS & NOTIFICATIONS
Create alerts on thresholds and get notified via Email, Webhook or Anypoint Platform.

LOGS EXPLORATION
Search and analyze application logs to troubleshoot issues quickly.

CUSTOM DASHBOARDS
Create custom dashboards with widgets to monitor what matters to you.

INTEGRATED PLATFORM
Works seamlessly with Runtime Manager, API Manager, CloudHub and on-premises.

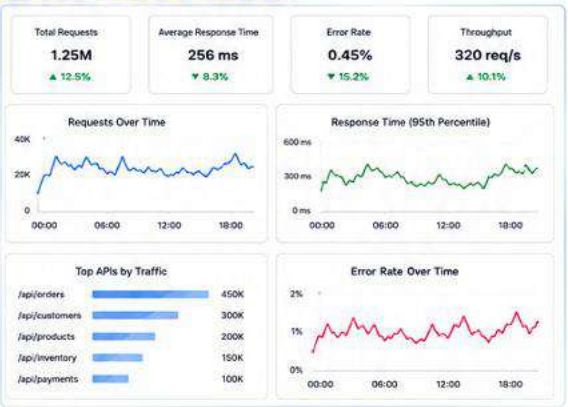
MONITORING ARCHITECTURE

Monitoring data is collected from runtime, processed and visualized in Anypoint Monitoring dashboards.

MONITORING DATA

DATA TYPE	DESCRIPTION
Metrics	Numerical values collected over time (e.g., response time, throughput).
Logs	Detailed runtime events and messages for troubleshooting.
Events	State changes and important occurrences (e.g., deployments, alerts).
Traces	End-to-end request flow for distributed tracing.

DASHBOARD OVERVIEW (EXAMPLE)



COMMON METRICS

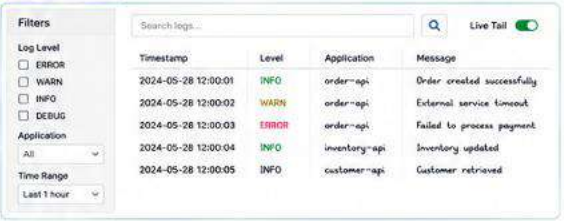
METRIC CATEGORY	EXAMPLES
Performance	Response Time, Latency, Throughput, CPU, Memory
Reliability	Error Rate, Faults, Timeouts, Availability
Usage	Total Requests, Successful Requests, API Invocations
Resources	CPU Utilization, Memory Usage, Threads, GC Time
Business	Custom business metrics using custom dashboards

ALERTS

ALERT TYPE	DESCRIPTION
Threshold Alerts	Triggered when a metric crosses a defined threshold.
Anomaly Alerts	Triggered when an unusual pattern is detected.
Log Alerts	Triggered based on log patterns or log levels.
Synthetic Monitor Alerts	Triggered when a synthetic monitor check fails.

ALERT NOTIFICATION CHANNELS

LOG EXPLORER



BEST PRACTICES

- Monitor key business flows and critical APIs.
- Set meaningful thresholds to avoid alert fatigue.
- Use custom dashboards for role-based visibility.
- Review logs and metrics regularly.
- Enable distributed tracing for complex flows.
- Clean up old data and optimize retention policies.

BENEFITS

- Real-time visibility into application and API health.
- Faster issue detection and resolution.
- Improved performance and reliability.
- Better user experience and customer satisfaction.
- Data-driven insights for capacity planning.

USE CASES

- Monitor API performance and SLA compliance.
- Detect and troubleshoot integration issues.
- Track system health and resource usage.
- Ensure business transactions are running smoothly.
- Capacity planning and usage trend analysis.

Anypoint Monitoring gives you complete visibility and actionable insights to keep your APIs and applications healthy, fast and reliable.

2.9.1 SECRETS MANAGER

MuleSoft from salesforce

Secrets Manager is used to securely store and manage sensitive information such as passwords, API keys, certificates, and tokens in Anypoint Platform.

WHAT IS SECRETS MANAGER?

- Secure storage service for sensitive information.
- Eliminates hard-coded credentials.
- Improves application security.
- Centralized secret management.
- Supports secure deployments.

WHY USE SECRETS MANAGER?

SECURITY
Protect sensitive information.

CENTRALIZED MANAGEMENT
Store secrets in one place.

ENCRYPTION
Secrets are securely encrypted.

COMPLIANCE
Meets security standards.

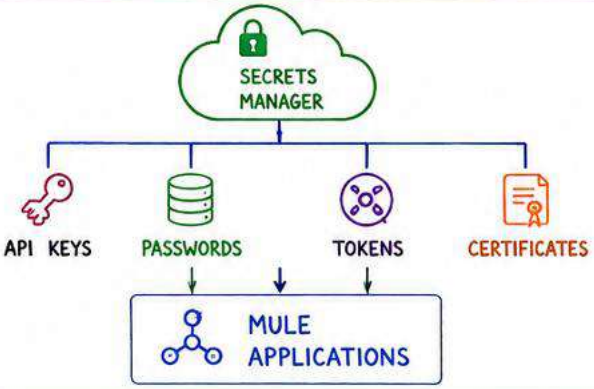
EASY ROTATION
Update secrets without code changes.

ACCESS CONTROL
Restrict access using roles.

SUPPORTED SECRETS

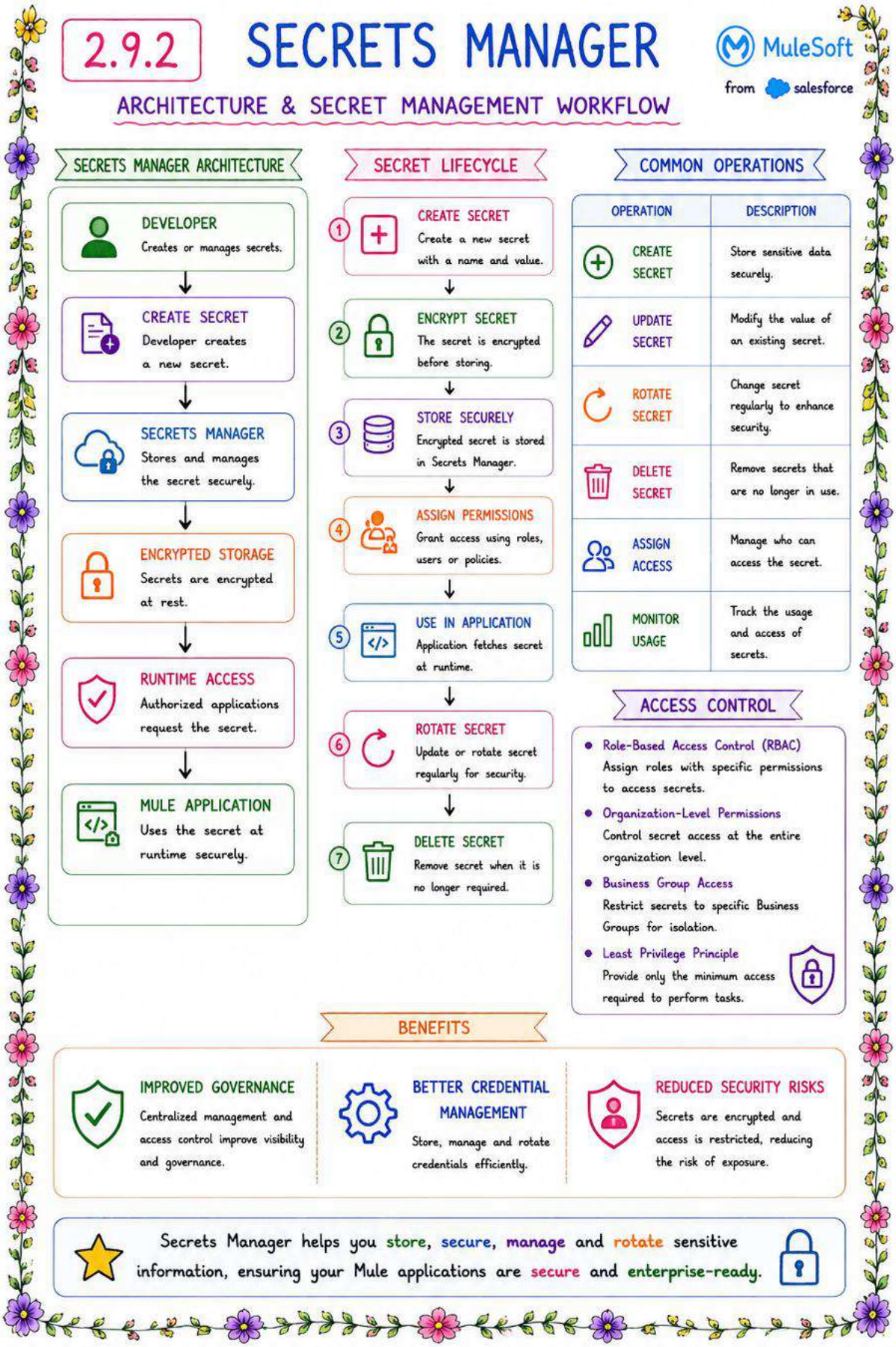
- Database Passwords
- API Keys
- OAuth Tokens
- Certificates
- Client Secrets
- Secure Properties

HIGH-LEVEL ARCHITECTURE



KEY TAKEAWAYS

- Centralized secret management.
- Improved application security.
- Secure and scalable deployments.


2.9.2
SECRETS MANAGER
MuleSoft
from salesforce
ARCHITECTURE & SECRET MANAGEMENT WORKFLOW
SECRETS MANAGER ARCHITECTURE
DEVELOPER
Creates or manages secrets.
CREATE SECRET
Developer creates a new secret.
SECRETS MANAGER
Stores and manages the secret securely.
ENCRYPTED STORAGE
Secrets are encrypted at rest.
RUNTIME ACCESS
Authorized applications request the secret.
MULE APPLICATION
Uses the secret at runtime securely.
SECRET LIFECYCLE
1 CREATE SECRET
Create a new secret with a name and value.
2 ENCRYPT SECRET
The secret is encrypted before storing.
3 STORE SECURELY
Encrypted secret is stored in Secrets Manager.
4 ASSIGN PERMISSIONS
Grant access using roles, users or policies.
5 USE IN APPLICATION
Application fetches secret at runtime.
6 ROTATE SECRET
Update or rotate secret regularly for security.
7 DELETE SECRET
Remove secret when it is no longer required.
COMMON OPERATIONS
OPERATION
DESCRIPTION
CREATE SECRET
Store sensitive data securely.
UPDATE SECRET
Modify the value of an existing secret.
ROTATE SECRET
Change secret regularly to enhance security.
DELETE SECRET
Remove secrets that are no longer in use.
ASSIGN ACCESS
Manage who can access the secret.
MONITOR USAGE
Track the usage and access of secrets.
ACCESS CONTROL
Role-Based Access Control (RBAC)
Assign roles with specific permissions to access secrets.
Organization-Level Permissions
Control secret access at the entire organization level.
Business Group Access
Restrict secrets to specific Business Groups for isolation.
Least Privilege Principle
Provide only the minimum access required to perform tasks.
BENEFITS
IMPROVED GOVERNANCE
Centralized management and access control improve visibility and governance.
BETTER CREDENTIAL MANAGEMENT
Store, manage and rotate credentials efficiently.
REDUCED SECURITY RISKS
Secrets are encrypted and access is restricted, reducing the risk of exposure.
Secrets Manager helps you store, secure, manage and rotate sensitive information, ensuring your Mule applications are secure and enterprise-ready.

2.9.3
SECRETS MANAGER
BEST PRACTICES & REAL-TIME USE CASES
MuleSoft
from salesforce
BEST PRACTICES
Never hard-code credentials in your application.
Use separate secrets for each environment.
Rotate secrets regularly to improve security.
Apply least privilege access for users and applications.
Use RBAC (Role-Based Access Control) for permissions.
Audit secret usage regularly for monitoring and compliance.
REAL-TIME USE CASES
Database Credential Management
Store and manage database usernames and passwords securely.
OAuth Client ID & Client Secret Management
Securely manage OAuth credentials for authentication flows.
API Authentication
Store API keys and tokens securely for external integrations.
Secure CloudHub Deployments
Inject secrets securely in CloudHub at deployment and runtime.
Salesforce Connected App Secrets
Store client id, client secret and other Salesforce credentials.
CI/CD Secure Deployments
Use Secrets Manager in CI/CD pipelines for secure build and deploy.
REAL-TIME EXAMPLE
DATABASE PASSWORD
SECRETS MANAGER
RUNTIME MANAGER
MULE APPLICATION
DATABASE CONNECTION
ENVIRONMENT-BASED SECRET MANAGEMENT
DEV
Database Password - DEV
QA
Database Password - QA
UAT
Database Password - UAT
PROD
Database Password - PROD
BENEFITS
BENEFIT
DESCRIPTION
SECURITY
Protect sensitive information from unauthorized access.
COMPLIANCE
Meets enterprise security and compliance standards.
SCALABILITY
Easily manage secrets across multiple environments.
MAINTAINABILITY
Update secrets without code changes or redeployments.
RELIABILITY
Secure and consistent access to secrets at runtime.
GOVERNANCE
Better visibility, control and auditing of secret usage.
FINAL TAKEAWAY
STORE
SECURE
ENCRYPT
MANAGE
ROTATE
Keep your Mule applications secure and enterprise-ready with Secrets Manager!

2.10 ANYPOINT MQ

MuleSoft from salesforce

Anypoint MQ is a fully managed message broker service that enables reliable and asynchronous communication between applications and systems. It ensures messages are delivered securely, in order, and at scale.

WHAT IS ANYPOINT MQ?

- A fully managed, cloud-native message broker service.
- Enables asynchronous, decoupled communication between producers and consumers.
- Guarantees message delivery, durability, and ordering.
- Supports request-response, pub-sub, and fire-and-forget patterns.
- Part of Anypoint Platform and integrates seamlessly with Mule applications.

KEY CAPABILITIES

FULLY MANAGED
No infrastructure to manage. High availability, scalability and automatic updates.

RELIABLE DELIVERY
Durable storage and acknowledgements ensure messages are not lost.

ORDERED MESSAGING
Preserve the order of messages within a queue or topic.

SECURE
Encryption in transit and at rest. Access controlled via IAM & policies.

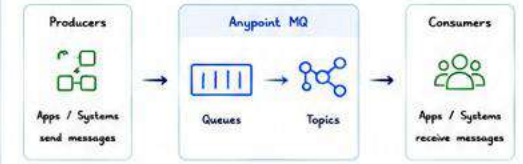
FLEXIBLE PATTERNS
Support request-response, publish-subscribe and point-to-point.

ELASTIC SCALE
Automatically scale throughput and storage based on demand.

MONITOR & ALERT
Real-time metrics, logs and alerts for operational visibility.

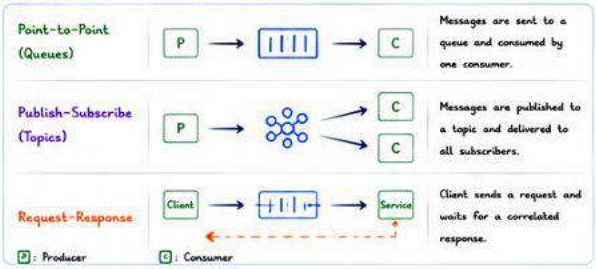
EASY INTEGRATION
Native connectors for Mule applications and Anypoint Platform.

HOW ANYPOINT MQ WORKS



Messages are persisted in MQ and delivered to consumers reliably and in order.

SUPPORTED MESSAGING PATTERNS



CORE COMPONENTS

Queues	Durable channels for point-to-point communication. Messages are consumed by one consumer only.
Topics	Durable channels for publish-subscribe communication. Messages are delivered to all subscriptions.
Messages	The data being transmitted. Messages are stored durably until successfully consumed.
Configurations	Settings for retention, TTL, delivery mode, deduplication, DLQ, permissions and encryption.

FEATURES HIGHLIGHTS

FEATURE	DESCRIPTION
Message Durability	Messages are persisted to disk for reliable delivery.
Ordering	Maintain strict order of messages within ia queue or partition.
Acknowledgements	Manual or auto-ack to confirm successful processing.
Dead Letter Queue (DLQ)	Undeliverable messages are moved to DLQ for review.
Message TTL	Automatically expire messages after a defined time.
Deduplication	Prevent duplicate message processing.
Message Priorities	Control the order in which messages are delivered.
Throughput & Scale	High throughput with automatic scaling.

COMMON USE CASES

System Integration	Order Processing	Payment Processing	Event Streaming	Microservices Communication
Decouple applications across systems.	Handle order events reliably.	Ensure reliable and ordered payments.	Stream events between services.	Enable async communication.

INTEGRATION WITH ANYPOINT PLATFORM

Use the Anypoint MQ Connector in Mule flows to publish and consume messages easily.

ANYPOINT MQ PLANS

PLAN	BEST FOR
Standard	General purpose workloads with variable traffic.
Premium	High throughput with low latency requirements.
Dedicated	Isolated capacity with dedicated virtual clusters for strict compliance and performance needs.

- Multiple regions
- High availability
- Encryption by default
- 99.99% SLA

BEST PRACTICES

- Use asynchronous messaging to decouple systems.
- Handle failures using retries and DLQ.
- Use correlation IDs for request-response flows.
- Set appropriate TTL and retention policies.
- Monitor metrics and set alerts for critical KPIs.
- Secure access using IAM and least privilege.

Anypoint MQ delivers reliable, secure and scalable messaging so your applications can communicate anytime, anywhere.

2.10.1 ANYPOINT MQ

MuleSoft from salesforce

Anypoint MQ is a fully managed, cloud-native message broker service that enables reliable and asynchronous communication between applications and systems. It ensures messages are delivered securely, in order, and at scale.

WHAT IS ANYPOINT MQ?

- Fully managed cloud-native message broker.
- Enables asynchronous communication.
- Guarantees message durability and ordering.
- Supports multiple messaging patterns.
- Seamlessly integrates with Mule applications.

KEY CAPABILITIES

FULLY MANAGED
No infrastructure management.

RELIABLE DELIVERY
Ensures message durability.

ORDERED MESSAGING
Maintains the order of messages.

SECURE
Encryption in transit and at rest. Access controlled via IAM & policies.

FLEXIBLE PATTERNS
Supports multiple messaging models.

ELASTIC SCALE
Automatically scales throughput and storage.

MONITOR & ALERT
Real-time metrics, logs and alerts for operational visibility.

EASY INTEGRATION
Native connectors for Mule applications and Anypoint Platform.

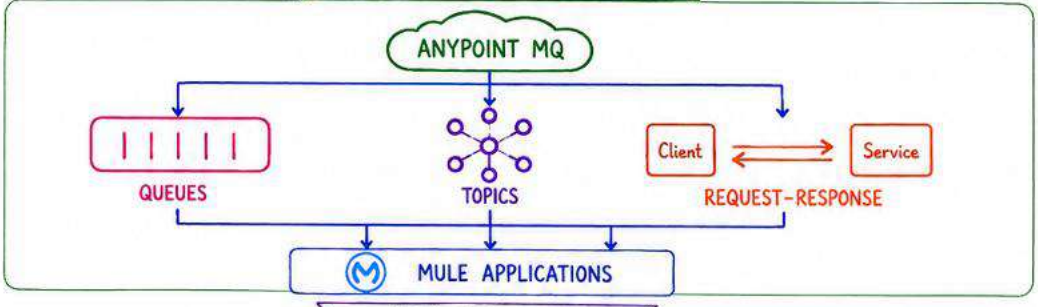
SUPPORTED MESSAGING PATTERNS

POINT-TO-POINT (QUEUES)
Messages are sent to a queue and consumed by one consumer.

PUBLISH-SUBSCRIBE (TOPICS)
Messages are published to a topic and delivered to all subscribers.

Client → Service
REQUEST-RESPONSE
Client sends a request and waits for a correlated response.

HIGH-LEVEL ARCHITECTURE



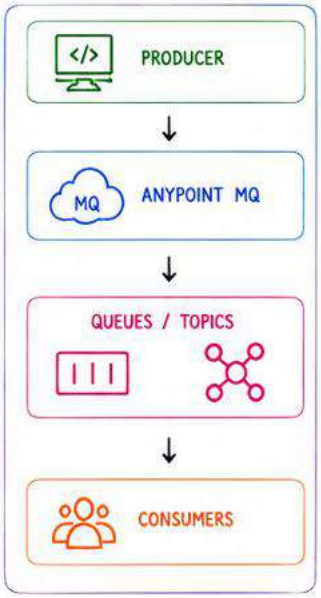
KEY TAKEAWAYS

- Reliable messaging.
- Asynchronous communication.
- Fully managed cloud service.

2.10.2 ANYPOINT MQ

MuleSoft from salesforce

HOW ANYPOINT MQ WORKS



MESSAGE FLOW

1. **APPLICATIONS** — Applications generate and publish messages.
2. **PUBLISH MESSAGES** — Messages are sent to Anypoint MQ.
3. **ANYPOINT MQ** — Messages are routed to Queues or Topics.
4. **MESSAGE PERSISTENCE** — Messages are stored durably and reliably.
5. **CONSUMERS RECEIVE MESSAGES** — Consumers subscribe and receive messages.

CORE COMPONENTS

COMPONENT	DESCRIPTION
QUEUES	Used for point-to-point communication. Messages are sent to a queue and consumed by one consumer.
TOPICS	Used for publish-subscribe communication. Messages are published to a topic and delivered to all subscribers.
MESSAGES	The data or information being transmitted between applications or systems.
CONFIGURATIONS	Settings for message retention, TTL, DLQ, deduplication, encryption and permissions.

FEATURE HIGHLIGHTS

FEATURE	DESCRIPTION
MESSAGE DURABILITY	Messages are persisted to disk to ensure reliable delivery even in case of failures.
ORDERING	Maintains strict order of messages within a queue or topic.
ACKNOWLEDGEMENTS	Manual or auto-ack to confirm successful processing of messages.
DEAD LETTER QUEUE (DLQ)	Undeliverable messages are moved to DLQ for review and reprocessing.
MESSAGE TTL	Automatically expire messages after a defined time (Time-To-Live).
DEDUPLICATION	Prevent duplicate message processing.
MESSAGE PRIORITIES	Control the order in which messages are delivered based on priority.
THROUGHPUT & SCALE	High throughput with automatic scaling based on demand.

BENEFITS

RELIABLE MESSAGE PROCESSING
Ensures messages are delivered securely and reliably.

IMPROVED SCALABILITY
Handles increased load with automatic scaling and elastic resources.

BETTER FAULT TOLERANCE
Messages are stored durably and survive failures.

LOOSE COUPLING BETWEEN SYSTEMS
Systems communicate asynchronously without being dependent on each other.

QUICK NOTES

- Messages are persisted and delivered in the order.
- Supports both Queues and Topics.
- Ensures high availability and disaster recovery.
- Built on cloud-native, highly available infrastructure.
- Fully managed service – no infrastructure to manage!

Anypoint MQ helps you build resilient, scalable and event-driven architectures with reliable message delivery.

2.10.3

ANYPOINT MQ

INTEGRATION, USE CASES & BEST PRACTICES

MuleSoft from salesforce

COMMON USE CASES

SYSTEM INTEGRATION
Integrate multiple systems in an asynchronous way.

ORDER PROCESSING
Process orders reliably and handle spikes in traffic.

PAYMENT PROCESSING
Handle payment transactions securely and reliably.

EVENT STREAMING
Stream real-time events between applications.

MICROSERVICES COMMUNICATION
Enable loose coupling and asynchronous communication.

INTEGRATION WITH ANYPOINT PLATFORM

MULE APPLICATION
Build and run your applications.

↓

MQ CONNECTOR
Use Anypoint MQ Connector to publish / consume messages.

↓

ANYPOINT MQ
Fully managed message broker (Queues / Topics).

↓

CONSUMERS / SYSTEMS
Downstream systems or applications consume the messages.

Use the Anypoint MQ Connector in Mule flows to publish and consume messages easily.

ANYPOINT MQ PLANS

PLAN	BEST FOR
STANDARD	General purpose workloads with variable traffic. Cost-effective and easy to get started.
PREMIUM	High throughput workloads with low latency requirements.
DEDICATED	Isolated capacity with dedicated virtual clusters for compliance and performance needs.

KEY BENEFITS

- Reliable and secure messaging
- Scalable and highly available
- Improved application resilience
- Asynchronous and decoupled communication
- Better performance and fault tolerance

REAL-TIME EXAMPLE – E-COMMERCE ORDER PLACEMENT

CUSTOMER PLACES ORDER
Customer places an order on the E-Commerce site.

→ **EXPERIENCE API**
Receives the order and validates the request.

→ **PROCESS API**
Processes the order and publishes it to Anypoint MQ.

→ **ANYPOINT MQ QUEUE**
Order message is stored in the queue reliably.

→ **INVENTORY SERVICE**
Consumes the message and updates inventory in real time.

→ **PAYMENT SERVICE**
Processes the payment and confirms the transaction.

→ **NOTIFICATION SERVICE**
Sends order confirmation to customer via Email / SMS / Push.

BEST PRACTICES

- Use asynchronous messaging to decouple systems.
- Handle failures using retries and Dead Letter Queue (DLQ).
- Use correlation IDs for request-response flows.
- Set appropriate Time to Live (TTL) for messages.
- Monitor metrics and set alerts for critical KPIs.
- Secure access using IAM and least privilege.
- Design idempotent consumers to avoid duplicates.
- Test and validate message flows thoroughly.

ADDITIONAL BENEFITS

DECOUPLED ARCHITECTURE
Systems are loosely coupled and can evolve independently.

IMPROVED SCALABILITY
Scale services independently based on demand.

RELIABLE EVENT PROCESSING
Messages are delivered reliably and in order.

ENHANCED FAULT TOLERANCE
System continues to operate even in case of failures.

FINAL TAKEAWAY

PUBLISH → QUEUE → DELIVER → PROCESS → SCALE

Build reliable, secure and scalable applications with Anypoint MQ – your fully managed messaging service!

Anypoint MQ empowers your applications with reliable, asynchronous and scalable messaging – anytime, anywhere!

MQ

2.11.1

BUSINESS GROUPS & ENVIRONMENTS

MuleSoft from salesforce

Business Groups help organize teams, resources, and permissions within an organization, while Environments help manage applications across different stages of the SDLC such as Development, QA, and Production.

WHAT ARE BUSINESS GROUPS?

- Logical containers within an organization.
- Organize users, teams, APIs, and resources.
- Provide isolation between business units.
- Simplify governance and access control.

WHAT ARE ENVIRONMENTS?

- Isolated spaces within Business Groups.
- Manage application lifecycle.
- Separate development and production workloads.
- Support secure deployments.

KEY CAPABILITIES

ORGANIZATION	Structure teams and resources.
LIFECYCLE MANAGEMENT	Manage application promotion across environments.
ISOLATION & SECURITY	Separate teams and environments with strong security.
GOVERNANCE & CONTROL	Apply policies, quotas, and SLAs across groups and environments.
DEPENDENCY MANAGEMENT	Manage dependencies and promote safely between environments.

HIGH-LEVEL OVERVIEW

ORGANIZATION
↓
BUSINESS GROUPS
↓
ENVIRONMENTS
↓
MULE APPLICATIONS

EXAMPLES

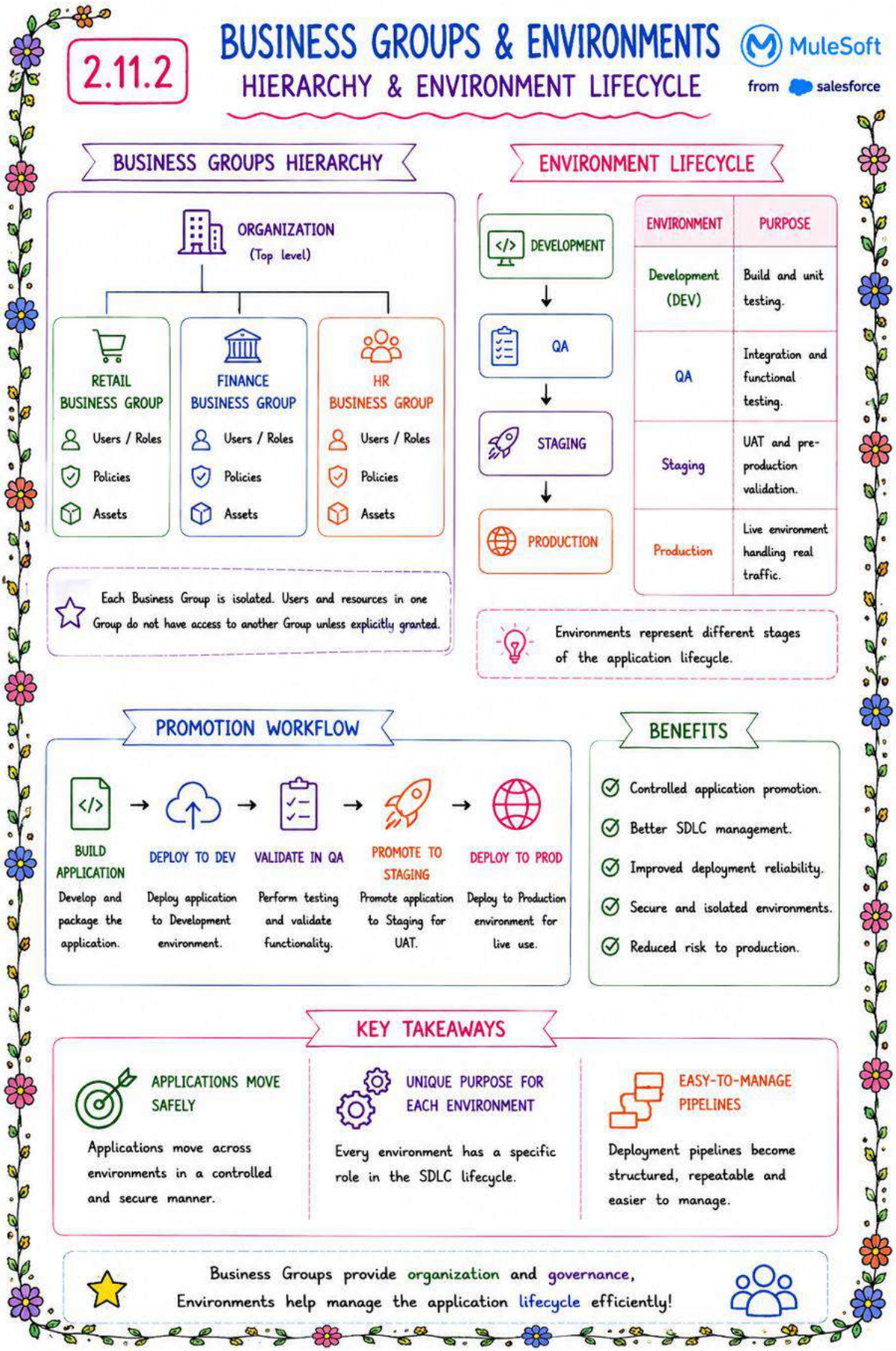
Business Group: Retail
Contains retail team, APIs, policies, and apps.

Environment: QA
Used for integration and functional testing before staging.

Environment: Production
Live environment handling real customer traffic securely.

KEY TAKEAWAYS

- Business Groups organize ownership of resources.
- Environments manage application deployments.
- Together they provide governance, isolation and security.


2.11.2
BUSINESS GROUPS & ENVIRONMENTS
HIERARCHY & ENVIRONMENT LIFECYCLE
MuleSoft
from salesforce
BUSINESS GROUPS HIERARCHY
ORGANIZATION
(Top level)
RETAIL BUSINESS GROUP
Users / Roles
Policies
Assets
FINANCE BUSINESS GROUP
Users / Roles
Policies
Assets
HR BUSINESS GROUP
Users / Roles
Policies
Assets
Each Business Group is isolated. Users and resources in one Group do not have access to another Group unless explicitly granted.
ENVIRONMENT LIFECYCLE
DEVELOPMENT
QA
STAGING
PRODUCTION
ENVIRONMENT
PURPOSE
Development (DEV)
Build and unit testing.
QA
Integration and functional testing.
Staging
UAT and pre-production validation.
Production
Live environment handling real traffic.
Environments represent different stages of the application lifecycle.
PROMOTION WORKFLOW
BUILD APPLICATION
Develop and package the application.
DEPLOY TO DEV
Deploy application to Development environment.
VALIDATE IN QA
Perform testing and validate functionality.
PROMOTE TO STAGING
Promote application to Staging for UAT.
DEPLOY TO PROD
Deploy to Production environment for live use.
BENEFITS
Controlled application promotion.
Better SDLC management.
Improved deployment reliability.
Secure and isolated environments.
Reduced risk to production.
KEY TAKEAWAYS
APPLICATIONS MOVE SAFELY
Applications move across environments in a controlled and secure manner.
UNIQUE PURPOSE FOR EACH ENVIRONMENT
Every environment has a specific role in the SDLC lifecycle.
EASY-TO-MANAGE PIPELINES
Deployment pipelines become structured, repeatable and easier to manage.
Business Groups provide organization and governance,
Environments help manage the application lifecycle efficiently!

2.11.3
BUSINESS GROUPS & ENVIRONMENTS
GOVERNANCE, PERMISSIONS & BEST PRACTICES
MuleSoft
from salesforce
ACCESS & PERMISSIONS HIERARCHY
ORGANIZATION ADMIN
Full access to organization.
BUSINESS GROUP ADMIN
Manage users, policies and resources in Business Group.
ENVIRONMENT ADMIN
Manage environments and deployments within BG.
USERS & DEVELOPERS
Build, deploy and access applications as per roles.
Permissions flow from top to bottom ensuring secure and controlled access.
BUSINESS GROUPS vs ENVIRONMENTS
ASPECT | BUSINESS GROUPS | ENVIRONMENTS
PURPOSE | Organize resources, teams and access. | Manage application lifecycle stages.
SCOPE | Organization-wide. | Within Business Groups.
ISOLATION | Isolates teams, resources and policies. | Isolates deployment stages (DEV, QA, PROD, etc.).
RESOURCES | Users, roles, policies, APIs, assets, etc. | Applications, configs, connections, etc.
GOVERNED BY | Business Group Admins. | Environment Admins.
EXAMPLE | Retail Business Group | DEV, QA, STAGING, PRODUCTION
GOVERNANCE & POLICIES
API POLICIES
Apply security, throttling, and access policies.
QUOTAS & RATE LIMITS
Control API usage with quotas and rate limits.
SLA MANAGEMENT
Define SLAs and monitor compliance.
SECURITY POLICIES
Enforce authentication, authorization and data protection.
MONITORING & ALERTS
Monitor usage, performance and get notified.
Policies ensure governance, security and compliance across Business Groups and Environments.
TYPICAL USE CASES
MULTI-TEAM ORGANIZATIONS
Manage multiple teams and their resources independently.
REGIONAL BUSINESS UNITS
Isolate business units by region with separate access.
PRODUCT-BASED TEAMS
Organize teams around products or services.
ENTERPRISE GOVERNANCE
Enforce policies, SLAs and security across the organization.
SECURE SDLC MANAGEMENT
Manage applications securely across all lifecycle stages.
BEST PRACTICES
Create meaningful Business Groups that reflect your organization structure.
Use Environments to manage SDLC stages effectively.
Apply least privilege access at both Business Group and Environment levels.
Promote applications through controlled deployment pipelines.
Monitor quotas, usage and SLAs regularly.
Keep Production environment isolated from Non-Production environments.
REAL-TIME EXAMPLE
RETAIL TEAM
RETAIL BUSINESS GROUP
DEV ENVIRONMENT
QA ENVIRONMENT
PROD ENVIRONMENT
RETAIL APIS & APPLICATIONS
FINAL TAKEAWAY
ORGANIZE TEAMS
Group teams and resources logically.
MANAGE RESOURCES
Control users, roles, policies and assets.
CONTROL ACCESS
Ensure secure and compliant access.
MANAGE DEPLOYMENTS
Deploy applications across environments.
PROMOTE SECURELY
Promote applications safely through SDLC.
Business Groups organize your organization.
Environments manage your application lifecycle.
Together they ensure governance, security and reliable deployments.

2.12.1 CONNECTED APPS

MuleSoft from salesforce

Connected Apps allow external applications to securely connect to Anypoint Platform. They provide standardized authentication and authorization using OAuth 2.0 so that applications can securely access APIs and platform resources in a controlled manner.

WHAT ARE CONNECTED APPS?

- A Connected App represents an external application that needs access to Anypoint Platform resources.
- It enables secure communication between external applications and MuleSoft services.
- Uses OAuth 2.0 and OpenID Connect (OIDC) for authentication and authorization.
- Provides granular access control using scopes, policies, and permissions.
- Helps organizations securely expose APIs and platform capabilities.

KEY CAPABILITIES

SECURE AUTHENTICATION	• Supports OAuth 2.0 and OIDC. • Provides standards-based authentication. • Uses access tokens for secure API access.
GRANULAR AUTHORIZATION	• Controls which resources an application can access. • Uses scopes for fine-grained permissions. • Prevents unauthorized access to APIs and services.
VISIBILITY & CONTROL	• Monitor usage of Connected Apps. • Track access patterns and API consumption. • Manage application lifecycle effectively.
LIFECYCLE MANAGEMENT	• Create and configure Connected Apps easily. • Update client credentials securely. • Revoke access whenever required.
INTEGRATION READY	• Works seamlessly with Mule applications. • Provides built-in support for Anypoint Platform services. • Simplifies third-party integrations.
POLICY ENFORCEMENT	• Apply security and access policies. • Configure rate limiting and access restrictions. • Improve API governance and compliance.

HIGH-LEVEL ARCHITECTURE

EXTERNAL APPLICATION
↓
CONNECTED APP
↓
OAUTH AUTHENTICATION
↓
ACCESS TOKEN
↓
ANYPOINT PLATFORM APIs
↓
{ } MULE APPLICATIONS

SUPPORTED AUTHENTICATION STANDARDS

AUTHENTICATION STANDARD	PURPOSE
OAuth 2.0	Secure authentication and authorization
OpenID Connect (OIDC)	User identity management
Access Token	Secure API access
Refresh Token	Renew access tokens
Client Credentials	Server-to-server authentication

KEY BENEFITS

- Secure API access.
- Centralized access management.
- Granular authorization controls.
- Easy integration with MuleSoft services.
- Improved governance and security.
- Enterprise-ready authentication mechanisms.

KEY TAKEAWAYS

- Connected Apps provide secure and controlled access to Anypoint Platform resources.
- OAuth 2.0 is the primary authentication mechanism used.
- Granular authorization improves API security.
- Connected Apps simplify external application integrations with MuleSoft.

FINAL SUMMARY

CONNECTED APPS

AUTHENTICATE → AUTHORIZE → ACCESS → MONITOR

SECURE • CONTROLLED • SCALABLE

Enables external applications to securely access MuleSoft APIs and platform resources.

2.12.2

CONNECTED APPS
OAUTH FLOWS & CONFIGURATION

MuleSoft from salesforce

OAUTH FLOW OVERVIEW

EXTERNAL APPLICATION
An external application requests access.

↓

AUTHENTICATE USER
User is authenticated using OAuth 2.0 / OIDC.

↓

CONNECTED APP
The Connected App is validated and authorized.

↓

GENERATE ACCESS TOKEN
Access token is issued for the requested scopes.

↓

ACCESS PLATFORM RESOURCES
Application uses the token to access APIs and resources.

OAUTH 2.0 FLOWS SUPPORTED

FLOW TYPE	DESCRIPTION
Authorization Code (Recommended)	Used by web and mobile applications where a user is present to grant access.
Client Credentials	Used by server-to-server applications without a user context.
Resource Owner Password Credentials	Used when the application has the user's credentials (legacy scenarios).
Refresh Token	Obtains new access tokens using a refresh token without re-authenticating.

CONNECTED APP CONFIGURATION

CONFIGURATION	PURPOSE
Basic Information	Name, description, logo, contact email.
Application Type	Web, Native, API, Server-to-Server, or Single Page App.
OAuth Settings	Grant types, redirect URIs, token settings, refresh token validity.
Scopes	Define scopes to control access to APIs and platform resources.
Policies	Apply and enforce policies for access control and security.
Client Credentials	Client ID and Client Secret (or certificate) used to authenticate the app.

CONNECTED APPS OVERVIEW (EXAMPLE)

Total Connected Apps	Active Apps	Inactive Apps	Revoked Apps
24	18	3	3

NAME	CLIENT ID	TYPE	SCOPES	STATUS	LAST MODIFIED
Mobile App	c1e2b3c4...	Native	api.read api.write	Active	May 02, 2024
Partner Portal	d4e5f6g7...	Web	api.read user.read	Active	May 01, 2024
Inventory Service	h7i8j9k0...	API (Server-to-Server)	api.read api.write	Active	Apr 30, 2024
Legacy App	l0m1n2o3...	Web	api.read	Inactive	Apr 28, 2024
Test App	p4q5r6s7...	Native	api.read	Revoked	Apr 10, 2024

BENEFITS

- Standardized authentication using OAuth 2.0 and OIDC.
- Flexible support for multiple application types.
- Granular scopes for fine-grained authorization.
- Centralized management of external applications.
- Better visibility and control over application access.

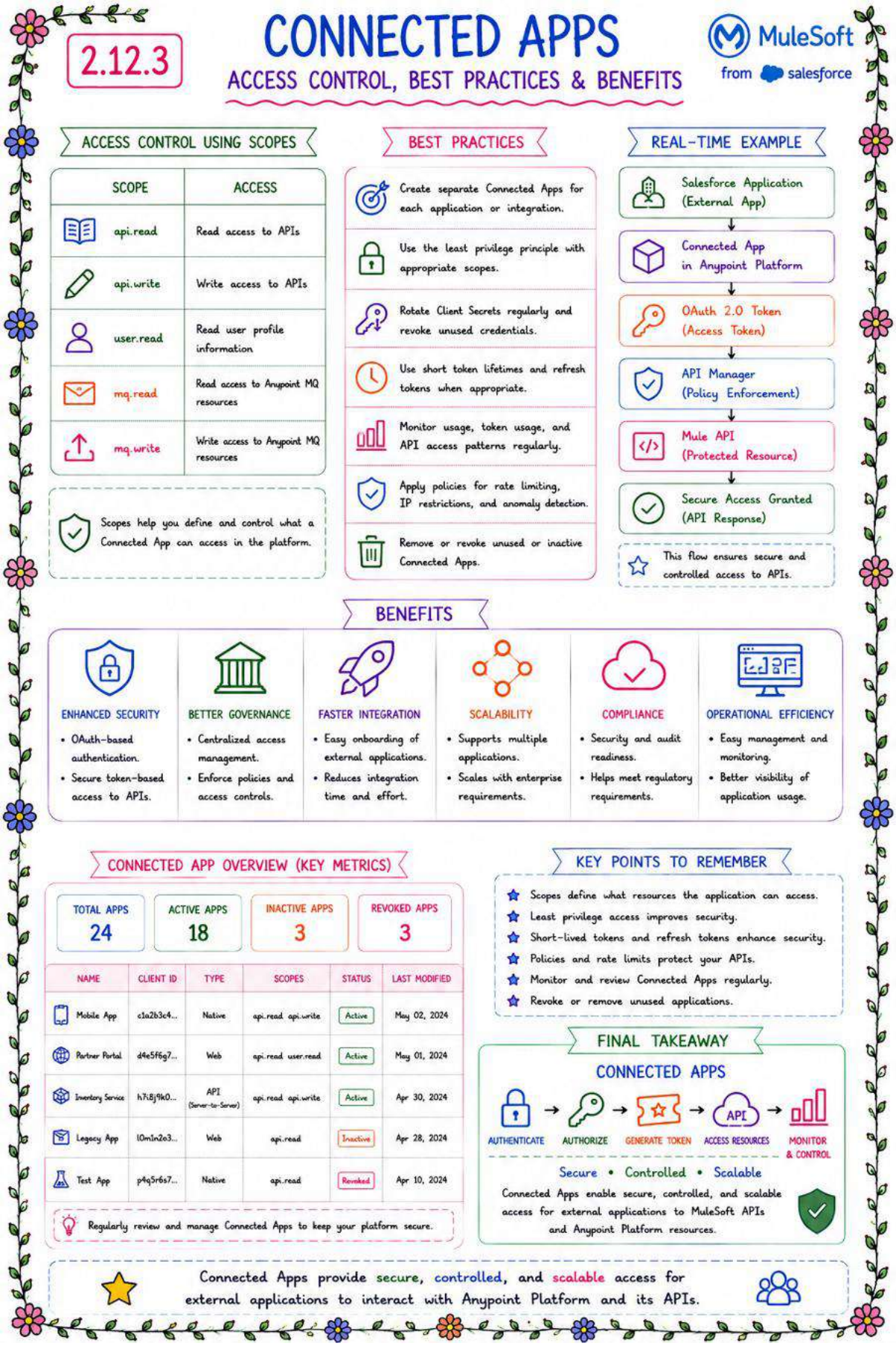
KEY POINTS

- Connected Apps act as a secure bridge between external applications and Anypoint Platform.
- OAuth 2.0 flows ensure standardized and secure access.
- Scopes and policies provide granular authorization.
- Proper configuration of tokens and credentials is essential for secure integrations.
- Monitor the status and usage of Connected Apps regularly.

QUICK SUMMARY

EXTERNAL APP — External application requests access. →
AUTHENTICATE — User authenticates using OAuth / OIDC. →
CONNECTED APP — Connected App is validated. →
ACCESS TOKEN — Access token is generated. →
PLATFORM ACCESS — Application accesses APIs and resources.

Connected App configuration and OAuth flows ensure secure, controlled, and reliable access to Anypoint Platform resources.


2.12.3
CONNECTED APPS
ACCESS CONTROL, BEST PRACTICES & BENEFITS
MuleSoft
from salesforce
ACCESS CONTROL USING SCOPES
SCOPE
ACCESS
api.read
Read access to APIs
api.write
Write access to APIs
user.read
Read user profile information
mq.read
Read access to Anypoint MQ resources
mq.write
Write access to Anypoint MQ resources
Scopes help you define and control what a Connected App can access in the platform.
BEST PRACTICES
Create separate Connected Apps for each application or integration.
Use the least privilege principle with appropriate scopes.
Rotate Client Secrets regularly and revoke unused credentials.
Use short token lifetimes and refresh tokens when appropriate.
Monitor usage, token usage, and API access patterns regularly.
Apply policies for rate limiting, IP restrictions, and anomaly detection.
Remove or revoke unused or inactive Connected Apps.
REAL-TIME EXAMPLE
Salesforce Application (External App)
Connected App in Anypoint Platform
OAuth 2.0 Token (Access Token)
API Manager (Policy Enforcement)
Mule API (Protected Resource)
Secure Access Granted (API Response)
This flow ensures secure and controlled access to APIs.
BENEFITS
ENHANCED SECURITY
• OAuth-based authentication.
• Secure token-based access to APIs.
BETTER GOVERNANCE
• Centralized access management.
• Enforce policies and access controls.
FASTER INTEGRATION
• Easy onboarding of external applications.
• Reduces integration time and effort.
SCALABILITY
• Supports multiple applications.
• Scales with enterprise requirements.
COMPLIANCE
• Security and audit readiness.
• Helps meet regulatory requirements.
OPERATIONAL EFFICIENCY
• Easy management and monitoring.
• Better visibility of application usage.
CONNECTED APP OVERVIEW (KEY METRICS)
TOTAL APPS 24
ACTIVE APPS 18
INACTIVE APPS 3
REVOKED APPS 3
NAME
CLIENT ID
TYPE
SCOPES
STATUS
LAST MODIFIED
Mobile App
c1a2b3c4...
Native
api.read api.write
Active
May 02, 2024
Partner Portal
d4e5f6g7...
Web
api.read user.read
Active
May 01, 2024
Inventory Service
h7i8j9k0...
API (Server-to-Server)
api.read api.write
Active
Apr 30, 2024
Legacy App
l0m1n2o3...
Web
api.read
Inactive
Apr 28, 2024
Test App
p4q5r6s7...
Native
api.read
Revoked
Apr 10, 2024
Regularly review and manage Connected Apps to keep your platform secure.
KEY POINTS TO REMEMBER
Scopes define what resources the application can access.
Least privilege access improves security.
Short-lived tokens and refresh tokens enhance security.
Policies and rate limits protect your APIs.
Monitor and review Connected Apps regularly.
Revoke or remove unused applications.
FINAL TAKEAWAY
CONNECTED APPS
API
AUTHENTICATE
AUTHORIZE
GENERATE TOKEN
ACCESS RESOURCES
MONITOR & CONTROL
Secure • Controlled • Scalable
Connected Apps enable secure, controlled, and scalable access for external applications to MuleSoft APIs and Anypoint Platform resources.
Connected Apps provide secure, controlled, and scalable access for external applications to interact with Anypoint Platform and its APIs.

3.1.1

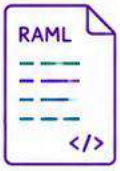
WHAT IS RAML?

MuleSoft from salesforce

RAML (RESTful API Modeling Language) is an open API specification language used to design, document and publish RESTful APIs in a standard, machine-readable format. It follows a design-first approach for building APIs on Anypoint Platform.

INTRODUCTION TO RAML

- RAML stands for RESTful API Modeling Language.
- It is a YAML-based language for describing RESTful APIs.
- It provides a standardized way to define API structure, resources, methods, requests, responses, parameters, security and more.
- RAML files have a .raml extension.
- RAML is the foundation for all MuleSoft APIs and is supported across Anypoint tools.



WHY RAML?

DESIGN-FIRST APPROACH
Design APIs before implementation.

STANDARDIZED
Uses a standard format for all APIs.

AUTOMATION READY
Machine-readable format enables automation.

BETTER COLLABORATION
Easy for teams to understand, review and collaborate.

REUSABILITY
Use libraries, traits and resource types to reuse.

RICH DOCUMENTATION
Auto-generates interactive API documentation.

HIGH-LEVEL OVERVIEW

DESIGN
Model and design APIs using RAML.

→

DOCUMENT
Document API structure and behavior.

→

PUBLISH
Publish APIs to Exchange and for consumers.

→

REUSE
Reuse components across multiple APIs.

RESTful APIs
Consistent • Standardized • Reusable

KEY TAKEAWAYS

- RAML promotes a design-first approach for building APIs.
- It is a YAML-based specification language for RESTful APIs.
- It helps create standardized, reusable and well-documented API definitions.

RAML helps you design, document and publish high-quality RESTful APIs in a consistent, secure and scalable way on Anypoint Platform.

3.1.2

WHAT IS RAML?

DESIGN-FIRST APPROACH & REST API DESIGN

MuleSoft from salesforce

DESIGN-FIRST APPROACH

1. **DESIGN API**
   Model and design your API using RAML.
2. **REVIEW API SPECIFICATION**
   Validate API design, get feedback and refine the specification.
3. **BUILD IMPLEMENTATION**
   Implement the API using Mule based on the RAML specification.
4. **PUBLISH API**
   Publish the API to Exchange and make it available for use.

Designing first ensures APIs are consistent, well-structured and aligned with business goals.

BENEFITS OF DESIGN-FIRST

- **Better API Governance**
  Ensures standardization and consistent API definitions.
- **Faster Development**
  Clear specifications speed up implementation.
- **Early Stakeholder Feedback**
  Review and validate the API before development.
- **Improved API Consistency**
  Standardized models reduce ambiguity and rework.
- **Simplified Collaboration**
  Shared contracts improve team communication.

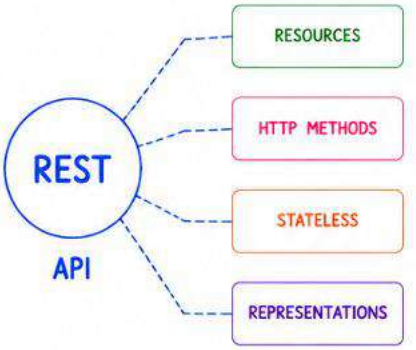
RESTFUL API DESIGN PRINCIPLES

PRINCIPLE	DESCRIPTION
Resources	Everything is a resource (noun).
GET HTTP Methods	Use standard HTTP methods: GET, POST, PUT, DELETE, etc.
Stateless	Each request is independent of others.
Representations	Data is represented in formats like JSON, XML, etc.
Uniform Interface	Consistent and uniform way to interact with APIs.

WHERE RAML IS USED

- **Anypoint Design Center**
  Design and validate RAML APIs visually and collaboratively.
- **API Manager**
  Apply policies, manage lifecycle and versions of APIs.
- **Runtime Manager**
  Deploy and manage APIs in runtime environments.
- **API Console**
  Generate interactive API documentation.
- **Exchange**
  Publish and share APIs with developers and consumers.

REST API DESIGN (OVERVIEW



- Everything is a resource (noun).
- Use standard HTTP verbs.
- Each request is independent of others.
- Data is represented in formats like JSON, XML.

KEY TAKEAWAYS

- RAML promotes a design-first approach for building APIs.
- It helps you design, document and publish APIs in a standard and machine-readable format.
- RAML ensures consistent, reusable and well-documented APIs across the API lifecycle.

Using RAML's design-first approach, you can create consistent, secure and scalable RESTful APIs on Anypoint Platform.

3.1.3

WHAT IS RAML?

BENEFITS, EXAMPLE & BEST PRACTICES

MuleSoft from salesforce

BENEFITS OF USING RAML

BENEFIT	DESCRIPTION
Consistency	Ensures consistent and standardized API definitions.
Faster Development	Auto-generates documentation and code scaffolds.
Better Governance	Enforces policies, standards and best practices.
Greater Reusability	Reuse traits, resource types, libraries and fragments.
Improved Collaboration	Clear API contracts for developers, testers and business stakeholders.
Future Ready	Industry standard (OAS compatible via conversion).

EXAMPLE: SIMPLE RESOURCE IN RAML

```
#%RAML 1.0
title: Sample API
version: v1
baseUri: https://api.example.com/{version}

/users:
  get:
    description: Get list of users
    responses:
      200:
        body:
          application/json:
            example:
              {
                "id": 1, "name": "John"
              }
```

WHAT THIS RAML DEFINES:

- API title, version and base URI
- A resource: /users
- A GET method to retrieve users
- A 200 OK response with JSON example
- A standard and machine-readable API contract

NOTES

- RAML is YAML-based.
- Indentation is important.
- Use 2 spaces per level.
- RAML 1.0 is the current stable version.

RAML BEST PRACTICES

FOLLOW DESIGN-FIRST APPROACH
Design and validate the API before writing the implementation.

USE MEANINGFUL RESOURCE NAMES
Use clear, consistent and business-friendly resource names.

REUSE TRAITS AND RESOURCE TYPES
Promote reusability and reduce duplication using traits and types.

MAINTAIN PROPER INDENTATION
Use 2 spaces per level and maintain correct YAML indentation.

USE RAML 1.0 SPECIFICATIONS
Use RAML 1.0 for better compatibility and future support.

DOCUMENT APIS THOROUGHLY
Add descriptions, examples, responses and error codes for clarity.

FINAL TAKEAWAY

RAML

DESIGN → REVIEW → BUILD → PUBLISH

CONSISTENT • SECURE • SCALABLE

Build better RESTful APIs using the Design-First approach with RAML on Anypoint Platform.

QUICK REFERENCE NOTES

- ✓ RAML is an open API specification language.
- ✓ It is YAML-based and easy to read.
- ✓ Supports design-first API development.
- ✓ Generates documentation and mocks automatically.
- ✓ Ensures consistent, reusable and well-documented APIs.
- ✓ Fully integrated with Anypoint Platform tools.

RAML helps you design, document and publish high-quality RESTful APIs in a consistent, secure and scalable way.

RAML helps you design, document and publish high-quality RESTful APIs in a consistent, secure and scalable way on Anypoint Platform.

3.2.1 RAML STRUCTURE OVERVIEW

MuleSoft from salesforce

INTRODUCTION

- A RAML file follows a standard structure to describe a RESTful API in a machine-readable format.
- It defines important API metadata such as the title, version, base URI, resources, methods, and reusable components.
- A well-structured RAML file improves readability, consistency, and API maintainability.

WHAT IS RAML STRUCTURE?

- RAML provides a standardized way to define the structure of REST APIs.
- It organizes API information in a clear and modular format.
- It supports API documentation, reusability, security definitions, and data modeling.
- RAML follows a Design-First approach, where the API contract is designed before implementation.
- It is widely used across MuleSoft and Anypoint Platform tools.

KEY ELEMENTS OF RAML STRUCTURE

ELEMENT	REQUIRED	PURPOSE
#%RAML 1.0	Yes	Specifies the RAML version
title	Yes	Defines the API name
version	Yes	Specifies the API version
baseUri	Yes	Defines the API base URL
mediaType	No	Defines the default payload type
documentation	No	Provides API documentation
resources	Yes	Defines API endpoints
securitySchemes	No	Defines authentication mechanisms
traits & resourceTypes	No	Supports reusable API components
libraries & includes	No	Imports external RAML modules

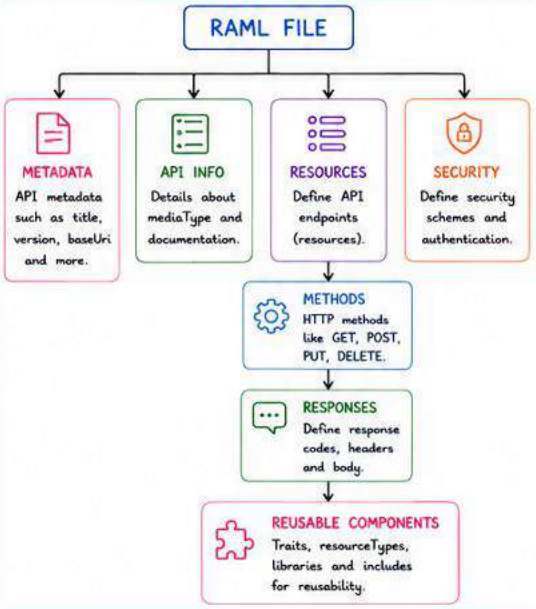
MANDATORY ELEMENTS

- #%RAML 1.0
- title
- version
- baseUri

OPTIONAL ELEMENTS

- mediaType
- documentation
- resources (practically required)
- securitySchemes
- traits
- resourceTypes
- libraries
- includes

HIGH-LEVEL RAML STRUCTURE



WHY IS RAML STRUCTURE IMPORTANT?

- **Standardization** — Provides a common structure for all APIs.
- **Better Maintainability** — Makes APIs easier to understand and manage.
- **Improved Collaboration** — Developers and architects can work using the same API contract.
- **Reusability** — Promotes the use of reusable components such as traits and libraries.
- **Better Documentation** — Enables automatic generation of API documentation.
- **API Governance** — Supports organizational API standards and policies.

KEY BENEFITS

- Consistent API design
- Easy API maintenance
- Improved collaboration among teams
- Supports modular API development
- Simplifies API documentation
- Provides better governance and standardization

QUICK NOTES

- RAML files are YAML-based.
- Use proper indentation throughout the file.
- Follow the Design-First approach whenever possible.
- Organize RAML files into reusable and modular components.
- Keep the structure simple and readable.

KEY TAKEAWAYS

- RAML follows a standardized API structure.
- Mandatory and optional elements provide flexibility.
- A well-designed RAML file improves API quality and maintainability.
- Proper organization makes APIs easier to build, document, and manage.

FINAL SUMMARY

Define → Organize → Reuse

Metadata → Resources → Methods

Responses

Documentation & Security

Clear • Modular • Maintainable APIs

A well-structured RAML file is the foundation of building consistent, reusable, and enterprise-grade RESTful APIs on Anypoint Platform.

3.2.2

RAML STRUCTURE EXPLAINED

MuleSoft from salesforce

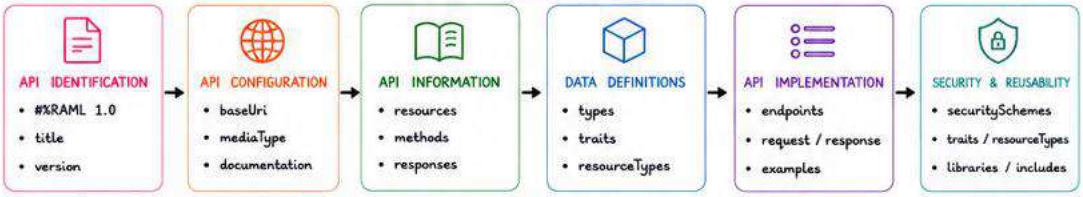
COMPLETE RAML STRUCTURE (EXAMPLE)

```
1  #%RAML 1.0
2  title: Sample API
3  version: v1
4  baseUri: https://api.example.com/{version}
5  mediaType: application/json
5  documentation:
     title: Overview
     content: |
       This API allows you to manage users.
6  /users:
     get:
       description: Get all users
       responses:
         200:
           body:
             application/json:
               type: User[]
8    post:
       description: Create a user
       body:
         application/json:
           type: User
9      responses:
         201:
           body:
             application/json:
10             type: User
```

STRUCTURE EXPLAINED

#	ELEMENT	DESCRIPTION
1	#%RAML 1.0	Specifies the RAML version (1.0).
2	title	Defines the name of the API.
3	version	Specifies the version of the API (e.g., v1, v2).
4	baseUri	The base URL for the API endpoints. {version} is a URI parameter.
5	mediaType	Default media type for requests and responses.
6	documentation	Provides additional information about the API.
7	/users	Resource (collection) that groups related endpoints.
8	methods (get, post)	HTTP methods available for the resource.
9	responses	Possible response status codes with response body.
10	body	Defines the request or response payload structure.

RAML STRUCTURE FLOW



The above flow shows the logical order of a RAML file. In actual files, sections can appear in different order.

BEST PRACTICES

- Keep the RAML file modular using libraries and includes.
- Use meaningful titles and descriptions.
- Define reusable data types, traits and resource types.
- Follow consistent naming conventions.
- Validate RAML using API Designer or CLI.
- Maintain the logical structure for readability.
- Document all sections with examples.

KEY TAKEAWAYS

- RAML has a well-defined structure.
- It starts with API identification and configuration.
- Resources and methods define the API behavior.
- Responses and data types provide the API contract.
- Reusable components improve maintainability.

A well-structured RAML file leads to clear API design, better collaboration and easier maintenance.

3.2.3

RAML STRUCTURE

EXAMPLES, BEST PRACTICES & RULES

MuleSoft from salesforce

MINIMAL RAML EXAMPLE

```
#%RAML 1.0
title: Minimal API
version: v1

/ping:
  get:
    responses:
      200:
        body:
          application/json:
            example:
              {
                "message": "pong"
              }
```

This is the smallest valid RAML API.

COMMON FILE EXTENSION

RAML files use the .raml extension.

WHERE RAML STRUCTURE IS USED

- **Anypoint Design Center** — Design, validate and visualize APIs collaboratively.
- **API Manager** — Apply policies and manage the lifecycle of APIs.
- **Runtime Manager** — Deploy and manage APIs in runtime environments.
- **Exchange** — Publish and share APIs with developers and consumers.
- **API Console** — Generate interactive API documentation.

INDENTATION RULES

CORRECT INDENTATION ✓

```
/users:
  get:
    responses:
      200:
        body:
          application/json:
            type: User[]
```

INCORRECT INDENTATION ✗

```
/users:
  get:
   responses:
     200:
       body:
         application/json:
           type: User[]
```

- Use 2 spaces for indentation (not tabs).
- Wrong indentation can cause parsing errors.

RAML BEST PRACTICES

- Use RAML 1.0 specifications.
- Maintain proper indentation.
- Keep APIs modular and reusable.
- Reuse traits and resource types.
- Define meaningful titles and descriptions.
- Document APIs thoroughly with examples.
- Follow standard naming conventions.
- Validate RAML using API Designer or CLI.
- Follow consistent data types and formats.
- Organize RAML files using libraries and includes.

Follow a consistent structure, proper indentation and best practices to build clear, reusable and maintainable APIs.

KEY TAKEAWAYS

- RAML structure starts with API identification and configuration.
- Resources and methods define API behavior and endpoints.
- Responses and data types define the API contract.
- Reusable components (traits, resource types, libraries) improve modularity.
- Proper indentation and structure ensure better readability.
- A well-structured RAML file improves collaboration and maintenance.

FINAL TAKEAWAY

IDENTIFY → CONFIGURE → DESIGN → IMPLEMENT → SECURE → PUBLISH

RAML STRUCTURE

Clear • Consistent • Reusable APIs

A well-structured RAML file makes APIs easier to design, build, document and manage.

A well-structured RAML file leads to better API design, easier collaboration and improved maintainability.

3.3 RESOURCES & METHODS

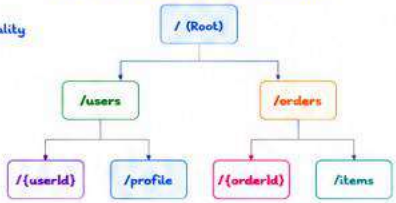
MuleSoft from salesforce

In RAML, a Resource represents an endpoint (URI) and Methods define the actions that can be performed on that resource using standard HTTP verbs.

WHAT ARE RESOURCES?

- Resources represent the data or functionality exposed by your API.
- Each resource is identified by a URI.
- Resources can be nested to represent hierarchical relationships.

EXAMPLE: RESOURCE HIERARCHY



COMMON HTTP METHODS IN RAML

GET	Retrieve data from a resource (Read operation)
POST	Create a new resource (Create operation)
PUT	Update an entire resource (Replace operation)
PATCH	Partially update a resource (Partial update)
DELETE	Delete a resource (Delete operation)
HEAD	Retrieve headers only (No response body)
OPTIONS	Get supported methods (Discover capabilities)

EXAMPLE: RAML RESOURCE STRUCTURE

```
/users:
  get:
    description: Get all users
  post:
    description: Create a new user
/{userId}:
  get:
    description: Get user by ID
  put:
    description: Update user by ID
  delete:
    description: Delete user by ID
```

- Collection Resource (List of users)
- Methods on Collection Resource
- Sub-Resource (Individual User)
- Methods on Sub-Resource

RESOURCE & METHOD EXAMPLES

RESOURCE	METHOD	DESCRIPTION	USE CASE
/users	GET	Get all users	List all users
	POST	Create a new user	Add a new user
/users/{userId}	GET	Get user by ID	Retrieve specific user
	PUT	Update user by ID	Replace user details
	PATCH	Partially update user	Update few fields
	DELETE	Delete user by ID	Remove user
/users/{userId}/orders	GET	Get orders of a user	List user's orders

COMPLETE RAML EXAMPLE WITH RESOURCES & METHODS

```
#%RAML 1.0
title: User Management API
version: v1
baseUri: https://api.example.com/{version}
/users:
    description: Collection of users
    get:
      description: Get all users
      responses:
        200:
          body:
            application/json:
              example: |
                [{ "id": 1, "name": "John" }]
    post:
      description: Create a new user
      body:
        application/json:
          example: |
            { "name": "John", "email": "john@example.com" }
/{userId}:
    description: Operations on a specific user
    get:
      description: Get user by ID
      responses:
        200:
          body:
            application/json:
              example: { "id": 1, "name": "John" }
    put:
      description: Update user by ID
    delete:
      description: Delete user by ID
```

- Collection Resource (/users)
- Methods on Collection
- Sub-Resource (/users/{userId})
- Methods on Sub-Resource

BEST PRACTICES

- Use nouns for resources (e.g., /users, /orders).
- Follow consistent naming conventions.
- Use plural for collections (e.g., /users, /products).
- Use appropriate HTTP methods for operations.
- Keep URIs clean, simple and meaningful.
- Avoid action words in URIs (e.g., avoid /getUsers).

URI PARAMETERS IN RESOURCES

- Use URI parameters to identify specific resources.
- Defined using curly braces {param}.

Example URI	/users/{userId}/orders/{orderId}
Meaning	Order of a specific user by userId and orderId

QUICK REFERENCE: HTTP METHODS

METHOD	PURPOSE	IDEMPOTENT
GET	Read/Retrieve data	Yes
POST	Create new resource	No
PUT	Replace/Update resource	Yes
PATCH	Partially update resource	Yes
DELETE	Remove resource	Yes

COMMON USE CASES

- /users → Get all users, Create user
- /users/{userId} → Manage a specific user
- /orders → Manage orders
- /products/{productId} → Product details

SUMMARY

- Resources define the API endpoints (URIs).
- Methods define the actions on resources.
- Use standard HTTP verbs for consistency.
- Organize resources hierarchically.
- Follow best practices for clean APIs.

Well-structured Resources and correct use of Methods make your APIs clear, consistent, easy to consume and maintain on Anypoint Platform.

3.3.1 WHAT ARE RESOURCES & METHODS?

MuleSoft from salesforce

In RAML, a Resource represents an API endpoint (URI), and Methods define the operations that can be performed on that resource using standard HTTP verbs.

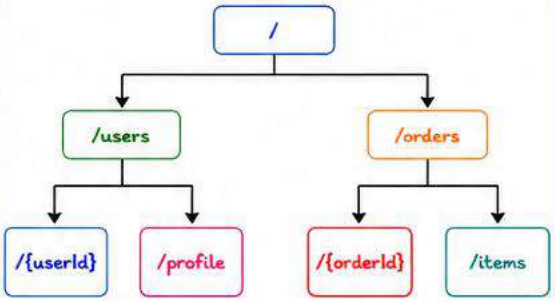
WHAT ARE RESOURCES?

- Resources represent API endpoints.
- Every resource is identified by a URI.
- Resources expose business functionality.
- Resources can contain sub-resources.

EXAMPLES OF RESOURCES

/users /orders /products

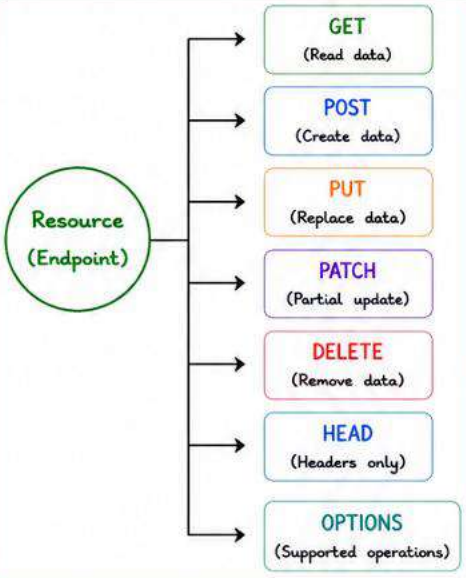
RESOURCE HIERARCHY



COMMON HTTP METHODS

METHOD	PURPOSE
GET	Retrieve data from a resource (Read operation).
POST	Create a new resource (Create operation).
PUT	Replace an entire resource (Replace operation).
PATCH	Update part of a resource (Partial update).
DELETE	Delete a resource (Delete operation).
HEAD	Retrieve headers only (No response body).
OPTIONS	Get supported methods (Discover capabilities).

RESOURCE & METHOD RELATIONSHIP



WHY RESOURCES & METHODS ARE IMPORTANT?

- They form the foundation of any RESTful API.
- Resources define what data is available.
- Methods define the actions that can be performed.
- Standard HTTP methods ensure consistency.
- They promote a clean and predictable API design.

KEY TAKEAWAYS

- Resources define API endpoints (URIs).
- Methods define the operations on resources.
- Use standard HTTP verbs for consistency.
- Follow RESTful principles for better APIs.
- A well-designed structure improves readability, maintainability and scalability.

Resources define API endpoints and Methods define actions.
Together they build powerful, consistent and easy-to-use APIs!

3.3.2 RAML RESOURCE STRUCTURE & EXAMPLES

MuleSoft from salesforce

RAML RESOURCE EXAMPLE

```
/users:
  get:
    description: Get all users

  post:
    description: Create user
```

This defines a collection resource "/users" with GET and POST methods.

SUB-RESOURCE EXAMPLE

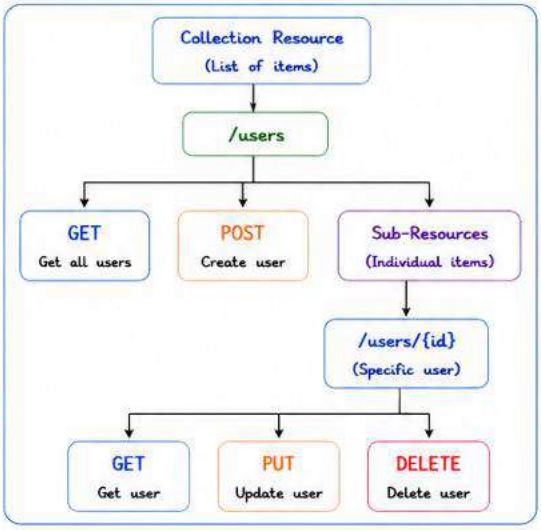
```
/users/{userId}:
  get:
    description: Get user details

  put:
    description: Update user

  delete:
    description: Delete user
```

This defines a sub-resource "/users/{userId}" with GET, PUT and DELETE methods.

COMPLETE RESOURCE STRUCTURE



RESOURCE & METHOD EXAMPLES

RESOURCE	METHOD	DESCRIPTION
/users	GET	Get all users
/users	POST	Create a new user
/users/{id}	GET	Get user by ID
/users/{id}	PUT	Update user details
/users/{id}	DELETE	Delete user by ID
/users/{id}/orders	GET	Get orders of a user

BENEFITS OF WELL-STRUCTURED RESOURCES

- Provides a clear and organized API structure.
- Improves readability and understandability.
- Makes APIs easier to extend and maintain.
- Promotes reusability through sub-resources.
- Supports RESTful design principles.

KEY TAKEAWAYS

- Resources represent endpoints (URIs) in an API.
- Methods define operations on those resources.
- Sub-resources help in representing hierarchical data.
- Use proper HTTP methods for each operation.
- A good structure leads to better API design.

A well-structured resource hierarchy and correct method usage result in clean, consistent and maintainable APIs.

3.3.3

RESOURCES & METHODS

QUICK REFERENCE, COMMON USE CASES & SUMMARY

MuleSoft from salesforce

QUICK REFERENCE – HTTP METHODS

HTTP METHOD	PURPOSE	IDEMPOTENT
GET	Read / Retrieve data	Yes
POST	Create a new resource	No
PUT	Replace / Update a resource	Yes
PATCH	Partially update a resource	Yes
DELETE	Remove a resource	Yes

- GET – Retrieves information from the API.
- POST – Creates a new resource.
- PUT – Replaces an existing resource.
- PATCH – Updates selected fields of a resource.
- DELETE – Removes an existing resource.

COMMON USE CASES

1. USER MANAGEMENT API

/users
/users/{userId}

- Create users
- Retrieve user info
- Update user profiles
- Delete users

2. ORDER MANAGEMENT API

/orders
/orders/{orderId}

- Manage customer orders
- Track order status
- Update order details
- View order history

3. PRODUCT MANAGEMENT API

/products
/products/{productId}

- Manage product catalogs
- Retrieve product info
- Update inventory details
- Manage product data

4. PAYMENT MANAGEMENT API

/payments
/payments/{paymentId}

- Process payments
- Track transactions
- Retrieve payment status

RESTFUL API DESIGN BEST PRACTICES

1. RESOURCE NAMING

Use meaningful and plural resource names.

RECOMMENDED
/users
/orders
/products
/customers
/payments

AVOID
/getUsers
/createUser
/deleteOrder
/updateProduct

2. URI DESIGN GUIDELINES

- Keep URIs short and meaningful.
- Use nouns instead of verbs.
- Follow a consistent naming convention.
- Use URI parameters for dynamic resources.

EXAMPLES
/users/{userId}
/orders/{orderId}
/products/{productId}
/users/{userId}/orders/{orderId}

3. HTTP METHOD GUIDELINES

Always use the appropriate HTTP method for the operation.

OPERATION	METHOD
Retrieve Data	GET
Create Resource	POST
Replace Resource	PUT
Partial Update	PATCH
Delete Resource	DELETE

4. GENERAL GUIDELINES

- Follow RESTful conventions.
- Use meaningful resource names.
- Keep APIs simple and reusable.
- Maintain consistency across APIs.
- Properly document all resources and methods.
- Test and validate APIs regularly.

BENEFITS OF FOLLOWING BEST PRACTICES

- Improves API readability.
- Enhances maintainability.
- Simplifies API consumption.
- Promotes RESTful standards.
- Improves API consistency.
- Supports scalable API design.
- Provides a better developer experience.

SUMMARY

RESOURCES	Define API endpoints. Represent business entities.
METHODS	Define operations on resources.
URI PARAMETERS	Identify specific resources dynamically.
BEST PRACTICES	Follow RESTful conventions. Use meaningful names. Keep APIs simple and reusable.

FINAL TAKEAWAY

Use Proper Resources
↓
Follow REST Standards
↓
Use Correct HTTP Methods
↓
Keep URIs Meaningful
↓
Build Clean & Scalable APIs

Clear • Consistent • Scalable APIs

Well-structured resources and the proper use of HTTP methods make APIs easier to design, consume, maintain and scale on Anypoint Platform.

3.4.1 URI PARAMETERS

MuleSoft from salesforce

URI Parameters (also called Path Parameters) are dynamic values that are included directly in the URI path. They are used to uniquely identify specific resources in RESTful APIs and help establish hierarchical relationships between resources.

WHAT ARE URI PARAMETERS?

- URI Parameters are dynamic values present in the URL path.
- They are enclosed within curly braces { } in RAML.
- They are used to uniquely identify a specific resource.
- URI Parameters are mandatory when accessing resource-specific data.
- They play a key role in designing RESTful and resource-centric APIs.

WHY USE URI PARAMETERS?

- Resource Identification – Helps identify a specific resource uniquely.
- Dynamic API Requests – Allows APIs to accept variable values in the URI.
- RESTful API Design – Supports resource-based API architecture.
- Hierarchical Relationships – Represents parent-child relationships between resources.
- Improved Readability – Makes API endpoints easy to understand and maintain.

HOW URI PARAMETERS WORK

KEY BENEFITS

- Makes APIs resource-centric.
- Supports RESTful API principles.
- Improves URI readability.
- Enables dynamic resource identification.
- Simplifies API design and maintenance.
- Improves API scalability.

KEY TAKEAWAYS

- URI Parameters uniquely identify resources.
- They are essential for RESTful API design.
- RAML provides built-in support for defining URI Parameters.
- Proper use of URI Parameters results in clean and scalable APIs.

URI PARAMETER SYNTAX IN RAML

```
/users/{userId}:
  uriParameters:
    userId:
      type: integer
      description: Unique ID of the user
      example: 12345
```

SYNTAX EXPLAINED

ELEMENT	DESCRIPTION
{userId}	URI parameter enclosed within curly braces
type	Specifies the parameter data type
description	Explains the purpose of the parameter
example	Provides a sample value
uriParameters	Defines all URI parameters for a resource

EXAMPLE URI TEMPLATES

URI TEMPLATE		EXAMPLE URI
/users/{userId}	→	/users/12345
/orders/{orderId}	→	/orders/987
/products/{productId}	→	/products/1001
/customers/{customerId}	→	/customers/501

COMMON USE CASES

- /users/{userId} → Retrieves information about a specific user.
- /orders/{orderId} → Retrieves information about a specific order.
- /products/{productId} → Retrieves details of a particular product.

QUICK NOTES

- URI Parameters are part of the URI path.
- Always use meaningful parameter names.
- They are enclosed within { }.
- Prefer resource names that are plural.
- Use URI Parameters only for identifying resources.

FINAL SUMMARY

URI Parameters make your APIs RESTful, resource-centric and easy to maintain. They are one of the fundamental building blocks of API design in MuleSoft and RAML.

3.4.2 URI PARAMETERS

(URI vs QUERY PARAMETERS & MULTIPLE URI PARAMETERS)

MuleSoft from salesforce

URI PARAMETERS vs QUERY PARAMETERS

ASPECT	URI PARAMETERS (Path Parameters)	QUERY PARAMETERS
Location	Included in the URI path	Included after '?' in the URI
Purpose	Identify a specific resource	Filter, search, sort, or paginate data
Mandatory	Usually required	Usually optional
Syntax	/users/{userId}	/users?name=John
RESTful Design	Resource-centric	Query-centric
Caching	Better support for caching	Less efficient for caching

WHEN TO USE URI PARAMETERS?

- Retrieve a specific resource.
- Update a specific record.
- Delete a particular resource.
- Design hierarchical resource structures.

Examples:

```
/users/{userId}
/orders/{orderId}
/products/{productId}
```

WHEN TO USE QUERY PARAMETERS?

- Filtering records.
- Searching data.
- Sorting results.
- Implementing pagination.

Examples:

```
/users?name=John
/orders?status=Shipped
/products?category=Electronics
/users?page=1&size=10
```

MULTIPLE URI PARAMETERS

RAML

```
/orders/{orderId}/items/{itemId}:
  uriParameters:
    orderId:
      type: integer
      description: Unique Order ID
      example: 987
    itemId:
      type: integer
      description: Unique Item ID
      example: 10
```

EXAMPLE URI

```
/orders/987/items/10
```

PARAMETER MAPPING

PARAMETER	VALUE
orderId	987
itemId	10

NESTED RESOURCE DESIGN

/users
↓
/users/{userId}
↓
/users/{userId}/orders
↓
/users/{userId}/orders/{orderId}

This structure improves:

- Readability
- Resource organization
- API maintainability

HOW MULTIPLE URI PARAMETERS WORK

Client Request → /orders/987/items/10 → Extract URI Parameters → orderId = 987, itemId = 10 → Retrieve Specific Item Details

BENEFITS OF USING URI PARAMETERS

- Supports RESTful API design.
- Makes resource identification easy.
- Improves API readability.
- Supports hierarchical resource relationships.
- Enables scalable API structures.

QUICK TIPS

USE URI PARAMETERS

```
/users/{userId}
/orders/{orderId}
```

When:

- Accessing a specific resource.
- Designing resource hierarchies.

USE QUERY PARAMETERS

```
/users?country=India
/orders?status=Delivered
```

When:

- Filtering data.
- Searching resources.
- Implementing pagination.

KEY TAKEAWAYS

- URI Parameters identify resources.
- Query Parameters filter or search resources.
- Multiple URI Parameters support nested APIs.
- Follow RESTful resource hierarchy for better API design.
- Use URI Parameters only when identifying resources.

FINAL SUMMARY

Identify Specific Resources → Support Nested Resource Structures → Use Query Parameters for Search & Filtering → Follow RESTful Standards → Build Clean & Scalable APIs

URI Parameters and Query Parameters complement each other in REST API design. Using them appropriately results in clean, intuitive and maintainable MuleSoft APIs.

3.4.3 URI PARAMETERS

(VALIDATION, BEST PRACTICES & COMMON USE CASES)

MuleSoft from salesforce

VALIDATION & CONSTRAINTS

CONSTRAINT	PURPOSE	EXAMPLE
type	Defines the data type	integer, string
minimum	Minimum numeric value	1
maximum	Maximum numeric value	999999
minLength	Minimum length of a string	3
maxLength	Maximum length of a string	10
pattern	Regular expression validation	[A-Z]{3}[0-9]{3}
enum	Restricts allowed values	Active, Inactive

URI PARAMETER VALIDATION EXAMPLE

```
/users/{userId}:
  uriParameters:
    userId:
      type: integer
      description: Unique ID of the user
      example: 123
      minimum: 1
      maximum: 999999
```

VALIDATION RULES

- Must be an integer.
- Must be greater than or equal to 1.
- Must be less than or equal to 999999.
- Must contain a valid user ID.

COMMON VALIDATION EXAMPLES

1. NUMERIC VALIDATION — minimum: 1, maximum: 1000
2. STRING LENGTH VALIDATION — minLength: 3, maxLength: 20
3. REGULAR EXPRESSION VALIDATION — pattern: "[A-Z]{3}[0-9]{3}", Example: ABC123
4. ENUM VALIDATION — enum: Active, Inactive, Pending. Only the specified values are accepted.

NESTED RESOURCES WITH URI PARAMETERS

RESOURCE HIERARCHY

/users
↓
/users/{userId}
↓
/users/{userId}/orders
↓
/users/{userId}/orders/{orderId}

EXAMPLE REQUEST PATH

/users/123/orders/987

PARAMETER VALUES

PARAMETER	VALUE
userId	123
orderId	987

URI Parameters help represent hierarchical relationships between resources.

BEST PRACTICES

1. USE MEANINGFUL NAMES

Good Examples
- userId
- orderId
- productId
- customerId
- invoiceId

Avoid
- id
- x
- param1
- value1

2. USE APPROPRIATE DATA TYPES

Examples
- type: integer
- type: string
- type: number

Always choose the most appropriate type.

3. APPLY VALIDATION CONSTRAINTS

Examples
- minimum
- maximum
- minLength
- maxLength
- pattern
- enum

Validation improves API reliability and reduces errors.

4. FOLLOW RESTFUL RESOURCE HIERARCHY

Good Examples
- /Users/{userId}
- /orders/{orderId}
- /products/{productId}

Avoid unnecessary nesting or overly complex URI structures.

5. DOCUMENT URI PARAMETERS

Always provide:
- Description
- Data type
- Example values
- Validation rules

Well-documented APIs are easier to consume and maintain.

COMMON USE CASES

/users/{userId}
Use Cases:
- Retrieve user details
- Update user information
- Delete a user

/orders/{orderId}
Use Cases:
- Retrieve order details
- Track order status

/products/{productId}
Use Cases:
- Retrieve product information
- Manage inventory

/invoices/{invoiceId}
Use Cases:
- Retrieve invoice details
- Download invoice information

BENEFITS OF USING VALIDATED URI PARAMETERS

- Improves API reliability.
- Enhances security.
- Supports RESTful API design.
- Reduces invalid API requests.
- Improves maintainability.
- Provides better API documentation.
- Enables scalable API structures.

QUICK TIPS

Use URI Parameters only for resource identification.
- Apply validation wherever possible.
- Keep resource hierarchies simple.
- Use meaningful parameter names.
- Provide examples and descriptions for every parameter.

KEY TAKEAWAYS

- URI Parameters uniquely identify resources.
- Validation constraints improve API quality.
- Proper documentation improves developer experience.
- Follow RESTful design principles consistently.
- Keep APIs simple, scalable, and maintainable.

FINAL SUMMARY

URI PARAMETERS
- Identify Resources
- Apply Validation
- Follow REST Principles
- Use Meaningful Names
- Build Secure & Scalable APIs

URI Parameters make your APIs RESTful, clear, and resource-centric. Proper validation and best practices help build secure, intuitive, and enterprise-grade MuleSoft APIs.

3.5.1 QUERY PARAMETERS

MuleSoft from salesforce

WHAT ARE QUERY PARAMETERS?

Query Parameters are optional values that are added to the end of a URI to customize the API response. They are commonly used for filtering, searching, sorting, and pagination without changing the actual resource path.
Unlike URI Parameters, Query Parameters do not identify a specific resource.
Instead, they control the data that should be returned by the API.

KEY CHARACTERISTICS

- Passed in the query string of the URI.
- Optional by default.
- Used to customize API responses.
- Do not identify specific resources.
- Multiple Query Parameters are separated using the & symbol.
- Extremely useful for filtering and pagination.

EXAMPLE OF QUERY PARAMETERS

`/users?name=John&status=active&page=1&limit=10`

↓

Parameter Breakdown

PARAMETER	VALUE	PURPOSE
name	John	Filter users by name
status	active	Retrieve active users
page	1	Return page number 1
limit	10	Return only 10 records

RAML SYNTAX

```
/users:
  get:
    queryParameters:
      name:
        type: string
        required: false
        description: Filter users by name
        example: John
      status:
        type: string
        required: false
        description: Filter users by status
        example: active
      page:
        type: integer
        required: false
        description: Page number
        minimum: 1
        example: 1
      limit:
        type: integer
        required: false
        description: Number of records per page
        default: 10
        maximum: 100
        example: 10
```

REQUIRED vs OPTIONAL PARAMETERS

OPTIONAL PARAMETERS (Default)

`required: false`

- Can be omitted from the API request.
- Most commonly used in REST APIs.
- Default values can be provided.

REQUIRED PARAMETERS

`required: true`

- Must be provided in the request.
- Missing parameters may result in 400 Bad Request error.
- Used only when absolutely necessary.

COMMON DATA TYPES

DATA TYPE	DESCRIPTION
string	Text values
integer	Whole numbers
number	Decimal values
boolean	True or False
array	List of values

WHEN SHOULD WE USE QUERY PARAMETERS?

	EXAMPLES
→ Filtering records.	• /users?status=active
→ Searching data.	• /users?name=John
→ Sorting results.	• /users?page=2&limit=20
→ Implementing pagination.	• /users?sort=createdDate
→ Limiting API responses.	• /users?search=MuleSoft
→ Customizing returned data.	

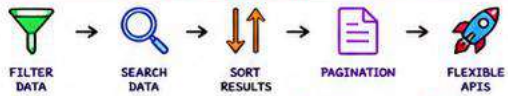
BEST PRACTICES

- ✓ Use meaningful parameter names.
- ✓ Use appropriate data types.
- ✓ Validate user inputs.
- ✓ Provide default values whenever possible.
- ✓ Clearly document all Query Parameters.
- ✓ Keep parameter names short and descriptive.

KEY TAKEAWAYS

- Query Parameters customize API responses.
- They are optional by default.
- They are ideal for filtering, searching, sorting and pagination.
- They improve API flexibility and usability.
- Proper documentation and validation are recommended.

FINAL SUMMARY



Query Parameters make APIs flexible, powerful, and user-friendly.

3.5.2 QUERY PARAMETERS

MuleSoft from salesforce

TYPES OF QUERY PARAMETERS & EXAMPLES

Query Parameters are used to filter, search, sort, paginate, and customize API responses. They help APIs return only the required data without changing the resource path.

COMMON DATA TYPES

DATA TYPE	DESCRIPTION
string	Text values
integer	Whole numbers
number	Decimal numbers
boolean	True or False values
array	List of values (comma separated)

TYPES OF QUERY PARAMETERS

TYPE	PURPOSE	DESCRIPTION
Filtering	Filter API responses	Return records that match specific conditions.
Searching	Search using keywords	Find records using keywords or text.
Sorting	Sort results	Sort data in ascending or descending order.
Pagination	Retrieve paged data	Return a specific page of results with a size limit.
Field Selection	Return selected fields	Return only required fields in the response.

EXAMPLES

- Filtering → /users?status=active
- Searching → /users?search=John
- Sorting → /users?sort=createdAt&order=desc
- Pagination → /users?page=2&limit=10
- Field Selection → /users?fields=id,name,email
- Multiple Filters → /users?role=admin&status=active
- Multiple Values → /users?status=active,inactive,pending

MULTIPLE VALUES EXAMPLE

You can pass multiple values for the same parameter using comma separated values.

/users?status=active,inactive,pending

RAML EXAMPLE

```
status:
  type: string[]
  description: Filter by multiple status values
  example:
    - active
    - inactive
    - pending
```

COMMON USE CASES

USE CASE	EXAMPLE URL
Filter by Name	/users?name=John
Filter by Status	/users?status=active
Pagination	/users?page=3&limit=20
Sorting	/users?sort=name&order=asc
Search Users	/users?search=API
Multiple Filters	/users?role=admin&status=active
Multiple Values	/users?status=active,inactive
Field Selection	/users?fields=id,name,email

KEY TAKEAWAYS

- Different types of Query Parameters help customize API responses.
- They make APIs flexible and powerful.
- You can combine multiple parameters to get precise results.
- Multiple values can be passed using comma separated values.
- Field selection helps in reducing the response size and improving performance.

FINAL SUMMARY

Query Parameters help you FILTER, SEARCH, SORT, PAGINATE, and CUSTOMIZE API responses. Use the right parameter type and follow best practices to build efficient, user-friendly, and scalable APIs.

3.5.3 QUERY PARAMETERS

MuleSoft from salesforce

VALIDATION, BEST PRACTICES & COMMON MISTAKES

Query Parameters make APIs flexible by allowing consumers to filter, search, sort, paginate, and customize responses. To build robust APIs, it's important to validate inputs, follow best practices, and avoid common mistakes.

1. VALIDATION & CONSTRAINTS

CONSTRAINT	DESCRIPTION	RAML EXAMPLE
required	Parameter is mandatory.	`required: true`
default	Default value used if not provided.	`default: 10`

enum	Restricts values to a set list.	`enum:`
		`- active`
		`- inactive`
		`- pending`

minimum	Minimum allowed value.	`minimum: 1`
maximum	Maximum allowed value.	`maximum: 100`
pattern	Validates value using regex.	`pattern: "^[a-zA-Z0-9]+$"`
type	Defines the data type.	`type: integer`

2. QUERY PARAMETERS vs URI PARAMETERS

	QUERY PARAMETERS	URI PARAMETERS
	Appear after ? in the URI.	Part of the URI path.
	Used to filter, search, sort, paginate, and customize data.	Used to identify a specific resource.
	Do not change the resource path.	Change the resource path.
	Optional by default.	Usually required.
	Multiple values can be passed for same parameter.	Usually single value for each URI parameter.

EXAMPLES

Query Parameters

```
/users?name=John
/users?page=2&limit=20
/users?status=active
/users?sort=name
/users?search=API
```

URI Parameters

```
/users/123
/users/John/orders
/products/789/details
```

3. COMMON MISTAKES

- Using Query Parameters to identify resources.
- Not validating input values.
- Not setting default values for optional parameters.
- Allowing too many optional parameters.
- Not documenting Query Parameters.
- Using unclear or inconsistent parameter names.
- Accepting invalid data types.
- Ignoring edge cases (null, empty, large values).

4. BEST PRACTICES

- Use meaningful, short, and consistent parameter names.
- Validate all input values and return proper errors.
- Use correct data types for each parameter.
- Set default values where applicable.
- Use enums for parameters with fixed choices.
- Support multiple values using comma separated values.
- Keep the number of optional parameters minimal.
- Document every Query Parameter with examples.
- Maintain consistency across all APIs.

5. BENEFITS OF QUERY PARAMETERS

- Provides flexibility to API consumers.
- Improves API usability and experience.
- Supports powerful filtering and searching.
- Enables efficient pagination.
- Helps retrieve only required data.
- Improves performance and scalability.
- Standard practice in RESTful APIs.

6. SAMPLE RAML WITH CONSTRAINTS

```
/users:
  get:
    queryParameters:
      status:
        type: string
        required: false
        enum: [active, inactive, pending]
        description: Filter users by status
      page:
        type: integer
        required: false
        minimum: 1
        default: 1
        description: Page number
      limit:
        type: integer
        required: false
        minimum: 1
        maximum: 100
        default: 10
        description: Number of records per page
```

EXAMPLE URI

```
/users?
status=active,
inactive,pending
&page=2&limit=20
```

RESPONSE EXAMPLE

```
{
  "status": "active",
  "page": 2,
  "limit": 20,
  "data": [
    { ... },
    { ... }
  ]
}
```

7. KEY TAKEAWAYS

- Query Parameters customize API responses.
- They are optional by default.
- They are ideal for filtering, searching, sorting, and pagination.
- They do not change the resource path.
- Validation and best practices make APIs reliable, secure, and user-friendly.
- Always document and test your Query Parameters.

Well designed Query Parameters make APIs FLEXIBLE, POWERFUL, and USER-FRIENDLY. Follow best practices, validate inputs, and avoid common mistakes to build SCALABLE and EFFICIENT MuleSoft APIs.

3.6.1 REQUEST & RESPONSE

MuleSoft from salesforce

In RAML, every API method can have a REQUEST (what the client sends) and one or more RESPONSES (what the server returns). Properly defining request and response structures is essential for building reliable and consumer-friendly APIs.

1 REQUEST BODY

- The body contains the data sent by the client.
- Used in POST, PUT, and PATCH methods.
- Specifies media type (e.g., application/json).
- Supports inline schemas and data types.

RAML SYNTAX EXAMPLE

```
post:
  body:
    application/json:
      type: object
      example:
        id: 101
        name: John
        email: john@example.com
```

KEY POINTS

- Used for sending data to APIs.
- Supports JSON and XML payloads.
- Helps validate incoming requests.
- Ensures data is in the correct format.

2 RESPONSE BODY

- The body contains the data returned by the server.
- Can have multiple status codes.
- Includes the response payloads.
- Helps the client process the returned data.

RAML SYNTAX EXAMPLE

```
responses:
  200:
    body:
      application/json:
        type: object
        example:
          id: 101
          name: John
          email: john@example.com
```

KEY POINTS

- Every response can have its own body.
- Responses should be properly documented.
- Supports reusable data types.
- Improves consumer experience.

3 COMMON HTTP STATUS CODES

CODE	MEANING	WHEN TO USE
200 OK	Success	Request succeeded.
201 Created	Created	Resource created successfully.
202 Accepted	Accepted	Request accepted for processing.
204 No Content	No Content	Success, but no data returned.
400 Bad Request	Client Error	Invalid request or parameters.
401 Unauthorized	Unauthorized	Authentication required/failed.
403 Forbidden	Forbidden	Client not allowed to access.
404 Not Found	Not Found	Resource not found.
409 Conflict	Conflict	Resource conflict (e.g., duplicate).
500 Internal	Server Error	Unexpected server error.
503 Service	Unavailable	Service temporarily unavailable.

BEST PRACTICES

- Use meaningful status codes.
- Return helpful error messages.
- Keep response structures consistent.
- Use appropriate headers.
- Validate request data and return proper errors.

SUMMARY

CLIENT — Sends Request

REQUEST BODY — Data sent by the client

API PROCESSING — Server processes the request

RESPONSE BODY — Data returned by the server

STATUS CODE — Indicates the result of request

KEY TAKEAWAYS

- Request Body is used to send data to the API.
- Response Body is used to return data from the API.
- HTTP Status Codes help in understanding the result of the request.
- Properly defining request and response structures makes APIs reliable and easy to consume.
- Always follow best practices for building robust and user-friendly APIs.

3.6.2 REQUEST & RESPONSE

MuleSoft from salesforce

4 HEADERS

Headers provide additional metadata for requests and responses.

REQUEST HEADERS

- Content-Type
- Authorization
- Accept
- User-Agent
- Custom Headers

RESPONSE HEADERS

- Content-Type
- RateLimit-Remaining
- Cache-Control
- Date
- Server

KEY POINTS

- Headers are used to send and receive metadata.
- They can be required or optional.
- Common headers improve API communication and control.

5 EXAMPLE: COMPLETE METHOD (POST /users)

```
/users:
  post:
    description: Create a new user
    headers:
      Content-Type:
        type: string
        required: true
    body:
      application/json:
        type: object
        example:
          id: 101
          name: John
          email: john@example.com
  responses:
    201:
      description: User created successfully
      body:
        application/json:
          type: object
          example:
            id: 101
            name: John
            email: john@example.com
    400:
      description: Invalid input
      body:
        application/json:
          type: object
          example:
            code: INVALID_REQUEST
            message: Email is required
```

KEY POINTS

- A complete method includes headers, body and responses.
- Supports multiple status codes.
- Provides clear examples for consumers.
- Improves API documentation and usability.

METHOD COMPONENTS

1. URI
2. Description
3. Headers
4. Request Body
5. Responses

6 DATA TYPE EXAMPLES

Example: User Data Type

```
User:
  type: object
  properties:
    id:
      type: integer
      example: 101
    name:
      type: string
      example: John
    email:
      type: string
      example: john@example.com
```

Data types help in reusability and ensure consistent data structures.

Example: Error Data Type

```
Error:
  type: object
  properties:
    code:
      type: string
      example: INVALID_REQUEST
    message:
      type: string
      example: Invalid input provided
    details:
      type: string
      example: Email is required
    timestamp:
      type: string
      example: 2024-05-18T10:15:30Z
    path:
      type: string
      example: /users
```

KEY POINTS

- Data types define the structure of request and response data.
- They improve reusability.
- Ensure schema validation.
- Reduce duplication of data structures.
- Make APIs more maintainable and scalable.

TIPS

- Use proper headers for authentication and content negotiation.
- Always define Content-Type.
- Use reusable data types.
- Keep method definitions clean and clear.

REMEMBER

- Headers carry metadata.
- Bodies carry payload.
- Data types ensure consistency.
- Good documentation = Better APIs.

Well-structured methods with proper headers, request/response bodies and data types lead to reliable and consumer-friendly APIs.

3.6.3 REQUEST & RESPONSE

MuleSoft from salesforce

7 REQUEST vs RESPONSE COMPARISON

	REQUEST (Client → Server)	RESPONSE (Server → Client)
	Sent by the client to the server.	Sent by the server to the client.
	Contains data or parameters.	Contains results or status.
	Includes headers and body.	Includes headers and body.
	Used to create, update or request data.	Used to return data or outcome.
	Examples: POST, PUT, PATCH, GET	Examples: 200 OK, 404 Not Found, etc.

8 MULTIPLE RESPONSES EXAMPLE (GET /users/{id})

```
/users/{id}:
  get:
    description: Get user by ID
    responses:
      200:
        description: Successful response
        body:
          application/json:
            type: object
            example:
              id: 101
              name: John
              email: john@example.com
      404:
        description: User not found
        body:
          application/json:
            type: object
            example:
              message: "User not found"
      500:
        description: Server error
        body:
          application/json:
            type: object
            example:
              message: "Something went wrong"
```

200 OK

Success

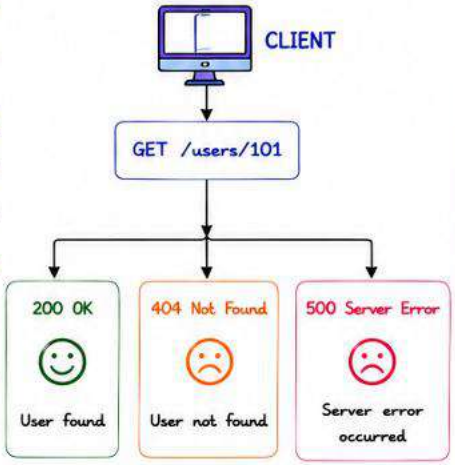
404 Not Found

User not found or resource not available.

500 Server Error

Something went wrong on the server.

9 RESPONSE FLOW



10 ERROR RESPONSE BEST PRACTICES

- Use a consistent error structure across all APIs.
- Return meaningful and human-readable error messages.
- Include application specific error codes.
- Include timestamp for better troubleshooting.
- Include the requested resource path.
- Provide additional details if required.
- Avoid exposing internal server information.

EXAMPLE ERROR STRUCTURE

```
type: object
properties:
  code:
    type: string
    description: Application specific error code
  message:
    type: string
    description: Human readable error message
  details:
    type: string
    description: Additional details about the error
timestamp:
  type: string
  format: date-time
  example: 2024-05-18T10:15:30Z
path:
  type: string
  description: The requested resource path
```

QUICK TIPS

- Use proper HTTP status codes.
- Keep error messages meaningful and user-friendly.
- Validate request payloads before processing.
- Always document request and response structures.
- Test APIs using tools like Postman or API Console.

NOTES

- Test your API endpoints thoroughly.
- Validate request and response schemas.
- Use data types and examples for better clarity.
- Keep API designs simple, consistent and consumer-friendly.

FINAL SUMMARY

Client Sends Request
↓
API Receives and Processes
↓
Server Returns Response
↓
Appropriate HTTP Status Code
↓
Clear & Consistent Communication
↓
Reliable & User-Friendly APIs

KEY TAKEAWAYS

- Request and Response are the core of API communication.
- Use proper headers, data types and status codes.
- Document every part clearly for better API usability.
- Good design leads to secure, reliable and scalable APIs.

3.7.1 DATA TYPES

MuleSoft from salesforce

RAML supports various built-in and custom data types that help define the structure of request and response bodies. Data types improve API validation, consistency and documentation.

1 BASIC DATA TYPES (BUILT-IN)

Data Type	Description	Example
string	Textual data	"John Doe"
number	Decimal numbers	25.50
integer	Whole numbers	100
boolean	True or False	true
date-only	Date	2024-05-20
time-only	Time	14:30:00
datetime	Date and Time	2024-05-20T14:30:00Z
datetime-only	Date and Time (No Time Zone)	2024-05-20T14:30:00
file	Binary File	File Content
nil	Null Value	null

2 RAML SYNTAX (TYPE DECLARATION)

```
types:
  User:
    type: object
    properties:
      id: integer
      name: string
      email: string
      isActive: boolean
      createdAt: datetime
```

KEY POINTS

- Define custom types under the 'types' section.
- Reuse data types across APIs.
- Improves API readability.
- Supports validation and auto-documentation.

3 USING DATA TYPES IN RAML

```
/users:
  post:
    body:
      application/json:
        type: User
        example:
          id: 101
          name: John
          email: john@example.com
          isActive: true
```

BENEFITS

- Promotes consistency in API design.
- Makes APIs reusable and modular.
- Improves maintainability.
- Provides better documentation.

TAKE AWAY

Using the right data types in RAML ensures well-defined, validated and well-documented APIs that are easy to understand and maintain.

3.7.2

DATA TYPES

MuleSoft from salesforce

Advanced data types in RAML help to model complex structures and flexible data.

4 OBJECT TYPE (EXAMPLE)

RAML Syntax:

```
types:
  Address:
    type: object
    properties:
      street: string
      city: string
      state: string
      zipCode: string
      country: string
```

Example (JSON):

```
{
  "street" : "221B Baker Street",
  "city" : "London",
  "state" : "London",
  "zipCode" : "NW1 6XE",
  "country" : "UK"
}
```

Object is used to group related properties.

5 ARRAY TYPE (EXAMPLE)

RAML Syntax:

```
types:
  UserList:
    type: array
    items: User
```

Example (JSON):

```
[
  {
    "id" : 101, "name" : "John"
  },
  {
    "id" : 102, "name" : "Jane"
  }
]
```

Array holds multiple values of the same type.

6 NESTED OBJECTS & ARRAYS

RAML Syntax:

```
types:
  Order:
    type: object
    properties:
      orderId: integer
      user: User
      items:
        type: array
        items: OrderItem
```

You can nest objects inside objects and arrays.

7 SCALAR TYPE CONSTRAINTS

Type	Constraints You Can Use	Example
string	minLength, maxLength, pattern, enum	minLength: 3 maxLength: 50 pattern: "^[A-Z].*"
number	minimum, maximum, multipleOf	minimum: 1 maximum: 100
integer	minimum, maximum, multipleOf	minimum: 1 maximum: 100
boolean	true or false	true / false
datetime	ISO format	2024-05-20T14:30:00Z

Constraints help to validate data.

8 ENUM TYPE

RAML Syntax:

```
types:
  OrderStatus:
    type: string
    enum:
      - PENDING
      - PROCESSING
      - SHIPPED
      - DELIVERED
      - CANCELLED
```

Example Values:
PENDING, PROCESSING, SHIPPED, DELIVERED, CANCELLED

Enum restricts values to a predefined list.

9 UNION TYPE

RAML Syntax:

```
types:
  ResponseMessage:
    type: string | nil
```

Examples:

- "Operation Successful."
- null

Allows one or more multiple possible types.

BENEFITS OF ADVANCED DATA TYPES

- Supports flexible and complex data modeling.
- Improves schema validation.
- Ensures data consistency.
- Makes APIs scalable and maintainable.
- Enhances API documentation.

KEY TAKEAWAYS

- Use Object to define structured data with properties.
- Use Array to hold multiple values of the same type.
- Use Enum to limit values to a predefined list.
- Use Union type for fields that can hold multiple possible types.
- Constraints ensure correct data and improve API reliability.

3.7.3

DATA TYPES

MuleSoft from salesforce

Special data types, reusability and best practices in RAML.

10 FILE TYPE

RAML Syntax:

```
types:
  FileUpload:
    type: object
    properties:
      file:
        type: file
```

Used For:

- File upload APIs
- Image upload APIs
- Document upload APIs

11 NIL TYPE

RAML Syntax:

```
OptionalNote:
  type: string | nil
```

Examples:

- "Delivery between 9 AM - 5 PM"

OR

- null

12 CUSTOM DATA TYPES (REUSABILITY)

RAML Syntax:

```
types:
  Money:
    type: object
    properties:
      currency: string
      amount: number
```

Benefits:

- Encourages reuse
- Ensures consistency
- Easy to maintain

13 REUSE IN RAML

RAML Syntax:

```
/payments:
  post:
    body:
      application/json:
        type: Money
        example:
          currency: "USD"
          amount: 99.99
```

This reuses the "Money" data type defined above.

14 COMPLETE DATA TYPES EXAMPLE

User

```
types:
  User:
    type: object
    properties:
      id: integer
      name: string
      email: string
      address: Address
      status: OrderStatus
      createdAt: datetime
```

Address

```
types:
  Address:
    type: object
    properties:
      street: string
      city: string
      state: string
      zipCode: string
      country: string
```

OrderStatus

```
types:
  OrderStatus:
    type: string
    enum:
      - PENDING
      - PROCESSING
      - SHIPPED
      - DELIVERED
      - CANCELLED
```

These types can be reused across all APIs for consistency.

15 EXAMPLE RESPONSE

Example JSON Response:

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@example.com",
  "address": {
    "street": "221B Baker Street",
    "city": "London",
    "state": "London",
    "zipCode": "NW1 6XE",
    "country": "UK"
  },
  "status": "ACTIVE",
  "createdAt": "2024-05-20T14:30:00Z"
}
```

16 BEST PRACTICES

- Define data types for all request bodies.
- Define data types for all response bodies.
- Use meaningful and consistent names.
- Apply constraints to validate data.
- Prefer reusable data types and modules.
- Use enums wherever possible.
- Always document with examples.

KEY TAKEAWAYS

- Data types are the foundation of RAML APIs.
- They ensure validation, consistency and reliability.
- Reuse data types to build scalable and maintainable APIs.
- Good data modeling leads to strong API contracts.

REMEMBER!

- Use the right data type for the right purpose.
- Validate incoming and outgoing data.
- Keep your API models clean, reusable and well-documented.

3.8.1 EXAMPLES (EXAMPLES IN RAML)

MuleSoft from salesforce

Examples in RAML are used to provide sample request and response payloads. They improve API documentation, help developers understand the expected data format and make APIs easier to test and consume.

1 WHAT ARE EXAMPLES?

- Help API consumers understand the expected data.
- Provide sample request and response payloads.
- Improve API documentation and readability.
- Demonstrate how APIs should be used.
- Used only for documentation (not at runtime).

2 RAML SYNTAX OVERVIEW

```
/users:
  get:
    responses:
      200:
        body:
          application/json:
            example:
              {
                "id": 1,
                "name": "John"
              }
```

3 WHERE CAN EXAMPLES BE USED?

REQ — Request Body
- POST, PUT, PATCH

RES — Response Body
- Success Responses
- Error Responses

HEAD — Headers
- Request Headers
- Response Headers

4 SINGLE EXAMPLE (INLINE)

Request Example

```
/users:
  post:
    body:
      application/json:
        example:
          {
            "name": "John",
            "email": "john@example.com"
          }
```

Response Example

```
/users:
  post:
    responses:
      200:
        body:
          application/json:
            example:
              {
                "id": 101,
                "name": "John",
                "email": "john@example.com"
              }
```

Use when only one sample is needed.

5 MULTIPLE EXAMPLES (INLINE)

```
/users:
  get:
    responses:
      200:
        body:
          application/json:
            examples:
              success:
                value:
                  {
                    "id": 101,
                    "name": "John"
                  }
              notFound:
                value:
                  {
                    "id": 102,
                    "name": "Jane"
                  }
```

Supports multiple valid responses for the same API.

6 NAMED EXAMPLES (INLINE)

```
/users:
  get:
    responses:
      200:
        body:
          application/json:
            examples:
              userFound:
                description: User found by ID"
                value:
                  {
                    "id": 101,
                    "name": "John"
                  }
              userNotFound:
                description: User not found
                value:
                  {
                    "message": "User not found"
                  }
```

Makes documentation meaningful and improves readability.

BEST PRACTICES

- Use meaningful example data.
- Keep examples realistic.
- Document both requests and responses.
- Use named examples wherever possible.
- Keep examples updated.
- Avoid sensitive information.

KEY TAKEAWAYS

- Examples improve API documentation.
- Help consumers understand payload structure.
- Multiple examples improve usability.
- Named examples make APIs easier to consume.
- Good examples enhance developer experience.

FINAL SUMMARY

RAML EXAMPLES
↓
Request Examples
↓
Response Examples
↓
Header Examples
↓
Better API Documentation

Well-written examples make your APIs self-explanatory, easier to test and easier for developers to integrate and consume.

3.8.2 EXAMPLES

MuleSoft from salesforce

1 EXTERNAL EXAMPLE FILES

```
/users:
  post:
    body:
      application/json:
        example: !include examples/createUser.json
```

Example File: createUser.json

```
{
  "name": "John",
  "email": "john@example.com",
  "status": "active"
}
```

Benefits:

- Keeps RAML files clean and organized.
- Promotes reuse of example payloads.
- Easier to maintain and update examples.

2 EXAMPLE WITH HEADERS

```
/users:
  get:
    responses:
      200:
        headers:
          Content-Type:
            example: application/json
          X-RateLimit-Limit:
            example: 100
        body:
          application/json:
            example:
              {
                "id": 101,
                "name": "John"
              }
```

Use Case: Document response headers clearly with examples.

3 EXAMPLE WITH QUERY PARAMETERS

```
/users:
  get:
    queryParameters:
      name:
        type: string
        example: John
      limit:
        type: integer
        example: 10
    responses:
      200:
        body:
          application/json:
            example:
              [
                { "id": 101,
                  "name": "John"
                }
              ]
```

Use Case: Show typical values for query parameters.

4 ADVANCED EXAMPLE (TYPES + EXAMPLES)

```
/users:
  post:
    body:
      application/json:
        type: User
        example:
          {
            "name": "John",
          } "email": "john@example.com"
```

Use Case: Reusable types with external examples for clean models.

5 EXAMPLE BEST PRACTICES

- Always use realistic data.
- Include both success and error examples.
- Use named examples wherever possible.
- Use external files for reusable examples.
- Keep examples updated.
- Avoid sensitive information.
- Document edge cases.

6 COMMON SCENARIOS

Scenario	What to Example
Create Resource	Request Body
Get Resource	Response Body
Update Resource	Request + Response
Delete Resource	Response
Error Handling	Error Response
Pagination	Response with Data

7 ERROR RESPONSE EXAMPLE

```
/users/999:
  get:
    responses:
      404:
        body:
          application/json:
            example:
              {
                "error": "User not found",
                "message": "No user exists with ID 999",
                "status": 404
              }
```

Benefits:

- Helps consumers handle API errors properly.
- Improves API usability and developer experience.

8 FINAL SUMMARY

RAML EXAMPLES
↓
Request Examples
↓
Response Examples
↓
Header Examples
↓
Query Parameter Examples
↓
Error Examples
↓
Better API Documentation

Good examples make your APIs self-explanatory, easy to test, and easier for consumers to understand and integrate.

TIP

Well-written examples support quick testing and improve developer experience!

REMEMBER

Examples don't run your APIs, they simply help in understanding and testing!

QUICK CHECKLIST

- Are examples realistic?
- Do examples cover all key scenarios?
- Are error examples included?
- Are examples easy to understand?

3.9.1 TRAITS

MuleSoft from salesforce

Traits in RAML are reusable pieces of metadata that can be applied to API resources, methods, or responses. They help enforce consistency, reduce duplication, and improve API maintainability.

1 WHAT ARE TRAITS?

Traits are reusable specifications that can contain:

- Headers
- Query Parameters
- Responses
- Security definitions
- Request validations
- Common API behaviors

KEY BENEFITS

- Promotes consistency across APIs.
- Reduces code duplication.
- Improves API readability.
- Speeds up API development.
- Encourages standardization.

2 HOW TRAITS WORK

DEFINE TRAIT </>

↓

APPLY TRAIT

↓

RESOURCE / METHOD {/}

↓

CONSISTENT BEHAVIOR

KEY POINTS

- Traits are defined inside the 'traits' section.
- Each trait has a name and body.
- Applied using the 'is' keyword.
- Multiple traits can be applied together.

3 RAML SYNTAX OVERVIEW

DEFINE A TRAIT

```
traits:
  <traitName>:
    <trait body>
```

APPLY A TRAIT

```
/users:
  get:
    is: [ client-id-required ]
```

You can apply one or more traits using an array.

4 WHERE CAN TRAITS BE APPLIED?

Element	Example
Resource	/users
Method	GET /users
Response	200 Response
API Root (Optional)	Security Trait

5 EXAMPLE: DEFINE TRAITS

```
traits:
  client-id-required:
    headers:
      client-id:
        type: string
        required: true
  rate-limit:
    headers:
      X-Rate-Limit:
        type: integer
        example: 100
```

6 APPLY TRAITS TO RESOURCES & METHODS

```
/users:
  get:
    is:
      [
        client-id-required,
        rate-limit
      ]
```

Traits are applied using the 'is' keyword inside methods or resources.

7 BENEFITS OF TRAITS

- Enforces API standards.
- Improves reusability.
- Makes RAML cleaner.
- Easier maintenance.
- Faster API development.

KEY TAKEAWAYS

- Traits are reusable pieces of metadata in RAML.
- They reduce duplication and enforce consistency.
- Traits are applied using the 'is' keyword.
- Multiple traits can be applied together.
- Traits improve maintainability and developer experience.

3.9.2 TRAITS

MuleSoft from salesforce

1 TRAITS WITH QUERY PARAMETERS

```
traits:
  paginated:
    queryParameters:
      page:
        type: integer
        example: 1
      limit:
        type: integer
        example: 20
```

APPLYING PAGINATION TRAIT

```
/users:
  get:
    is: [
        client-id-required,
        paginated,
        rate-limit
      ]
```

Use Case: Reusable query parameters for pagination.

2 MULTIPLE TRAITS & OVERRIDING

A. MULTIPLE TRAITS

```
/orders:
  get:
    is: [
        client-id-required,
        rate-limit,
        paginated
      ]
```

B. OVERRIDING TRAIT PROPERTIES

```
/orders:
  get:
    is: [ paginated ]

    queryParameters:
      limit:
        type: integer
        example: 50
```

Element-level definitions override trait properties when the same property exists.

3 TRAIT WITH SECURITY SCHEME

```
traits:
  oauth2-secured:
    securedBy:
      [ oauth_2_0 ]
```

SECURITY SCHEME EXAMPLE

```
securitySchemes:
  oauth_2_0:
    type: OAuth 2.0
```

APPLYING SECURITY TRAIT

```
/secure-data:
  get:
    is: [
        oauth2-secured,
        standard-error-response
      ]
```

Use Case: Secure APIs using reusable security traits.

4 COMMON TRAITS USED IN APIS

Trait Name	Purpose
client-id-required	Validate client-id
rate-limit	API rate limiting
standard-error-response	Common error responses
oauth2-secured	Apply OAuth security
cors-headers	CORS support
request-id	Request tracing
audit-info	Audit headers

5 TRAITS BEST PRACTICES

- Create small and reusable traits.
- Use meaningful and consistent names.
- Avoid very large traits.
- Keep traits generic (avoid business logic).
- Document all traits clearly.
- Version traits if reused across multiple APIs.

6 TRAITS VS DATA TYPES

TRAITS	DATA TYPES
Reusable metadata	Data structures
Applied using is keyword	Used in request/response bodies
Used for API behavior	Used for payload validation
Reduces duplication and enforces consistency	Defines the shape and structure of data

7 ERROR RESPONSE EXAMPLE

```
/users/999:
  get:
    responses:
      404:
        body:
          application/json:
            example:
              {
                "error": "User not found",
                "message": "No user exists with ID 999",
                "statusCode": 404
              }
```

Use Case: Standard error responses can be reused using traits.

Benefits
- Improves API consistency.
- Helps consumers handle errors predictably.

8 FIINAL SUMMARY

RAML TRAITS
↓
Define Traits
↓
Reusable Logic
↓
Apply Traits
↓
Resources & Methods
↓
Consistent APIs
↓
Secure & Maintainable APIs

TIP

Great examples and reusable traits support quick implementation and improve developer experience!

REMEMBER

- Traits reduce repetition and improve API consistency.
- Use traits to enforce standards and security.
- Keep traits reusable, small and well-documented.
- Good traits make your APIs scalable and maintainable!

3.9.3

TRAITS

(ADVANCED USAGE & PRACTICAL EXAMPLES)

MuleSoft from salesforce

Traits make RAML reusable, consistent and easy to maintain. This page covers composite traits, parameterized traits, conditional usage, practical examples, and tips for effective trait design.

1 COMPOSITE TRAITS

A trait can use other traits inside its definition to build a composite trait.

```
traits:
  secured-trait:
    is: [ client-id-required, oauth2-secured ]

  client-id-required:
    headers:
      client-id:
        type: string
        required: true

  oauth2-secured:
    securedBy: [ oauth_2_0 ]
```

Use Case:
Combine multiple traits into one for easier reuse.

2 PARAMETERIZED TRAITS

Traits can accept parameters to make them flexible and reusable.

```
traits:
  header-required:
    (headerName):
      headers:
        <<headerName>>:
          type: string
          required: true

/orders:
  get:
    is: [ header-required: "X-Client-Id" ]
```

Use Case:
Use the same trait for different headers, values, or inputs.

3 CONDITIONAL TRAITS (WHEN/IF)

Traits can be applied conditionally using when with expressions.

```
traits:
  internal-access-only:
    headers:
      internal:
        type: string
        required: true

/admin:
  get:
    is: [ internal-access-only ]
    when: "%environment == 'production'"
```

Use Case:
Apply traits only in specific environments or scenarios.

4 TRAITS WITH RESPONSES

Define standard responses in a trait and reuse them.

```
traits:
  standard-error-response:
    responses:
      400:
        body:
          application/json:
            example:
              { "error": "Bad Request" }
      500:
        body:
          application/json:
            example:
              { "error": "Internal Server Error" }

/users:
  post:
    is: [ standard-error-response ]
```

Use standardized error responses across all APIs.

5 PRACTICAL EXAMPLE: COMPLETE API USING MULTIPLE TRAITS

```
traits:
  client-id-required:
    headers:
      client-id:
        type: string
        required: true
  rate-limit:
    headers:
      X-Rate-Limit:
        type: integer
        example: 100
  standard-error-response:
    responses:
      400:
        body:
          application/json:
            example: { "error": "Bad Request" }
      500:
        body:
          application/json:
            example: { "error": "Internal Server Error" }

/products:
  get:
    is: [ client-id-required, rate-limit, standard-error-response ]
    responses:
      200:
        body:
          application/json:
            example: { "id": 1, "name": "Laptop" }
```

6 OVERRIDE TRAIT AT RESOURCE / METHOD LEVEL

You can override or extend trait properties when needed.

```
traits:
  paginated:
    queryParameters:
      page:
        type: integer
        default: 1
      limit:
        type: integer
        default: 20

/users:
  get:
    is: [ paginated ]
    queryParameters:
      limit:
        default: 50   # Override default
```

Override values to suit specific resource requirements.

7 TIPS FOR EFFECTIVE TRAIT DESIGN

- Keep traits small, focused and reusable.
- Use meaningful and descriptive names.
- Avoid too many parameters.
- Make traits generic and independent of specific business logic.
- Document traits clearly with examples.
- Test traits independently before use.
- Version traits when used across multiple APIs.

8 WHEN TO USE TRAITS?

- When the same headers or query parameters are used across multiple APIs.
- When standard responses are common.
- When security requirements are repeated.
- When you want to keep RAML clean and maintainable.
- When you want to enforce consistency across teams.

9 TRAITS DESIGN CHECKLIST

- Is the trait reusable?
- Is the trait name clear?
- Is the trait generic and not tightly coupled?
- Are parameters used only when necessary?
- Is the trait well-documented?
- Have you tested the trait?

KEY TAKEAWAY

Traits are powerful tools in RAML to build reusable, consistent and maintainable APIs. Use them smartly to improve productivity, enforce standards and reduce duplication.

3.10.1 RESOURCE TYPES

MuleSoft from salesforce

Resource Types in RAML let you define **reusable** resource templates with common configurations. They help you avoid **repetition** and ensure **consistency** across your API.

1. WHAT ARE RESOURCE TYPES?

- A template for a resource.
- Can include URI parameters, methods, query parameters, headers, traits, etc.
- Can be applied to one or more resources.
- Promote reuse and standardization.
- Support consistent documentation and implementation.

Think of resource types as blueprints for similar API resources.

2. HOW RESOURCE TYPES WORK?

1. **Define** — Create a resource type
2. **Apply** — Use the type on one or more resources
3. **Customize** — Override or extend if needed
4. **Reuse** — Apply again wherever needed

KEY POINTS

- Defined under the resourceTypes section.
- Applied with type: resourceTypeName.
- Element-level properties can be overridden.
- Encourages DRY (Don't Repeat Yourself) principles.

3. SYNTAX OVERVIEW

Define a Resource Type

```
resourceTypes:
  typeName:
    <trait body or declaration>
```

Use a Resource Type

```
/resourcePath:
  type: typeName
    <additional overrides>
```

Important

You can apply resource types to one or more resources.

4. EXAMPLE: DEFINE A RESOURCE TYPE

```
#%RAML 1.0
title: Sample API
version: v1
resourceTypes:
  collection:
    description: Collection of <resourcePathName>
    get:
      description: Get all <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: []
  item:
    description: Resource in <resourcePathName>
    get:
      description: Get a <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: {}
    post:
      description: Create a <resourcePathName>
      responses:
        201:
          body:
            application/json:
              example: {}
```

Explanation

- resourceTypes: Defines reusable resource templates.
- collection: Template for a collection of resources.
- item: Template for a single resource.
- Includes methods (GET, POST), responses, and example payloads.
- Helps maintain consistency in structure across similar resources.

Use Case

Use resource types when multiple resources share the same structure and behavior. It reduces duplication and improves maintainability.

3.10.2 RESOURCE TYPES

MuleSoft from salesforce

1. EXAMPLE: APPLY RESOURCE TYPE RESOURCES

```
/users:
  type: collection
  resourcePathName: Users

/products:
  type: collection
  resourcePathName: Products

/orders:
  type: collection
  resourcePathName: Orders
```

Use a resource type on one or more resources for consistent structure.

2. OVERRIDING AND EXTENDING

Override a specific property

```
/users:
  type: collection
  resourcePathName: Users
  get:
    description: Get all users (overridden)
```

Add or change query parameter

```
/products:
  type: collection
  resourcePathName: Products
  get:
    queryParameters:
      page:
        type: string
        required: false
        example: 1
```

Disable a method (remove POST)

```
/orders:
  type: collection
  resourcePathName: Orders
  post: !include disabled.raml
```

Overrides apply at the element level.

3. RESOLVED (EFFECTIVE) VIEW

```
/users:
  get: { ... }
  post: { ... }

/products:
  get: { ... }
  post: { ... }

/orders:
  get: { ... }
  post: { ... }
```

Explanation

The resolved view shows the final, expanded version after applying the resource type to the resources.

4. RESOURCE TYPE WITH TRAITS

```
resourceTypes:
  securedCollection:
    get:
      is: [ paginated, secured ]
    post:
      description: Get all <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: {}
```

Traits can be included in resource types for reusability.

5. NESTED RESOURCE TYPES

```
/accounts:
  type: securedCollection
/{accounts/(id)}:
  type: item
  get:
    description: Get a <resourcePathName>
/users:
  type: collection
  resourcePathName: User
  get:
    queryParameters:
      limit:
        type: integer
```

Resource types can be used at multiple levels (parent and child resources).

6. COMMON USE CASES

CRUD Operations
Define standard CRUD behavior once and reuse across multiple resources.

Security
Apply common security schemes or traits to multiple resources.

Pagination & Filtering
Standardize query parameters like limit, offset, sort, etc.

Consistent Responses
Ensure consistent response structures and status codes.

7. RESOURCE TYPE BEST PRACTICES

- Create resource types for commonly used patterns.
- Keep resource types focused and cohesive.
- Use meaningful and consistent names.
- Override only what is necessary.
- Document resource types clearly.
- Version your resource types if used across APIs.

8. RESOURCE TYPES VS TRAITS

Aspect	Resource Types	Traits
Purpose	Template for resources	Reusable metadata/behavior
Applies to	Resources (endpoints)	Methods, resources, components, etc.
Structure	Contain resources, declarations (methods, responses, etc.)	Contain properties, headers, queries, responses, etc.
Example	Users, Orders, Products, Items, SecureItem	Paginated, Cacheable, Secured, RateLimited
Use when	You want to standardize resource structures	You want to add reusable behavior or rules

9. QUICK REFERENCE

Define:
```
resourceTypes: <name>: <declaration>
```

Apply:
```
/path:
  type: <name>
```

Override:
```
/path:
  type: <name>
    <override properties>
```

Add Traits:
```
is: [ <trait1>, <trait2> ]
```

Best For:
Reusability, consistency and maintainability across APIs.

Resource Types help you build consistent, reusable and maintainable APIs by reusing standards across your resources, methods and responses.

3.10.3 RESOURCE TYPES

MuleSoft from salesforce

10. COMMON USE CASES

CRUD Operations
Define standard CRUD behavior once and reuse across multiple resources.

Security
Apply common escurity schemes or traits to multiple resources.

Pagination & Filtering
Standardize query parameters like limit, offset, sort, etc.

Consistent Responses
Ensure consistent response structures and status codes.

11. RESOURCE TYPE BEST PRACTICES

- Create resource types for commonly used patterns.
- Keep resource types focused and cohesive.
- Use meaningful and consistent names.
- Override only what is necessary.
- Document resource types clearly.
- Version your resource types if used across APIs.

Following best practices ensures reusability, consistency and easier maintenance.

12. RESOURCE TYPES VS TRAITS

Aspect	Resource Types	Traits
Purpose	Template for resources	Reusable metadata/behavior
Applies to	Resources (endpoints)	Methods, resources, components, etc.
Structure	Contain resources, declarations (methods, responses, etc.)	Contain properties, headers, queries, responses, etc.
Example	Users, Orders, Products, Items, SecureItem	Paginated, Cacheable, Secured, RateLimited
Use when	You want to standardize resource structures	You want to add reusable behavior or rules

Key Difference
Resource Types define the structure of resources, while Traits define reusable behavior or metadata.

Remember
Use both together to build consistent, reusable and well-structured RAML APIs.

13. QUICK REFERENCE

Define	`resourceTypes: <name>: <declaration>`
Apply	`/path: type: <name>`
Override	`/path: type: <name> <override properties>`
Add Traits	`is: [ <trait1>, <trait2> ]`
Best For	Reusability, consistency and maintainability across APIs.

Example

```
resourceTypes:
  collection:
    description: Collection of <resourcePathName>
    get:
      description: Get all <resourcePathName>
      responses:
        200:
          body:
            application/json:
              example: []
/users:
  type: collection
  resourcePathName: Users
```

Resource Types help you build consistent, reusable and maintainable APIs by reusing standards across your resources, methods and responses.

3.11.1 LIBRARIES

MuleSoft from salesforce

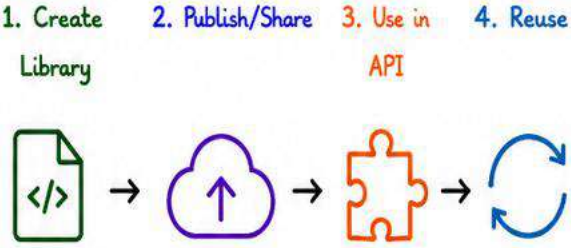
Libraries in RAML let you extract and reuse fragments of RAML across multiple APIs.
They promote reusability, consistency and modular design.

1. WHAT ARE LIBRARIES?

- A library is an external RAML file that contains reusable fragments like types, traits, resource types, security schemes, examples and more.
- Imported into an API and referenced using a namespace.
- Encourages reuse and prevents duplication.
- Centralizes common definitions and simplifies maintenance.

Think of libraries as building blocks that can be reused across many APIs.

2. HOW LIBRARIES WORK



2. Key Points

- Libraries follow DRY (Don't Repeat Yourself).
- Define once, use many times.
- Version your libraries for safe evolution.
- Namespaces prevent naming conflicts.

Libraries are the key to building consistent, scalable and maintainable APIs by reusing common resource patterns in a single place.

3.11.2 LIBRARIES

MuleSoft from salesforce

3. SYNTAX OVERVIEW

Define Library (library.raml)

```
#%RAML 1.0 Library
usage: Provides reusable types, traits,
resource types, security schemes, etc.
```

Use Library in an API (api.raml)

```
#%RAML 1.0
uses:
  lib: libraries/library.raml
# Use components with the "lib" prefix
/users:
  get:
    200: lib.SuccessResponse
```

Use a local or remote RAML file as a library.

4. WHAT CAN BE IN A LIBRARY?

Element	Description
Types	Data type definitions (objects, arrays, enums).
Traits	Reusable request/response fragments.
Resource Types	Templates for resources and methods.
Parameters	Standardized request parameters.
Examples	Reusable example payloads.
Security Schemes	Security definitions (OAuth, API Key, etc.).
Annotations	Custom annotations and metadata.

NAMESPACE AND REFERENCES

```
uses:
  common: libraries/common-types.raml
  paging: libraries/paging.raml

# Referencing
common.User                    # Type
common.paginated               # Traits
common.SuccessResponse         # Response
paging.PagingRequest           # Another library
paging.Pagination              # Resource type
```

Always use namespaces to reference items from a library.

3.11.3 LIBRARIES

MuleSoft from salesforce

1. EXAMPLE: DEFINE A LIBRARY

File: libraries/common-types.raml

```
#%RAMPL 1.0 Library
types:
  User:
    type: object
    properties:
      id: integer
      name: string
      email: string
      active: boolean
      age?: integer
    example:
      id: 101
      name: "John"
      email: "john@example.com"
      active: true
traits:
  paginated:
    queryParameters:
      page:
        type: integer
        required: false
      limit:
        type: integer
        required: false
responses:
  SuccessResponse:
    description: Request was successful
    body:
      application/json:
        example: { "message": "Success" }
  NotFoundResponse:
    description: Resource not found
    body:
      application/json:
        example: { "message": "Not Found" }
```

2. EXAMPLE: USE LIBRARY IN AN API

File: api.raml

```
#%RAML 1.0
uses:
  common: libraries/common-types.raml
/users:
  get:
    is: [ common.paginated ]
    description: Get all users
    responses:
      200:
        body:
          application/json:
            type: common.User
            example: { "message": "Success" }
      404:
        body:
          application/json:
            type: common.NotFoundResponse
/users/{id}:
  get:
    is: [ common.paginated ]
    description: Get user by ID
    responses:
      200:
        body:
          application/json:
            type: common.User
      404:
        body:
          application/json:
            type: common.NotFoundResponse
```

Use "is" to include resources like traits, responses, headers, resource types, etc.

3. EXAMPLE: LIBRARY WITH RESOURCE TYPE

File: libraries/resource-types.raml

```
#%RAML 1.0 Library
resourceTypes:
  collection:
    get:
      is: [ paginated ]
      responses:
        200:
          body:
            application/json:
              type: array
              items: <<item>>
              example: []
```

Use "<<item>>" to pass the resource type when using this resource type.

4. USING RESOURCE TYPE FROM LIBRARY

File: api.raml

```
#%RAML 1.0
uses:
  common: libraries/resource-types.raml
  products: libraries/common-types.raml
/products:
  type: collection
  get:
    description: Get all products
    responses:
      200:
        body:
          application/json:
            type: common.Product
```

Pass parameters to resource types using "type".

Libraries are the key to building consistent, scalable and maintainable APIs by reusing common resource patterns in a single place.

3.12.1 FRAGMENTS

MuleSoft from salesforce

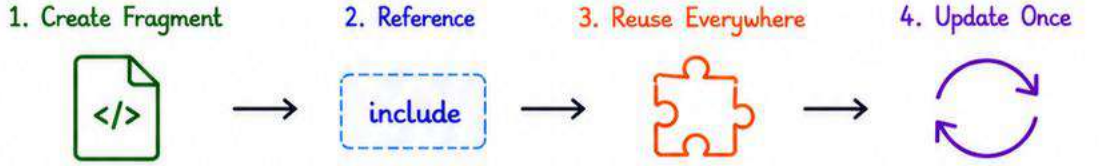
Fragments in RAML let you extract and reuse any YAML content across your API. They help you follow DRY (Don't Repeat Yourself), improve consistency, and simplify maintenance.

PART 1 OF 3 1. WHAT ARE FRAGMENTS?

- A fragment is a reusable piece of YAML content.
- Can be used for data types, traits, headers, query parameters, responses, examples, etc.
- Stored in separate YAML files.
- Reused in multiple APIs to reduce duplication.
- Promotes reusability, consistency and modularity.

Think of fragments as reusable puzzle pieces that can be used wherever needed.

PART 2 OF 3 2. HOW FRAGMENTS WORK



Create reusable content in a separate YAML file.

Reference the fragment using include.

Use the fragment in any RAML API multiple times.

Update the fragment once, changes reflect everywhere.

Key Benefits

- Eliminates duplication and reduces errors.
- Enforces consistency across your API.
- Simplifies updates and maintenance.
- Encourages modular and scalable API design.

PART 3 OF 3 3. SYNTAX OVERVIEW

Define a Fragment File

```yaml
# File: fragments/common-headers.raml
headers:
  X-Correlation-ID:
    type: string
    description: Correlation ID for tracing
  Accept:
    type: string
    description: Accepted token
```

Use a Fragment in RAML

```yaml
# RAML 1.0
uses:
  h: fragments/common-headers.raml

/users:
  get:
    headers: h
```

Fragment path is relative to the RAML file (including sub-folders).

Fragments help you build clean, consistent and maintainable RAML APIs. Define once, reuse everywhere!

3.12.2 FRAGMENTS

MuleSoft from salesforce

Fragments in RAML let you extract and reuse any YAML content across your API. They help you follow DRY (Don't Repeat Yourself), improve consistency, and simplify maintenance.

PART 1 OF 3 1. CREATE FRAGMENT

Create a reusable YAML fragment and store it in a separate file.

File: fragments/common-headers.raml

```
headers:
  X-Correlation-ID:
    type: string
    description: Correlation ID for tracing
  Accept:
    type: string
    description: Accepted token
```

Keep fragments focused, reusable and well-structured.

PART 2 OF 3 2. REFERENCE FRAGMENT

include

Reference the fragment inside your RAML file using the include keyword.

File: api.raml

```
#%RAML 1.0
uses:
  common-headers: fragments/common-headers.raml
/users:
  get:
    description: Get all users
    headers: !include common-headers.headers
    responses:
      200:
        body:
          application/json:
            type: array
```

Use !include to include a fragment (or part of it) at any place in your RAML.

PART 3 OF 3 3. BENEFITS

Eliminates Duplication
Avoid repeating the same content across multiple APIs or endpoints.

Enforces Consistency
Ensure consistent headers, parameters, responses and data types.

Simplifies Updates
Make changes in one place and it reflects everywhere.

Encourages Modular Design
Build small, reusable pieces and compose APIs easily.

Improves Maintainability
Easier to manage, debug and maintain large APIs.

Scalable and Reusable
Reuse fragments across teams, projects and APIs.

Fragments help you build clean, consistent and maintainable RAML APIs.
Define once, reuse everywhere!

3.12.3 FRAGMENTS

MuleSoft from salesforce

Fragments in RAML let you extract and reuse any YAML content across your API. They help you follow DRY (Don't Repeat Yourself), improve consistency, and simplify maintenance.

PART 1 OF 3 1. SYNTAX OVERVIEW

Define a Fragment File

```
# File: fragments/common-headers.raml
headers:
  X-Correlation-ID:
    type: string
    description: Correlation ID for tracing
  Accept:
    type: string
    description: Accepted token
```

Use a Fragment in RAML

```
# File: api.raml
#%RAML 1.0
uses:
  h: fragments/common-headers.raml

/users:
  get:
    headers: h
```

Fragment path is relative to the RAML file (including sub-folders).

PART 2 OF 3 2. EXAMPLE: USING FRAGMENTS IN RAML

Fragment File

```
# File: fragments/common-headers.raml
headers:
  X-Request-ID:
    type: string
    description: Unique request ID
  Authorization:
    type: string
    description: Bearer token
```

Another Fragment File

```
# File: fragments/pagination.raml
queryParameters:
  page:
    type: integer
    description: Page number
    default: 1
  limit:
    type: integer
    description: Items per page
    default: 10
```

Main RAML File Using Fragments

```
# File: api.raml
#%RAML 1.0
uses:
  hdr: fragments/common-headers.raml
  pg:  fragments/pagination.raml
/users:
  get:
    description: Get all users
    headers: !include hdr.headers
    queryParameters: !include pg.queryParameters
    responses:
      200:
        body:
          application/json:
            type: array
            items: !include fragments/user-response.raml
```

Use `!include` to include fragments (or parts of them) at any place in your RAML.

PART 3 OF 3 3. BENEFITS OF USING FRAGMENTS

Eliminates Duplication
Avoid repeating the same content across multiple APIs or endpoints.

Enforces Consistency
Keep headers, parameters, responses and data types consistent across your API.

Simplifies Updates
Change content in one place and it reflects everywhere it is used.

Encourages Modular Design
Break large APIs into small, reusable pieces that are easy to understand.

Scalable and Reusable
Reuse fragments across teams, projects and multiple APIs.

Improves Maintainability
Makes APIs easier to manage, debug and maintain over time.

Fragments help you build clean, consistent and maintainable RAML APIs.
Define once, reuse everywhere!

3.13.1 INCLUDES

MuleSoft from salesforce

Includes in RAML let you split your API definition into multiple files and bring them together. They promote modularity, reusability and easier maintenance.

PART 1 OF 3 1. WHAT ARE INCLUDES?

- Include brings content from another RAML file into the current file.
- Helps to split large RAML files into smaller, manageable parts.
- Improves readability and maintainability.
- Supports reuse of common definitions across multiple APIs.

Think of includes as building blocks that you assemble to build your API.

PART 2 OF 3 2. HOW INCLUDES WORK

Create reusable RAML fragments.

Use !include to bring them in.

RAML processor merges the content.

Final RAML is ready to be used.

Key Points

- Included content is merged at the location of the !include.
- Relative paths are resolved from the including file.
- Includes can be nested.
- Avoid circular includes.

PART 3 OF 3 3. SYNTAX OVERVIEW

```
!include <relative-path-to-file.raml>
```

Example

```
types:
  User: !include types/user.raml
traits:
  paginated: !include traits/paginated.raml
/users: !include resources/users.raml
```

The included file can contain any valid RAML fragment (types, traits, resources, etc.).

3.13.2 INCLUDES

MuleSoft from salesforce

Includes in RAML let you split your API definition into multiple files and bring them together. They promote modularity, reusability and easier maintenance.

PART 1 OF 3 — 1. EXAMPLE: PROJECT STRUCTURE

```
api.raml
├── types/
│   └── user.raml
├── common.raml
├── traits/
│   ├── paged.raml
│   └── secured.raml
├── resources/
│   ├── users.raml
│   └── orders.raml
└── responces/
    └── common-responses.raml
```

Key Points

- Organize your RAML using small, focused files.
- Group related fragments by domain or purpose.
- Keeps main RAML file clean and easy to read.
- Encourages reuse and consistency across APIs.

Think of includes as building blocks that you assemble to build your API.

PART 2 OF 3 — 2. EXAMPLE: MAIN FILE (api.raml)

```
#%RAML 1.0
title: Sample API
version: v1
baseUri: https://api.example.com/{version}
mediaType: application/json
uses:
  types: types/
  traits: traits/
  responses: responses/common-responses.raml

/users: !include resources/users.raml
/orders: !include resources/orders.raml
```

- The main file imports types, traits and common responses.
- Resource files are included where the resources are defined.
- Keeps the main file simple and focused.

PART 3 OF 3 — 3. EXAMPLE: INCLUDED FILES

types/user.raml

```
User:
  type: object
  properties:
    id: integer
    name: string
    email: string
    status: string
```

traits/paged.raml

```
paged:
  queryParameters:
    page:
      type: integer
      required: false
      example: 1
    limit:
      type: integer
      required: false
      example: 10
```

traits/secured.raml

```
secured:
  headers:
    Authorication:
      type: string
      description: Bearer token
      required: true
```

resources/users.raml

```
/users:
  is: [paged, secured]
  get:
    description: Get all users
    responses:
      200: !include ../responses/success.raml
  post:
    description: Create a new user
    responses:
      201: !include ../responses/created.raml
```

resources/orders.raml

```
/orders:
  is: [paged, secured]
  get:
    description: Get all orders
    responses:
      200: !include ../responses/success.raml
  post:
    description: Create a new order
    responses:
      201: !include ../responses/created.raml
```

responses/common-responses.raml

```
success:
  200:
    description: Successful response
    body:
      application/json:
        type: User
created:
  201:
    description: Resource created
    body:
      application/json:
        type: User
```

Structure your RAML using includes to build clean, modular and maintainable APIs.
Small files, big impact!

3.13.3 INCLUDES

MuleSoft from salesforce

Includes in RAML let you split your API definition into multiple files and bring them together.
They promote modularity, reusability and easier maintenance.

PART 1 OF 3 — 1. SYNTAX OVERVIEW

```
!include <relative-path-to-file.raml>
```

Syntax Rules

- Use !include to bring content from another RAML file.
- The path is relative to the file containing the include.
- The included content is merged at the location of the include.
- Includes can be nested (an included file can include others).
- Avoid circular includes.

Example

api.raml

```
#%RAML 1.0
title: Sample API
version: v1
baseUri: https://api.example.com/{version}
mediaType: application/json

uses:
  types: types/user.raml
  traits: traits/pagination.raml

/users:
  get:
    is: [ pagination ]
    responses:
      200:
        body:
          application/json:
            type: User
```

Tip
Think of includes as building blocks that you assemble to build your API.

Note
The included file can contain any valid RAML fragment (types, traits, resources, etc.).

PART 2 OF 3 — 2. HOW INCLUDES ARE RESOLVED

1. **Include Found** — RAML processor finds the !include statement.
2. **File Loaded** — The referenced file is loaded from the given relative path.
3. **Content Merged** — The content of the included file is inserted at the exact location of the !include.
4. **Final RAML Ready** — After resolving all includes, the complete RAML is ready to be used.

Example Resolution

Before (api.raml)

```
/users:
  get:
    is: [ pagination ]
    responses:
      200:
        body:
          application/json:
            type: User

/orders:
  !include resources/
           orders.raml
```

+ Included File (resources/orders.raml)

```
/orders:
  get:
    description: Get all
                 orders
    responses:
      200:
        body:
          application/json:
            type: Order
  post:
    description: Create
                 a new order
    responses:
      201:
        body:
          application/json:
            type: Order
```

= After Resolution (Final RAML)

```
/users:
  get:
    is: [ pagination ]
    responses:
      200:
        body:
          application/json:
            type: User
/orders:
  get:
    description: Get all
                 orders
    responses:
      200:
        body:
          application/json:
            type: Order
  post:
    description: Create
                 a new order
    responses:
      201:
        body:
          application/json:
            type: Order
```

All included content behaves as if it was written directly in the main file.

PART 3 OF 3 — 3. BEST PRACTICES

- **Keep It Modular** — Split RAML into small, focused files by domain or purpose.
- **Reuse Common Fragments** — Store shared types, traits and responses in common files and include them where needed.
- **Use Clear Structure** — Organize files in a logical folder structure for easy navigation and maintenance.
- **Avoid Deep Nesting** — Too many nested includes can make it hard to trace and maintain.
- **Avoid Circular Includes** — Circular references will cause errors. Plan your include dependencies carefully.
- **Test After Changes** — After modifying included files, test your API to ensure everything works as expected.

Includes help you build clean, modular and maintainable RAML APIs.
Small files. Big impact!

3.14.1 SECURITY SCHEMES

MuleSoft from salesforce

Security schemes in RAML let you define how your API is secured.
They promote consistency, reuse and clear documentation of authentication methods.

PART 1 OF 3 1. WHAT ARE SECURITY SCHEMES?

- Define the authentication and authorization methods used to access your API.
- Can be referenced globally or at resource, method or trait level.
- Encourage reuse and consistency.
- Improve API documentation and developer experience.
- Help enforce security best practices.

Define once, use everywhere.
Keep your API secure and consistent.

Key Points

- Define once and reuse in multiple places.
- Choose the right scheme for your API.
- Multiple schemes can be defined and used.
- Secure by default, but allow flexibility.
- Document clearly for better API consumer experience.

PART 2 OF 3 2. HOW SECURITY SCHEMES WORK

1. Define → Define a security scheme in the securitySchemes section.
2. Apply → Apply the scheme to resources, methods or traits as needed.
3. Authenticate → Clients authenticate using the defined security scheme.
4. Authorize → The API validates tokens/credentials and authorizes access.

Security schemes help enforce authentication and authorization consistently across your API.

PART 3 OF 3 3. SYNTAX OVERVIEW

Define a security scheme

```
securitySchemes:
  <schemeName>:
    description: <description>
    type: <schemeType>
    describedBy:
      headers:
        <headerName>:
          description: <header description>
          type: <string|integer|boolean>
          required: <true|false>
```

Example

```
securitySchemes:
  customScheme:
    description: Custom authentication using
        API key in header
    type: x-custom
    describedBy:
      headers:
        x-api-key:
          description: API Key for authentication
          type: string
          required: true
```

Common Properties

Property	Description
securitySchemes	Root node to define one or more security schemes.
<schemeName>	Name of the security scheme (e.g., customScheme, basicAuth, oauth_2_0).
type	Type of the scheme (e.g., basic, digest, oauth_2_0, x-custom).
describedBy	Provides additional details (e.g., headers, query parameters, responses).
headers	Used to define HTTP headers required for authentication/authorization.

Note

Different scheme types have different properties. Refer to RAML specification for details on each type.

Security schemes help you define authentication methods clearly, ensuring consistent, reusable and well-documented API security.

3.14.2 SECURITY SCHEMES

MuleSoft from salesforce

Security schemes in RAML let you define how your API is secured.
They promote consistency, reuse and clear documentation of authentication methods.

PART 1 OF 3 — 1. SECURITY SCHEME TYPES

Type	Description	Common Use	Example
basic	HTTP Basic Authentication	Simple user + password	Authorization: Basic <token>
digest	HTTP Digest Authentication	Enhanced password handling	Authorization: Digest <token>
oauth_1_0	OAuth 1.0	Legacy OAuth 1.0 flows	OAuth 1.0 signed requests
oauth_2_0	OAuth 2.0	Modern OAuth 2.0 flows	Bearer <access_token>
x509	X.509 (mutual TLS)	Secure API to API communication	Client certificate
x509-sni	X.509 Client Certificate (with SNI)	Mutual TLS in enterprise environments	Client certificate + SNI
custom	Custom authentication mechanism	Proprietary or custom auth	Custom headers / logic

Key Points

- Choose the scheme that best matches your security needs.
- Multiple schemes can be defined and used.
- Security schemes can be applied at API root, resource or method level.
- Secure by default, but allow flexibility.

Define once, use everywhere. Keep your API secure and consistent.

PART 2 OF 3 — 2. EXAMPLE: DEFINE SECURITY SCHEMES

File: security.raml

```
#%RAML 1.0
securitySchemes:
  basicAuth:
    type: Basic Authentication
    description: Basic auth using username and password

  oauth2_0:
    type: OAuth 2.0
    description: OAuth 2.0 using Authorization Code Grant
    describedBy:
      headers:
        Authorization:
          type: string
          description: "OAuth 2.0 access token"
  apiKeyAuth:
    type: x-api-key
    describedBy:
      headers:
        X-API-Key:
          type: string
          description: "API Key for authentication"
  customAuth:
    type: Custom Authentication
    describedBy: API Key passed in request header
```

Explanation

This file defines four security schemes:

- basicAuth -> HTTP Basic Authentication
- oauth2_0 -> OAuth 2.0 (Authorization Code Grant)
- apiKeyAuth -> API Key in request header
- customAuth -> Custom authentication mechanism

These schemes can now be applied to your API, resources or methods.

Best Practices

- Define schemes in a separate file (e.g., security.raml).
- Use clear and descriptive names.
- Document required headers or parameters.
- Reuse schemes across APIs and resources.
- Secure by default, override when necessary.

PART 3 OF 3 — 3. APPLY SECURITY SCHEMES

Example 1: Apply to Entire API

```
#%RAML 1.0
title: Sample API
version: v1
baseUri: https://api.example.com/{version}
securitySchemes: !include security.raml
securedBy: [ basicAuth ]   # Apply to entire API

/users:
  get:
    description: Get all users
```

Example 2: Apply to Resource

```
#%RAML 1.0
title: Sample API
version: v1
baseUri: https://api.example.com/{version}
securitySchemes: !include security.raml
/users:
  securedBy: [ oauth2_0 ]   # Apply to /users
  get:
    description: Get all users
  post:
    description: Create a new user
```

Example 3: Apply to Method

```
#%RAML 1.0
title: Sample API
version: v1
baseUri: https://api.example.com/{version}
securitySchemes: !include security.raml
/products:
  get:
    description: Get products (public)
  post:
    securedBy: [ apiKeyAuth ]   # Only this method
    description: Create a new product
```

Important Notes

- Multiple schemes can be applied using an array: securedBy: [basicAuth, apiKeyAuth]
- You can override a scheme at a lower level (resource or method).
- If no scheme is applied, the resource or method is considered public.

Tip

Use security schemes to enforce consistent authentication and authorization across all your APIs.

Use security schemes to protect your APIs with the right authentication methods.
Define once, reuse everywhere, and keep your APIs secure and consistent!

3.14.3 SECURITY SCHEMES

MuleSoft from salesforce

Security schemes in RAML let you define how your API is secured.
They promote consistency, reuse and clear documentation of authentication methods.

PART 1 OF 3 — 1. EXAMPLE: APPLY SECURITY SCHEMES

File: api.raml

```
#%RAML 1.0
title: User API
version: v1
baseUri: https://api.example.com/{version}
securitySchemes: !include security.raml

/users:
  get:
    description: Get all users
    securedBy: [ basicAuth ]    # Apply to entire API
    responses:
      200:
        description: Successful response
        body:
          application/json:
            type: array
            items: User
  post:
    description: Create a new user
    securedBy: [ oauth2_0 ]     # Override for this method
    body:
      application/json:
        type: User
/products:
  get:
    description: Get products (public)
    # No security applied -> public
  post:
    description: Create a new product
    securedBy: [ apiKeyAuth ]   # Only this method
```

How It Works

1. **Apply at API root**
   Use `securedBy` at the root level to apply a scheme to all resources/methods.
2. **Override when needed**
   Use `securedBy` at resource or method level to override the applied scheme.
3. **Method level security**
   Apply a scheme only to specific methods that require protection.
4. **Public access**
   If no scheme is applied, the resource or method is considered public.

Define once, use everywhere.
Keep your API secure and consistent.

PART 2 OF 3 — 2. INHERITANCE AND OVERRIDE

Inheritance Flow

API Root
`securedBy: [ basicAuth ]`
↓
/users (Resource)
`securedBy: [ oauth2_0 ]`
Overrides root for /users
↓
/users/post (Method)
`securedBy: [ apiKeyAuth ]`
Overrides resource for this method
↓
/products (Resource)
No securedBy
Resource is public
↓
/products/post (Method)
`securedBy: [ apiKeyAuth ]`
Secured only this method

Effective Security

Level	Example	Secured By	Result
API Root	All resources/methods	[basicAuth]	All inherit basicAuth
Resource Override	/users	[oauth2_0]	/users and its methods inherit oauth2_0
Method Override	/users/post	[apiKeyAuth]	Only this method uses apiKeyAuth
No Security	/products	(none)	Public access
Method Only	/products/post	[apiKeyAuth]	Only this method is secured

Important Notes

- Method level `securedBy` always overrides resource and API root level.
- Resource level `securedBy` overrides API root level.
- If no `securedBy` is defined at any level, the resource or method is public.
- Choose the most restrictive level that meets your security requirements.

PART 3 OF 3 — 3. BEST PRACTICES

Use the Right Scheme

Choose the scheme that fits your security needs:

- basic – Simple username/password
- digest – Enhanced password handling
- oauth2_0 OAuth 2.0 flows
- x509 – Mutual TLS / Certificates
- custom – Custom authentication mechanims

Reuse Schemes

- Define schemes once in a security file and reuse across APIs.
- Ensures consistency and reduces duplication.
- Easier to maintain and update.

Document Clearly

- Provide clear descriptions for each scheme and property.
- Document required headers or parameters.
- Helps developers understand how to authenticate.

Secure by Default

- Apply security at the highest practical level.
- Override or remove only when necessary.

Test Your Security

- Verify that schemes are applied as expected.
- Test both authorized and unauthorized access.

Stay Consistent

- Use consistent naming conventions.
- Keep scheme definitions versioned and reviweed.
- Follow your organization's security standards.

Security schemes help you clearly define and apply authentication methods across your API.
Define once, reuse everywhere, and protect your APIs with confidence.

3.15 RAML BEST PRACTICES

MuleSoft from salesforce

Following best practices helps you build RAML APIs that are consistent, reusable, secure and easy to maintainable.

WHY BEST PRACTICES MATTER?

- Improve consistency across APIs and teams.
- Enhance readability and maintainability.
- Promote reuse and reduce duplication.
- Make onboarding and collaboration easier.
- Lead to better API quality and consumer experience.

CORE PRINCIPLES

Consistency — Use consistent structure, naming and style.

Reuse — Reuse types, traits, responses and resources.

Security — Apply proper security and protect data.

Clarity — Be clear and concise in documentation.

Maintainability — Design for change and easy maintenance.

QUICK SYNTAX REMINDERS

```yaml
title: My API
version: v1
mediaType: application/json
types:
  User:
    type: object
    properties:
      id: integer
      name: string
traits:
  paginated:
    queryParameters:
      page: integer
      size: integer
/users:
  get:
    is: [ paginated ]
    responses:
      200:
        body:
          application/json:
            type: User[]
```

BEST PRACTICES CHECKLIST

1. **Organize with Consistent Structure**
   - Follow a clear folder and file structure.
   - Group related resources and types.
   - Keep RAML files focused and modular.
2. **Use Meaningful and Consistent Naming**
   - Use clear, descriptive names for resources, types, traits, and methods.
   - Follow naming conventions (e.g., kebab-case for resources).
   - Avoid abbreviations unless widely accepted.
3. **Reuse with Libraries, Includes, Types and Traits**
   - Extract reusable components to libraries.
   - Use !include and !include fragments to avoid duplication.
   - Define common patterns as traits and use them.
4. **Keep It DRY (Don't Repeat Yourself)**
   - Avoid repeating examples, responses, or headers.
   - Use types and traits to centralize common definitions.
5. **Write Clear and Helpful Documentation**
   - Provide a good description at API, resource and method level.
   - Use annotations and Markdown for detailed docs.
   - Include request/response examples.
6. **Design Consistent Responses**
   - Use consistent response codes and structures.
   - Define standard error responses and reuse them.
   - Document all possible responses.
7. **Apply Security Best Practices**
   - Define security schemes at the top level.
   - Use least-privilege access.
   - Do not expose sensitive data.
8. **Version Your API**
   - Include version in the title or baseUri.
   - Follow semantic versioning (e.g., v1, v1.1, v2).
   - Maintain backward compatibility.
9. **Use Appropriate Resource Modeling**
   - Model resources as nouns, methods as actions.
   - Use proper HTTP methods (GET, POST, PUT, DELETE, etc.).
   - Use sub-resources and collections correctly.
10. **Validate and Test Your RAML**
    - Validate RAML using a linter (e.g., RAML CLI, API Designer).
    - Test examples and schemas.
    - Catch issues early.
11. **Collaborate and Review**
    - Use version control (Git) for RAML files.
    - Review changes and follow code review practices.
    - Maintain a changelog.
12. **Plan for Maintainability**
    - Keep files small and focused.
    - Document decisions and assumptions.
    - Refactor and clean up regularly.

GOOD VS AVOID

Good Practice	Avoid
Use meaningful names	Using unclear or abbreviated names
Reuse types and traits	Duplicating schemas and definitions
Consistent response structures	Different structures for same scenarios
Secure by design	Exposing sensitive data or endpoints
Clear documentation and examples	Missing or unclear documentation

EXAMPLE: GOOD PRACTICE

```yaml
#%RAML 1.0
title: User Management API
version: v1
baseUri: https://api.example.com/{version}
mediaType: application/json

uses:
  common: libraries/common.raml

/users:
  description: Manage users
  get:
    is: [ common.paginated ]
    responses:
      200:
        body:
          application/json:
            type: common.User[]

  post:
    body:
      application/json:
        type: common.User
    responses:
      201:
        body:
          application/json:
            type: common.User
```

TOOLS THAT HELP

- **API Designer** — Design and validate RAML visually.
- **RAML CLI** — Validate, lint and bundle RAML files.
- **VS Code Extensions** — RAML language support and snippets.
- **CI/CD Pipelines** — Automate validation and quality checks.

COMMON PITFALLS

- Overusing includes making it hard to navigate.
- Creating overly large RAML files.
- Ignoring validation warnings and errors.
- Inconsistent naming and formatting.
- Not documenting error responses.

FINAL TAKEAWAY

Follow best practices to build RAML APIs that are consistent, secure, reusable, and a joy to maintain.

Good RAML today,
Great APIs tomorrow!

Good RAML is not just about valid syntax – it's about clarity, reuse and value for your consumers.
Follow best practices and build APIs that stand the test of time!

4.1 WHAT IS DATAWEAVE?

MuleSoft from salesforce

DataWeave is MuleSoft's powerful data transformation language. It enables you to transform, map and format data between different formats and structures with ease.

INTRODUCTION

- DataWeave is the data transformation language of MuleSoft.
- It is used to manipulate and transform data in a readable, concise and declarative way.
- Introduced in Mule 4 and used across the Mule runtime.
- Replaces the legacy MEL (Mule Expression Language) for transformations.

WHY DATAWEAVE?

- Easy to learn and use
- Less code, more output
- Readability & maintainability
- Powerful built-in functions
- Supports multiple data formats
- High performance & streaming
- Strong typing & error handling

SIMPLE EXAMPLE

Input (JSON)

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@demo.com",
  "skills": ["MuleSoft", "Java"]
}
```

Output (JSON)

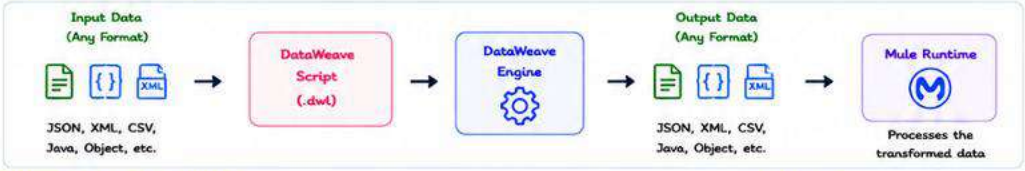
```
{
  "id": 101,
  "fullName": "John Doe",
  "contact": "john@demo.com",
  "skillCount": 2
}
```

DataWeave Script

```
%dw 2.0
output json
---
{
  id: payload.id,
  fullName: payload.name,
  contact: payload.email,
  skillCount: sizeOf(payload.skills)
}
```

DataWeave turns complex transformations into simple and expressive scripts.

DATAWEAVE ARCHITECTURE



DATAWEAVE OF DATAWEAVE

- **Declarative** — Focus on what to transform, not how to do it.
- **Powerful Functions** — Rich set of built-in functions for strings, arrays, objects, dates, and more.
- **Multiple Data Formats** — Supports JSON, XML, CSV, Java, and more. Easily convert between formats.
- **High Performance** — Optimized execution engine with streaming support for large payloads.
- **Easy to Read** — Human-friendly syntax. Less code, more clarity.
- **Strong Typing** — Type-safe scripts with early error detection.
- **Extensible** — Create custom functions and reuse across projects.
- **Maintainable** — Clean, modular and easy to maintain transformations.

USE CASES

- Transform API requests and responses
- Map between different data models
- Data enrichment and aggregation
- Format data for reports and documents
- Integrate with multiple systems
- Convert between different data formats
- Handle complex business transformations

BEFORE & AFTER COMPARISON

Before (MEL / Java)

```
#[
  {
    "id": payload.id,
    "fullName": payload.name,
    "contact": payload.email,
    "skillCount": payload.skills.size()
  }
]
```

With DataWeave

```
%dw 2.0
output json
---
{
  id: payload.id,
  fullName: payload.name,
  contact: payload.email,
  skillCount: sizeOf(payload.skills)
}
```

- ✗ Verbose and complex
- ✗ Harder to read and maintain
- ✗ More code for simple tasks
- ✗ Difficult for non-developers
- ✓ Concise and expressive
- ✓ Easy to read and maintain
- ✓ Less code, more output
- ✓ Accessible to all integration developers

DATAWEAVE IN MULE FLOW

DataWeave sits at the heart of data transformation in Mule flows.

QUICK SYNTAX OVERVIEW

Concept	Syntax Example
Script Header	%dw 2.0
Output Directive	output json | output xml | output application/java
Payload Access	payload
Variable Declaration	var name = "John"
Object	{ key: value, key2: value2 }
Array	[1, 2, 3]
Function Call	upper("mule")

KEY POINTS

- DataWeave is MuleSoft's transformation language.
- It helps you transform data between any formats.
- It is simple, powerful and highly performant.
- It improves productivity and reduces development time.
- It is the recommended way to do data transformations in Mule 4.

TOOLS THAT HELP

- **API Designer** — Design and validate DW scripts visually.
- **RAML CLI** — Validate, lint and bundle RAML files.
- **VS Code Extension** — RAML language support and snippets.
- **CI/CD Pipelines** — Automate validation and quality checks.

COMMON PITFALLS

- Overusing includes making it hard to navigate.
- Creating overly large RAML files.
- Ignoring validation warnings and errors.
- Inconsistent naming and formatting.
- Not documenting error responses.

FINAL TAKEAWAY

Follow best practices to build RAML APIs that are consistent, secure, reusable, and easy to maintain.

Good RAML today, Great APIs tomorrow!

Good RAML is not just about valid syntax – it's about clarity, reuse and value for your consumers. Follow best practices and build APIs that stand the test of time!

4.2 DATAWEAVE SYNTAX

MuleSoft from salesforce

DataWeave uses a simple and expressive syntax to transform data.
A DataWeave script is made of a header and a body.

SCRIPT STRUCTURE

```
%dw 2.0
output application/json
var x = 10
---
{
  message: "Hello",
  value: x
}
```

Header (Configuration)

Body (Transformation)

Everything before --- is the header.
Everything after --- is the body.

SYNTAX OVERVIEW

Element	Description
%dw	Declares DataWeave version
output	Specifies the output data format
var	Declares a variable
---	Separates between header and body
{ }	Represents an object
[ ]	Represents an array
:	Key-value separator in objects
,	Separator between elements
( )	Used in functions
" " or ' '	Used for string literals
//	Single line comment
/* */	Multi line comment

BASIC SYNTAX EXAMPLE

```
%dw 2.0
output application/json
---
{
  name: "John",
  age: 30,
  city: "New York"
}
```

This script returns a JSON object with name, age and city.

HEADER DETAILS

- %dw 2.0 — Declares DataWeave version 2.0
- output <media-type> — Defines the output format (json, xml, application/json, etc.)
- var <name> = <value> — Declares a variable
- import * from dw::core::Strings — Imports modules (optional)

Example:

```
%dw 2.0
output application/json
import * from dw::core::Strings
var greeting = "HiSoft"
var name = "MuleSoft"
---
{ message: greeting ++ " " ++ name }
```

Header configures how the script should behave.

BODY BASICS

- Contains the transformation logic
- Produces the output structure
- Can be an object { }, array [ ], value, or any expression
- Can use variables, functions, operators and keywords

Example:

```
%dw 2.0
output application/json
var name = "MuleSoft"
---
{
  upperName: upper(name),
  length: sizeOf(name)
}
```

Body defines what the output should look like.

DATA LITERALS (VALUES)

Literal Type	Syntax	Example	Result
String	"text"	"Hello"	Hello
Number	123	45.67	45.67
Boolean	true / false	true	true
Null	null	null	null
Array	[a, b, c]	[1, 2, 3]	Array of 3 items
Object	{ key: value }	{ a: 1, b: 2 }	Object with 2 keys

COMMENTS

Single line comment

```
%dw 2.0
output application/json
// This is a single line comment
var x = 10
---
x
```

Multi line comment

```
%dw 2.0
output application/json
/*
  This is a
  multi line comment
*/
---
"Done"
```

Comments are ignored during execution.

COMMON OUTPUT DIRECTIVES

Directive	Description	Example
output application/json	Outputs JSON	output application/json
output application/xml	Outputs XML	output application/xml
output application/java	Outputs Java / POJO	output application/java
output text/csv	Outputs CSV	output text/csv
output application/yaml	Outputs YAML	output application/yaml

If no output directive is specified, application/java is used by default.

VARIABLE DECLARATION

- Declared in the header using var
- Can use any expression as value
- Immutable (cannot be reassigned)
- Supports strings, numbers, arrays, objects and expressions

Example:

```
%dw 2.0
output application/json
var a = 100
var name = "MuleSoft"
var list = [1, 2, 3]
var obj = { x: 10, y: 20 }
---
{a: a, name: name, list: list, obj: obj }
```

COMPLETE SYNTAX EXAMPLE

```
%dw 2.0
output application/json
import * from dw::core::Strings
// Variable declarations
var first = "Mule"
var last = "Soft"
var full = first ++ " " ++ last
var nums = [1, 2, 3, 4]
---
{
  fullName: full,
  upperName: upper(full),
  numbers: nums,
  count: sizeOf(nums)
}
```

Result (JSON)

```
{
  "fullName": "MuleSoft",
  "upperName": "MULESOFT",
  "numbers": [1, 2, 3, 4],
  "count": 4
}
```

Tip:
Use meaningful variable names and keep scripts simple and readable.

KEY TAKEAWAYS

- A DataWeave script has two parts: Header and Body.
- Header configures the script (version, output, variables, imports).
- Body contains the transformation logic and returns the output.
- DataWeave syntax is simple, expressive and easy to learn.
- Use variables, functions and literals to build powerful transformations.
- Comments and formatting improve readability.
- Always test and keep your scripts clean and maintainable.

A clear syntax leads to clear transformations.
Write simple, readable and reusable DataWeave scripts!

4.3 DATA TYPES

MuleSoft from salesforce

DataWeave supports a rich set of data types to represent and transform data.
Every value in DataWeave is an expression that has a specific data type.

OVERVIEW

- Data types define the kind of value a variable or expression can hold.
- DataWeave is strongly typed.
- Types help in validation, transformation and function usage.
- Data types can be primitive, complex or special.

CATEGORIES OF DATA TYPES

Primitive Types	Complex Types	Special Types
• String • Number • Boolean • Null • Date • DateTime	• Array • Object	• Binary • Period • Duration
Basic single values	Collections of values	Used for advanced scenarios

HOW TO CHECK TYPE

```
// Using typeOf() function
typeOf("Hello")          // String
typeOf(123)              // Number
typeOf(true)             // Boolean
typeOf(null)             // Null
typeOf([1, 2, 3])        // Array
typeOf({a: 1})           // Object
typeOf(2024-05-20)       // Date
typeOf(2024-05-20T10:30:00Z)  // DateTime
```

typeOf() returns the data type of any value or expression.

DATAWEAVE DATA TYPES IN DETAIL

STRING
Sequence of characters enclosed in double quotes.

Example
```
"MuleSoft"
"Hello World"
```
Result: String

NUMBER
Represents numeric values. Supports integers and decimals.

Example
```
10
-10
45.67
3.1415
```
Result: Number

BOOLEAN
Represents true or false value.

Example
```
true
false
```
Result: Boolean

NULL
Represents an empty or missing value.

Example
```
null
```
Result: Null

ARRAY
Ordered collection of values. Elements can be of different types.

Example
```
[1, 2, "Mule", true]
["a", "b", "c"]
```
Result: Array

OBJECT
Unordered collection of key-value pairs.

Example
```
{
  name: "John",
  age: 30,
  city: "New York"
}
```
Result: Object

DATE
Represents date without time component.
Format: [YYYY-MM-DD]

Example
```
|2024-05-20|
```
Result: Date

DATETIME
Represents date with time and timezone.
Format: [YYYY-MM-DDTHH:mm:ssZ]

Example
```
|2024-05-20T10:15:30Z|
```
Result: DateTime

BINARY
Represents binary data such as files, images and streams.

Example
```
write("Hello" as Binary)
```
Result: Binary

PERIOD
Represents a period of time (date range).

Example
```
|P1Y2M10D|
|P3M|
```
Result: Period

DURATION
Represents a length of time (time duration).

Example
```
|PT1H30M|
|PT45M|
```
Result: Duration

TYPE CONVERSION (CASTING)

From	To	Example	Result
String	Number	("123" as Number)	123
String	Boolean	("true" as Boolean)	true
String	String	("abc" as String)	"abc"
String	Date	("2024-05-20" as Date)	|2024-05-20|
String	DateTime	("2024-05-20T10:30:00Z" as DateTime)	|2024-05-20T10:30:00Z|
Any	Binary	write(any as Binary)	Binary representation

Use the as operator to convert values from one type to another.

TYPE COMPATIBILITY

Operation	Compatible Types
String Concatenation (+)	String + String
Arithmetic (+, -, *, /, %)	Number + Number
Comparison (==, !=, <, >, <=, >=)	Number, Date, DateTime, String
Logical (and, or, not)	Boolean
Array Functions	Array
Object Functions	Object

QUICK REFERENCE

String	"abc"
Number	10, -10, 45.67
Boolean	true, false
Null	null
Array	[1, 2, "a", true]
Object	{ name: "John" }
Date	|2024-05-20|
DateTime	|2024-05-20T10:15:30Z|
Binary	write("Hello" as Binary)
Period	|P1Y2M10D| (1y 2m 10d)
Duration	|PT1H30M| (1h 30m)

EXAMPLES IN DATAWEAVE SCRIPT

```
var id = 10
var name = "John"                          // String
var isActive = true                        // Boolean
var score = 98.5                           // Number
var tags = ["api", "integration", "cloud"] // Array
var user = {                               // Object
    id: id,
    name: name,
    created: |2024-05-20|,
    updated: |2024-05-20T10:15:30Z|
}
---
{ name, count: sizeOf(tags), active: isActive, user }
```

KEY POINTS

- DataWeave is strongly typed and every value has a specific data type.
- Use the right data type for your data to avoid errors.
- Types ensure validation and consistency.
- Objects store key-value pairs.
- Date, DateTime, Period and Duration handle time-related data.
- Use typeOf() to check the type of any value.
- Use as operator for explicit type conversion when required.

TIPS

- Combine multiple schemes if different clients need different methods.
- Always document how to obtain tokens or keys.
- Test security setups in API gateways and clients.
- Keep security schemes updated and reviewed.

Understanding data types is the foundation for writing correct and efficient DataWeave scripts.

4.4 VARIABLES

MuleSoft from salesforce

Variables in DataWeave store values that can be reused in your script.
They make your transformations readable, reusable and easy to maintain.

OVERVIEW

- Variables store values for reuse.
- Declared in the header section using the var keyword.
- Improve readability and avoid repeating expressions.
- Can hold any DataWeave data type.
- Support scope: global (header) and local (inside functions/blocks).

Variables are reusable. Once assigned, their values can be used in the script.

SYNTAX

```
%dw 2.0
output application/json
var variableName = expression

// Use variableName in the body
{
  "result": variableName
}
```

WHERE TO DECLARE?

- Variables are declared in the header section, before the --- separator.
- Multiple variables are separated by commas or new lines.

MULTIPLE VARIABLES

```
%dw 2.0
output application/json
var a = 10,
    b = "MuleSoft",
    c = [1, 2, 3]
---
{
  "a": a,
  "b": b,
  "c": c
}
```

Use meaningful names for variables to make scripts self-explanatory.

TYPES OF VARIABLES

GLOBAL VARIABLES (Header Scope)

- Declared in the header using var.
- Accessible throughout the entire script.
- Best for values used multiple times.

```
%dw 2.0
output application/json
var taxRate = 0.10
---
{
  "price": 100,
  "tax": price * taxRate,
  "total": 100 + (100 * taxRate)
}
```

LOCAL VARIABLES (Inside Functions/Blocks)

- Declared inside functions, objects or arrays.
- Accessible only within the local block.
- Best for intermediate or temporary values.

```
%dw 2.0
output application/json
---
(payload map ((item, index) -> do {
    var discounted = item.price * 0.9
    ---
    item ++ { price: discounted }
  }))
}
```

CONSTANTS

- Use the const keyword to define constants.
- Helps in situations that a value will not change.

```
%dw 2.0
output application/json
const PI = 3.14159
---
{
  "pi": PI
}
```

const variables are also immutable.

EXAMPLES

Example 1: Simple Variable

```
%dw 2.0
output application/json
var name = "MuleSoft"
---
{
  "message": "Welcome " ++ name ++ "!"
}
```

Output

```
{ "message": "Welcome MuleSoft!" }
```

Example 2: Reusing Variables

```
%dw 2.0
output application/json
var price = 250,
    discount = 0.1
---
{
  "price": price,
  "discount": discount,
  "finalPrice": price - (price * discount)
}
```

Output

```
{ "price": 250, "discount": 0.1, "finalPrice": 225 }
```

Example 3: Using Expressions

```
%dw 2.0
output application/json
var items = [10, 20, 30],
    total = (items sum ((num) -> num))
---
{
  "items": items, "total": total
}
```

Output

```
{ "items": [10, 20, 30], "total": 60 }
```

Example 4: Local Variable in Function

```
%dw 2.0
output application/json
---
(payload map ((item) -> do {
    var tax = item.amount * 0.18
    ---
    item ++ { tax: tax }
  })
```

Output

Depends on the input payload

SCOPE RULES

Syntax	Declared Where	Accessible Where	Example
Global	Header (before ---)	Throughout the entire script	Configuration values, constants
Local	Inside functions, do blocks, objects	Only within that block	Intermediate calculations, temporary values

VARIABLE SHADOWING

A local variable with the same name as a global variable will override the global one inside the local block.

```
%dw 2.0
output application/json
var x = "global"
---
{
  "outer": x,
  "inside": (do { var x = "local" --- x })
}
```

Output

```
{ "outer": "global", "inside": "local" }
```

BEST PRACTICES

- Use meaningful and descriptive names.
- Declare variables at the top of the header.
- Use const for values that never change.
- Use var for values that should not change.
- Limit the scope of variables.
- Avoid too many variables if not required.

Good variable usage leads to clean, maintainable and efficient scripts.

COMMON MISTAKES

- ✗ Forgetting to declare variables in the header.
- ✗ Trying to modify a variable (variables are immutable).
- ✗ Using improper names such as x, temp.
- ✗ Declaring variables that are not used.

QUICK REFERENCE

Syntax	Example
Declare variable	var x = 10
Declare multiple variables	var a = 1, b = 2
Use variable	x or a + b
Local variable	do { var y = 5 --- y }

KEY TAKEAWAYS

- Variables store values for reuse.
- Declared in the header using var (or const).
- Global variables are accessible throughout the script.
- Local variables are accessible only within their scope.
- Variables improve readability, reusability and maintainability.

Use variables wisely to write clean, efficient and professional DataWeave scripts.

4.5 OPERATORS

MuleSoft from salesforce

Operators in DataWeave are used to perform operations on values and expressions.
They help you build conditions, calculations and transformations easily.

OVERVIEW

- Operators work on literals, variables, arrays, objects and expressions.
- DataWeave supports arithmetic, comparison, logical, conditional, string and other operators.
- Most operators return a value.
- Use parentheses ( ) to control evaluation order.

OPERATOR CATEGORIES

+ - * /	Arithmetic — Math operations
= != >	Comparison — Compare values
&& ||	Logical — Boolean logic
? :	Conditional — Conditional logic
++ --	Increment / Decrement
& | ^ ~	Bitwise — Bitwise operations
..	Range — Create ranges
=~	Pattern — Match operator

BASIC SYNTAX EXAMPLE

```
%dw 2.0
output application/json
var a = 10
var b = 5
---
{
  sum: a + b,
  isGreater: a > b,
  isEqual: a == b
}
```

Operators make expressions powerful and easy to read.

ARITHMETIC OPERATORS

Operator	Name	Description	Example	Result
+	Addition	Adds two values	10 + 5	15
-	Subtraction	Subtracts right from left	10 - 5	5
*	Multiplication	Multiplies two values	10 * 5	50
/	Division	Divides left by right	10 / 5	2
%	Modulo	Returns remainder	10 % 3	1
^	Exponent	Power (left to the power right)	2 ^ 3	8
- (unary)	Unary Minus	Negates a value	-10	-10
+ (unary)	Unary Plus	Returns the positive value	+10	10

COMPARISON OPERATORS

Operator	Name	Description	Example	Result
==	Equal	Checks equality	10 == 10	true
!=	Not Equal	Checks inequality	10 != 5	true
>	Greater Than	Checks if left is greater	10 > 5	true
<	Less Than	Checks if left is less	10 < 5	false
>=	Greater or Equal	Checks if greater or equal	10 >= 5	true
<=	Less or Equal	Checks if less or equal	10 <= 5	false

LOGICAL OPERATORS

Operator	Name	Description	Example	Result
&&	Logical AND	True if both are true	true && false	false
||	Logical OR	True if any one is true	true || false	true
!	Logical NOT	Inverts the value	!true	false

CONDITIONAL (TERNARY) OPERATOR

Operator / Syntax	Description	Example	Result
condition ? valueIfTrue : valueIfFalse	Returns valueIfTrue if condition is true, otherwise valueIfFalse	10 > 5 ? "Yes" : "No"	"Yes"
		3 > 10 ? "Yes" : "No"	"No"

INCREMENT / DECREMENT OPERATORS

Operator	Name	Description	Example	Result
++	Increment	Increases value by 1	var x = 5; ++x	6
--	Decrement	Decreases value by 1	var x = 5; --x	4

RANGE OPERATOR

Operator	Description	Example	Result
..	Creates a range from start to end (inclusive)	0 .. 5	[0, 1, 2, 3, 4, 5]
..<	Creates a range from start to end (exclusive)	1 ..< 4	[1, 2, 3]

STRING OPERATORS

Operator	Name	Description	Example	Result
++	Concatenation	Joins strings	"Hello" ++ "World"	"HelloWorld"
=~	Contains (Regex)	Checks regex match	"hello" =~ /he/	true
!~	Not Contains (Regex)	Checks no regex match	"hello" !~ /he/	false

OTHER USEFUL OPERATORS

Operator	Description	Example	Result
default	Returns default if value is null or empty	null default "N/A"	"N/A"
as	Casts a value to a specific data type	"123" as Number	123
in	Checks if value exists in an array	3 in [1, 2, 3, 4]	true
not in	Checks if value does not exist in an array	5 not in [1, 2, 3, 4]	true

OPERATORS PRECEDENCE (HIGH TO LOW)

Precedence	Category	Operators
1	Grouping	( )
2	Unary	- + ++ -- (unary)
3	Exponent	^
4	Multiplication	* / %
5	Addition	+ -
6	Comparison	== != > < >= <=
7	Logical AND	&&
8	Logical OR	||
9	Conditional	?:

EXAMPLES IN DATAWEAVE

```
%dw 2.0
output application/json
var a = 10
var b = 5
var name = "MuleSoft"
---
{
  arithmetic: { sum: a + b, diff: a - b, mul: a * b, mod: a %b},
  comparison: { greater: a > b, equal: a == b, lessOrEqual: a <= 10 },
  logical: { and: a > 5 && b < 10, or: a > 15 || b < 0, not: !false },
  conditional: a > b ? "a is greater" : "b is greater or equal",
  range: [1 .. 5],
  concat": "Hello " ++ name
}
```

Output

```
{
  "arithmetic": {"sum": 15, "diff": 5, "mul": 50, "mod": 0},
  "comparison": {"greater": true, "equal": false, "lessOrEqual": true },
  "logical": { "and": false, "or": true, "not": false },
  "conditional": "a is greater",
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello MuleSoft"
}
```

BEST PRACTICES

- Use meaningful and descriptive names.
- Declare variables at the top of the header.
- Use const for values that never change.
- Use var for values that should not change.
- Limit the scope of variables.
- Avoid too many variables if not required.

Good variable usage leads to clean, maintainable and efficient scripts.

COMMON MISTAKES

- ✗ Forgetting operator precedence.
- ✗ Not using parentheses for complex conditions.
- ✗ Ignoring data types while comparing values.
- ✗ Not handling null cases in expressions.

SCOPE RULES

Scope	Declared Where	Accessible Where	Example
+ - * / % ^	Header (before ---)	Throughout the entire script	Configuration values, constants
== != > < >= <=	Comparison	Only within that block	Intermediate calculations, temporary values
&& || !	Logical		
?:	Conditional (Ternary)		a > b ? x : y
.. ..<	Range		0 .. 5
in not in	Membership		3 in [1,2,3]
default	Default Value		null default "N/A"
as	Type Cast		"123" as Number

QUICK REFERENCE

Syntax	Example
Declare variable	var x = 10
Declare multiple variables	var a = 1, b = 2
Use variable	x or a + b
Local variable	do { var y = 5 --- y }

KEY TAKEAWAYS

- Variables store values for reuse.
- Declared in the header using var (or const).
- Global variables are accessible throughout the script.
- Local variables are accessible only within their scope.
- Variables improve readability, reusability and maintainability.

Mastering operators helps you write efficient, readable and bug-free DataWeave scripts.

4.6.1 CONDITIONAL STATEMENTS

MuleSoft from salesforce

Conditional statements in DataWeave allow you to make decisions and create different expressions based on conditions.
They are essential for building dynamic and intelligent transformations.

OVERVIEW

- Used to execute different logic based on conditions.
- Makes scripts dynamic and data-driven.
- Can be used with any expression that returns true or false.
- Frequently used in validation, mapping and filtering.

TYPES OF CONDITIONAL STATEMENTS

1. if-else

Simple condition with two paths.

2. else-if ladder

Multiple conditions checked in order.

3. when (switch)

Match a value against multiple cases.

4. default

Falback case used inside when expressions.

SYNTAX OVERVIEW

Construct	Syntax
if-else	if (condition) do A else B
else-if	if (c1) do A else if (c2) do B else C
when	value1 -> A, value2 -> B
default	else C

1. IF - ELSE

Syntax

```
if (condition) do
    expression1
else
    expression2
```

Example

```
%dw 2.0
output application/json
var age = 20
---
if (age >= 18)
    "Adult"
else
    "Minor"
```

Output

```
{
  "status":"Adult"
}
```

2. ELSE IF LADDER

Syntax

```
if (condition1) do
    expr1
else if (condition2) do
    expr2
else if (condition3) do
    expr3
...
else
    exprN
```

Example

```
%dw 2.0
output application/json
var score = 75
---
if (score >= 90) "A"
else if (score >= 75) "B"
else if (score >= 50) "C"
else "F"
```

Output

```
{
  "grade":"B"
}
```

Use if-else and else-if statements whenever your transformation logic depends on one or more conditions.

4.6.2 CONDITIONAL STATEMENTS (Part 2)

MuleSoft from salesforce

3. WHEN (SWITCH)

Syntax

```
when
value case1 -> result1
value case2 -> result2
...
else defaultResult
```

Example

```
%dw 2.0
output application/json
var day = "Mon"
---
when day
 "Mon", "Tue", "Wed", "Thu", "Fri"
-> "Weekday"
 "Sat", "Sun" -> "Weekend"
 else "Invalid Day"
```

Output

```
{ "type" : "Weekday" }
```

4. DEFAULT (IN WHEN)

Syntax

```
when
value case1 -> result1
value case2 -> result2
...
else defaultResult
```

Example

```
%dw 2.0
output application/json
var role = "Guest"
---
when role
  "Admin" -> "Full Access"
  "User" -> "Limited Access"
else "Read Only"
```

Output

```
{ "access" : "Read Only" }
```

5. NESTED CONDITIONALS

Example

```
%dw 2.0
output application/json
var customer = { type: "Premium", active: true }
---
if (customer.active) do
   if (customer.type == "Premium") do
       "Welcome Premium Customer"
    else "Welcome Customer"
  else "Invctive Customer"
```

Output

```
{ "message" : "Welcome Premium Customer" }
```

6. CONDITIONALS WITH BOOLEANS

Example

```
%dw 2.0
output application/json
var isMember = true
---
if (isMember) do "Member: true and valid"
else false
```

Output

```
{ "isMemberEligible" : false }
```

7. CONDITIONALS WITH NULL CHECK

Example

```
%dw 2.0
output application/json
var name = null
---
if (isEmpty(name) or name == null)
  do "Guest"
else name
```

Output

```
{ "displayName" : "Guest" }
```

PRACTICAL SCENARIOS

Scenario	Example Use Case
Data Validation	Check if required fields are present.
Mapping Values	Map codes to meaningful descriptions.
Business Rules	Apply different logic based on conditions.
Status Calculation	Calculate status based on multiple criteria.
Error Handling	Provide default values or error messages.

BEST PRACTICES

- ✓ Keep conditions simple and readable.
- ✓ Use meaningful variable and field names.
- ✓ Prefer when for multiple discrete values.
- ✓ Handle all possible cases using else or default.
- ✓ Use isEmpty, isBlank or null to check for null/empty.
- ✓ Test all branches to ensure expected results.

COMMON MISTAKES

- ✗ Forgetting else case to handle no match.
- ✗ Using = instead of == for equality.
- ✗ Ignoring data types while comparing values.
- ✗ Not checking for null or empty values.
- ✗ Returning inconsistent data types in branches.

Good use of conditional statements leads to smart, flexible and maintainable DataWeave scripts.

4.6.3 CONDITIONAL STATEMENTS (Part 3)

MuleSoft from salesforce

OPERATORS PRECEDENCE (HIGH TO LOW)

Precedence	Category	Operators	Example
1	Grouping	( )	(a + b) * c
2	Unary	! , + , - , -- (unary)	-a , !flag
3	Exponent	^	a ^ 2
4	Multiplicative	* , / , %	a * b , a % b
5	Additive	+ , -	a + b , a - b
6	Comparison	== , != , > , < , >= , <=	a >= b
7	Logical AND	&&	a > 0 && b > 0
8	Logical OR	||	a > 0 || b > 0
9	Conditional	? :	condition ? x : y

Operators with higher precedence are evaluated first.
Use parentheses ( ) to control evaluation order.

SCOPE RULES

Scope	Declared Where	Accessible Where	Example
Global	Header (before ---)	Throughout the entire script	var x = 10 // accessible everywhere

Local	Inside functions, do blocks, objects	Only within that block	do {
			var y = 5
			--- y
			}

Default Value	As variable initialization	Where the variable is used	var n = 0 default 10
Type Cast	During initialization	Where the variable is used	var age = "20" as Number // age is number

Local variables cannot be accessed outside their scope.
Global variables are accessible across the entire script.

EXAMPLES IN DATAWEAVE

```
%dw 2.0
output application/json
var a = 10
var b = 5
var name = "MuleSoft"
---
{
  "arithmetic": { "sum": a + b, "diff": a - b,          // 15, 5, 50, 2, 0
                  "mul": a / b, "mod": a % b },
  "comparison": { "greater": a > b, "equal": a == b,    // true, false, false
                  "lessOrEqual": a <= b },
  "logical":  { "and": (a > 5 && b > 2), "or": (a >     // true, false, true
                '> 15 || b < 0), "not": !(a == b) },
  "conditional": "a is greater"                         // ternary operator
                 ((a > b) ? "a is greater" : "b is greater or equal"),
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello " ++ name                            // string concatenation
}
```

Output

```
{
  "arithmetic": { "sum": 15, "diff": 5, "div": 2, "mod": 0 },
  "comparison": { "greater": true, "equal": false, "lessOrEqual": false
  "logical": { "and": true, "or": false, "not": true },
  "conditional": "a is greater",
  "range": [1, 2, 3, 4, 5],
  "concat": "Hello MuleSoft"
}
```

QUICK REFERENCE

Syntax	Example
if - else	if (condition) do A else B
else-if ladder	if (c1) do A else if (c2) do B else C
when (switch)	value1 -> A | value2 -> B | else C
default (in when)	else C
Nested conditions	if (cond1) do if (cond2) do X else Y else Z
Boolean operators	&& (AND), || (OR), ! (NOT)
Comparison operators	==, !=, >, <, >=, <=
Null check	isEmpty(value) or value == null

KEY TAKEAWAYS

- Conditional statements help make decisions in your DataWeave scripts.
- Use if-else for simple conditions and else-if for multiple conditions.
- Use when (switch) for matching a value against multiple cases.
- Always handle unmatched cases using default or else.
- Use parentheses to control evaluation order and avoid errors.
- Check for null or empty values to prevent runtime issues.
- Keep conditions simple, readable and well-structured.
- Well-written conditions make scripts robust, maintainable and efficient.

COMMON MISTAKES

- ✗ Forgetting else case in if-else or else-if ladder.
- ✗ Using = instead of == for equality comparison.
- ✗ Ignoring data types while comparing values.
- ✗ Not checking for null or empty values.
- ✗ Returning inconsistent data types in different branches.
- ✗ Overly complex conditions - break into smaller parts.
- ✗ Not using parentheses, leading to wrong results.

BEST PRACTICES

- ✓ Use meaningful variable and field names.
- ✓ Keep conditions short and easy to understand.
- ✓ Prefer early return / default to reduce nesting.
- ✓ Validate input data before applying conditions.
- ✓ Use functions like isEmpty(), isBlank() and default for safety.
- ✓ Test all possible branches with sample data.
- ✓ Comment complex conditions for future reference.

Use conditional statements to write smart, flexible and maintainable DataWeave scripts.

4.7.1 ARRAY FUNCTIONS (Part 1)

MuleSoft from salesforce

- Array functions in DataWeave help you process, transform and analyze array data easily.
- They are powerful, expressive and can be combined to solve complex scenarios.

OVERVIEW

- DataWeave provides rich array functions.
- Functions are immutable and return new arrays.
- Functions can be chained together.
- Support mapping, filtering, reducing and grouping.
- Useful in real-time integration transformations.

COMMON ARRAY FUNCTIONS

Function	Purpose
map()	Transform each element
filter()	Filter elements
reduce()	Reduce array to single value
pluck()	Extract field values
flatten()	Flatten nested arrays
distinctBy()	Remove duplicates based on key
groupBy()	Group by key
orderBy()	Sort values

SYNTAX OVERVIEW

map()	array map ((item,index) -> expression)
filter()	array filter ((item,index) -> condition)
reduce()	array reduce ((item,acc,index) -> expression)
pluck()	array pluck(key)
flatten()	array flatten()
distinctBy()	array distinctBy((item) -> key)
groupBy()	array groupBy((item) -> key)
orderBy()	array orderBy((item) -> key)

1. map()

Description

Transforms every element using an expression.

Example

```
var nums = [1, 2, 3, 4]
---
nums map ((n) -> n * 2)
```

Output

```
[2, 4, 6, 8]
```

2. filter()

Description

Returns elements matching a condition.

Example

```
var nums = [1, 2, 3, 4, 5, 6]
---
nums filter ((n) -> n mod 2 == 0)
```

Output

```
[2, 4, 6]
```

3. reduce()

Description

Reduces an array into a single value.

Example

```
var nums = [1, 2, 3, 4]
---
nums reduce ((acc, n) -> acc + n)
```

Output

```
10
```

map(), filter() and reduce() are the three most commonly used array functions in DataWeave and are heavily used in MuleSoft projects.

4.7.3 ARRAY FUNCTIONS (Part 3)

MuleSoft from salesforce

OPERATORS PRECEDENCE (HIGH TO LOW)

Precedence	Category	Operators	Example
1	Grouping	( )	(a + b) * c
2	Unary	! , +, -, -- (unary)	-a , !flag
3	Exponent	^	a ^ 2
4	Multiplicative	* , / , %	a * b , a % b
5	Additive	+ , -	a + b , a - b
6	Comparison	== , != , >, < , >=, <=	a >= b
7	Logical AND	&&	a > 0 && b > 0
8	Logical OR	||	a > 0 || b > 0
9	Conditional	? :	condition ? x : y

Operators with higher precedence are evaluated first.
Use parentheses ( ) to control evaluation order.

SCOPE RULES

Scope	Declared Where	Accessible Where	Example
Global	Header (before ---)	Throughout the entire script	var x = 10 // accessible everywhere

Local	Inside functions, do blocks, objects	Only within that block	do {
			var y = 5
			--- y
			}

Default Value	As variable initialization	Where the variable is used	var n = 0 default 10
Type Cast	During initialization	Where the variable is used	var age = "20" as Number // age is number

Local variables cannot be accessed outside their scope.
Global variables are accessible across the entire script.

QUICK REFERENCE

Syntax	Example
map()	var a = [1,2,3] map ($*2)
filter()	var a = [1,2,3] filter ($ > 1)
reduce()	var a = [1,2,3] reduce ((acc,n)->acc+n, 0)
pluck()	var a = [{"n":1},{"n":2}] pluck ("n")
flatten()	var a = [[1],[2]] flatten()
distinctBy()	var a = [{id: 1},{id: 1}] distinctBy $.id
groupBy()	var a = [{c:"A"},{c:"B"},{c:"A"}] groupBy $.c
orderBy()	var a = [{s:30},{s:10}] orderBy $.s
flatten()	var a = [[1],[2]] flatten()

EXAMPLES IN DATAWEAVE

```
%dw 2.0
output application/json
var a = 10
var b = 5
var name = "MuleSoft"
---
{
  "arithmetic":  { "sum": a + b, "diff": a - b,                  // 15, 5, 50, 2, 0
                   "mul": a * b, "mod": a % b },
  "comparison": { "greater": a > b, "equal": a == b;             // true, false, false
                   "lessOrEqual": a <= b },
  "logical":    { "and": (a > 5 && b > 2), "or": (a > 7 ),       // true, false, true
                  "not": !(a == b) },
  "conditional": "a is greater",                                 // ternary operator
                 ((a > b) ? "a is greater" : "b is greater or equal"),
  "range":      [1, 2, 3, 4, 5],
  "concat":     "Hello " ++ name                                 // string concatenation
}
```

Output

```
{
  "arithmetic":  { "sum": 15, "diff": 5, "mul": 50, "mod": 0 },
  "comparison": { "greater": true, "equal": false, "lessOrEqual": false },
  "logical":    { "and": true, "or": true, "not": true },
  "conditional": "a is greater",
  "range":      [1, 2, 3, 4, 5],
  "concat":     "Hello MuleSoft"
}
```

KEY TAKEAWAYS

- Array functions are the backbone of DataWeave transformations.
- They help you filter, transform, aggregate and organize array data.
- Combine multiple functions to solve complex business problems.
- Functions are immutable and return new arrays.
- Chaining functions makes scripts clean, readable and efficient.
- Leverage functions to improve performance and maintainability.

COMMON MISTAKES

- ✖ Using map() for filtering (use filter() instead).
- ✖ Forgetting to return accumulator in reduce().
- ✖ Using distinctBy() without proper key selector.
- ✖ Sorting without considering data types.
- ✖ Over-nesting functions makes script hard to read.
- ✖ Not handling empty arrays in transformations.

BEST PRACTICES

- Always use the right function for the task.
- Prefer filter() before map() to reduce unnecessary processing.
- Use distinctBy() to remove duplicates by a key.
- Use orderBy() to sort data for reporting and analysis.
- Chain functions for expressive and concise transformations.
- Avoid heavy processing in large arrays when performance matters.

Master array functions to write concise, powerful and performant DataWeave transformations!

4.8.1 OBJECT FUNCTIONS

- Object functions in DataWeave help you transform, extract and analyze data stored in objects (key-value pairs).
- They make your scripts cleaner, reusable and more powerful.

OVERVIEW

- Objects in DataWeave are collections of key-value pairs.
- Object functions help manipulate and transform object data.
- These functions return new objects, arrays or values.
- Widely used in mapping, filtering and data transformations.

COMMON OBJECT FUNCTIONS

Function	Purpose
mapObject()	Transform each key-value pair
pluck()	Extract values from keys
keysOf()	Get all object keys
valuesOf()	Get all object values
namesOf()	Get names of all keys

SYNTAX OVERVIEW

mapObject()	object mapObject((value,key,index) -> expression)
pluck()	object pluck(key)
keysOf()	object keysOf()
valuesOf()	object valuesOf()
namesOf()	object namesOf()

⭐ All object functions are immutable. They never modify the original object.

1. mapObject()

- Applies an expression to every key-value pair.
- Returns a new transformed object.

Example:

```
var obj = {
    name : "John",
    age  : 30,
    city : "NY"
}
---
obj mapObject((value, key, index) ->
    {(key) : upper(value as String)}
)
```

Output:

```
{
    "name" : "JOHN",
    "age"  : "30",
    "city" : "NY"
}
```

2. pluck()

- Extracts values from one or more object keys.
- Returns an array.

Example:

```
var obj = {
    name : "John",
    age  : 30,
    city : "NY"
}
---
obj pluck ["name", "city"]
```

Output:

```
[
    "John",
    "NY"
]
```

mapObject() and pluck() are among the most frequently used object functions in MuleSoft DataWeave transformations.

4.8.2 OBJECT FUNCTIONS (Part 2)

MuleSoft from salesforce

Object functions help you extract information, get keys and values, and work with object data efficiently.

3. keysOf()

- Returns an array of all keys in the object.
- Useful to get the list of keys.

Example:

```
var obj = {
    name : "John",
    age  : 30,
    city : "NY"
}
---
obj keysOf()
```

Output:

```
[ "name", "age", "city" ]
```

4. valuesOf()

- Returns an array of all values in the object.
- Useful to get all values together.

Example:

```
var obj = {
    name : "John",
    age  : 30,
    city : "NY"
}
---
obj valuesOf()
```

Output:

```
[ "John", 30, "NY" ]
```

5. namesOf()

- Returns an array of key names.
- Similar to keysOf(), used for clarity in some scenarios.

Example:

```
var obj = {
    name : "John",
    age  : 30,
    city : "NY"
}
---
obj namesOf()
```

Output:

```
[ "name", "age", "city" ]
```

USING FUNCTIONS WITH COMPLEX OBJECT

Example:

```
var employee = {
  id      : 101,
  name    : "Alice",
  details : {
    role     : "Architect",
    salary   : 75000,
    location : "Bangalore"
  },
  skills  : ["DW", "Mule", "Java"]
}
---
employee keysOf()
employee valuesOf()
```

Output:

```
["id", "name", "details", "skills"]
[101, "Alice", {...}, ["DW","Mule","Java"]]
```

NESTED OBJECT SCENARIO

Example:

```
var data = {
  order    : { id : 1, name : "Book",
               active : true },
  customer : { name : "Bob",
               balance : 150 }
}
---
var orderInfo = data.order pluck ["id","name"]
var customerInfo = data.customer valuesOf()
var allKeys = data keysOf()
```

Output:

```
orderInfo    : { "id": 1, "name": "Book" }
customerInfo : [ "Bob", 150 ]
allKeys      : [ "order", "customer"]
```

COMBINING FUNCTIONS

Example:

```
var obj = {
  a : 10,  b : 20,  c : 30
}
---
// Get keys and then get values
(obj keysOf())
  map ((k) -> obj[k])
```

Output:

```
[10, 20, 30]
```

BEST PRACTICES

- Use keysOf() to iterate over object keys.
- Use valuesOf() when you need all values.
- Use pluck() to fetch specific fields as an array.
- Combine functions (e.g., keysOf() + map()) for advanced transformations.
- Always handle nested objects carefully.

COMMON MISTAKES

- Using pluck() with wrong key name.
- Assuming order of keys in objects (objects are unordered).
- Forgetting that object functions return new results.
- Not handling null values in object fields.
- Overusing functions instead of simple mapping.

Use object functions wisely to write clean, efficient and powerful DataWeave transformations!

4.8.3 OBJECT FUNCTIONS (Part 3)

MuleSoft from salesforce

Object functions are powerful tools in DataWeave. They can be chained and combined to solve complex transformation requirements.

KEY TAKEAWAYS

- Object functions help transform, extract and analyze object data.
- They return new objects, arrays or values.
- These functions are immutable and do not modify the original object.
- Functions can be chained together for powerful transformations.

FUNCTIONS RECAP

Function	Purpose
mapObject()	Transform each key-value pair
pluck()	Extract values from keys
keysOf()	Get all object keys
valuesOf()	Get all object values
namesOf()	Get names of all keys

CHEAT SHEET (SYNTAX)

mapObject()	object mapObject((value, key, index) -> expression)
pluck()	object pluck(key)
keysOf()	object keysOf()
valuesOf()	object valuesOf()
namesOf()	object namesOf()

ADVANCED USAGE OF OBJECT FUNCTIONS

1. Rename all keys using mapObject()

Example:

```
var obj = {
    firstName: "John",
    lastName: "Doe",
    age      : 30
}
---
obj mapObject((value, key, index) ->
    {(upper(key)): value} )
```

Output:

```
{
    FIRSTNAME: "John",
    LASTNAME : "Doe",
    AGE      : 30
}
```

2. Filter object by keys and values

Example:

```
var obj = {
    name : "John",
    age  : 30,
    city : "NY",
    role : "Developer"
}
---
obj filterObject ((value, key, index) ->
 (key != "age" and value != "NY") )
```

Output:

```
{
   name : "John",
   role : "Developer"
}
```

3. Create object from keys and values

Example:

```
var keys   = ["id", "name", "city"]
var values = [101, "Alice", "Boston"]
---
keys zip values
reduce ((acc, item, index) ->
    acc ++ {(item[0]): item[1]}, {})
```

Output:

```
{
  id   : 101,
  name : "Alice",
  city : "Boston"
}
```

REAL-WORLD SCENARIO

Transform customer data object to return only required fields with formatted values.

Example:

```
var customer = {
  id     : 501,
  name   : "Alice Johnson",
  email  : "alice@example.com",
  city   : "new york",
  joined : "2024-01-15",
  active: true
}
---
customer
  filterObject ((value,key) -> key != "active")
  mapObject ((value,key) ->
    if (key == "city") {(key): upper(value)}
    else {(key): value})
```

Output:

```
{
  id     : 501,
  name   : "Alice Johnson",
  email  : "alice@example.com",
  city   : "NEW YORK",
  joined : "2024-01-15"
}
```

We filtered out "active" field and converted city value to uppercase using mapObject().

COMBINING MULTIPLE FUNCTIONS

Example:

```
var obj = { a: 10, b: 20, c: 30,d: 40 }
---
obj
  mapObject ((value,key) -> {(key): value * 2})
  filterObject ((value, key) -> value >= 40)
  pluck (keysOf($)[0])  // get first key's value
```

Output:

```
80
```

BEST PRACTICES

- Use keysOf() and valuesOf() to explore object data.
- Use mapObject() for conditional formatting and renaming.
- Chain functions for concise and powerful transformations.
- Always handle nullable or empty objects safely.
- Write reusable functions for common object operations.

COMMON MISTAKES

- Modifying original objects (functions return new objects).
- Using pluck() with wrong or non-existing key.
- Not handling missing keys in objects.
- Overusing mapObject() when simple pluck() is enough.
- Assuming order in objects (objects are unordered).

Master object functions to write clean, efficient and powerful DataWeave transformations!

4.9.1 STRING FUNCTIONS (Part 1)

MuleSoft from salesforce

String functions in DataWeave help you manipulate, transform and work with text data efficiently.
They are widely used for data cleaning, formatting and integration tasks.

OVERVIEW

- Strings are sequences of characters enclosed in quotes.
- DataWeave provides rich and powerful functions to work with text.
- Helps in cleaning, validation, formatting and data preparation.
- Essential in real-world integration and API payloads.

COMMON STRING FUNCTIONS

ABC upper() Convert to uppercase	abc lower() Convert to lowercase	trim() Remove leading and trailing whitespace	123 sizeOf() Returns length of the string
splitBy() Split string into an array	joinBy() Join array of strings	replace() Replace text or pattern	contains() Check if string contains text

SYNTAX OVERVIEW

Function	Syntax
upper	upper(text)
lower	lower(text)
trim	trim(text)
sizeOf	sizeOf(text)
splitBy	splitBy(text, separator)
joinBy	joinBy(array, separator)
replace	replace(text, oldText, newText)
substring	substring(text, fromIndex[, toIndex])
contains	contains(text, substring)
startsWith	startsWith(text, prefix)
endsWith	endsWith(text, suffix)
matches	matches(text, regex)

1. upper()

Converts all characters to uppercase.

Example

```
var str = "MuleSoft Rocks"
---
upper(str)
```

Output

```
"MULESOFT ROCKS"
```

2. lower()

Converts all characters to lowercase.

Example

```
var str = "MULESOFT ROCKS"
---
lower(str)
```

Output

```
"mulesoft rocks"
```

3. trim()

Removes leading and trailing whitespace.

Example

```
var str = " Hello World "
---
trim(str)
```

Output

```
"Hello World"
```

4. sizeOf()

Returns the number of characters in the string.

Example

```
var str = "DataWeave 2.0"
---
sizeOf(str)
```

Output

```
12
```

5. splitBy()

Splits the string into an array using the given separator.

Example

```
var str = "a,b,c,d"
---
splitBy(str, ",")
```

Output

```
[ "a", "b", "c", "d" ]
```

These are some of the most commonly used string functions in DataWeave for text manipulation!

4.9.2 STRING FUNCTIONS (Part 2)

MuleSoft from salesforce

MORE USEFUL STRING FUNCTIONS

6 joinBy()

Joins an array of strings using the given separator.

Example:

```
var arr = ["Jan","Feb","Mar"]
---
joinBy(arr, " | ")
```

Output:

```
"Jan | Feb | Mar"
```

7 replace()

Replaces all occurrences of a pattern with another string.

Example:

```
var str = "I love MuleSoft"
---
replace(str, "MuleSoft",
    "DataWeave")
```

Output:

```
"I love DataWeave"
```

8 substring()

Extracts a substring using start index and length (or end index).

Example:

```
var str = "Hello DataWeave"
---
substring(str, 0, 5)
```

Output:

```
"Hello"
```

9 contains()

Checks if a string contains the specified substring.

Example:

```
var str = "Welcome to MuleSoft"
---
contains(str, "MuleSoft")
```

Output:

```
true
```

10 startsWith()

Checks if string starts/ends with the given prefix.

Example:

```
var str = "Hello World"
---
startsWith(str, "Hello")
```

Output:

```
true
```

11 endsWith()

Checks if string starts/ends with the given suffix.

Example:

```
var str = "Hello World"
---
endsWith(str, "World")
```

Output:

```
true
```

REAL-WORLD SCENARIOS

Scenario	Function(s) Used	Description
Standardize text	upper(), lower()	Normalize text format
Extract names	splitBy(), trim()	Split and clean text to get names
Calculate length	sizeOf()	Get total number of characters
Clean text	trim(), replace()	Remove spaces and unwanted text
Remove duplicates	distinctBy()	Eliminate duplicate records
Join values	joinBy()	Combine array elements
Format reports	replace(), padLeft()	Format text for reports/tables

BEST PRACTICES

- Use trim() to clean user inputs before validations.
- Prefer splitBy() over complex regex for parsing simple data.
- Use replace() with regex carefully for complex text patterns.
- Avoid hardcoding text; use variables or lookups where possible.
- Use startsWith() / endsWith() for prefix/suffix validations.
- Avoid heavy processing in large strings when performance matters.

Use the right string function at the right time to write clean, efficient and powerful DataWeave scripts!

4.9.3 STRING FUNCTIONS (Part 3)

MuleSoft from salesforce

COMMON MISTAKES

1. **Forgetting strings are immutable.**
   String functions return new strings.
   They do not change the original string.
2. **Not handling null values.**
   Always check for null before applying string functions.
3. **Overusing replace().**
   Using replace() too often can make scripts hard to read and maintain.
4. **Assuming contains() is case-sensitive.**
   contains() is case-sensitive by default.
   Use lower()/upper() for case-insensitive search.
5. **Modifying original strings.**
   Do not try to change strings directly.
   Always store the result in a new variable.

QUICK REFERENCE TABLE

Function	Purpose
upper()	Convert text to uppercase
lower()	Convert text to lowercase
trim()	Remove leading and trailing spaces
sizeOf()	Return length of the string
splitBy()	Split string into an array
joinBy()	Join array of strings
replace()	Replace text or pattern
substring()	Extract part of the string
contains()	Check if string contains text
startsWith()	Check if string starts with prefix
endsWith()	Check if string ends with suffix
matches()	Check if string matches regex

SYNTAX CHEAT SHEET

Function	Syntax
upper(text)	upper(text)
lower(text)	lower(text)
trim(text)	trim(text)
sizeOf(text)	sizeOf(text)
splitBy(text, separator)	splitBy(text, separator)
joinBy(array, separator)	joinBy(array, separator)

Function	Syntax
replace(text, oldText, newText)	replace(text, oldText, newText)
substring(text, fromIndex, toIndex)	substring(text, fromIndex, toIndex)
contains(text, substring)	contains(text, substring)
startsWith(text, prefix)	startsWith(text, prefix)
endsWith(text, suffix)	endsWith(text, suffix)
matches(text, regex)	matches(text, regex)

KEY TAKEAWAYS

- String functions help you extract, transform and manipulate text data efficiently.
- They return new strings and are immutable.
- Use the right function for the task to keep your scripts clean and readable.
- Combine multiple functions to solve complex transformation requirements.
- Write clean, efficient and maintainable DataWeave scripts using string functions.

EXAMPLE COMBO

```
var str = " Hello MuleSoft "
---
trim(str)                  // "Hello MuleSoft"
lower(trim(str))           // "hello mulesoft"
upper(trim(str))           // "HELLO MULESOFT"
sizeOf(trim(str))          // 15
splitBy(trim(str), " ")    // ["Hello", "MuleSoft"]
replace(trim(str), "MuleSoft", "World") // "Hello World"
```

Master string functions to write clean, efficient and maintainable DataWeave transformations!

4.10.1 DATE & TIME FUNCTIONS

MuleSoft from salesforce

Date & Time functions in DataWeave help you create, parse, format, manipulate and compare dates and times.
They are essential for handling time zones, durations and real-world integrations.

OVERVIEW

- DataWeave uses Java Time API under the hood.
- All date and time values are immutable.
- Supports parsing, formatting, arithmetic, comparison and time zone handling.
- Key concepts include:
  - Date
  - Time
  - DateTime
  - LocalDate
  - LocalTime
  - TimeZone
  - Duration
  - Period

COMMON DATE & TIME FUNCTIONS

now()	Current date and time
today()	Current date
format()	Format date/time
parse()	Parse string into date/time
add()	Add duration/period
subtract()	Subtract duration/period
withZone()	Convert time zone
between()	Difference between dates
isBefore()	Compare dates (before)
isAfter()	Compare dates (after)
atZone()	Attach time zone to date/time

SYNTAX OVERVIEW

- now()
- format(dateTime, pattern)
- parse(string, pattern)
- add(dateTime, duration/period)
- subtract(dateTime, duration/period)
- withZone(dateTime, zone)
- between(date1, date2, unit)
- isBefore(date1, date2)
- isAfter(date1, date2)
- atZone(localDateTime, zone)

All Date & Time functions do not modify the original object.
They return new results.

1. now()

Returns the current date & time in ISO-8601 format.

Example :

```
%dw 2.0
output application/json
---
now()
```

Output (example) :

```
2024-05-20T14:35:22.123+04:00
```

2. today()

Returns the current date without the time portion.

Example :

```
%dw 2.0
output application/json
---
today()
```

Output (example) :

```
2024-05-20
```

3. format()

Formats a date or datetime using a specified pattern.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T14:35:22Z"
---
format(dt,"yyyy-MM-dd HH:mm")
```

Output :

```
2024-05-20 14:35
```

4. parse()

Parses a string into a Date or DateTime value.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20 14:35:22"
---
parse(dt,"yyyy-MM-dd HH:mm:ss")
```

Output :

```
2024-05-20T14:35:22
```

The most frequently used Date & Time functions in MuleSoft projects are now(), today(), format() and parse().

4.10.2 DATE & TIME FUNCTIONS

MuleSoft from salesforce

Advanced Date & Time functions help you perform arithmetic, time zone conversions, comparisons and duration calculations.

5. add()

Adds a duration or period to a date/time.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T10:00:00Z"
---
add(dt, |P1D|)
```

Output :

```
2024-05-21T10:00:00Z
```

6. subtract()

Subtracts a duration or period from a date/time.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T10:00:00Z"
---
subtract(dt, |PT5H|)
```

Output :

```
2024-05-20T05:00:00Z
```

7. withZone()

Converts a date/time to a different time zone.

Example :

```
%dw 2.0
output application/json
var dt = "2024-05-20T10:00:00Z"
---
withZone(dt, "America/New_York")
```

Output :

```
2024-05-20T06:00:00-04:00
```

8. between()

Returns the difference between two date/times in the given unit.

Example :

```
%dw 2.0
output application/json
var d1 = "2024-05-20"
var d2 = "2024-05-25"
---
between(d1, d2, "days")
```

Output :

```
5
```

9. isBefore() / isAfter()

Compares two date/times.

Example :

```
%dw 2.0
output application/json
var d1 = "2024-05-20"
var d2 = "2024-05-25"
---
{
  before : isBefore(d1, d2),
  after  : isAfter(d1, d2)
}
```

Output :

```
{ before: true, after: false }
```

10. atZone()

Attaches a time zone to a local DateTime (without zone).

Example :

```
%dw 2.0
output application/json
var localDt = "2024-05-20T10:00:00"
---
atZone(localDt, "Asia/Kolkata")
```

Output :

```
2024-05-20T10:00:00+05:30
```

These functions help you perform powerful date & time operations efficiently and build real-world, time-aware solutions.

DATE & TIME UNITS FOR between()

Unit	Description
years	Difference in years
months	Difference in months
days	Difference in days
hours"	Difference in hours
minutes	Difference in minutes
seconds"	Difference in seconds

DURATION & PERIOD EXAMPLES

Format	Example
|P1D|	// 1 day
|PT5H|	// 5 hours
|PT30M|	// 30 minutes
|P7D|	// 7 days
|P1M|	// 1 month
|P1Y|	// 1 year
|P1DT2H|	// 1 day 2 hours
|P2DT3H|	// 2 days 3 hours

COMMON TIME ZONES

Time Zone ID	Description
UTC	Coordinated Universal Time
America/New_York	Eastern Time (US)
Europe/London	London Time
Asia/Kolkata	India Standard Time
Asia/Tokyo	Japan Standard Time
Australia/Sydney	Australian Eastern Time

DATE & TIME SCENARIOS

- ✓ Log timestamps in UTC and convert at the edges.
- ✓ Calculate SLAs and timeouts.
- ✓ Handle user time zones for reports and notifications.
- ✓ Schedule jobs and reminders.
- ✓ Validate data within specific date ranges.

COMMON MISTAKES

- ✗ Parsing without specifying the correct pattern.
- ✗ Mixing up Duration and Period.
- ✗ Ignoring time zones and using server local time.
- ✗ Assuming between() returns fractional values.
- ✗ Not handling null or invalid date strings.

Leverage Date & Time functions to build accurate, time-aware and reliable DataWeave solutions!

4.10.3 DATE & TIME FUNCTIONS

MuleSoft from salesforce

Master Date & Time functions to write accurate, time-aware and reliable DataWeave solutions!

QUICK REFERENCE TABLE

Function	Purpose
now()	Current date and time
today()	Current date
format()	Format date/time
parse()	Parse string into date/time
add()	Add duration/period
subtract()	Subtract duration/period
withZone()	Convert to different time zone
between()	Difference between dates
isBefore()	Check if date1 is before date2
isAfter()	Check if date1 is after date2
atZone()	Attach zone to local date/time

• SYNTAX CHEAT SHEET •

now()	now()
today()	today()
format()	format(dateTime, pattern)
parse()	parse(string, pattern)
add()	add(dateTime, duration/period)
subtract()	subtract(dateTime, duration/period)
withZone()	withZone(dateTime, "Zone/Id")
between()	between(date1, date2, unit)
isBefore()	isBefore(date1, date2)
isAfter()	isAfter(date1, date2)
atZone()	atZone(localDateTime, "Zone/Id")

REAL-WORLD SCENARIOS

Log timestamps in UTC and convert at the edges.	Functions: now(), withZone()	Example: Store in UTC, display in user's time zone.
Calculate SLAs and timeouts.	Functions: add(), between(), today()	Example: Check if SLA breached in hours.
Handle user time zones for reports and notifications.	Functions: withZone(), atZone()	Example: Show dates in user's local time.
Schedule jobs and reminders.	Functions: add(), format()	Example: Send reminder after 2 days.
Validate data within specific date ranges.	Functions: isBefore(), isAfter(), between()	Example: Validate order date in range.

BEST PRACTICES

- ✓ Always use UTC for storing timestamps.
- ✓ Convert to user time zone only for display.
- ✓ Use ISO-8601 format for consistency.
- ✓ Handle daylight saving changes carefully.
- ✓ Validate date inputs before processing.

COMMON MISTAKES

- ✗ Parsing without specifying the correct pattern.
- ✗ Mixing up Duration and Period.
- ✗ Ignoring time zones and using server local time.
- ✗ Assuming between() returns fractional values.
- ✗ Not handling null or invalid date strings.

EXAMPLE COMBINATION

```
%dw 2.0
output application/json
var dt = "2024-05-20T14:35:22Z"
---
{
  localTime: withZone(dt, "Asia/Kolkata"),      // convert zone
  formatted: format(dt, "yyyy-MM-dd HH:mm"), // format
  tomorrow: add(today(), |P1D|),                // add 1 day
  diffInDays: between("2024-05-20", "2024-05-25", "days")
              // difference in days
}
```

KEY TAKEAWAYS

- ★ Date & Time functions make your scripts time-aware.
- ★ They help in calculations, comparisons and scheduling.
- ★ Use the right function for the right task.
- ★ Consistent handling of time zones ensures accuracy.
- ★ Combine multiple functions for powerful transformations.

SAMPLE OUTPUT

```
{
  "localTime": "2024-05-20T20:05:22+05:30",
  "formatted": "2024-05-20 14:35",
  "tomorrow": "2024-05-21",
  "diffInDays": 5
}
```

Leverage Date & Time functions to build accurate, time-aware and reliable DataWeave solutions!

4.11 ALL DATAWEAVE FUNCTIONS

MuleSoft from salesforce

DataWeave provides a rich set of built-in functions to transform, manipulate, query and validate data easily. These functions are grouped by category.

1. STRING FUNCTIONS

1) upper()
2) lower()
3) trim()
4) sizeOf()
5) splitBy(text, separator)
6) joinBy(array, separator)
7) replace(text, old, new)
8) substring(text, from, to)
9) contains(text, substring)
10) startsWith(text, prefix)
11) endsWith(text, suffix)
12) matches(text, regex)

2. ARRAY FUNCTIONS

1) sizeOf(array)
2) isEmpty(array)
3) map(array, (item, index) -> ...)
4) filter(array, (item, index) -> ...)
5) reduce(array, (acc, item, index) -> ..., initialValue)
6) pluck(array, key)
7) distinctBy(array, criteria)
8) orderBy(array, (item) -> ...)
9) groupBy(array, (item) -> ...)
10) flatten(array)
11) slice(array, from, to)
12) take(array, n)
13) drop(array, n)

3. OBJECT FUNCTIONS

1) keys(object)
2) values(object)
3) pluck(object, key)
4) mapObject(object, (value, key) -> ...)
5) filterObject(object, (value, key) -> ...)
6) ++ (obj1, obj2)
7) update(object, key, value)
8) removeKey(object, key)
9) isEmpty(object)
10) sizeOf(object)

4. DATE & TIME FUNCTIONS

1) now()
2) today()
3) format(dateTime, pattern)
4) parse(string, pattern)
5) add(dateTime, duration/period)
6) subtract(dateTime, duration/period)
7) withZone(dateTime, zone)
8) between(date1, date2, unit)
9) isBefore(date1, date2)
10) isAfter(date1, date2)
11) atZone(localDateTime, zone)

5. MATH FUNCTIONS

1) abs(number)
2) round(number)
3) floor(number)
4) ceil(number)
5) max(number1, number2, ...)
6) min(number1, number2, ...)
7) sum(array)
8) avg(array)
9) random()

6. LOGICAL FUNCTIONS

1) not(value)
2) and(value1, value2, ...)
3) or(value1, value2, ...)
4) default(value, defaultValue)
5) isEmpty(value)
6) isBlank(value)
7) isNull(value)
8) isNumber(value)
9) isString(value)
10) isBoolean(value)

7. TYPE & CONVERSION FUNCTIONS

1) asString(value)
2) asNumber(value)
3) asBoolean(value)
4) asArray(value)
5) asObject(value)
6) asDate(value, pattern)
7) asDateTime(value, pattern)
8) asLocalDate(value)
9) asLocalTime(value)
10) asPeriod(value)
11) asDuration(value)

8. NULL & DEFAULT FUNCTIONS

1) default(value, defaultValue)
2) whenNull(value, defaultValue)
3) whenEmpty(value, defaultValue)
4) isNull(value)
5) isEmpty(value)
6) isBlank(value)

9. QUERY FUNCTIONS

1) filter(array, condition)
2) find(array, condition)
3) findIndex(array, condition)
4) any(array, condition)
5) every(array, condition)
6) none(array, condition)

10. PAYLOAD NAVIGATION FUNCTIONS

1) payload
2) attributes
3) vars
4) correlationId
5) messageId
6) error
7) error.description
8) error.cause

11. XML & NAMESPACE FUNCTIONS

1) localName(name)
2) namespace(name)
3) prefix(name)
4) namespaceUri(name)
5) qName(prefix, localName)

12. MISCELLANEOUS FUNCTIONS

1) uuid()
2) now()
3) $$ (current context)
4) write(value, format)
5) read(payload, format)
6) cache(key, ttl, factory)
7) do(expression)

KEY TAKEAWAYS

- DataWeave functions are IMMUTABLE and return new values.
- Choose the ri t function for the right task.
- Use functions to write clean, readable and maintainable scripts.
- Combine multiple functions for powerful transformations.
- Refer to official MuleSoft documentation for more details.

SYNTAX REMINDER

- functionName(arg1, arg2, ...) // most functions
- Use named parameters where available.
- Functions are case-sensitive.
- All functions return new results (do not modify originals).

Master DataWeave functions and build efficient, clean and scalable MuleSoft integrations!

4.12.1 READ & WRITE FORMATS

DataWeave supports multiple data formats for reading input and writing output payloads.
Choosing the right format ensures seamless integration and optimal performance.

OVERVIEW

- Read formats parse incoming payloads into DataWeave data.
- Write formats serialize DataWeave data into the desired output.
- Formats can be configured using metadata or inline script directives.
- Custom properties provide fine-grained control over parsing and writing.

SUPPORTED FORMATS

Format	Description
JSON	JavaScript Object Notation
XML	Extensible Markup Language
CSV	Comma-Separated Values
JAVA	Java Objects
PROPERTIES	Java Properties Files
TEXT	Plain Text
BINARY	Binary Data (Base64)
YAML	YAML Ain't Markup Language

SYNTAX OVERVIEW

Read Format

```
%dw 2.0
input payload <formatName>
---
payload
```

Write Format

```
%dw 2.0
output <formatName>
---
<data>
```

- Format names are case-insensitive.
- Default read format is application/java.
- Default write format is application/java.

READ FORMATS (INPUT FORMATS)

1. JSON

Example Input

```
{
  "name": "John",
  "age": 30
}
```

DataWeave

```
%dw 2.0
input payload json
---
payload
```

Output

```
{
  "name": "John",
  "age": 30
}
```

2. XML

Example Input

```
<person>
  <name>John</name>
  <age>30</age>
</person>
```

DataWeave

```
%dw 2.0
input payload xml
---
payload
```

Output

```
{ "person": {
    "name": "John",
    "age": "30"
  }
}
```

3. CSV

Example Input

```
name,age,city
John,30,NY
Jane,25,LA
```

DataWeave

```
%dw 2.0
input payload csv
---
payload
```

Output

```
[
  {"name":"John","age":"30","city":"NY"},
  {"name":"Jane","age":"25","city":"LA"}
]
```

Understanding input formats is the first step toward building robust DataWeave transformations.

4.12.2 READ & WRITE FORMATS

MuleSoft from salesforce

DataWeave supports multiple data formats for reading input payloads and writing output payloads. Choosing the right format ensures seamless integration and optimal performance.

READ FORMATS (INPUT FORMATS) – PART 2

4. PROPERTIES

Example Input

```
host = localhost
port = 8080
env = dev
```

DataWeave

```
%dw 2.0
input payload properties
---
payload
```

Output (DataWeave Data)

```
{ "host": "localhost", "port": "8080", "env": "dev" }
```

5. BINARY

Example Input

```
(Binary file / Base64 encoded content)
```

DataWeave

```
%dw 2.0
input payload binary
---
payload
```

Output (DataWeave Data)

```
(Binary data as Base64 string)
"U29tZUJhc2U2NERhdGE="
```

6. TEXT

Example Input

```
Hello World!
Welcome to DataWeave.
```

DataWeave

```
%dw 2.0
input payload text
---
payload
```

Output (DataWeave Data)

```
"Hello World!\nWelcome to DataWeave."
```

7. YAML

Example Input

```
user:
  name: John
  age: 30
  city: New York
```

DataWeave

```
%dw 2.0
input payload yaml
---
payload
```

Output (DataWeave Data)

```
{ "user": { "name": "John", "age": "30",
  "city": "New York" } }
```

WRITE FORMATS (OUTPUT FORMATS)

1. JSON

DataWeave

```
%dw 2.0
output application/json
---
{ "name": "John", "age": 30 }
```

2. XML

DataWeave

```
%dw 2.0
output application/xml
---
{ "name": "John", "age": 30 }
```

3. CSV

DataWeave

```
%dw 2.0
output application/csv
---
[{"name": "John", "age": 30}]
```

4. PROPERTIES

DataWeave

```
%dw 2.0
output application/properties
---
{ "host": "localhost",
  "port": "8080" }
```

5. TEXT

DataWeave

```
%dw 2.0
output application/text
---
"Hello World!"
```

6. BINARY

DataWeave

```
%dw 2.0
output application/binary
---
(Binary data / Base64)
```

7. YAML

DataWeave

```
%dw 2.0
output application/yaml
---
{ "user": { "name": "John" } }
```

8. JAVA

DataWeave

```
%dw 2.0
output application/java
---
{ "name": "John", "age": 30 }
```

KEY TAKEAWAYS

- Use input payload <format> to read data in a specific format.
- Use output application/<format> to write data in the desired format.
- Choose the right format for better performance and maintainability.
- Leverage custom properties for fine-grained control.

Remember: Format names are case-insensitive.

4.12.3 READ & WRITE FORMATS

MuleSoft from salesforce

DataWeave supports multiple data formats for reading input payloads and writing output payloads.
Choosing the right format ensures seamless integration and optimal performance.

WRITE FORMATS (OUTPUT FORMATS) – PART 2

Format	Description	DataWeave Syntax (Output)	Example Output
JAVA	Serializes data as Java Objects.	%dw 2.0 output application/java --- payload	(Java Object representation of the payload) e.g. Person{name=John, age=30}
CSV	Serializes data into Comma-Separated Values.	%dw 2.0 output application/csv --- payload	name,age,city John,30,NY Jane,25,LA
PROPERTIES	Serializes data as Java Properties (key=value pairs).	%dw 2.0 output application/properties --- payload	host=localhost port=8080 env=dev
TEXT	Serializes data as plain text.	%dw 2.0 output application/text --- payload	Hello World! Welcome to DataWeave.
BINARY	Serializes data as binary (Base64 encoded).	%dw 2.0 output application/binary --- payload	QmluYXJ5IGRhdGEgYXMg QmFzZTY0IQ== (Base64 encoded content)
YAML	Serializes data as YAML Ain't Markup Language.	%dw 2.0 output application/yaml --- payload	user: name: John age: 30 city: New York

EXAMPLES SUMMARY (OUTPUT FORMATS)

1. JSON

```
%dw 2.0
output application/json
---
payload
```

Example Output

```
{ "name": "John",
  "age": 30, "city": "NY" }
```

2. XML

```
%dw 2.0
output application/xml
---
payload
```

Example Output

```
<user>
  <name>John</name>
  <age>30</age>
  <city>NY</city>
</user>
```

3. CSV

```
%dw 2.0
output application/csv
---
payload
```

Example Output

```
name,age,city
John,30,NY
Jane,25,LA
```

4. PROPERTIES

```
%dw 2.0
output application/properties
---
payload
```

Example Output

```
host=localhost
port=8080
env=dev
```

5. TEXT

```
%dw 2.0
output application/text
---
payload
```

Example Output

```
Hello World!
Welcome to DataWeave.
```

6. BINARY

```
%dw 2.0
output application/binary
---
payload
```

Example Output

```
QmluYXJ5IGRhdGEgYXMg
QmFzZTY0IQ==
(Base64 encoded content)
```

7. YAML

```
%dw 2.0
output application/yaml
---
payload
```

Example Output

```
user:
  name: John
  age: 30
  city: New York
```

8. JAVA

```
%dw 2.0
output application/java
---
payload
```

Example Output

```
(Java Object representation
of the payload)
e.g. Person{name=John, age=30}
```

KEY TAKEAWAYS

- Use output <format> to write data in the desired format.
- Choose the right format for interoperability.
- Custom properties help control serialization behavior.
- Consistent formats ensure smooth system integrations.

REMEMBER

- Format names are case-insensitive.
- Default write format is application/java.
- Test with sample payloads for accurate results.
- Refer to MuleSoft DataWeave docs for advanced configuration.

4.13.1 XML TRANSFORMATIONS

XML transformations in DataWeave help you convert, reshape and map XML data into the required structures. They ensure clean, consistent and integration-ready payloads for real-world applications.

OVERVIEW

- DataWeave makes XML transformations simple and powerful.
- It helps you extract, restructure, rename and enrich XML nodes and values.
- Supports filtering, sorting, aggregation and complex mappings.
- Produces valid, clean, namespace-aware XML output.

COMMON XML TRANSFORMATION TASKS

Element Mapping Map source elements to target elements	**Attribute Mapping** Map and create attributes	**Filtering** Filter elements or values
Restructuring Change hierarchy or structure	**Namespaces** Handle and declare namespaces	**Conditional Logic** Apply conditions in mapping
Aggregation Group and aggregate data	**Arrays & Repetition** Handle arrays and repeat elements	

SYNTAX OVERVIEW

Root Element
`< rootName ...> ... </ rootName >`

Child Element
`< childName ...> ... </ childName >`

Attribute
`@ attributeName = "value"`

Namespace
`ns default "uri"`

Filter
`filter (condition)`

Map Array
`array map ((item, index) -> ...)`

Conditional
`if (condition) ... else ...`

Text (Value)
`"text value" or value`

Always output well-formed XML with a single root element. Declare namespaces for accuracy.

EXAMPLES – XML TRANSFORMATIONS (BASIC)

1. Simple Element Mapping

Input (XML)

```
<book>
  <id>123</id>
  <title>DataWeave Guide</title>
</book>
```

DataWeave

```
output application/xml
---
{
  bookId: payload.book.id,
  name: payload.book.title
}
```

Output (XML)

```
<library>
  <bookId>123</bookId>
  <name>DataWeave Guide</name>
</library>
```

2. Attribute Mapping

Input (XML)

```
<user id="U1001">
  <name>John</name>
</user>
```

DataWeave

```
output application/xml
---
{
  user: {
    @id: payload.user.@id,
    fullName: payload.user.name
  }
}
```

Output (XML)

```
<customer id="U1001">
  <fullName>John</fullName>
</customer>
```

3. Filtering Elements

Input (XML)

```
<orders>
  <order id="1" status="NEW" />
  <order id="2" status="CLOSED" />
  <order id="3" status="NEW" />
</orders>
```

DataWeave

```
output application/xml
---
orders: payload.orders filter
    ($.status == "NEW")
```

Output (XML)

```
<orders>
  <order id="1" status="NEW" />
  <order id="3" status="NEW" />
</orders>
```

These basic transformations form the foundation for advanced XML mappings in DataWeave.

4.13.2 XML TRANSFORMATIONS

Part 2 of 3

EXAMPLES – XML TRANSFORMATIONS (CONTINUED)

4. Restructuring

Input (XML)

```
<employee>
  <name>John</name>
  <dept>IT</dept>
  <role>Dev</role>
</employee>
```

DataWeave

```
output application/xml
---
{
 staff: {
  "person": payload.employee.name,
  "department": payload.employee.dept,
  "designation": payload.employee.role
 }
}
```

Output (XML)

```
<staff>
  <person>John</person>
  <department>IT</department>
  <designation>Dev</designation>
</staff>
```

5. Namespaces

Input (XML)

```
<ns:root xmlns:ns="http://ns.com">
 <ns:item>Values</ns:item>
</ns:root>
```

DataWeave

```
output application/xml
---
ns default "http://ns.com"
---
{
  items: {
    item: "Values"
  }
}
```

Output (XML)

```
<items xmlns="http://ns.com">
 <item>Values</item>
</items>
```

Use namespaces to avoid name collisions and ensure XML validity.

6. Array Mapping

Input (XML)

```
<users>
 <user><name>Ac</name></user>
 <user><name>Bc</name></user>
</users>
```

DataWeave

```
output application/xml
---
{
  users:
    payload.users map ((u) -> {
      userName: u.name
    })
}
```

Output (XML)

```
<users>
  <userName>Ac</userName>
  <userName>Bc</userName>
</users>
```

Use map to transform each element in an array.

7. Conditional Mapping

Input (XML)

```
<user>
  <age>25</age>
  <status>Active</status>
</user>
```

DataWeave

```
output application/xml
---
<user>
  <age>{ payload.user.age }</age>
  <status>
   %if (payload.user.age >= 18)
     "Adult"
   %else
     "Minor"
   %end
  </status>
</user>
```

Output (XML)

```
<user>
  <age>25</age>
  <status>Adult</status>
</user>
```

Use conditions to apply business rules in mapping.

ADVANCED TRANSFORMATION EXAMPLES

8. Aggregation

Input (XML)

```
<sales>
  <item amount="100" />
  <item amount="200" />
  <item amount="150" />
</sales>
```

DataWeave

```
output application/xml
---
<summary>
  <total>{ (payload.sales.item.@amount
           default [1] sum as Number }</total>
  <count>{ sizeOf(payload.sales.item) }</count>
</summary>
```

Output (XML)

```
<summary>
  <total>450</total>
  <count>3</count>
</summary>
```

Use sum, count, sizeOf to perform aggregations on XML data.

9. Complex Transformation

Input (XML)

```
<orders>
  <order id="1" name="TV">
    <qty>2</qty> <price>300</price>
  </order>
  <order id="2" name="Table">
    <qty>1</qty> <price>120</price>
  </order>
</orders>
```

DataWeave

```
output application/xml
---
<invoice>
  <header>Invoice 001</header>
  <items>
    %for (o in payload.orders) {
      <product>{o.name}</product>
      <quantity>{o.qty}</quantity>
      <amount>{o.qty * o.price}</amount>
    }
  </items>
</invoice>
```

Output (XML)

```
<invoice>
  <header>Invoice 001</header>
  <items>
      <product>TV</product>
      <quantity>2</quantity>
      <amount>600</amount>
      <product>Table</product>
      <quantity>1</quantity>
      <amount>120</amount>
  </items>
</invoice>
```

Tips

- Use meaningful element names and attributes.
- Declare namespaces explicitly.
- Filter data early for better performance.
- Avoid deep nesting in XML whenever possible.
- Validate output XML using tools or schemas.
- Leverage DataWeave functions for mapping, filtering, aggregation and restructuring.

Remember!
Clean and well-structured XML leads to reliable integrations and easier maintenance.

Master XML transformations in DataWeave to build clean, reliable and integration-ready solutions!

4.13.3 XML TRANSFORMATIONS

Part 3 of 3

SYNTAX OVERVIEW

Item	DataWeave Syntax
Root Element	< rootName ... > ... </ rootName >
Child Element	< childName ... > ... </ childName >
Attribute	@ attributeName = "value"
Namespace	ns default "uri"
Filter	filter (condition)
Map Array	array map ((item, index) -> ...)
Conditional	if (condition) ... else ...
Text (Value)	"text value" or value

Always output well-formed XML with a single root element. Declare namespaces for accuracy and interoperability.

BEST PRACTICES

- Use clear and meaningful element and attribute names.
- Declare namespaces explicitly.
- Filter data early for better performance.
- Avoid deep nesting; keep XML structure clean.
- Validate output XML using tools or schemas.

KEY TAKEAWAYS

- XML transformations convert source data into structured, usable XML output.
- DataWeave provides powerful mapping, filtering, aggregation and restructuring.
- Handle namespaces and attributes carefully for accurate integrations.
- Clean and optimized transformations lead to reliable and performant solutions.

COMMON MISTAKES

- ✗ Forgetting the single root element.
- ✗ Not declaring namespaces (leading to invalid XML).
- ✗ Incorrect use of arrays and indexes.
- ✗ Missing attributes or wrong data types.
- ✗ Ignoring empty or null values.

QUICK REFERENCE

Element

Syntax
```
<name> ... </name>
```
Example
```
<book>
  <title>DataWeave
  Guide</title>
</book>
```

Attribute

Syntax
```
@name = "value"
```
Example
```
<book id="123">
  <title>Book</title>
</book>
```

Namespace

Syntax
```
ns default "uri"
```
Example
```
ns default "http://
example.com"
<root>
  <child/>
</root>
```

Filter

Syntax
```
filter (condition)
```
Example
```
orders filter
($.status ==
"NEW")
```

Map Array

Syntax
```
array map ((item,
index) -> ...)
```
Example
```
items map ((item) -> {
<item>{item.name}</item>
})
```

Conditional

Syntax
```
if (condition) ...
else ...
```
Example
```
if ($.age > 18)
  "Adult"
else
  "Minor"
```

Text Value

Syntax
```
"text" or value
```
Example
```
<name>{user.name}
</name>
```

FULL WORKED EXAMPLE - INVENTORY REPORT

Input (XML)
```
<inventory>
  <category name="Electronics">
    <product id="P1" name="TV" price="300" qty="2" />
    <product id="P2" name="Laptop" price="800" qty="1" />
  </category>
  <category name="Accessories">
    <product id="P3" name="Mouse" price="20" qty="5" />
  </category>
</inventory>
```

DataWeave
```
output application/xml
---
<report>
  <generatedDate>{now()}</generatedDate>
  <summary>
    <totalProducts>{payload..product.sizeOf()}</totalProducts>
    <totalValue>{payload..product map ((p) ->
      (p.@price as Number) * (p.@qty as Number))
       reduce ((acc, item) -> acc + item, 0)}
    </totalValue>
  </summary>
  <products>
    {payload..product map ((p) ->
      <product>
        <id>{p.@id}</id>
        <name>{p.@name}</name>
        <price>{p.@price}</price>
        <quantity>{p.@qty}</quantity>
        <value>{(p.@price as Number) * (p.@qty as Number)}</value>
      </product>
    )}
  </products>
</report>
```

Output (XML)
```
<report>
  <generatedDate>2024-05-16T10:30:00Z</generatedDate>
  <summary>
    <totalProducts>3</totalProducts>
    <totalValue>1460</totalValue>
  </summary>
  <products>
    <product>
      <id>P1</id>
      <name>TV</name>
      <price>300</price>
      <quantity>2</quantity>
      <value>600</value>
    </product>
    <product>
      <id>P2</id>
      <name>Laptop</name>
      <price>800</price>
      <quantity>1</quantity>
      <value>800</value>
    </product>
    <product>
      <id>P3</id>
      <name>Mouse</name>
      <price>20</price>
      <quantity>5</quantity>
      <value>100</value>
    </product>
  </products>
</report>
```

Note

..product selects all product elements depth-first.

- sizeOf() counts total products.
- reduce() calculates total value.
- map() builds the products list.

With the right techniques, XML transformations in DataWeave become clean, efficient and integration-ready!

3

4.14.1 CSV TRANSFORMATIONS

CSV (Comma Separated Values) transformations in DataWeave help you parse, write and manipulate delimited data. They are simple, lightweight and ideal for flat-file integrations and data exchange.

OVERVIEW

- CSV is a plain text format where values are separated by delimiters (default comma).
- DataWeave reads CSV data into arrays of objects.
- You can transform, filter, sort and reshape CSV data easily.
- DataWeave can also write data back to CSV using custom options.

COMMON CSV TRANSFORMATION TASKS

- Read / Parse CSV
- Write CSV
- Filter Rows
- Sort Data
- Select / Rename Columns
- Add / Remove Columns
- Join / Lookup Data
- Aggregate Data
- Group / Pivot Data
- Custom Delimiters & Options

SYNTAX OVERVIEW

Purpose	DataWeave Syntax
Read CSV	read(url, "application/csv")
Write CSV	write(payload, "application/csv")
Separator	separator : "|"
Header	header : true
Quote Character	quote : "\""
Escape Character	escape : "\""
Line Separator	lineSeparator : "\n"
Skip Header	skipHeader : true
Null Values	nullValue : ""
Encoding	UTF-8

Default settings:

- separator = ","
- header = true
- quote = "\""
- escape = "\""
- lineSeparator = "\n"

EXAMPLES - READ CSV (Parsing CSV to Data)

EXAMPLE INPUT (employees.csv)

```
id,name,dept,role,salary
1,John,IT,Developer,85000
2,Alice,HR,Analyst,60000
3,Bob,IT,Architect,105000
4,Carol,Finance,Manager,75000
```

1. Basic Read

```
%dw 2.0
output application/json
---
read(payload,"application/csv")
```

Output (JSON)

```
[
  {
    "id"    : "1",
    "name"  : "John",
    "dept"  : "IT",
    "role"  : "Developer",
    "salary": "85000"
  }
  ... more rows
]
```

2. Read Without Header

```
%dw 2.0
output application/json
---
read(payload,"application/csv",
 {
   header : false
 }
)
```

3. Custom Delimiter

```
%dw 2.0
output application/json
---
read(payload,
   "application/csv",
 {
   separator : "|"
 })
```

4. Skip Header Row

```
%dw 2.0
output application/json
---
read(payload,
   "application/csv",
 {
   skipHeader : true
 })
```

Reading CSV files is one of the most common DataWeave operations in MuleSoft integrations. Understanding headers, delimiters, and parsing options is essential for working with flat-file data.

4.14.2 CSV TRANSFORMATIONS

EXAMPLES – WRITE CSV (DATA To CSV)

1. Basic Write (Default)

DataWeave

```
%dw 2.0
output application/csv
---
payload
```

Input Data (Payload)

```
[
  {id: 1, name: "John",
   dept: "IT", role: "Dev",
   salary: 85000},
  {id: 2, name: "Alice",
   dept: "HR", role: "Analyst",
   salary: 60000}
]
```

Output (CSV)

```
id,name,dept,role,salary
1,John,IT,Dev,85000
2,Alice,HR,Analyst,60000
```

2. Write Without Header

DataWeave

```
%dw 2.0
output application/csv
header: false
---
payload
```

Output (CSV)

```
1,John,IT,Dev,85000
2,Alice,HR,Analyst,60000
```

Use header: false to remove the header row in the output CSV.

3. Custom Delimiter (|)

DataWeave

```
%dw 2.0
output application/csv
separator: "|"
---
payload
```

Output (CSV)

```
id|name|dept|role|salary
1|John|IT|Dev|85000
2|Alice|HR|Analyst|60000
```

Change separator to any character you need (e.g. |, ;, ~).

4. Custom Quote & Escape

DataWeave

```
%dw 2.0
output application/csv
quote: """
escape: "\\"
---
payload
```

Output (CSV)

```
'id','name','dept','role','salary'
'1','John','IT','Dev','85000'
'2','Alice','HR','Analyst','60000"
```

Customize quote and escape characters for special requirements.

COMMON TRANSFORMATIONS WITH CSV DATA

1. Filter Rows

DataWeave

```
%dw 2.0
output application/csv
---
payload filter ($.dept == "IT")
```

Output (CSV)

```
id,name,dept,role,salary
1,John,IT,Dev,85000
3,Bob,IT,Architect,105000
```

2. Sort By Salary (Desc)

DataWeave

```
%dw 2.0
output application/csv
---
payload orderBy (-$.salary)
```

Output (CSV)

```
id,name,dept,role,salary
3,Bob,IT,Architect,105000
1,John,IT,Dev,85000
4,Carol,Finance,Mgr,75000
2,Alice,HR,Analyst,60000
```

3. Select / Rename Columns

DataWeave

```
%dw 2.0
output application/csv
---
payload map {
  id: $.id,
  employee: $.name,
  department: $.dept,
  compensation: $.salary
}
```

Output (CSV)

```
id,employee,department,compensation
1,John,IT,85000
2,Alice,HR,60000
3,Bob,IT,105000
4,Carol,Finance,75000
```

4. Add / Remove Column

DataWeave

```
%dw 2.0
output application/csv
---
payload map (item) ->
 item ++ {
   bonus: item.salary * 0.1
 }
```

Output (CSV)

```
id,name,dept,role,salary,bonus
1,John,IT,Dev,85000,8500
2,Alice,HR,Analyst,60000,6000
3,Bob,IT,Architect,105000,10500
4,Carol,Finance,Mgr,75000,7500
```

5. Group / Aggregate

DataWeave

```
%dw 2.0
output application/csv
---
payload groupBy $.dept
map (dept, items) -> {
  department: dept,
  totalSalary: (items sum $.salary)
}
```

Output (CSV)

```
department,totalSalary
IT,190000
HR,60000
Finance,75000
```

CSV FORMAT OPTIONS

Option	Description	Example
separator	Field delimiter	"," or "|"
header	Print header row	true / false
quote	Quote character	" or '
escape	Escape character	"\\" or "\\"
lineSeparator	Line ending	"\n" or "\r\n"
nullValue	String for nulls	"" or "NULL"
skipHeader	Skip first row	true / false
encoding	Character encoding	UTF-8

BEST PRACTICES

- ✓ Always specify header: true unless not needed.
- ✓ Use explicit data types when doing calculations.
- ✓ Handle nulls and empty values using nullValue.
- ✓ Validate delimiters, quotes and escape settings.
- ✓ Use streaming (readUrl) for large CSV files.

COMMON MISTAKES

- ✗ Forgetting header: true leading to wrong field names.
- ✗ Using wrong delimiter causing parsing errors.
- ✗ Not handling quotes and escapes in text fields.
- ✗ Assuming all values are numbers or non-null.
- ✗ Writing large data without streaming or batching.

KEY TAKEAWAYS

- ★ CSV transformations in DataWeave are simple and efficient.
- ★ Read CSV into arrays of objects, transform and write back easily.
- ★ Flexible options help handle different CSV formats.
- ★ Good practices ensure clean, reliable and maintainable CSV integrations.

Transform flat files into valuable data with DataWeave CSV transformations!

4.14.3 CSV TRANSFORMATIONS

— ADVANCED TRANSFORMATIONS & REAL-WORLD EXAMPLES —

1. Advanced Transformations with CSV Data

A) Filter by Salary (> 70000)

DataWeave

```
%dw 2.0
output application/csv
---
payload filter
($.salary as Number > 70000)
```

Output (CSV)

```
id,name,dept,role,salary
1,John,IT,Developer,85000
3,Bob,IT,Architect,105000
```

B) Sort by Salary (Descending)

DataWeave

```
%dw 2.0
output application/csv
---
payload orderBy
(-$.salary as Number)
```

Output (CSV)

```
id,name,dept,role,salary
3,Bob,IT,Architect,105000
1,John,IT,Developer,85000
4,Carol,Finance,Manager,75000
2,Alice,HR,Analyst,60000
```

C) Select / Rename Columns

DataWeave

```
%dw 2.0
output application/csv
---
payload map {
  ID          : $.id,
  Employee    : $.name,
  Department  : $.dept,
  Compensation: $.salary
}
```

Output (CSV)

```
ID,Employee,Department,Compensation
1,John,IT,85000
2,Alice,HR,60000
3,Bob,IT,105000
4,Carol,Finance,75000
```

D) Add / Remove Column

DataWeave

```
%dw 2.0
output application/csv
---
payload map (item) -> {
 item ++ { Bonus: item.salary * 0.1 }
}
```

Output (CSV)

```
id,name,dept,role,salary,Bonus
1,John,IT,Developer,85000,8500
2,Alice,HR,Analyst,60000,6000
3,Bob,IT,Architect,105000,10500
4,Carol,Finance,Manager,75000,7500
```

E) Group / Aggregate by Department (Total Salary)

DataWeave

```
%dw 2.0
output application/csv
---
payload groupBy $.dept
map (dept, items) -> {
  Department: dept,
  TotalSalary: (items sum $.salary)
}
```

Output (CSV)

```
Department,TotalSalary
IT,190000
HR,60000
Finance,75000
```

2. CSV FORMAT OPTIONS (Reference)

Option	Description	Example
separator	Field delimiter	";", "|", ";"
header	Print header row	true / false
quote	Quote character	" "
escape	Escape character	"\"
lineSeparator	Line ending	"\n" or "\r\n"
nullValue	String for nulls	"" or "NULL"
skipHeader	Skip first row	true / false
encoding	Character encoding	UTF-8

3. REAL-WORLD SCENARIOS

- Import and transform flat files from legacy systems.
- Generate CSV reports for business users.
- Exchange data with partners using CSV files.
- Convert CSV data to JSON/XML for API processing.
- Cleanse, enrich and reshape CSV data for analytics.
- Load CSV data into databases or data lakes.

4. BEST PRACTICES

- Always specify header: true unless not needed.
- Use explicit data types when doing calculations.
- Handle nulls and empty values using nullValue.
- Validate delimiters, quotes and escape settings.
- Use streaming (readUrl) for large CSV files.
- Keep transformations simple and test with samples.

5. COMMON MISTAKES

- Forgetting header: true leading to wrong field names.
- Using wrong delimiter causing parsing errors.
- Not handling quotes and escapes in text fields.
- Assuming all values are numbers or non-null.
- Writing large data without streaming or batching.

6. QUICK REFERENCE

Task	DataWeave Example
Read CSV	read(url, "application/csv")
Write CSV	write(payload, "application/csv")
Filter Rows	payload filter (condition)
Sort Rows	payload orderBy ($.field)
Add Column	payload map (item) -> item ++ {...}
Group / Agg.	payload groupBy $.field
Select Columns	payload map { field1: $.a, ... }

7. KEY TAKEAWAYS

- CSV transformations in DataWeave are simple, powerful and efficient.
- Use read() to parse CSV into arrays and write() to export back to CSV.
- Advanced operations like filter, sort, group, aggregate and column manipulations are easy.
- Proper options and best practices ensure clean, reliable and scalable CSV integrations.

Master CSV transformations in DataWeave – Transform flat files into valuable data!

4.15 ADVANCED DATAWEAVE

Advanced DataWeave helps you solve complex integration challenges with reusable functions, user-defined types, modules, higher-order functions and advanced expression. Write concise, maintainable and high-performance transformations.

OVERVIEW

- Go beyond basic mapping with functions, modules and types.
- Build reusable, maintainable and testable transformations.
- Leverage higher-order functions for dynamic data handling.
- Handle complex data structures, logic and constraints.
- Improve performance and consistency using modular code and reusability.

ADVANCED CAPABILITIES

- User Defined Functions
- User Modules
- User Defined Types
- Lambda Expressions
- Higher-Order Functions
- Recursion
- Pattern Matching
- Conditional Expressions
- Namespaces & Imports
- Metadata Access
- Dynamic Keys
- Error Handling & Defaults

SYNTAX OVERVIEW

Function	Syntax (Example)
Import	`import * from dw::core::Strings`
Define function	`fun name(p1, p2, ...) = ...`
Lambda	`(item) -> item + 1`
User defined type	`type Name = { field: String, age: Number }`
Array of objects	`[{ name: "John", age: 30 }]`
Map (Object)	`{ key1: "value1", key2: "value2" }`
Default (null coalesce)	`payload.name default "Unknown"`
If / Else	`if (condition) "Yes" else "No"`
Match (pattern)	`match (payload) { case "A" -> 1 }`
Try / Catch	`try { ... } catch (e) { ... }`

EXAMPLES – ADVANCED PATTERNS

1. User Defined Function
```
fun toTitle(str) = str upper
```
2. Module Import & Use
```
import * from modules::strUtils
```
3. Lambda & Map
```
[1, 2, 3] map ((n) -> n * 2)
```
4. Higher-Order Function
```
payload filter ((n) -> n > 10)
```
5. Pattern Matching
```
match payload.type
case "user" -> "USER DATA"
case "admin" -> "ADMIN DATA"
else -> "UNKNOWN"
```
6. Try / Catch
```
try { payload map (n) -> n.id }
catch (e) { [] }
```

Write once, reuse everywhere. Modularize and simplify complex transformations.

BUILT-IN FUNCTIONS (Advanced)

Function	Description	Example
sizeOf()	Length of array/string	sizeOf([1,2,3]) -> 3
flatten()	Flatten nested arrays	flatten([1],[2]]) -> [1,2]
distinctBy()	Unique by key	distinctBy($, $.id)
groupBy()	Group by key	groupBy($, $.cat)
orderBy()	Sort by key	orderBy($, $.age)
filter()	Filter items	arr filter ($.age > 18)
pluck()	Extract field	arr pluck $.name

BEST PRACTICES

- Use functions and modules to avoid duplication.
- Keep business logic and mapping separate.
- Use proper types for better validation.
- Reuse built-in functions for writing concise code.
- Handle nulls and exceptions explicitly.
- Keep transformations modular and testable.

COMMON MISTAKES

- Parsing without specifying the correct pattern.
- Mixing up brackets and parentheses.
- Ignoring time zones and using server local time.
- Assuming all values are numbers or non-null.
- Not handling null or invalid data strings.

KEY TAKEAWAYS

- Advanced DataWeave enables powerful and reusable solutions.
- Use functions, types and modules to write clean, maintainable code.
- Leverage higher-order functions and pattern matching for flexible transformations.
- Modular code leads to consistent and high-performance integrations.

Master Advanced DataWeave – Build reusable, reliable and high-performance integrations!

4.16 ERROR HANDLING IN DATAWEAVE

DataWeave provides powerful constructs to handle errors gracefully, prevent runtime failures and return meaningful defaults or custom messages, ensuring resilient and robust integrations.

OVERVIEW

- Runtime errors happen when expressions fail due to invalid inputs, missing data, etc.
- Error handling improves flow reliability and ensures predictable outputs.
- Use built-in functions, try/catch, defaults and custom logic to handle errors.
- Keep transformations safe, clean and user-friendly.

TYPES OF RUNTIME ERRORS

Null Access
Occurs when a field or function on null is accessed.
Example: payload.name

Type Mismatch
Occurs when performed an incompatible data type.
Example: "abc" + 10

Invalid Cast
Cannot cast a value to the target type.
Example: "abc" as Number

Invalid Function Argument
Passing incorrect arguments to functions.
Example: sizeOf(null)

Other Runtime Errors
Index out of bounds, invalid regex, division by zero, etc.

SYNTAX OVERVIEW

Construct	Syntax / Example
Try / Catch	try { <expression> } catch (e) { <fallback> }
Try / Catch (with variable)	try { <expression> } catch (e) { <fallback> }
Default operator (when null)	<expression> default <value>
Default operator (when empty)	<expression> default <value> when "N/A"
Exists operator	is Number(payload.age) is String(payload.name)
Type check	type Name

TIP

- Prefer safe navigation (?) and defaults operator to avoid unnecessary errors.
- Use try/catch for complex logic that may fail.

EXAMPLES — TRY / CATCH

1. Basic Try / Catch

```
try {
  "Hello " ++ "John"
}
catch (e) {
  "Unknown"
}
```

Output
"Hello John"

2. Using Error Variable

```
try {
  payload.age as Number
}
catch (e) {
  "Error: " ++ e.message
}
```

Output
"Error: Cannot cast 'abc' to Number"

3. Nested Try / Catch

```
try {
  try { payload.date when
  is Number(payload.date)
  } catch (e) { 0 }
}
catch (e) { -1 }
```

Output
0 or -1 or number

4. Handling Function Errors

```
try {
  sizeOf(payload.items)
} catch (e) { 0 }

try {
  100 / sizeOf(payload.items)
} catch (e) { 0 }
```

Output
100 or 0 or use of array

5. Safe Navigation — Try / Catch

```
try {
  payload.order.customer?.name
} catch (e) {
  "N/A"
}
```

Output
"N/A" when anything fails

DEFAULTING FUNCTIONS

1. Default when Null
payload.name default "N/A"
Output: "N/A"

2. Default when Empty
payload.items default []
Output: [] (empty array)

3. Exists Operator
is Number(payload.age)
Output: true or false

4. Type Check
is Number(payload.age)
Output: true or false

CUSTOM ERROR MESSAGES

```
try {
  payload.order.total as Number
} catch (e) {
  error "VAL:01" : "Invalid total.
    Please provide a valid number."
}
```

Result
"Invalid total: Please provide a valid number."

COMMON MISTAKES

- ✗ Not handling errors leads to flow failures.
- ✗ Ignoring null and runtime errors.
- ✗ Using try/catch for simple validations.
- ✗ Returning wrong data types in fallbacks.
- ✗ Returning errors instead of using values.

BEST PRACTICES

- Prefer safe navigation (?) and default operator.
- Use try/catch only for complex operations or expressions.
- Handle nulls and exceptions explicitly.
- Log or propagate error messages when needed.

KEY TAKEAWAYS

- Advanced DataWeave enables powerful, resilient integrations.
- Use functions, modules, types and higher-order functions.
- Write clean, efficient and failure-ready transformations.
- Handle errors gracefully for robust and user-friendly flows.

Handle errors gracefully. Build resilient, reliable and user-friendly integrations with DataWeave!

5.1 MULE FLOW

MuleSoft from salesforce

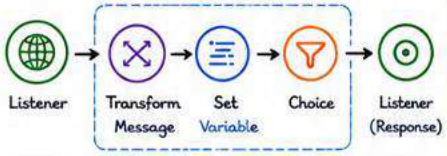
A Mule Flow is the fundamental building block of any Mule application. It defines a sequence of components that process incoming events to produce the desired output.

WHAT IS A FLOW?

- A flow is a message processing pipeline that starts with an event source and ends with an event.
- A flow contains a sequence of processors that act on the event.
- Each flow must have exactly one source.
- A flow can have zero or more message processors.
- A flow can have an optional error handling strategy.

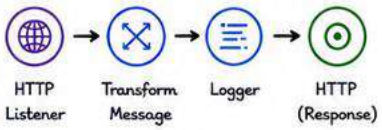
FLOW STRUCTURE

A typical Mule Flow consists of the following elements:



- **Source** : Entry point of the flow. It receives an event.
- **Processors** : Components that process/transform/filter/route the event.
- **End Event** : The event is sent back to the caller or another flow.
- **Error Handling** : Defines how errors in the flow will be handled.

SIMPLE FLOW EXAMPLE

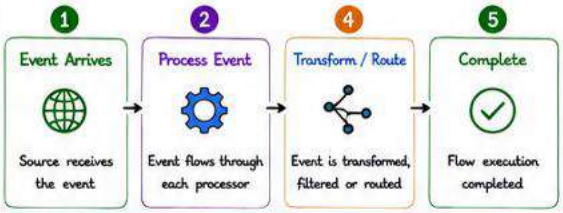


TIP

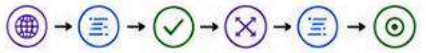
- Prefer safe navigation (?) and defaults operator to avoid unnecessary errors.
- Use try/catch for complex logic that may fail.

EVENT PROCESSING IN A FLOW

1. An event is received by the source (e.g., HTTP Listener).
2. The event passes through each processor in sequence.
3. Each processor can transform, filter, enrich, or route the event.
4. The event is returned by the end event (e.g., HTTP Response).
5. If any error occurs, the error handler is triggered.



FLOW EXAMPLE - REAL WORLD



Key Points:

- Flow starts with HTTP Listener.
- Order is validated.
- Transformation is done using DataWeave.
- Request is sent to external system.
- Response is logged and returned to caller.

ERROR HANDLING IN FLOW

A flow can have an error handler to manage exceptions.

You can define specific or global error handlers.

Common Errors Handled

- Null or missing data
- Invalid data type
- Invalid cast
- External system failure
- Timeout or connection error

BEST PRACTICES

- Keep flows short and focused on a single responsibility.
- Use meaningful names for flows and components.
- Use Logger components for monitoring and debugging.
- Handle errors gracefully using error handling strategies.
- Externalize configurations and endpoints.
- Reuse logic using Sub Flows or Flow References.

COMMON USE CASES

- Expose APIs
- Data Transformation
- System Integration
- Message Routing
- File / Data Processing

INTERVIEW QUESTIONS

1. What is a Mule Flow?
2. Can a flow have multiple sources?
3. How does event processing work in a flow?
4. What is the difference between a flow and a sub flow?
5. How do you handle errors in a flow?
6. Can a flow return a response? If yes, how?

QUICK REFERENCE

Flow	A sequence of components that processes an event.
Source	Entry point of the flow.
Processor	Components that perform actions on the event.
End Event	Returns the event to the caller.
Error Handler	Handles exceptions in the flow.

Flows are the heart of Mule applications - Keep them simple, reusable and reliable!

5.2 SUB FLOW

MuleSoft from salesforce

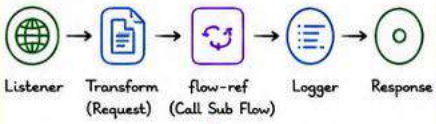
A Sub Flow is a reusable flow that can be called from any flow in the same application. It promotes reusability, modularity and maintainability of Mule applications.

WHAT IS A SUB FLOW?

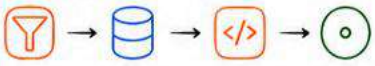
- A Sub Flow is a flow defined with the <sub-flow> tag.
- It can be invoked using the flow-ref component.
- It runs in the same thread and shares the same event.
- It cannot be called from another application.
- It does not have a source or listener.

HOW SUB FLOW WORKS

Main Flow (Calls Sub Flow)



Sub Flow



Execution returns to the calling point in the main flow after the sub flow completes.

SUB FLOW EXAMPLE

1) Main Flow
- HTTP Listener receives request.
- Calls Sub Flow using flow-ref.
- Logs response and returns to client.

2) Sub Flow Definition
- Validates order details.
- Performs database operation.
- Transforms data.
- Returns control to main flow.

FLOW INVOCATION USING flow-ref

Attribute	Description
name	Name of the Sub Flow to invoke
doc:name	Display name in Studio
variables	Optional. Pass variables to sub flow

VARIABLES IN SUB FLOW

- Sub Flow can access and modify flow variables.
- Variables passed using "variables" are available inside the sub flow.
- Changes made to variables in sub flow are visible to the calling flow.

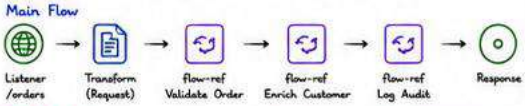
SUB FLOW vs PRIVATE FLOW

Aspect	Sub Flow	Private Flow
Visibility	Can be called from any flow in the same app	Can be called only within the same flow
Called using	flow-ref	flow-ref
Use case	Reuse logic across multiple flows	Break a large flow into smaller parts
Scope	Application level	Flow level (private)
Best for	Common business logic reuse	Improving readability of a single flow

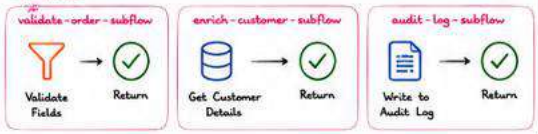
REAL WORLD EXAMPLE

Scenario: Validate Order, Enrich Customer & Log Audit

Main Flow



Sub Flows



SUB FLOW BEST PRACTICES

- Keep sub flows small, focused and single-purpose.
- Use meaningful names and documentation.
- Avoid complex transformations inside sub flows.
- Use variables to pass data clearly.
- Handle errors inside sub flows and return meaningful errors.
- Test sub flows independently.

COMMON USE CASES

- Reusable business logic (e.g., validation, enrichment)
- Database operations
- Common transformations
- Audit logging
- Security checks
- External service calls

ERROR HANDLING IN SUB FLOW

A sub flow can have an error handler to manage exceptions.

Define specific or global error handlers.

Common errors handled:
- Null or missing data
- Invalid data type
- Invalid cast
- External system failure
- Timeout or connection error

INTERVIEW QUESTIONS

1. What is a Sub Flow in Mule?
2. How do you invoke a Sub Flow?
3. Can a Sub Flow be called from another application?
4. How are variables shared between main flow and sub flow?
5. What is the difference between Sub Flow and Private Flow?
6. Can a Sub Flow have a listener?
7. What are the best practices for using Sub Flows?

QUICK REFERENCE

Concept	Description
Sub Flow	A reusable flow called from any flow in the same application.
Invoke Sub Flow	Using flow-ref component.
Variables	Use "variables" to pass data to sub flow.
Return	Control returns to main flow after sub flow completes.

Sub Flows make your integrations modular, reusable and easy to maintain!

5.3 PRIVATE FLOW

MuleSoft from salesforce

A Private Flow is a flow that is visible and callable only within the same Mule application.
It is similar to a Sub Flow but can contain its own sources or listeners.

WHAT IS A PRIVATE FLOW?

- A flow that is private to the current application.
- Can be called only from other flows in the same app.
- Can have a source (e.g., Listener).
- Not accessible from other applications.
- Ideal for breaking a large flow into smaller, reusable and independent triggers if needed.

PRIVATE FLOW CONCEPT

Main Flow

Private Flow

Private Flow can have its own source or be invoked only using flow-ref. It cannot be called from outside the app.

PRIVATE FLOW EXAMPLE

1) Main Flow → calls Private Flow
- Receives order request.
- Calls Private Flow using flow-ref.
- Logs response and returns to client.

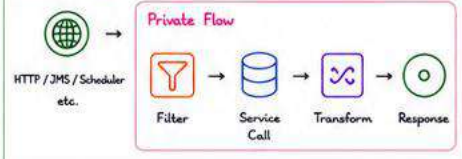
2) Private Flow → with Listener (optional)
- Processes order: validates, updates DB, sends email.
- Returns result to main flow.
- Not accessible from outside the application.

SUB FLOW vs PRIVATE FLOW

Aspect	Sub Flow	Private Flow
Visibility	Can be called from any flow in the app	Can be called only within the same app
Called using	flow-ref	flow-ref
Use case	Reuse logic across multiple flows	Break a large flow into smaller parts
Source	No source or listener	Can have source (Listener optional)
Scope	Application level	Application level (Private)
Best for	Common business logic reuse	Independent triggers, modularity & flexibility

PRIVATE FLOW WITH LISTENER (INDEPENDENT TRIGGER)

External Trigger



Notes
- A Private Flow with a listener behaves like any other flow when triggered.
- It is still private to the application and cannot be invoked from other apps.

WHEN TO USE PRIVATE FLOW?

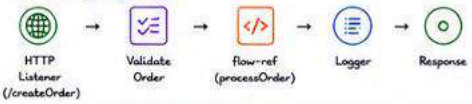
- When a flow needs to be invoked both internally and externally.
- When you want to encapsulate a complete process behind a listener.
- When you need to isolate business logic.
- When modularizing an application with reusable independent triggers.

Tip: Prefer Private Flow over Sub Flow when independent invocation is needed.

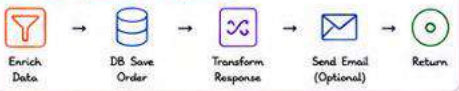
EXAMPLE: ORDER PROCESSING APPLICATION

Scenario: Create an order and send confirmation email.

Main Flow (Entry Point)



Private Flow (Reusable + Independent)



SUB FLOW BEST PRACTICES

- Keep private flows small, focused and single-purpose.
- Use meaningful names and documentation.
- Avoid complex transformations inside private flows.
- Use variables to pass data clearly.
- Handle errors inside private flows and return meaningful results.
- Test private flows independently.

COMMON USE CASES

- Reusable business logic (validation, enrichment)
- Database operations
- Common transformations
- Audit logging
- Security checks
- External service calls

ERROR HANDLING IN PRIVATE FLOW

Handle errors within the private flow and propagate custom error if needed.

Handle:
- Null or missing data
- Invalid data type
- Invalid cast
- External system failure
- Timeout or connection error

INTERVIEW QUESTIONS

1. What is a Private Flow in Mule?
2. How do you invoke a Private Flow?
3. Can a Private Flow be called from another application?
4. Can a Private Flow have a source or listener?
5. How are variables shared between main flow and private flow?
6. What is the difference between Sub Flow and Private Flow?
7. What are the best practices for using Private Flows?

QUICK REFERENCE

Concept	Description
Private Flow	A reusable flow visible and callable only within the same application.
Invoke Private Flow	Using flow-ref component.
Source / Listener	Optional. Makes the private flow independently triggerable.
Variables	Use "variables" to pass data between flows.
Return	Automatic return to caller flow after completion.

Private Flows give you flexibility and modularity with independent triggers!

5.4 FLOW REFFERENCE

MuleSoft from salesforce

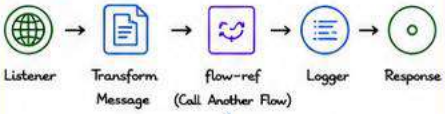
The flow-ref component is used to invoke a Sub Flow or a Private Flow. It promotes reusability, modularity and maintainability in Mule applications.

WHAT IS FLOW REFERENCE?

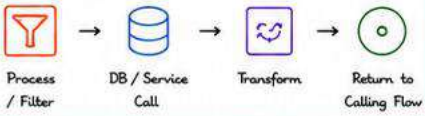
- flow-ref is a component used to call a Sub Flow or Private Flow.
- It passes the current Mule Event to the referenced flow.
- When the referenced flow completes, control returns to the calling flow.
- It helps in reusing business logic and keeping flows clean and manageable.

HOW FLOW-REF WORKS

Calling Flow



Referenced Flow (Sub Flow or Private Flow)



Execution jumps to the referenced flow and returns back after its completion with the event.

SYNTAX

Basic Syntax

```
<flow-ref name="flow-name" />
```

With Variables

```
<flow-ref name="flow-name">
 <variables>
  <variable key="var1" value="{payload.id}" />
  <variable key="var2" value="{vars.customer}" />
 </variables>
</flow-ref>
```

Attribute	Description
name	Name of the Sub Flow or Private Flow to invoke.
doc:name	Display name in Anypoint Studio.
variables	Optional variables passed to the referenced flow.

PASSING VARIABLES TO FLOW-REF

Example

- Pass variables like orderId, payload, or sourceSystem using <variables>.
- Access in the referenced flow using vars.orderId, vars.sourceSystem.

RETURNING DATA FROM REFERENCED FLOW

- The referenced flow can set payload, variables or attributes.
- When it completes, the event (with updated payload/variables) returns to the calling flow.
- No additional configuration is required to return data.

FLOW-REF WITH SUB FLOW vs PRIVATE FLOW

Aspect	Sub Flow	Private Flow
Visibility	Visible only in current app	Visible within the same app
Called using	flow-ref	flow-ref
Source / Listener	Not allowed	Allowed (optional)
Reuse Scope	Across multiple flows	Within the same flow
Best For	Common reusable logic	Break large flows into small parts

REAL WORLD EXAMPLE

Scenario: Place Order and Send Confirmation (using Sub Flow)

Main Flow (Entry Point)

Sub Flow: place-order-subflow

ERROR HANDLING WITH FLOW-REF

Errors in the referenced flow propagate back to the calling flow.

Handle using Try scope or Error Handler.

Common Errors:

- Flow not found
- Invalid variables
- Required variables missing
- Data transformation errors
- Business logic exceptions

BEST PRACTICES

- Use flow-ref to build reusable flows.
- Keep sub flows small and focused.
- Use meaningful names.
- Pass only necessary variables.
- Handle errors inside referenced flows and return meaningful messages.
- Avoid deep nesting of flow-refs.
- Prefer Private Flow for internal modularization.

WHEN TO USE PRIVATE FLOW?

- When a flow needs to be invoked both internally and externally.
- When you want to encapsulate a complete process behind a listener.
- When you need to isolate business logic.
- When modularizing an application with independent triggers.

Tip: Prefer Private Flow over Sub Flow when independent invocation is needed.

COMMON USE CASES

- Reusable business logic (validation, enrichment)
- Database operations
- Common transformations
- Audit logging
- Security / Authorization checks
- External service calls

QUICK REFERENCE

Concept	Description
flow-ref	Invokes a Sub Flow or Private Flow.
Sub Flow	Reusable flow with no source or listener.
Private Flow	Flow with optional source/listener; callable internally and externally.
Variables	Pass data between calling flow and referenced flow.
Return	Control returns to the calling flow after completion.

Use flow-ref to build reusable, modular and easy to maintain Mule applications!

5.5 LOGGER COMPONENT

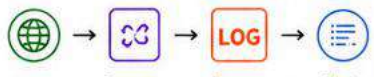
MuleSoft from salesforce

Logger component is used to log messages at various levels to monitor or debug Mule applications. It helps in troubleshooting and tracking flow execution, errors and data.

WHAT IS LOGGER COMPONENT?

- Logger writes messages to the specified log.
- It helps in monitoring, debugging and auditing.
- It does not change the message payload.
- It can log static text or dynamic data using DataWeave expressions.
- Available in all Mule editions.

HOW LOGGER WORKS



Logger listens to the flow and writes the provided message at the specified level without affecting the event.

BASIC SYNTAX

```
<logger level="LEVEL" message="MESSAGE" />
```

Where,

level : Logging level (INFO, DEBUG, ERROR, WARN, TRACE)

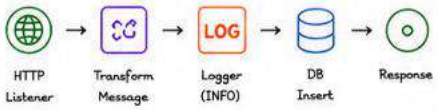
message : Message to be logged (String or DataWeave expression)

Attribute	Description
level	Specifies the log level.
message	The message to log. Can be static text or DataWeave expression.

LOG LEVELS

Level	Purpose
TRACE	Detailed diagnostic information
DEBUG	Detailed debug information
INFO	General operational messages
WARN	Warning messages
ERROR	Error messages

LOGGER IN A FLOW



Example

```
<logger level="INFO" message="Order received successfully!" />
```

LOGGER EXAMPLES

1. Static Message

```
<logger level="INFO"
  message="Order received successfully!" />
```

2. Dynamic Message Using DataWeave

```
<logger level="INFO"
  message="Order ID is " ++ payload.id
    ++ ", Customer: " ++ payload.customer />
```

3. Log Error Details

```
<logger level="ERROR"
  message="Error: " ++ vars.errorName
    ++ ", Action: " ++ vars.action />
```

LOG FILE PATTERN

Mule applications log to files in the following location:

`${MULE_HOME}/logs/<app-name>.log`

Sample Log Output

```
2024-05-20 10:15:20,123 INFO  [create-order-flow] Logger - Order created successfully
2024-05-20 10:15:20,456 DEBUG [create-order-flow] Logger - Payload after transformation
2024-05-20 10:15:20,789 ERROR [create-order-flow] Logger - Failed to create order: Duplicate key
```

BEST PRACTICES

- Use meaningful log messages.
- Use appropriate log levels.
- Avoid logging sensitive information.
- Use DataWeave to log only required information.
- Remove or lower log levels in production.
- Use correlation IDs to trace requests.
- Keep logger messages short and clear.

COMMON USE CASES

- Debugging and troubleshooting
- Monitoring flow execution
- Auditing business transactions
- Tracking request/response
- Performance analysis
- Error tracking and analysis

CONFIGURE LOGGER LEVEL (log4j2.xml)

You can control which levels are written to logs.

```
<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d [%t] %-5level %logger{36} - %msg%n" />
    </Console>
  </Appenders>
  <Loggers>
    <Root level="INFO" additivity="false">
      <AppenderRef ref="Console" />
    </Root>
  </Loggers>
</Configuration>
```

Only messages at the configured level and above will be written to the logs.

CONDITIONAL LOGGING

Log only when condition is true.

```
<choice>
  <when expression="#[payload.isError]">
    <logger level="ERROR"
      message="Logging error: " ++ payload.errorMsg />
  </when>
  <otherwise>
    <logger level="INFO"
      message="Processing completed successfully." />
  </otherwise>
</choice>
```

Logger does not impact the flow. It is a non-blocking component and has no output.

INTERVIEW QUESTIONS

1. What is Logger component in Mule?
2. What are the different log levels available in Mule?
3. How does Logger help in debugging?
4. Can Logger change the message payload?
5. How can you log dynamic content using Logger?
6. How can you control Logger levels?
7. How does Logger help in production environments?

QUICK REFERENCE

Element	Syntax / Example
Logger Component	`<logger level="INFO" message="Hello" />`
Log with Variable	`<logger level="DEBUG" message="ID: " ++ vars.id />`
Log Error	`<logger level="ERROR" message="Error: " ++ vars.error />`
Log with Condition	Use `<choice>` or `<when>` for conditional logging
Log File Location	`${MULE_HOME}/logs/<app-name>.log`

KEY TAKEAWAYS

- Logger component is used to log messages without impacting the flow.
- Choose proper log levels for effective monitoring and debugging.
- Use DataWeave for dynamic and meaningful log messages.
- Avoid logging sensitive data and unnecessary details.
- Logger is essential for building maintainable and production-ready applications.

Use Logger wisely to make your Mule applications easy to monitor, debug and support!

5.6 TRANSFORM MESSAGE

MuleSoft from salesforce

Transform Message component is used to transform the payload, attributes or variables using DataWeave 2.0. It is the most powerful way to shape and map data in Mule.

WHAT IS TRANSFORM MESSAGE?

- Uses DataWeave 2.0 for data transformation.
- Transforms payload, attributes or variables.
- Supports complex mappings, functions, filtering and formatting.
- Preferred over Set Payload for non-trivial transformations.

HOW IT WORKS

Source (Payload / Variables) → Input → Transform Message (DataWeave 2.0) → Output (Transformed Data) → Next Processor

DataWeave scripts read the input and produce the transformed output based on the defined mapping logic.

BASIC SYNTAX

```
%dw 2.0
output application/json
---
{
  key: value
  // use input payload
  otherKey: payload.id
}
```

Element	Description
%dw 2.0	Main Transform Message component.
output	Defines the output format.
---	Separates header from body.
payload	Refers to the input payload.

INPUT SOURCES IN DATAWEAVE

Input Object	Description
payload	The current message payload.
attributes	Attributes of the Mule event.
vars	Variables stored in the event.
flowVars	Flow variables.
sessionVars	Session variables (if used).
Mule	Mule functions.
Mule::uuid()	Generates UUID.

TRANSFORM MESSAGE EXAMPLES

1. Simple Payload Transformation

```
%dw 2.0
output application/json
---
{
  id: payload.id,
  name: payload.name,
  amount: payload.price as Number
}
```

2. Filtering and Mapping

```
%dw 2.0
output application/json
---
payload.orders map (item) -> {
  orderId: item.id,
  total: item.price * item.qty
}
```

3. Using Variables

```
%dw 2.0
output application/json
---
{
  user: vars.currentUserId,
  orderCount: sizeOf(payload.orders)
}
```

4. Creating Nested Structure

```
%dw 2.0
output application/json
---
{
  customer: {
    id: payload.id,
    details: payload.details
  }
}
```

SET ATTRIBUTES USING DATAWEAVE

```
<set-attributes value="#[{
  'correlationId': uuid(),
  'timestamp': now()
}]"
doc:name="Set Attributes Example"/>
```

```
%dw 2.0
output application/json
---
{
  correlationId: attributes.correlationId,
  timestamp: now()
}
// use in next steps
```

TRANSFORM MESSAGE IN A FLOW

HTTP Listener → Logger (Request) → Transform Message (DataWeave) → Logger (Response) → HTTP Response

Sample DataWeave Script

```
%dw 2.0
output application/json
---
{
  orderId: payload.id,
  customer: payload.customer.name,
  totalAmount: payload.amount,
  currency: payload.currency default "USD"
}
```

Input Payload

```
{
  "id": "ORD123",
  "customer": { "name": "John Doe" },
  "amount": 250,
  "currency": "USD", "qty": 2
}
```

Output Payload

```
{
  "orderId": "ORD123",
  "customer": "John Doe",
  "totalAmount": 250,
  "currency": "USD"
}
```

Key Points

- DataWeave script to format output structure.
- Supports complex mapping, functions and expressions.
- Keeps flows short and reusable.
- Powerful with DataWeave 2.0 capabilities.

BEST PRACTICES

- Use meaningful names for Transform component.
- Keep DataWeave scripts modular and reusable.
- Use variables for dynamic values.
- Handle null and defaults using default operator.
- Validate input before transforming.
- Add comments in DataWeave for complex logic.
- Prefer Transform Message over multiple Set Payloads.

COMMON USE CASES

- Data format conversion (JSON ⇄ XML ⇄ CSV)
- API request/response transformation
- Data enrichment (add, remove, rename fields)
- Filtering and aggregation
- Creating complex structures
- Mapping between systems

ERROR HANDLING

If DataWeave script has an error, it throws an exception.

Handle it using Error Handler.

INTERVIEW QUESTIONS

1. What is the purpose of Transform Message component?
2. What language is used for transformation?
3. How is Transform Message different from Set Payload?
4. What are input sources in DataWeave?
5. How do you use variables in DataWeave?
6. How do you handle null values in DataWeave?

QUICK REFERENCE

Item	Use
payload	Access message payload
attributes	Access message attributes
vars	Access variables
now()	Current date & time
uuid()	Generate unique ID
sizeOf()	Get size of array/string

Transform Message is the heart of data shaping in Mule.
Master DataWeave, master integration!

5.7 SET PAYLOAD

MuleSoft from salesforce

Set Payload component is used to set or replace the current message payload with a static value, DataWeave expression, variable, or the result of a function.

WHAT IS SET PAYLOAD?

- Sets or replaces the current message payload.
- Accepts static values, expressions, variables or functions.
- Useful for initializing, formatting or preparing data for next steps.
- Does not change attributes or variables.
- Important component in any Mule flow.

HOW SET PAYLOAD WORKS

Set Payload takes the provided value or expression and makes it the new payload for the event.

BASIC SYNTAX

```
<set-payload value="{expression}"
 mediaType="application/json" />
or
<set-payload>
  <set-payload value="{expression}" />
</set-payload>
```

Attribute	Description
value	The value or DataWeave expression to set as payload.
mediaType (optional)	MIME type of the payload (default: application/json).

TYPES OF PAYLOAD YOU CAN SET

Type	Description
Static Value	"Hello Mule"
DataWeave Expression	#[payload.name]
Variable	#[vars.var1]
Function / Operation	#[upper(payload)]
Object / Array / JSON	#[{ "id": 101, "name": "John" }]

SET PAYLOAD EXAMPLES (DATAWEAVE 2.0)

1. Set Static String
```
<set-payload value="Hello Mule" mediaType="text/plain" />
```
2. Set Number
```
<set-payload value="100" mediaType="application/json" />
```
3. Set Using DataWeave Expression
```
<set-payload value="#[payload.user.name]" />
```
4. Set JSON Object
```
<set-payload value="#[%dw 2.0
output application/json
---
{ status: "SUCCESS",
  message: "Payload set successfully" }]" />
```

SET PAYLOAD IN A FLOW

Example: DataWeave Payload
```
%dw 2.0
output application/json
---
{
  "id": payload.id,
  "name": payload.name,
  "status": "SUCCESS"
}
```

COMMON USE CASES

- Initializing payload with default values.
- Creating response structures.
- Preparing payload for next formats.
- Mapping payload for REST/API calls.
- Replacing current data with computed results.
- Returns custom response to client.

SET PAYLOAD WITH DIFFERENT DATA TYPES

Data Type	Example Expression	Result
String	"MuleSoft Developer"	"MuleSoft Developer"
Number	100	100
Boolean	true	true
Array	#[1,2,3,4]	[1,2,3,4]
Object	#[{name: "John", age: 30}]	{"name":"John","age":30}
Null	null	null

BEST PRACTICES

- Use DataWeave expressions for dynamic values.
- Keep payload small and structured.
- Use meaningful names for variables.
- Avoid hardcoding large JSON in flows.
- Validate payload before sending to external systems.

REAL WORLD EXAMPLE

Scenario: API response formatting

Input Payload
```
{
 "id": 101,
 "name": "John Doe",
 "email": "johndoexample.com"
}
```

Set Payload (Transform Response)
```
%dw 2.0
output application/json
---
{
  "id": "101",
  "name": "John Doe",
  "status": "Customer found"
}
```

Output Payload
```
{
 "id": "101",
 "name": "John Doe",
 "status": "Customer found"
}
```

SET PAYLOAD vs TRANSFORM MESSAGE

Aspect	Set Payload	Transform Message
Purpose	Replace payload only	Transform payload, attributes or variables
Attributes Support	No	Yes
Use Case	Simple payload update or initialization	Complex mapping and transformation
Performance	Slightly lighter	Slightly heavier
Best For	Quick payload changes	Complete data transformation

ERROR HANDLING

Set Payload throws an error in case the expression is invalid.

Example Error (Invalid Expression):
```
<set-payload value="#[payload.name.first]" />
```
Error:
EXPRESSION: Cannot resolve property 'name' on null

INTERVIEW QUESTIONS

1. What is the purpose of Set Payload component?
2. Can Set Payload replace the current payload?
3. What data types can be set as payload?
4. How do you set a JSON object as payload?
5. Difference between Set Payload and Transform Message?
6. When would you use Set Payload in a flow?
7. What happens if the expression in Set Payload is invalid?

QUICK REFERENCE

Concept	Syntax / Example
Set static string	`<set-payload value="Hello" />`
Set number	`<set-payload value="100" />`
Set using variable	`<set-payload value="#[vars.var1]" />`
Set JSON object	`<set-payload value="#[{id:101,name:'John'}]" />`
Set using DataWeave	`<set-payload value="#[upper(payload.name)]" />`
Set null	`<set-payload value="null" />`

Use Set Payload to shape, initialize and control your data flow effortlessly!

5.8 SET VARIABLE

MuleSoft from salesforce

Set Variable component is used to store data in variables for later use in the same flow or even acros flows. It helps in maintaining state and ensuring performance.

WHAT IS SET VARIABLE?

- Creates or updates a variable with specified value.
- Variables can be used anywhere in the same flow.
- Helps in storing information temporarily.
- Does not modify the payload.
- Supports DataWeave expressions for dynamic values.

HOW SET VARIABLE WORKS

The variable is stored in the event and can be accessed using vars.variableName.

BASIC SYNTAX

```
<set-variable variableName="varName"
  value="#(expression)" />
or
<set-variable variableName="varName"
  value="staticValue" />
```

Where,

Attribute	Description
variableName (required)	Name of the variable.
value (required)	Value or expression to assign to the variable.

EXAMPLE USAGE

1. Set a static value
```
<set-variable variableName="status"
value="SUCCESS" />
```
2. Set a value using DataWeave expression
```
<set-variable variableName="orderCount"
value="#[sizeOf(payload.orders)]" />
```
3. Set a variable using function result
```
<set-variable variableName="currentTime"
value="#[now()]" />
```

ACCESSING VARIABLES

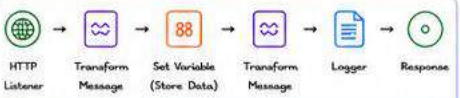
Variables are accessed using the vars object.

- vars.variableName // Use in expressions
- vars.variableName // Use in DataWeave

Example in Transform Message

```
%dw 2.0
output application/json
---
{
  orderId: vars.orderId,
  status: vars.status,
  time: vars.currentTime
}
```

SET VARIABLE IN A FLOW



Example: Store Support Info

```
%dw 2.0
output application/json
---
{
  "id": payload.id,
  "name": payload.name,
  "status": vars.status
}
```

TYPES OF VARIABLES

Type	Description
Flow Variable	Lives within the flow.
Session Variable	Lives acros multiple flows (if sessionVars used).
Inbound Properties	Read-only properties from the inbound request.

VARIABLE LIFECYCLE

- Created or updated when Set Variable is executed.
- Lives in the event until the flow completes.
- Not visible outside the flow unless shared (e.g., in Object Store or external system).
- Lost when the flow execution ends.

BEST PRACTICES

- Use meaningful and consistent variable names.
- Avoid storing large payloads in variables.
- Use variables for frequently used data.
- Clear variables if not needed further.
- Prefer Object Store for cross-flow persistence.

DATAWEAVE USAGE WITH VARIABLES

1. Using Variable in Expression
```
%dw 2.0
output application/json
---
{
  "userId": vars.userId,
  "orderCount": vars.orderCount
}
```

2. Using Variable in Flow
```
%dw 2.0
output application/json
---
{
  "createdAt": vars.currentTime,
  "status": vars.status
}
```

3. Using Variable as Condition
```
%dw 2.0
output application/json
---
{
  "isActive": vars.status == "SUCCESS"
}
```

COMMON USE CASES

- Store IDs from requests for later use
- Save configuration values
- Share results between steps
- Maintain flags or counters
- Pass calculated values to subsequent steps

REAL WORLD EXAMPLE

Scenario: Get customer details and status flag

SET VARIABLE vs SET PAYLOAD

Aspect	Set Variable	Set Payload
Purpose	Store data for later use	Replace entire payload
Affects Payload	No	Yes
Use Case	Share data inside flow	Tranaform or change output
Performance	Lightweight	Heavy if large payload
Best For	Temporary data storage	Complete data tranaformation

ERROR HANDLING

Set Variable does not throw an error.

Errors occur in the expression.

Example Error
```
<set-variable value="#[payload.name.first]" />
```
Error:
EXPRESSION: Cannot resolve property 'first' on null

INTERVIEW QUESTIONS

1. What is Set Variable component in Mule?
2. How does Set Variable differ from Set Payload?
3. Where are variables stored in Mule?
4. How do you access variables in DataWeave?
5. Can variables persist across flows?

QUICK REFERENCE

Action	Syntax / Example
Set static value	`<set-variable variableName="status" value="SUCCESS" />`
Set using expression	`<set-variable variableName="count" value="#[sizeOf(payload)]" />`
Access variable	`vars.variableName`
Set using function	`<set-variable variableName="currentTime" value="#[now()]" />`

KEY TAKEAWAYS

- Use Set Variable to store data and reuse it in the flow.
- Variables help maintain state and improve performance.
- Keep variables small and meaningful.
- Use clear names and good practices for maintainable flows.

Use Set Variable to store, reuse and control data in your Mule flows efficiently!

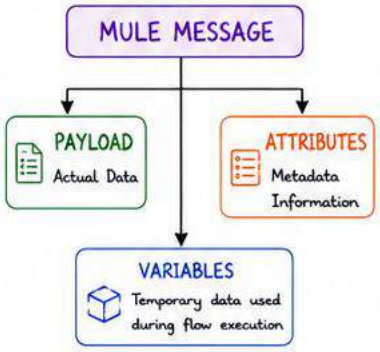
5.9 MULE MESSAGE STRUCTURE

MuleSoft from salesforce

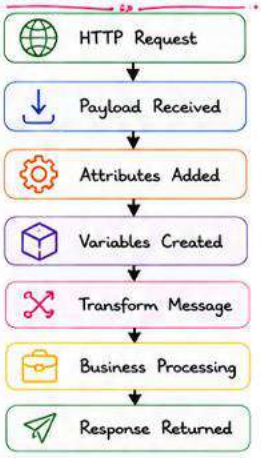
WHAT IS MULE MESSAGE?

- Mule Message is the data container used throughout a Mule Flow.
- It carries all the information required during processing.
- Every Mule event contains a Mule Message.
- Mule Message is immutable (changes create a new message internally).

MULE MESSAGE COMPONENTS



HOW MULE MESSAGE FLOWS



MESSAGE STRUCTURE OVERVIEW

PAYLOAD

Stores the actual business data.

Examples:

- JSON
- XML
- CSV
- Java Objects
- Text Files
- Binary Data

ATTRIBUTES

Stores metadata related to the message.

Examples:

- HTTP Method
- Query Parameters
- URI Parameters
- Headers
- File Information
- Status Codes

VARIABLES

Used to store temporary values.

Examples:

- Customer ID
- Order Number
- Authentication Token
- Processing Flags
- Calculated Values

COMMON OPERATIONS

Component	Purpose
Payload	Stores actual data
Attributes	Stores metadata
Variables	Stores temporary information
Transform Message	Modifies payload
Set Variable	Creates variables
Logger	Logs message details

PAYLOAD EXAMPLES

- **JSON Payload** — Customer Details
- **XML Payload** — Product Information
- **CSV Payload** — Employee Records
- **Java Object** — Application Data
- **Binary Payload** — PDF / Image Files

ATTRIBUTES EXAMPLES

HTTP Headers → Content-Type → Authorization Token → Request Path → Query Parameters → Response Status

VARIABLES USE CASES

- Store API responses.
- Maintain state across processors.
- Pass values between components.
- Store temporary calculations.
- Improve reusability of data.

REAL WORLD SCENARIOS

- **API Integrations** — Store: Customer Information, Order Details, Payment Information
- **Salesforce Integrations** — Store: Account IDs, Contact Details, Access Tokens
- **Database Operations** — Store: Query Results, Generated IDs, Processing Status

BEST PRACTICES

- Keep payload clean and meaningful.
- Use variables for temporary values.
- Do not store large payloads in variables.
- Transform payload only when required.
- Log important information carefully.
- Follow meaningful naming conventions.

COMMON MISTAKES

- Confusing payload with variables.
- Modifying attributes unnecessarily.
- Storing everything in variables.
- Logging sensitive information.
- Performing unnecessary payload transformations.

INTERVIEW QUESTIONS

1. What is Mule Message?
2. What are the components of Mule Message?
3. What is the difference between Payload and Attributes?
4. What are Variables used for?
5. Is Mule Message mutable or immutable?
6. How is Mule Message processed inside a Mule Flow?
7. Where should temporary values be stored?

QUICK REFERENCE

Component	Usage
Payload	Business Data
Attributes	Metadata
Variables	Temporary Values
Transform Message	Data Transformation
Set Variable	Store Values
Logger	Debugging

KEY TAKEAWAYS

- Mule Message is the heart of every Mule application.
- It consists of Payload, Attributes and Variables.
- Payload stores business data.
- Attributes store metadata.
- Variables store temporary values.
- Understanding Mule Message is essential for MuleSoft interviews and real-world integrations.

"Master Mule Message Structure to build clean, scalable and maintainable MuleSoft integrations."

5.10 CHOICE ROUTER

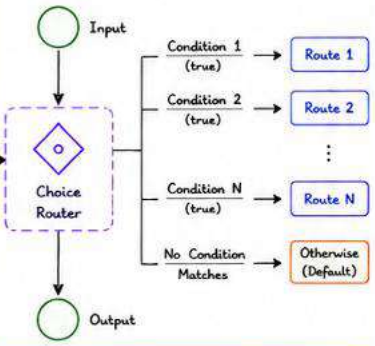
MuleSoft from salesforce

Choice Router routes the flow to one of the multiple routes based on the first condition that evaluates to true.
If no condition matches, the default route is executed (if defined).

WHAT IS CHOICE ROUTER?

- Evaluates conditions in sequential order.
- Routes to the first matching condition.
- Supports multiple when conditions and one otherwise (default) route.
- Useful for decision making based on message content.
- Only one route is executed per Choice.

HOW CHOICE ROUTER WORKS



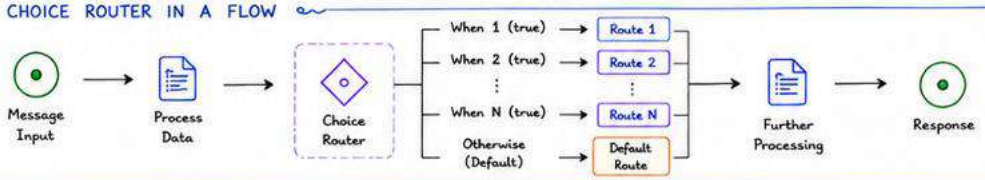
Conditions are evaluated in order.
Only the first condition that is true is executed.

BASIC SYNTAX

```
<choice>
  <when expression="#{expression1}">
    <!-- Route for condition 1 -->
  </when>
  <when expression="#{expression2}">
    <!-- Route for condition 2 -->
  </when>
  ...
  <otherwise>
    <!-- Default route -->
  </otherwise>
</choice>
```

Element	Description
choice	Defines the Choice router.
when	Defines a condition and its route.
expression	DataWeave expression that returns boolean.
otherwise	Default route if no condition matches.

CHOICE ROUTER IN A FLOW



EXAMPLE FLOW SNIPPET

```
"Flow: order-processing-flow"
- When payload.type = "EXPRESS"
  -> Route to "Processing Express Order"
- When payload.type = "STANDARD"
  -> Route to "Processing Standard Order"
- When payload.priority = "HIGH"
  -> Route to "Processing High Priority Order"
- Otherwise
  -> Route to "Processing Default Order"
```

EXAMPLE PAYLOADS & RESULTS

Payload	Evaluation	Route Executed
{ "type": "EXPRESS" }	payload.type == "EXPRESS" = true	Route 1
{ "type": "STANDARD" }	First = false Second = true	Route 2
{ "priority": "HIGH" }	First = false Second = false Third = true	Route 3
{ "type": "BULK" }	All conditions false	Default Route

DATAWEAVE EXPRESSION EXAMPLES

- #[payload.type == "EXPRESS"]
- #[payload.amount > 1000]
- #[attributes.queryParams.action == "create"]
- #[vars.isValid == true]
- #[sizeOf(payload.items) > 0]

CONFIGURATION OPTIONS

Option	Description
Order of when conditions	Conditions are evaluated top to bottom.
Otherwise (default)	Executed only if none of the when conditions are true.
Expression Type	Must return boolean (true/false).
Route Execution	Only one route is executed per Choice.

BEST PRACTICES

- Order conditions from most specific to least specific.
- Keep expressions simple and efficient.
- Always define an otherwise route for unexpected cases.
- Use meaningful logging in each route.
- Avoid overlapping conditions.

COMMON USE CASES

- Routing based on message type.
- Handling different customer tiers.
- Processing based on priority or status.
- Conditional flow for system calls.
- Data validation and branching logic.

MONITORING & LOGGING

- Log which route is executed.
- Monitor condition hits.
- Use custom metrics for analysis.
- Alert on unexpected default route executions.

ERROR HANDLING

Handle errors within each route using error-handler.

- Use On Error Continue / Propagate as required.

INTERVIEW QUESTIONS

1. What is Choice Router in MuleSoft?
2. How does Choice Router evaluate conditions?
3. What happens if multiple conditions are true?
4. What is the purpose of otherwise route?
5. How do you debug Choice Router logic?

QUICK REFERENCE

Task	Syntax / Example
Choice start	<choice>
When condition	<when expression="true">
Otherwise (default)	<otherwise>
End choice	</choice>

KEY TAKEAWAYS

- Choice Router makes smart routing decisions.
- First matching condition wins.
- Always include a default route.
- Helps build flexible and efficient Mule flows.

Use Choice Router to make the right decisions and build intelligent, efficient, and scalable Mule applications!

5.11 SCATTER GATHER

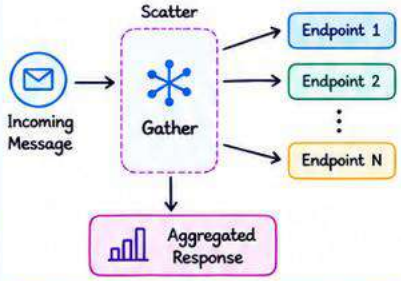
MuleSoft from salesforce

Scatter Gather allows you to send a message to multiple endpoints in parallel (scatter) and aggregate their responses (gather) into a single result.
It improves performance by parallel processing and combines results efficiently.

WHAT IS SCATTER GATHER?

- Sends a request to multiple endpoints concurrently.
- Waits for all responses and aggregates them.
- Improves performance by parallel execution.
- Supports configurable timeout and fail strategy.
- Useful for calling multiple services and combining results.

HOW SCATTER GATHER WORKS



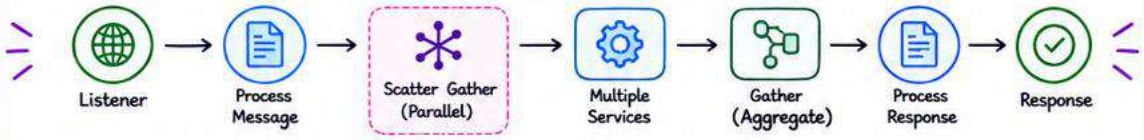
Requests are sent in parallel and responses are collected and merged.

BASIC SYNTAX

```
<scatter-gather>
  <route> <processor-chain/> </route>
  <route> <processor-chain/> </route>
  ...
  <aggregator>
    <collection-size>#[vars.size]</collection-size>
    <timeout>5000</timeout>
    <fail-strategy>FAIL</fail-strategy>
    <expression>#[payload]</expression>
  </aggregator>
</scatter-gather>
```

Element	Description
route	Defines a route to execute in parallel.
aggregator	Aggregates all route responses.
collection-size	Number of expected responses.
timeout	Max time to wait for responses.
fail-strategy	Action if any route fails.

SCATTER GATHER IN A FLOW



EXAMPLE FLOW SNIPPET

Routes execute in parallel:

1. Call Customer Service
2. Call Order Service
3. Call Inventory Service

↳ Responses are combined into a single payload.

AGGREGATOR EXPRESSION EXAMPLES

Expression	Result
#[payload]	List of all payloads
#[payload joinBy ","]	Comma separated string
#[payload map (item) -> item.id]	List of IDs
#[payload reduce (acc, item) -> acc + item.total]	Sum of all totals

FAIL STRATEGY OPTIONS

Strategy	Description
FAIL	Fail if any route fails.
IGNORE	Ignore failed routes.
RETURN_PARTIAL	Return successful responses only.

CONFIGURATION OPTIONS

Option	Description
maxConcurrency	Max parallel calls.
timeout	Max wait time for responses.
collectionSize	Expected number of responses.
routeingStrategy	ROUND_ROBIN or FIRST_COMPLETED.

BEST PRACTICES

- Use for independent service calls.
- Set appropriate timeouts.
- Choose the right fail strategy.
- Limit max concurrency.
- Keep aggregation logic simple.

COMMON USE CASES

- Calling multiple services in parallel
- Aggregating data from multiple sources
- Improving response time of composite APIs
- Dashboard or report data aggregation
- Collecting results from external systems

INTERVIEW QUESTIONS

1. What is Scatter Gather in MuleSoft?
2. How does it improve performance?
3. What are the fail strategy options?
4. What is the difference between ROUND_ROBIN and FIRST_COMPLETED?
5. How do you aggregate responses?

QUICK REFERENCE

Task	Syntax / Example
Scatter start	<scatter-gather>
Add a route	<route>...</route>
Set timeout	<timeout>5000</timeout>
Set fail strategy	<fail-strategy>FAIL</fail-strategy>
End scatter	</scatter-gather>

KEY TAKEAWAYS

- Enables parallel execution of routes.
- Combines multiple responses efficiently.
- Provides better performance & scalability.
- Use with proper timeouts and fail strategy.
- Ideal for composite and integration scenarios.

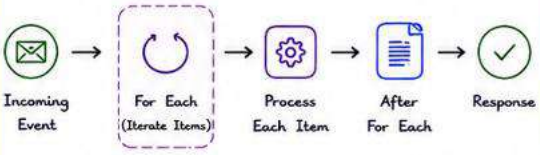
Use Scatter Gather to make the right decisions and build efficient, scalable and high performance Mule applications!

5.12.1 FOR EACH

MuleSoft from salesforce

For Each scope is used to iterate over a collection (Array, List, Number, Cursor, etc.) and process each item sequentially.
It improves reusability and keeps flows simple and easy to maintain.

HOW FOR EACH WORKS



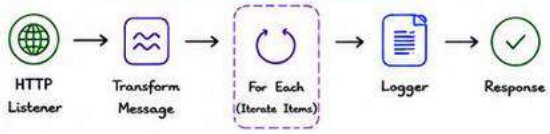
The For Each scope iterates through each item in the collection, executes the enclosed processors, and continues until all items have been processed.

BASIC SYNTAX

```
<for-each collection="#[payload]" item="item" index="index">
  <!-- Process Each Item -->
  <logger level="INFO" message="Processing #[item]
    at index #[index]"/>
  <!-- More Processors -->
</for-each>
```

Attribute	Description
collection (required)	The collection to iterate over.
item (optional)	Name of the variable that holds the current item. Default: "item"
index (optional)	Name of the variable that holds the current index. Default: "index"
varName (optional)	Name of the For Each scope.

FOR EACH IN A FLOW



Example Flow Snippet

```
<for-each collection="#[payload]" item="item" index="index">
  <logger level="INFO" message="Item #[item] at index #[index]"/>
  <!-- Process each item -->
</for-each>
```

AVAILABLE VARIABLES INSIDE FOR EACH

Variable	Description
item (default)	The current item in the collection.
index (default)	The current index (starting from 0).
vars	All user-defined variables.
vars.size	Total number of items in the collection.
vars.current	Reference flag for For Each status.

COLLECTION TYPES SUPPORTED

- Array
- List
- Set
- Map (iterator over values)
- Stream
- Cursor (with streaming)

Note: If the payload is not a collection, MuleSoft treats it as a single item collection and executes the scope once.

OUTPUT BEHAVIOR

By default, For Each returns a collection of results produced by each iteration.

Scenario	Result
Inner processor returns a value	List of all returned values.
Inner processor does not return payload	Original payload is returned.
Inner processor returns null	List of nulls is returned.
No. of input payloads match for Each	Overrides item count.

EXAMPLE

Input:
["A", "B", "C"]
Each item processed
↓
Output:
["Result A", "Result B", "Result C"]

BEST PRACTICES

- Use meaningful variable names (item, index).
- Prefer streaming mode for large collections.
- Avoid heavy operations inside the loop.
- Handle error scenarios inside the loop.
- Keep inner flow lightweight.

COMMON USE CASES

- Processing items from arrays or lists.
- Calling APIs for each item.
- Sending emails or validating each record.
- Inserting or updating multiple records.

INTERVIEW QUESTIONS

1. What is the purpose of For Each in MuleSoft?
2. How does For Each handle incoming payload?
3. What variables are available inside For Each?
4. What is the difference between For Each and Map?
5. What are the collection types supported?

KEY TAKEAWAY

For Each helps you iterate over collections sequentially, process each item, and optionally collect results.

Use it to build simple, reusable and efficient Mule flows!

Use For Each to iterate, process, and achieve more – one item at a time!

5.13 PARALLEL FOR EACH

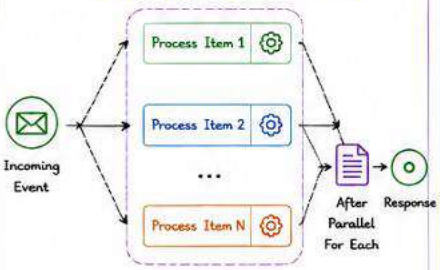
MuleSoft from salesforce

Parallel For Each scope processes items in a collection concurrently. It improves performance by leveraging multiple threads, while keeping the flow logic simple and reusable.

WHAT IS PARALLEL FOR EACH?

- Iterates over each item in a collection.
- All items are processed concurrently.
- Each item is handled in a separate thread.
- Improves throughput and reduces overall time.
- Maintains variables like index and size.
- Returns a collection of results once all threads complete.
- Best for independent, non-blocking operations.

HOW PARALLEL FOR EACH WORKS



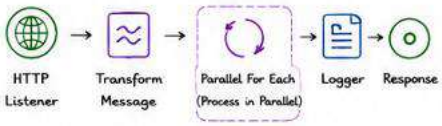
All items are processed concurrently. Once all threads complete, results are aggregated and the flow continues.

BASIC SYNTAX

```
<parallel-for-each collection="#[payload]"
  maxConcurrency="value"
  waitForAll="true">
  <!-- Process / Components -->
</parallel-for-each>
```

Attribute	Description
collection	The collection to iterate over.
maxConcurrency	Max number of threads to run in parallel.
waitForAll	Wait until all threads complete.
maxConcurrency (optional)	Default number of threads = 2

PARALLEL FOR EACH IN A FLOW



Sample Flow Snippet

```
<parallel-for-each collection="#[payload.items]"
  maxConcurrency="5" waitForAll="true">
  <!-- Process each item in parallel -->
</parallel-for-each>
```

AVAILABLE VARIABLES INSIDE PARALLEL FOR EACH

Variable	Description
item (default)	The current item in the collection.
index (default)	The current index (starting from 0).
vars	All user-defined variables.
vars.size	Total number of items in collection.
vars.current	Reference flag for Parallel For Each.

Example Usage

- item.id → Current item
- index → Current index
- vars.size → Total items

COLLECTION TYPES SUPPORTED

Parallel For Each works with:

- Array
- List
- Set
- Map (iterator over values)
- Iterator
- Cursor (streaming supported)

Note: If payload is not a collection, MuleSoft treats it as a single item collection and executes the scope once.

CONFIGURATION OPTIONS

Attribute	Description
maxConcurrency	Limits the number of threads in parallel.
timeout	Max time to wait for all threads.
failStrategy	Action when a thread fails.
maxConcurrency (default)	Default threads = 2

RESULTS AGGREGATION

By default, results from each iteration are aggregated into a List.

Example Output (Aggregated Result):

```
[ "Item 1", "Item 2", "Item 3",
  "Item 4", "Item 5" ]
```

If any thread fails, overall result depends on the configured failStrategy.

STREAMING SUPPORT

For large collections, enable streaming to avoid loading all items in memory.

```
<parallel-for-each collection="#[payload]"
  streaming="true" maxConcurrency="5" >
  <!-- Process items in streaming mode -->
</parallel-for-each>
```

Streaming is recommended for large datasets or low memory scenarios.

EXAMPLES

1. Iterate Over Array
Input : ["A", "B", "C"]
Output : List of results (Each item processed in parallel)

2. Iterate With Index
Input : [10, 20, 30]
Output : index → 0, 1, 2
item → 10, 20, 30

3. Iterate Over Map
Input : {"a":1, "b":2}
Output : Processes values (1, 2) in parallel

4. Streaming Large Data
Input : Large Collection
Output : Memory efficient streaming

5. Iterate Over Cursor
Input : Database Cursor
Output : Records processed concurrently

COMMON USE CASES

- Processing large datasets.
- Calling multiple APIs in parallel.
- Data aggregation.
- Batch processing.

BEST PRACTICES

- Use meaningful variable names.
- Keep operations independent.
- Prefer streaming for large data.
- Handle errors in each thread.

ERROR HANDLING

Handle errors inside the Parallel For Each scope to avoid stopping the entire flow.

PARALLEL vs FOR EACH

Aspect	For Each	Parallel For Each
Execution	Sequential	Concurrent
Performance	Slower	Faster
Use Case	Small Data	Large Data
Threads	Single Thread	Multiple Threads

KEY TAKEAWAY

Parallel For Each helps you process collections faster by using multiple threads. Use it for better performance and scalability!

Use Parallel For Each to process more, in less time – safely and efficiently!

5.14 UNTIL SUCCESSFUL

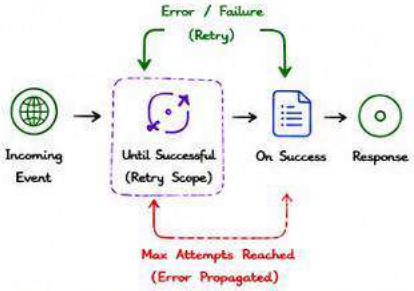
MuleSoft from salesforce

Until Successful scope repeatedly executes the enclosed processor until it completes successfully, fails max retries or until the maximum number of attempts is reached. It is ideal for handling transient errors.

WHAT IS UNTIL SUCCESSFUL?

- Retries the enclosed processor until it succeeds.
- Handles transient errors automatically.
- Configurable max attempts and retry interval between tries.
- Optional exponential backoff for increasing delays.
- If all attempts fail, the error is propagated.

HOW UNTIL SUCCESSFUL WORKS



The enclosed processor is executed until it succeeds or the maximum number of attempts is reached.

BASIC SYNTAX

```
<until-successful maxRetries="3"
    millisBetweenRetries="2000"
    exponentialBackoff="true">
    <!-- Flow / Components -->
    <processor .... />
    <error-handler ref="POST_path" />
</until-successful>
```

Attribute	Description
maxRetries (required)	Maximum number of retry attempts.
millisBetweenRetries (required)	Wait time between retries in milliseconds.
exponentialBackoff (optional)	Enable exponential backoff (true/false).
backoffFactor (optional)	Multiplier for wait time when backoff is enabled.

UNTIL SUCCESSFUL IN A FLOW

Sample Flow Snippet

```
<until-successful maxRetries="3" millisBetweenRetries="2000"
    exponentialBackoff="true" backoffFactor="2.0">
    <!-- Your processors here -->
    <error-handler ref="POST_path" />
</until-successful>
```

CONFIUGRATION OPTIONS

Option	Description
maxRetries	Number of retry attempts after failure.
millisBetweenRetries	Initial wait time between retries (in ms).
exponentialBackoff	Enables increasing wait time after each retry.
backoffFactor	Multiplier for wait time when backoff is enabled.

Example (BackoffFactor = 2.0)

- Attempt 1 → Wait 2000 ms
- Attempt 2 → Wait 4000 ms
- Attempt 3 → Wait 8000 ms
- Attempt 4 → Wait 16000 ms
- ...

HOW RETRIES WORK

1. Enclosed processor is executed.
2. If an error occurs, retrying starts after wait time.
3. If it succeeds, Mule exits the scope.
4. If retries exceed maxRetries, error is thrown.
5. Response is returned.

ERROR HANDLING

If all retry attempts fail, the original error is raised and can be handled by error-handler.
Use <error-handler> inside the scope.

COLLECTION TYPES SUPPORTED

Until Successful can be used with:

- Array (fixed size)
- List
- Set
- Map (fixed size)
- Object Store
- Custom business logic

COMMON USE CASES

1. Retrying external API calls.
2. Handling temporary database connectivity issues.
3. Dealing with network timeouts.
4. Processing messages with transient failures.
5. Importing/validating external data.

BEST PRACTICES

- Use for transient errors only.
- Set appropriate maxRetries and wait times.
- Use exponential backoff to reduce load.
- Log each attempt for better observability.

OUTPUT BEHAVIOR

On Success

The result of the successful execution is returned.

On Failure

The error from the last attempt is propagated.

EXAMPLES

1. Retrying API Call

Retry API call until success or max retries reached.

2. Database Operation

Retry DB operation on deadlock or connection issues.

3. File Processing

Retry file read/write if file is temporarily locked.

4. Message Processing

Retry message processing on transient failures from external systems.

INTERVIEW QUESTIONS

1. What is Until Successful in MuleSoft?
2. How does Until Successful work?
3. What is maxRetries used for?
4. What is the use of exponential backoff?
5. How is error handled in Until Successful?

QUICK REFERENCE

Item	Purpose
maxRetries	Max retry attempts after failure
millisBetweenRetries	Wait time between retries
exponentialBackoff	Increase wait time after each retry
backoffFactor	Multiplier for backoff wait time

KEY TAKEAWAY

Until Successful helps you handle temporary failures gracefully by retrying automatically until success or until the configured limit. Use it wisely to build reliable and resilient Mule flows!

Use Until Successful to build resilient and reliable Mule flows!

5.15 ASYNC SCOPE

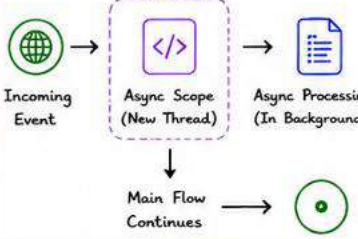
MuleSoft from salesforce

Async Scope processes events asynchronously in a separate thread, without blocking the main flow.
It improves responsiveness and enables parallel execution of independent tasks.

WHAT IS ASYNC SCOPE?

- Executes a set of processors asynchronously.
- Does not block the main flow.
- Improves system responsiveness.
- Useful for fire-and-forget tasks (logging, notifications, audit trails).
- Errors can be handled within the scope.

HOW ASYNC SCOPE WORKS



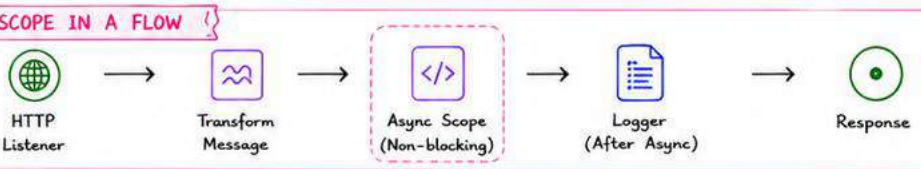
The Async Scope runs in a separate thread.
The main flow continues immediately.

BASIC SYNTAX

```
<async>
  <async-executor name="myExecutor"
                  maxThreads="10" />
  <!-- Flow / Components -->
  <logger level="INFO"
          message="Processing
          in async scope" />
</async>
```

Attribute	Description
async-executor	Reference to the async executor configuration.
maxThreads	Maximum number of threads in the pool.

ASYNC SCOPE IN A FLOW



ERROR HANDLING

Handle errors inside Async Scope using on-error-continue to prevent them from affecting the main flow.

```
<async>
  <error-handler>
    <on-error-continue type="ANY">
      <logger level="ERROR"
        message="Async error: #[error.description]"/>
    </on-error-continue>
  </error-handler>
  <!-- Processors -->
</async>
```

COMMON USE CASES

- Sending emails or notifications
- Writing logs or audit records
- Updating external systems
- Triggering reporting or analytics
- Fire-and-forget operations

EXAMPLE FLOW

```
<flow name="order-processing-flow">
  <http:listener config-ref="HTTP_Listener_config"
      path="/order" />
  <transform message="#[payload]" />
  <async>
    <async-executor name="asyncExecutor"
      maxThreads="10" />
    <logger level="INFO" message="Processing order
      async: #[payload.orderId]" />
    <db:insert config-ref="DB_Config" />
    <email:send to="user@example.com" />
  </async>
  <logger level="INFO" message="Order request accepted" />
</flow>
```

ASYNC EXECUTOR CONFIG EXAMPLE

```
<async:config name="Async_Config" doc:name="Async Config">
  <async:async-executor name="asyncExecutor"
                        maxThreads="10"
                        keepAliveTime="60"
                        queueCapacity="100" />
</async:config>
```

BEST PRACTICES

- Use Async Scope for non-blocking, independent tasks.
- Handle errors within the Async Scope.
- Configure thread pool (maxThreads) based on load.
- Avoid long-running or blocking operations.
- Ensure idempotent operations when needed.

OUTPUT BEHAVIOR

On Success:
Main flow continues and response is returned.

On Failure:
Error is handled inside the Async Scope and does not impact the main flow.

INTERVIEW QUESTIONS

1. What is Async Scope in MuleSoft?
2. How does Async Scope differ from a normal flow?
3. When should you use Async Scope?
4. How are errors handled in Async Scope?
5. How do you configure Async Executor?

QUICK REFERENCE

Component	Purpose
Async Scope	Executes tasks asynchronously (non-blocking)
Async Executor	Manages thread pool for Async Scope
maxThreads	Maximum number of threads in the pool
on-error-continue	Handles errors within Async Scope

Async Scope helps you build responsive, scalable and high-performing Mule applications!

Use Async Scope to run tasks in the background and keep your flows responsive!

5.16 BATCH PROCESSING

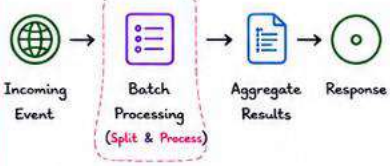
MuleSoft from salesforce

Batch Processing scope divides large payloads into smaller batches and processes them in chunks.
It improves performance, memory usage, and reliability for bulk data operations.

WHAT IS BATCH PROCESSING?

- Splits large payloads into smaller batches.
- Processes each batch sequentially.
- Reduces memory consumption.
- Handles bulk data efficiently.
- Supports error handling, aggregation, and custom batch sizes.

HOW BATCH PROCESSING WORKS



Data is split into batches → Each batch is processed → Results are aggregated → Final response is returned.

BASIC SYNTAX

```
<batch:job jobName="batchJob"
        maxFailedRecords="1">
  <batch:process-records>
    <!-- Process each record -->
  </batch:process-records >
  <batch:aggregator>
    <!-- Aggregate results -->
  </batch:aggregator>
</batch:job>
```

Attribute	Description
jobName	Name of the batch job.
maxFailedRecords	Max number of records that can fail. (-1 = no limit)
batchSize	Number of records per batch.

BATCH PROCESSING IN A FLOW

SAMPLE FLOW SNIPPET

```
<flow name="bulk-processing-flow">
  <http:listener config-ref="HTTP_Listener_config" path="/processBulk"/>
  <set-payload value="#[payload]"/>
  <batch:job jobName="batchJob" maxFailedRecords="1"
             batchSize="100">
    <batch:process-records>
      <logger level="INFO" message="Processing item: #[payload.id]"/>
      <!-- Business logic or DB operations -->
    </batch:process-records>
    <batch:aggregator>
      <logger level="INFO" message="Batch processing completed"/>
    </batch:aggregator>
  </batch:job>
  <logger level="INFO" message="Bulk processing completed"/>
</flow>
```

CONFIGURATION OPTIONS

Option	Description
batchSize	Number of records in each batch.
maxFailedRecords	Maximum number of failed records allowed. (-1 = no limit)
continueOnError	Whether to continue processing after an error. (true/false)
target	Target component for batch aggregation.

COMMON USE CASES

- Processing large CSV or JSON files
- Bulk database inserts/updates
- Data migration and ETL jobs
- Bulk API integrations
- Mass email or notification processing

ERROR HANDLING

Handle errors using maxFailedRecords and on-error-continue.

```
<batch:job jobName="batchJob" maxFailedRecords="1">
  <batch:process-records>
    <logger level="ERROR"
        message="Error processing item: #[error.description]"/>
  </batch:process-records>
</batch:job>
```

BEST PRACTICES

- Choose an optimal batch size.
- Use continueOnError wisely.
- Monitor memory and performance.
- Aggregate results efficiently.
- Log failed records for troubleshooting.

INTERVIEW QUESTIONS

1. What is Batch Processing in MuleSoft?
2. How does Batch Processing work?
3. What is the purpose of batchSize?
4. How do you handle errors in Batch Processing?
5. How do you aggregate batch results?

KEY TAKEAWAY

Batch Processing helps you handle large volumes of data efficiently by processing them in smaller chunks, reducing memory usage and improving performance.

Use Batch Processing to handle large data sets efficiently and reliably!

5.17 PARALLEL FOR EACH

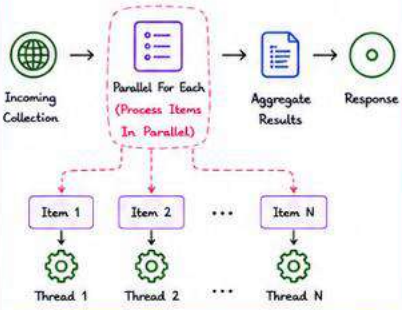
MuleSoft from salesforce

Parallel For Each scope processes each item in a collection concurrently using multiple threads. It improves performance and reduces overall processing time for large datasets.

WHAT IS PARALLEL FOR EACH?

- Processes items in a collection in parallel using multiple threads.
- Reduces total processing time significantly.
- Improves system throughput and responsiveness.
- Ideal for independent item processing.
- Supports error handling and aggregation.
- Configurable concurrency (maxConcurrency).

HOW PARALLEL FOR EACH WORKS



Each item is processed by a separate thread.
Results are aggregated after all threads complete.

BASIC SYNTAX

```
<foreach collection="# [payload.items]">
  <parallel-foreach
     maxConcurrency="5">
    <!-- Process each item in parallel -->
    <logger level="INFO"
       message="Processing item #[item]" />
  </parallel-foreach>
  <!-- Aggregate results -->
</foreach>
```

Attribute	Description
collection	Collection of items to iterate.
maxConcurrency	Maximum number of concurrent threads.
target	Optional target to store aggregated results.
continueOnError	Whether to continue processing remaining items on error.

PARALLEL FOR EACH IN A FLOW

EXAMPLE FLOW SNIPPET

```
<flow name="parallel-foreach-flow">
  <http-listener config-ref="HTTP_Listener_config" path="/processItems" />
  <set-payload value="#[payload.items]" />

  <foreach collection="#[payload]">
    <parallel-foreach maxConcurrency="5"
       target="processedItems" continueOnError="true">
      <logger level="INFO" message="Processing item #[item.id]" />
      <!-- Business logic for each item -->
      <set-variable variableName="result" value="#[processItem(item)]" />
    </parallel-foreach>
    <logger level="INFO"
       message="All items processed. Count: #[sizeOf(vars.processedItems)]" />
  </foreach>
</flow>
```

CONFIGURATION OPTIONS

Option	Description
maxConcurrency	Maximum number of threads to process items in parallel.
target	Variable to store the list of results.
continueOnError	true: Continue processing other items on error. false: Stop processing on first error.
timeout	Maximum time to wait for all threads to complete.
waitForCompletion	true: Wait for all threads to finish (default). false: Continue without waiting.

ERROR HANDLING

Use on-error-continue or on-error-propagate inside the scope.

```
<parallel-foreach maxConcurrency="5">
  <!-- Process item -->
  <try>
    <error-handler>
      <on-error-continue type="ANY">
        <logger level="ERROR"
          message="Error processing item #[item.id]:
                   #[error.description]" />
      </on-error-continue>
    </error-handler>
  </try>
</parallel-foreach>
```

COMMON USE CASES

- Processing large lists of records from database or APIs.
- Bulk data transformations or validations.
- Parallel calls to external systems or microservices.
- Generating reports or documents for many items.
- Real-time data enrichment at scale.

BEST PRACTICES

- Choose an optimal maxConcurrency based on system capacity.
- Ensure items are independent to avoid data conflicts.
- Handle errors per item to avoid failing the entire batch.
- Avoid heavy operations inside threads; keep them efficient.
- Monitor thread usage and system resources.

INTERVIEW QUESTIONS

1. What is Parallel For Each in MuleSoft?
2. How does Parallel For Each differ from For Each?
3. How do you control the number of threads?
4. How are results aggregated in Parallel For Each?
5. How do you handle errors in Parallel For Each?

KEY TAKEAWAY

Parallel For Each helps you process items concurrently, reducing overall processing time and improving performance for large datasets when items are independent.

Use Parallel For Each to process more items in less time and build high-performance Mule applications!

5.18 VALIDATION MODULE

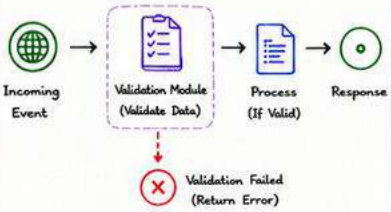
MuleSoft from salesforce

Validation Module scope validates data against defined rules or schemas. It ensures data quality, prevents invalid data from processing further, and provides clear error messages.

WHAT IS VALIDATION MODULE?

- Validates incoming data against rules or schemas.
- Supports built-in and custom validation rules.
- Prevents invalid data from moving forward.
- Provides meaningful error messages.
- Easily reusable and configurable.

HOW VALIDATION MODULE WORKS



If the data is valid, it proceeds to the next step.
If invalid, an error is returned with details.

BASIC SYNTAX

```
<validation:validate
    doc:name="Validate Data"
    config-ref="Validation_Config" />
```

Attribute	Description
doc:name	Name of the validation module.
config-ref	Reference to the validation configuration.

VALIDATION MODULE IN A FLOW

EXAMPLE FLOW SNIPPET

```
<flow name="user-validation-flow">
    <http:listener config-ref="HTTP_Listener_config" path="/user" />
    <set-payload value="#[payload]" />

    <validation:validate doc:name="Validate User"
            config-ref="User_Validation_Config" />

    <logger level="INFO" message="User data is valid." />
    <flow-ref name="process-user-flow" />

    <error-handler>
      <on-error-continue type="VALIDATION:INVALID_INPUT_DATA">
          <logger level="ERROR"
                  message="Validation failed: #[error.description]" />
          <set-payload value="{{"status": "error",
                                "message": "#[error.description]"}" />
      </on-error-continue>
    </error-handler>
</flow>
```

VALIDATION CONFIGURATION EXAMPLE

```
<validation:config name="User_Validation_Config" doc:name="User Validation Config">
  <validation:rule name="Validate User Payload">
    <validation:all>
      <validation:rule doc:name="Name is Required">
        <validation:not-null value="#[payload.name]" />
      </validation:rule>
      <validation:rule doc:name="Email Format is Valid">
        <validation:expression
          expression="#[payload.email matches '^[^@]+@[^@]+\.[^@]+$"]" />
      </validation:rule>
      <validation:rule doc:name="Age is Greater than 18">
        <validation:expression expression="#[payload.age > 18]" />
      </validation:rule>
    </validation:all>
  </validation:rule>
</validation:config>
```

ERROR HANDLING

Validation errors are handled using on-error-continue or on-error-propagate.

```
<error-handler>
  <on-error-continue type="VALIDATION:INVALID_INPUT_DATA">
    <logger level="ERROR"
            message="Validation failed: #[error.description]" />
    <set-payload value="{{"status": "error",
                          "message": "#[error.description]"}" />
  </on-error-continue>
</error-handler>
```

COMMON USE CASES

- Validating API request payloads.
- Ensuring required fields are present.
- Checking data formats (email, phone, date, etc.).
- Verifying business rules (age > 18, amount > 0, etc.).
- Preventing bad data from entering systems.

BEST PRACTICES

- Keep validation rules simple and focused.
- Use descriptive error messages.
- Reuse validation configurations across flows.
- Fail fast to avoid unnecessary processing.
- Version validation configs for maintainability.

INTERVIEW QUESTIONS

1. What is Validation Module scope in MuleSoft?
2. How does Validation Module work?
3. What are the different validation rules supported?
4. How do you handle validation errors?
5. Can you reuse validation configurations?

KEY TAKEAWAY

Validation Module ensures only valid and quality data moves forward in your integration flows, improving reliability and data integrity.

Use Validation Module to ensure data quality, prevent errors, and build reliable integrations!

5.18 ERROR HANDLING

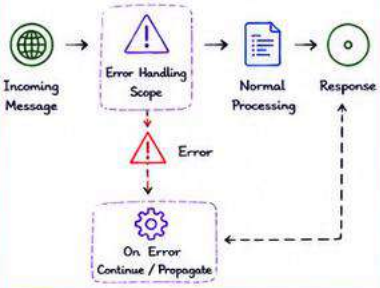
MuleSoft from salesforce

Error Handling scope manages errors that occur during message processing. It allows you to catch, handle, and respond to errors gracefully without disrupting the flow, improving reliability and user experience.

WHAT IS ERROR HANDLING?

- Catches errors that occur in a flow.
- Prevents application crashes and ensures graceful degradation.
- Returns meaningful error responses to clients.
- Supports different error types and strategies.
- Centralizes error handling logic.

HOW ERROR HANDLING WORKS



If an error occurs → it is caught by the Error Handling scope → handled accordingly → flow continues or returns an error response.

BASIC SYNTAX

```
<error-handler>
  <on-error-continue type="ANY">
    <!-- Handle the error -->
    <logger level="ERROR"
      message="An error occurred: #[error.description]"/>
    <!-- Build custom error response -->
  </on-error-continue>
  <on-error-propagate type="HTTP:NOT_FOUND">
    <!-- Propagate specific errors -->
  </on-error-propagate>
</error-handler>
```

Element	Description
on-error-continue	Handles the error and continues processing the flow.
on-error-propagate	Propagates the error to the caller.
type	Error type to catch (e.g., ANY, HTTP:NOT_FOUND).

ERROR HANDLING IN A FLOW

EXAMPLE FLOW SNIPPET

```
<flow name="create-order-flow">
  <http:listener config-ref="HTTP_Listener_config" path="/orders"/>
  <set-payload value="#[payload]"/>
  <error-handler>
    <on-error-continue type="ANY">
      <logger level="ERROR"
        message="Error occurred: #[error.description]"/>
      <set-variable variableName="errorMessage"
        value="#[error-description]"/>
      <set-payload value="#[{'status':'error',
                            'message': vars.errorMessage}]"/>
      <set-property propertyName="httpStatus" value="500"/>
    </on-error-continue>
  </error-handler>
  <db:insert config-ref="DB_Config"/>
  <set-payload value="{"status":"success"}" mediaType="application/json"/>
</flow>
```

ERROR TYPES EXAMPLES

Error Type	Description
ANY	Catches all types of errors.
DEF:VALIDATION	DataWeave validation errors.
HTTP:NOT_FOUND	HTTP 404 errors.
HTTP:BAD_REQUEST	HTTP 400 errors.
DB:CONNECTIVITY	Database connectivity errors.
CUSTOM	Custom application errors.

ERROR HANDLING STRATEGIES

- Use on-error-continue for recoverable errors.
- Use on-error-propagate for critical errors.
- Return meaningful error messages to clients.
- Log errors for monitoring and troubleshooting.
- Set appropriate HTTP status codes.

ERROR HANDLING SNIPPET (ON-ERROR-PROPAGATE)

```
<error-handler>
  <on-error-propagate type="HTTP:NOT_FOUND"/>
</error-handler>
```

ERROR RESPONSE STRUCTURE (EXAMPLE)

```
{
  "status": "error",
  "message": "Resource not found",
  "errorCode": "404",
  "timestamp": "2024-05-24T10:30:00Z"
}
```

COMMON USE CASES

- Handling system or integration errors.
- Returning user-friendly error responses.
- Managing validation and business rule errors.
- Handling timeouts and connectivity issues.
- Ensuring graceful degradation of services.

BEST PRACTICES

- Handle specific errors before using ANY.
- Provide clear and consistent error messages.
- Avoid exposing internal error details to clients.
- Log errors with sufficient context.
- Monitor error rates and trends.

INTERVIEW QUESTIONS

1. What is Error Handling scope in MuleSoft?
2. How does Error Handling scope work?
3. What is the difference between on-error-continue and on-error-propagate?
4. How do you handle different types of errors?
5. How do you return custom error responses in MuleSoft?

KEY TAKEAWAY Error Handling scope helps you catch, manage, and respond to errors gracefully, ensuring application reliability and a better user experience.

Handle errors gracefully, respond meaningfully, and build reliable Mule applications!

5.19 OBJECT STORE

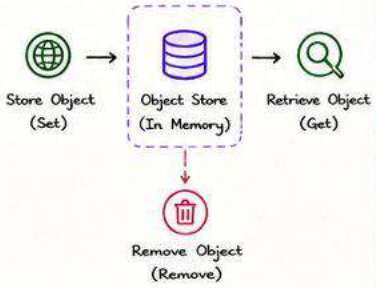
MuleSoft from salesforce

Object Store scope stores and retrieves Java objects in memory during the application runtime. It enables caching and sharing data across flows and requests for better performance and efficiency.

WHAT IS OBJECT STORE?

- Stores Java objects in memory during runtime.
- Provides fast access to frequently used data.
- Enables data sharing across flows and requests.
- Supports local or distributed storage.
- Ideal for caching, session management, and reference data.

HOW OBJECT STORE WORKS



Objects are stored as key-value pairs. Values can be any Java object.

BASIC SYNTAX

```
<object-store:config name="Object_Store_Config">
  <object-store:object-store
      type="local"
      persistent="true"
      entryTtl="Server Object" />
</object-store:object-store>
<object-store:set key="myKey"
    target="Store Object"
    value="#[payload]" />
<object-store:get key="myKey"
    target="Retrieve Object" />
<object-store:remove key="myKey"
    target="Remove Object" />
</object-store:config>
```

Attribute	Description
key	Unique key used to store or retrieve the object.
value	The Java object to store.
target	Variable to store the retrieved object.

OBJECT STORE IN A FLOW

EXAMPLE FLOW SNIPPET

```
<flow name="user-session-flow">
  <http:listener config-ref="HTTP_Listener_config" path="/login-data" />
  <set-payload value="#[payload]" />
  <!-- Store object in Object Store -->
  <object-store:set key="userData" value="#[payload]" />
  <!-- Retrieve object from Object Store -->
  <object-store:get key="userData" target="StoredData" defaultValue="Not Object" />
  <logger level="INFO" message="Stored Data: #[vars.storedData]" />
  <!-- Remove object from Object Store -->
  <object-store:remove key="userData" target="Remove Object" />
</flow>
```

KEY ATTRIBUTES

Attribute	Description
key	Unique identifier for the object.
value	Java object to be stored.
target	Variable to store the retrieved object.
store-scope	Scope type (local or application (default: local)
expiration-interval	Interval after which objects expire automatically.
max-entries	Maximum number of objects that is removed.
persistent	Whether the store is persistent (or distributed stores).

BEST PRACTICES

- Use meaningful keys for easy identification.
- Avoid storing large objects.
- Set appropriate expiration and idle times.
- Remove objects when no longer needed.
- Use distributed Object Store for clustered environments.
- Monitor memory usage and clean up regularly.

COMMON USE CASES

- Caching reference data for faster access.
- Storing user session or preference data.
- Sharing data between flows in the same application.
- Storing temporary results for batch processing.
- Rate limiting and throttling counters.

CONFIGURATION OPTIONS

Option	Description
store-scope	Store objects in local or distributed scope.
expiration-interval	Time interval to automatically expire objects (ms).
max-entries	Max number of objects allowed in the store.
persistent	Enables persistence across restarts (if supported).
entry-ttl	Default validity period for stored objects.

ERROR HANDLING

Handle errors while storing or retrieving objects.

```
<on-error-continue type="ANY">
  <logger level="ERROR"
      message="Error in Object Store operation:
          #[error.description]" />
</on-error-continue>
```

INTERVIEW QUESTIONS

1. What is Object Store in MuleSoft?
2. How does Object Store differ from variables or properties?
3. When should you use Object Store?
4. What are the key attributes of Object Store?
5. How does distributed Object Store work?

KEY TAKEAWAY

Object Store scope helps you store and retrieve Java objects in memory during runtime, enabling caching, sharing data across flows, improving performance and efficiency.

Use Object Store to cache, share, and reuse data efficiently and build high-performance Mule applications!

5.21 CACHE SCOPE

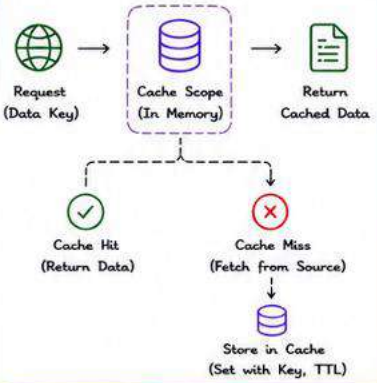
MuleSoft from salesforce

Cache scope stores data temporarily in memory for fast retrieval.
It improves performance by reducing external calls and processing time.
It supports different cache types, TTL expiration, and size limits.

WHAT IS CACHE SCOPE?

- Stores data in memory for fast access.
- Reduces external service calls and latency.
- Supports TTL (time to live) expiration.
- Multiple cache types: OBJECT, STRING.
- Configurable size limit and entry count.
- Supports local or distributed caching.

HOW CACHE SCOPE WORKS



If data is in cache and not expired → return it.
If not in cache or expired → fetch, store, and return the data.

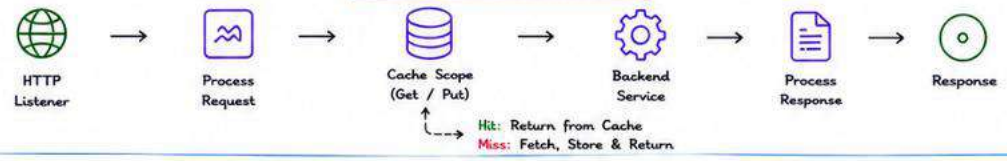
CACHE TYPES

Cache Type	Description
OBJECT	Stores any Java object.
STRING	Stores string values.

BASIC ATTRIBUTES

Attribute	Description
key	Unique key for the cache entry.
value	Value to be stored in cache.
ttl	Time to live for the entry.
timeUnit	Time unit for TTL.
unless	Condition to skip caching.
target	Variable to store the cached value.
defaultValue	Value to return if key not found.

CACHE SCOPE IN A FLOW



CONFIGURATION OPTIONS

Option	Description
entry-count	Maximum number of entries in cache.
time-to-live	Default TTL for entries.
time-unit	Time unit for TTL (SECONDS, MINUTES, etc.).
overflow-policy	Policy when cache is full (e.g., LRU, FIFO).
persistent	Whether cache is persistent (for distributed cache).
store-by-reference	Store object reference instead of value (true/false).

BEST PRACTICES

- Use meaningful and unique cache keys.
- Set appropriate TTL based on data freshness.
- Avoid caching large or frequently changing data.
- Use unless condition to skip null or invalid data.
- Clear cache when underlying data is updated.
- Monitor cache hit/miss ratio for optimization.

COMMON USE CASES

- Caching reference data (countries, currencies, etc.).
- Reducing database or API calls.
- Improving response time for read-heavy flows.
- Storing configuration or lookup data.
- Storing temporary results for batch processing.
- Rate limiting and throttling counters.

ERROR HANDLING

- Use on-error-continue or on-error-propagate.
- Log and handle cache operation failures.
- Set alerts for high error rates.

MONITORING & METRICS

- Track cache hits, misses, and hit ratio.
- Monitor memory usage and entry count.
- Set alerts for high miss ratio or low memory.
- Review and tune TTL and size limits.

INTERVIEW QUESTIONS

1. What is Cache scope in MuleSoft?
2. How does Cache scope improve performance?
3. What is the difference between cache hit and miss?
4. How does TTL work in Cache scope?
5. How can you clear or remove cache entries?

Cache scope stores data in memory for fast retrieval, reduces latency, and improves efficiency.
Use it wisely with proper TTL, size limits, and key strategy for the best results.

Use Cache scope to store, retrieve, and reuse data efficiently and build high-performance Mule applications!

5.22 FILE AND FTP COMPONENTS

MuleSoft from salesforce

File and FTP components enable reading from and writing to files and FTP servers.
They support operations like read, write, list, move, copy, delete, and streaming large files efficiently.

WHAT ARE FILE & FTP COMPONENTS?

- File component works with files in the local file system.
- FTP component works with files on FTP/FTPS servers.
- Support a wide range of operations.
- Handles text, binary, and streaming data.
- Useful for file transfers, data exchange, and batch processing.

HOW IT WORKS

FILE COMPONENT

Source / Destination (File System) → File Component → Process Data → Response

FTP COMPONENT

FTP Server → FTP Component → Process Data → Response

Read / List – Get data from File/FTP
Write – Create or overwrite data
Move / Copy / Delete – Manage files

KEY ATTRIBUTES

- File based (local or FTP).
- Supports operations: read, write, list, move, copy, delete.
- Handles headers, query params, and payloads.
- Supports streaming and timeouts.
- Fully integrated with Mule error handling and security.
- Useful for transfers, exchange, and batch processing.

FLOW OF FILE & FTP COMPONENTS

FILE FLOW

File (Read / Write / List / Move / Delete) → Process Data → Transform / Enrich → Response

FTP FLOW

FTP Server (Read / Write / List / Move / Delete) → Process Data → Transform / Enrich → Response

OPERATIONS SUPPORTED

FILE COMPONENT OPERATIONS

- read : Reads a file from the local file system.
- write : Writes data to a file.
- list : Lists files in a directory.
- move : Moves a file to another location.
- copy : Copies a file to another location.
- delete : Deletes a file.

FTP COMPONENT OPERATIONS

- read : Reads a file from FTP server.
- write : Writes data to FTP server.
- list : Lists files in a directory on FTP server.
- move : Moves a file on FTP server.
- copy : Copies a file on FTP server.
- delete : Deletes a file on FTP server.

CONFIGURATION & EXAMPLE POINTS

FILE CONFIGURATION OPTIONS

- workingDir : Base directory (file component).
- connection : Connection working directory.
- recursive : Enable recursive search.
- maxIdleTime : Max idle time before removing connection.
- persistent : Whether the store is persistent.

FTP CONFIGURATION OPTIONS

- host : FTP server host.
- port : FTP server port (default 21).
- user : FTP user name.
- password : FTP user password.
- passiveMode : Use passive mode in FTP.
- timeout : Timeout (FTP) in ms.

BEST PRACTICES

- Use absolute paths for clarity.
- Handle large files using streaming.
- Use passive mode for FTP.
- Close streams properly.
- Handle exceptions and retry if needed.
- Use secure FTP (FTPS) when possible.

COMMON USE CASES

- File based data exchange.
- Loading or unloading flat files.
- SFTP/FTP file transfers.
- Batch file processing.
- Archiving and file management.
- Integration with legacy systems.

KEY TAKEAWAY

File and FTP components make it easy to work with local files and FTP servers. Use them to read, write, transfer files efficiently and handle large data smoothly!

Use File and FTP components to move, manage, and process files efficiently and build reliable Mule applications!

5.23 HTTP COMPONENTS

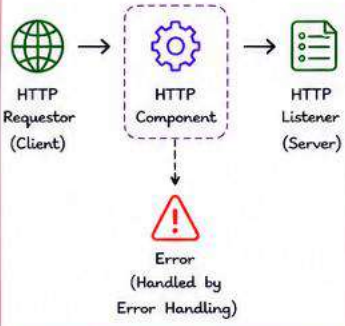
MuleSoft from salesforce

HTTP components allow you to send and receive HTTP requests and responses. They support REST, SOAP over HTTP, streaming, authentication, and error handling to build robust and scalable integrations.

WHAT ARE HTTP COMPONENTS?

- Enable communication over HTTP/HTTPS protocols.
- Support REST, SOAP, and custom APIs.
- Handle requests, responses, headers, query params, and payloads.
- Support authentication, streaming, and timeouts.
- Built-in error handling and status code support.

HOW HTTP COMPONENTS WORK



HTTP Component acts as a client to send requests and receive responses from HTTP services.

BASIC SYNTAX

HTTP REQUESTOR

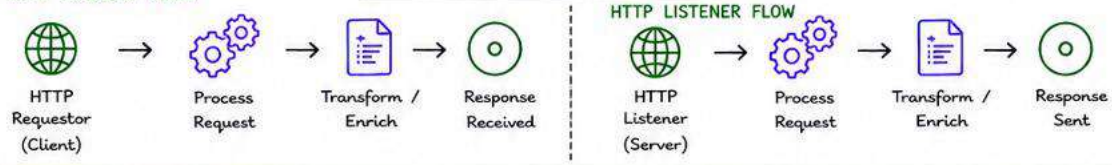
- Sends HTTP requests (e.g., GET, POST).
- Configure path, headers, query params, and timeouts.

HTTP LISTENER

- Listens for incoming HTTP requests.
- Configure path, allowed methods, and timeouts.

Attribute	Description
config-ref	Reference to HTTP configuration.
method	HTTP method (GET, POST, PUT, DELETE).
path	Resource path.
headers	Request or response headers.
body	Request payload.
responseTimeout	Timeout for responses (in ms).

HTTP COMPONENTS IN A FLOW



COMMON HTTP METHODS

- GET · Retrieve data from the server.
- POST · Create a new resource.
- PUT · Update an existing resource.
- PATCH · Partially update a resource.
- DELETE · Delete a resource.
- HEAD · Retrieve headers only.
- OPTIONS Get supported HTTP methods.

COMMON STATUS CODES

- 200 · OK – Request successful.
- 201 · Created – Resource created.
- 204 · No Content – Success, no content.
- 400 · Bad Request – Invalid request.
- 401 · Unauthorized – Auth required.
- 403 · Forbidden – Access denied.
- 404 · Not Found – Resource not found.
- 500 · Internal Server Error.

AUTHENTICATION TYPES

- Basic Authentication
- Digest Authentication
- OAuth 1.0 / 2.0
- Client Certificate (mTLS)
- API Key (Header or Query Param)
- Custom Authentication

BEST PRACTICES

- Use absolute paths for clarity.
- Set appropriate timeouts.
- Handle all status codes and errors.
- Use HTTPS for secure communication.
- Reuse configurations for better performance.

COMMON USE CASES

- Integrate with external APIs and web services.
- Consume REST and SOAP services.
- Streaming large payloads.
- File transfers and data exchange.
- Batch processing and automation.

ERROR HANDLING

- Use on-error-continue and on-error-propagate.
- Log meaningful error details.
- Return proper HTTP status codes.
- Set alerts for high error rates.

HTTP components simplify communication over HTTP/HTTPS, enabling you to build scalable, secure, and efficient integrations with REST APIs and web services.

Use HTTP components to connect, communicate, and create powerful integrations with the world of APIs!

5.24 WEB SERVICE CONSUMER (SOAP)

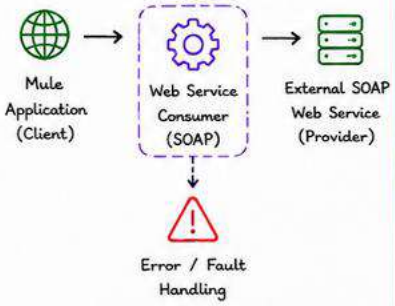
MuleSoft from salesforce

Web Service Consumer (SOAP) allows your Mule application to call external SOAP web services.
It uses WSDL to understand the service contract and supports request/response operations with features like headers, timeouts, security, and error handling.

WHAT IS SOAP CONSUMER?

- Invokes operations on external SOAP web services.
- Uses WSDL to generate service metadata.
- Supports request / response communication.
- Handles SOAP headers, fault responses, and attachments.
- Fully integrated with Mule error handling and security.

HOW IT WORKS



The consumer sends a SOAP request based on the WSDL contract and receives a SOAP response or a fault.

KEY FEATURES

- Communicates over HTTP/HTTPS.
- Supports REST, SOAP, and custom APIs.
- Handles headers, query params, and payloads.
- Supports authentication, streaming, and timeouts.
- Built-in error handling and status code support.

FLOW OF SOAP CONSUMER

SUPPORTED OPERATIONS

- ✓ read : Reads a file from the server.
- ✓ write : Writes data to a file.
- ✓ list : Lists files in a directory.
- ✓ move : Moves a file to another location.
- ✓ copy : Copies a file to another location.
- ✓ delete : Deletes a file.

COMMON HTTP METHODS

- GET : Retrieve data.
- POST : Create a resource.
- PUT : Update a resource.
- PATCH : Partially update a resource.
- DELETE : Delete a resource.
- HEAD : Retrieve headers only.
- OPTIONS : Get supported methods.

COMMON STATUS CODES

- 200 : OK
- 201 : Created
- 400 : Bad Request
- 401 : Unauthorized
- 403 : Forbidden
- 404 : Not Found
- 500 : Internal Server Error

SECURITY SUPPORT

- ✓ WS-Security (UsernameToken).
- ✓ Basic authentication over HTTPS.
- ✓ Client certificates (mTLS).
- ✓ API Key (Header or Query Param).
- ✓ Custom SOAP headers for tokens.

BEST PRACTICES

- ✓ Use WSDL from a reliable source.
- ✓ Set appropriate timeouts.
- ✓ Handle all status codes and errors.
- ✓ Use secure endpoints (HTTPS).
- ✓ Reuse configurations.

COMMON USE CASES

- ✓ Integrate with external enterprise systems.
- ✓ Consume legacy SOAP services.
- ✓ Communication with CRM, ERP, or mainframe systems.
- ✓ Batch processing and data exchange.

Web Service Consumer (SOAP) enables your Mule application to reliably call external SOAP services using WSDL, handle responses, faults, and security efficiently.

MuleSoft
from salesforce
Thank You!
For reading and learning with us.
We hope this MuleSoft Anypoint Platform Components Guide – Volume 1 has helped you build a stronger foundation and boost your integration skills.
Keep practicing, keep building, and keep innovating.
You're now one step closer to becoming a MuleSoft integration expert!
Thank you for being a part of this journey!
Your curiosity and commitment drive your growth and inspire continuous learning.
Volume 2 is coming soon!
More components, more insights, and more real-world scenarios to level up your integration expertise.
Stay tuned!